

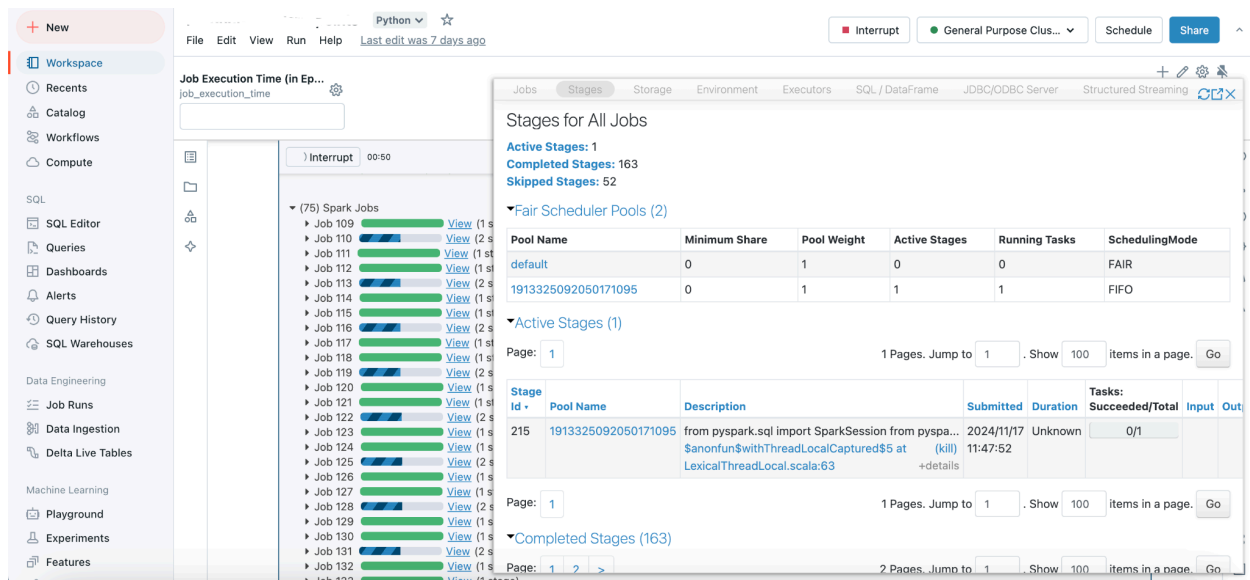
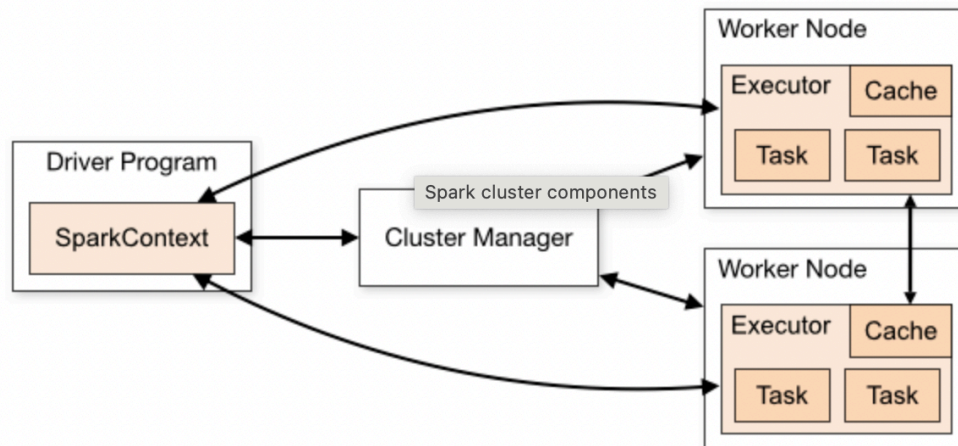
TechStackQs

Many of similar tech stack questions can be checked on
GPT/Open-source-github-repos.

Spark

PySpark

Batch Pipelines



- Spark turns the user's data processing commands into a Directed Acyclic Graph, or DAG, which determines the execution plan, i.e what tasks are executed on what nodes and in what sequence.
- Spark runs in a distributed fashion by combining a Driver core process that splits a Spark application into tasks & distributes them among many Executor processes that do the work. Driver sends tasks to executors via Cluster Manager.
- OpenSource vs Databricks managed Spark:

- **Native Spark** (open-source):
 - If running mode: `--deploy-mode cluster`:
 - spark-submit job → Resource Manager
 - submit-job always talks to RM first.
 - You must configure beforehand: driver-memory, driver-cores, num-executors, executor-memory, executor-cores, dynamic allocation, cluster manager settings, etc., when submitting a job.
 - Local machine can disconnect ("fire and forget") - Prod use.
 - Resource Manager launches Driver (random node inside the cluster)
 - Driver starts running your Spark app
 - Driver requests executors from Resource Manager
 - Resource Manager allocates executors on worker nodes
 - Executors register back with Driver
 - **Note:** If you request more Spark executor cores or instances than your infrastructure actually has, the resource manager would still allocate what is physically available rather than crashing the system.
 - If running mode: `--deploy-mode client`:
 - Driver runs on your machine
 - submit-job starts driver locally, then driver calls RM (next point).
 - In client mode, you "skip" the step of asking the Resource Manager to find a home for the Driver because it's already living on your machine.
 - Local machine must stay online for the job to finish - Local use.
 - Driver talks to Resource Manager
 - Resource Manager allocates executors

- **Databricks (managed Spark):** User submits job → Databricks handles almost EVERYTHING. You only configure high-level things.
- **SparkContext vs SparkSession:**
 - **SparkContext:** Only ONE per JVM; Core connection to cluster; Low level.
 - **SparkSession:** Multiple possible; Wrapper around SparkContext; Abstracted level.
 - `getOrCreate()` → returns existing session if present (same object)
 - `newSession()` → creates a new SparkSession (different object, same SparkContext)
 - `spark.stop()` → required to fully reset everything
- **Data Processing:**
 - **Resilient Distributed Dataset (RDD):** Low-level abstraction.
 - Compile-time checking before the job runs.

```
For ex., type-safety checked:
val df = Seq((1, "Alice")).toDF("id", "name")
df.select("nam").show()
// Typo in column name: "nam" instead of "name".
// Compilation error thrown in start, before job runs
```

- NOT Catalyst/Tungsten engine optimised hence comparatively slow.
 - Use - Low level data transformations, custom partitioning data etc.
- **DataFrame:** Medium-level abstraction.
 - NO compile-time checking.
 - Catalyst/Tungsten optimised.
 - Use - Analytical queries, SQL-like aggregations, joins, filters.
- **Dataset:** High-level abstraction.
 - Compile-time checking.
 - Catalyst/Tungsten optimised.
 - Hence best of dataframe & RDD.
 - Use - Type safety + functional + SQL benefits (Scala/Java only).

- Components: Driver node creates jobs/stages/tasks, and workers execute or process data partitions.
 - **Application** - A user program built on Spark using its APIs. It consists of a driver program and executors/workers on the cluster.
 - **Job** - Refers to a sequence of transformations on data in response to a Spark Action. Each job is converted into a DAG with execution plan/stages.
 - Execution: **Sequential**, unless you explicitly trigger multiple actions asynchronously (e.g., via threads, async APIs, or multiple Spark contexts).
 - Actions: **collect(), count(), take(n)/head(n)/tail(), first(), foreach(func), foreachPartition(func), saveAsTextFile(), saveAsParquet(), saveAsTable(), reduce(func), fold(), aggregate(), countByKey(), collectAsMap(), toPandas(), write.format("").save(), show(), rdd.takeSample(), rdd.saveAs...(), spark.sql("SELECT ...").show(), writeStream.start()**.
 - We see from earlier that **writes trigger a job**. But in case of: **spark.sql("SELECT ...") or spark.read.parquet("path") will only build a logical plan, no action**. If you couple it with **.show()**, then action is triggered.
 - **Exception**: Generally, when reading data, spark would know the schema like if reading parquet data, then from metadata of file, schema is known. But assume, reading CSV (comma-separated-values) then spark may inferSchema. **If it does then spark.read.csv("file.csv", inferSchema=True) could trigger 1 job** unlike our previously discussed point.
 - Note:
 - **count()** is an action that triggers a job. Computation happens on executors, and only a final aggregated count is returned to the driver → memory safe but requires full data scan.
 - **collect()** brings all data to the driver → can cause OOM, should be avoided for large datasets.

- `head(n)` is an action that retrieves first `n` rows and may stop early after scanning enough partitions → more **efficient** than full scan.
 - `tail(n)` is **inefficient** in distributed systems since Spark may need to scan the entire dataset → generally avoided.
- **Stage** - Each job gets divided into smaller sets of tasks called stages, in which a sequence of transformations can be executed in a single pass, i.e., without any shuffling of data. **The boundary between two stages is due to data shuffling.** Transformations never “bring data” immediately (they’re lazy - will discuss). But they define how tasks and stages will run once an action triggers them.
- Execution: **Sequential**
 - Transformations:
 - **Wide (creates a new stage):** `reduceByKey()`, `groupByKey()`, `groupByKey()`, `join(df, on="id", how="left")`, `distinct()`, `repartition()`, `sortBy()`, `orderBy()`, `sortWithinPartitions()`, `cogroup()`, `cartesian()`, `aggregateByKey()`.
 - **Narrow:** `map()`, `filter()`, `union()`, `flatMap()`, `withColumn()`, `drop() / alias()`, `coalesce()`, `mapPartitions()`, `sample()`, `mapValues()`, `keyBy()`, `select() / where() / limit()`, `cache() / persist() / checkpoint()`, `createOrReplaceTempView()`.
 - Note:
 - If you do: `coalesce(numPartitions, shuffle=True)` - **then it becomes a wide transformation.** Acts like `repartition` only.
 - `limit(n)` is tricky: narrow if no global order, wide if combined with `orderBy`.

- **Task** - A task is the smallest unit of execution in Spark, executed on a single core of an executor. Each stage is divided into multiple tasks, where each task processes one partition of the data. A task runs all transformations in that stage on its assigned partition. The number of tasks equals the number of partitions, and the number of tasks that can run in parallel depends on the total available cores in the cluster.
 - Execution: Parallel
- Example:

```
spark = SparkSession.builder \
    .appName("MultipleJobsStagesTasksExample") \
    .master("local[4]") \
    # local[4] → uses 4 cores → at most 4 tasks can run in
parallel
    # local[*] → uses all available cores on the machine
    # NOTE:
    # - This controls PARALLELISM (concurrent task execution), NOT
number of partitions directly
    # - spark.default.parallelism may be derived from cores if not
explicitly set
    .config("spark.executor.memory", "2g") \
    .config("spark.driver.memory", "1g") \
    .config("spark.sql.shuffle.partitions", "16") \
    # Default for spark.sql.shuffle.partitions = 200 → overridden
to 16 (affects shuffle stages like groupBy)
    .getOrCreate()

# DATA SOURCE
df = spark.createDataFrame(data, columns)

# NOTE:
# - createDataFrame from in-memory data → usually creates 1
partition
# - "size / 128MB" rule DOES NOT apply here
# - That rule applies when reading from files (parquet/orc/etc.):
#     partitions ≈ total_data_size /
```

```

spark.sql.files.maxPartitionBytes (default 128MB)

# WIDE TRANSFORMATION → causes shuffle
df = df.repartition(8, "group_id")
# After this → df has 8 partitions

# TRANSFORMATIONS (lazy)
df_filtered = df.filter(col("amount") > 20)
# narrow transformation → no shuffle → pipelined
df_mapped = df_filtered.withColumn("amount_plus_5", col("amount")
+ 5)
# still narrow → stays in same stage

# WIDE TRANSFORMATION → causes shuffle
df_grouped = df_mapped.groupBy("group_id") \
.agg(avg("amount_plus_5").alias("avg_amount"))

# ACTION 1 → triggers Job 1
df_grouped.show()

# ACTION 2 → triggers Job 2 (recomputes lineage since no cache)
df_mapped.count()

# -----
# Job 1 → df_grouped.show()

# Stage 1:
# createDataFrame → repartition(8, group_id) → shuffle (map
side)
# tasks = initial partitions of df (likely 1, since
createDataFrame)

# Stage 2:
# repartition(8, group_id) → shuffle (reduce side)
# + filter + withColumn → narrow transformations (pipelined)
# tasks = 8 (new partitions after repartition)

```

```
# Stage 3:
# groupBy → shuffle (map side)
# tasks = 8 (input partitions)

# Stage 4:
# groupBy → shuffle (reduce side / aggregation)
# tasks = 16 (controlled by spark.sql.shuffle.partitions)

# Driver collects limited rows (default 20) for show()

# -----
# Job 2 → df_mapped.count()

# Stage 1:
# createDataFrame → repartition(8, group_id) → shuffle (map
side)
# tasks = initial partitions (likely 1)

# Stage 2:
# repartition(8, group_id) → shuffle (reduce side)
# + filter + withColumn → pipelined
# tasks = 8

# Final:
# Each partition computes partial count
# → tree aggregation → final result returned to driver

# -----
# NOTES:
# 1. Partition vs Parallelism:
#   - Partitions → define number of tasks
#   - Cores (local[4]) → define how many tasks run in parallel
# 2. File-based reads:
#   - partitions ≈ data_size / 128MB (default)
#   - controlled by: spark.sql.files.maxPartitionBytes
# 3. In-memory (createDataFrame):
#   - usually 1 partition unless explicitly repartitioned
```

```
# 4. repartition(n):
#   - full shuffle
#   - sets number of partitions to n
# 5. spark.sql.shuffle.partitions:
#   - controls number of partitions AFTER shuffle (e.g., groupBy)
# -----
```

- Spark does **lazy evaluation**. Transformations (filter, map, groupBy) do NOT execute immediately. They build a logical plan (DAG). Only when you call an action like show(), count(), write(); Spark triggers execution.
 - Plans are of 2 types: Logical + Physical.
 - Evaluation Engines:
 - **Catalyst** (Spark's **query optimization** engine that converts a logical plan into an optimized physical execution plan using rule-based and cost-based optimizations)
 - **Tungsten** (Spark's **physical execution** engine that focuses on CPU and memory efficiency using code generation, off-heap memory, and vectorized processing)
 - Plan lifecycle:

```
Unresolved Logical Plan
  ↓ (Catalyst: Analysis)
Analyzed Logical Plan
  ↓ (Catalyst: Logical Optimization)
Optimized Logical Plan
  ↓ (Catalyst: Physical Planning + CBO)
Physical Plan
  ↓ (Tungsten: Whole-Stage Codegen)
Optimized Physical Plan (with fused operators)
  ↓ (Tungsten: Execution Engine)
Execution:
- Off-heap memory management
- UnsafeRow (binary format)
- Cache-aware execution
- Vectorized processing (batch execution)
  ↓
Tasks run on Executors (1 task per partition)
```


↓
Result returned to Driver

- **Unresolved Logical Plan:** Just the code what you wrote.
- **Analyzed Logical Plan:** Resolves: Column names, Data types, Table references
- **Optimized Logical Plan:** Rule-based and cost-based optimizations applied by Catalyst like:
 - **Predicate Pushdown:** filter early → less data read
 - **Projection Pushdown (Column pruning):** read only required columns
 - **Constant Folding:** simplify expressions at compile-time rather than execution-time. Eg: `SELECT * FROM users WHERE age > (10 + 5)` --> `SELECT * FROM users WHERE age > (15)`; Other ex: `SELECT name FROM users WHERE age > 30 AND 1 = 1` --> Removing the `1=1`.
 - **Null Propagation:** If it detects that a transformation will inevitably result in null because one of the constants is null, it can skip the calculation entirely.
 - **Join Optimization:** Choosing joins (broadcast, shuffle, etc. - will discuss); Avoiding unnecessary shuffles.
 - **Operation Reordering:** Eg: `map → join → filter` to `filter → map → join` to reduce dataset size
 - **Pipeline Fusion:** Here Spark fuses multiple consecutive operations (like filter, map, and project) into a single, highly optimized function. So instead of moving data from one function to the next, data stays in CPU registers as long as possible.
- **Physical Plan:** Spark chooses how to execute based on cost (**Cost Based Optimizer**). Cost includes: Data size (bytes), Row count, Shuffle cost (network I/O), CPU cost, Memory usage. Config to enable it: `spark.conf.set("spark.sql.cbo.enabled", True)`.
- **Optimized Physical Plan:**
 - **Whole-Stage Code Generation:** Instead of processing data row-by-row with multiple function calls, Spark: generates

one optimized Java program for the entire stage - via Janino (JIT compiler) and runs directly on JVM. So: No virtual function calls, No object creation per row, Tight CPU loop → better performance.

Eg:

> Old (Iterator Model -- slow)

Each operator runs separately

Too many function calls

Row → Filter → Row → Map → Row → Output

```
while (iterator.hasNext()) {  
    Row row = iterator.next();  
    if (row.value > 10) {  
        output.append(row.value * 2);  
    }  
}
```

> New (Whole-stage codegen -- fast)

Spark merges all operations into one loop

```
for (int i = 0; i < numRows; i++) {  
    if (column[i] > 10) {  
        output[i] = column[i] * 2;  
    }  
}
```

- **Off-Heap Memory Management**: Instead of storing data as Java objects (which cause GC overhead): Spark stores data in raw binary format outside JVM heap. Uses low-level APIs like `sun.misc.Unsafe`.
- **Cache-Aware Execution (UnsafeRow)**: Data is stored in contiguous memory blocks. Instead of: Row → object → pointer → value; Spark uses: Flat memory → direct offset access. So no pointer chasing and faster data access.
 - **UnsafeRow is the internal binary format** Spark uses to represent a row of data in memory.
- **Vectorized Processing**: Instead of processing 1 row at a time: Spark processes batches of rows (column-wise). Files like parquet/ORC are perfect for vectorized processing due to columnar format.

- **Execution Runtime:**

- **Adaptive Query Execution (AQE)** happens at runtime, after the physical plan is created. Enable AQE via: `spark.conf.set("spark.sql.adaptive.enabled", "true")`. It does:

- **Dynamic Join Strategy Change:** Say planned was Shuffle Join but may switch to Broadcast Join if data is small.
- **Coalesce Shuffle Partitions:** Reduce 200 → 20 partitions to avoid too many small tasks.
- **Skew Join Optimization:** Detects skewed partitions (very large ones); Splits them → avoids slow tasks
- **Better Aggregation Decisions.**

- Partition is a logical chunk of data that Spark processes in a task on a core. Ideal number: 2-3 partitions on a core. Default partitioning: automatically partitions data based on the underlying file system block size — typically 128 MB or 256 MB per block in HDFS/S3. Eg: Reading 3 files of 1 GB → $\approx 8 \times 3 = 24$ partitions (1 GB / 128 MB ≈ 8).

- Types of Partitioning:

- **Hash Partitioning:** Takes your key, applies a hash function, and assigns it to a partition by: `partition = hash(key) % numPartitions`. Deterministic distribution but if one key repeats a lot, that partition maybe skewed. Eg: `join()`, `groupByKey()`, `reduceByKey()`, `distinct()`. Code ex:

```
rdd = sc.parallelize([("A", 1), ("B", 2), ("A", 3), ("C", 4)], numSlices=3)
```

```
grouped = rdd.groupByKey() # triggers hash partitioning
```

Other ex:

Say data: 1,2,3,4,5,6... If hash partition, i.e `hash(id)%4` (Assume 4 constant used) then data would go accordingly to:

```
1 in partition p1
2 in partition p2
3 in partition p3
4 in partition p0
5 in partition p1, etc...
```

- **Range Partitioning:** Sorts keys and divides them into contiguous ranges, so each partition gets a range of keys, say:

partition 0: keys < 100, partition 1: 100–199, etc. Eg:
sortByKey(), Range-based joins, etc. Code ex:

```
data = [(5, "A"), (1, "B"), (10, "C"), (20, "D"), (100, "E")]
rdd = sc.parallelize(data, 2)
sorted_rdd = rdd.sortByKey(numPartitions=3) # internally uses
RangePartitioner
```

- **Custom Partitioning**: By extending the `Partitioner` class (RDD-level only, not DataFrame).
- Adjust partitions using:
 - **repartition(n)** – increase/decrease partitions with full shuffle (expensive but creates balanced, equal-sized partitions).
 - **coalesce(n)** – reduce partitions without full shuffle (cheap but may create uneven partitions; good for writing smaller outputs). It has a `shuffle=True/False` flag.
 - Sidenote: You may see a similar partition function called: **partitionBy()**. It is NOT the same as `repartition`. **partitionBy() is used only during writes**. Physical data layout on disk is affected (organizing files), not execution-time partitioning. Useful for query pruning. It triggers a shuffle. Eg:

```
df.write.partitionBy("country").parquet("/path").
This creates directory structure like:
/path/
country=US/
country=IN/
country=UK/
```

- Partition strategy:
 - Count: If high -> excessive overhead in managing many small tasks, If low -> underutilised cores in cluster.
 - Size: If large -> long computation time, slow write times, executor/worker OOM, If small -> slow read time downstream, large task creation overhead, driver OOM
 - Avoid partition on high-cardinality keys (e.g., `user_id`) → causes small files problem, skew, catalogue explosion, and slow query planning.

- **Caching:** Cache intermediate DataFrames to avoid recomputation across stages. Although the number of stages/tasks created would be in expected lines and not reduced, but skipped. Cache vs Persist vs Checkpoint:
 - **Cache:** Stores the DataFrame in memory (RAM) by default. It's equivalent to: `df.persist(StorageLevel.MEMORY_ONLY)`. To remove cache ~ `df.unpersist()`
 - **Persist:** More flexible version of `cache()` — you can choose where and how to store data: Memory only, Memory + Disk, Disk only, Serialized, etc.
 - **Checkpoint:** It's like a savepoint. Breaks the lineage of a DataFrame (cuts off its dependency graph); Writes data to disk (HDFS/S3) as a new materialized copy; Used for fault tolerance and state management (streaming or long transformations). Forces a job execution. **Use a checkpoint if lineage is too long.** Eg:


```
# Set the checkpoint dir
spark.sparkContext.setCheckpointDir("/mnt/checkpoints")
# Doing the checkpoint
df.checkpoint()
```

Eg:

```
from pyspark.sql.functions import col
df = spark.read.parquet("big_data.parquet")
df1 = df.filter(col("amount") > 100)          # narrow
df2 = df1.withColumn("x", col("amount") * 2)  # narrow
# df2.cache() # caching if required
# Reused multiple times
df3 = df2.groupBy("category").count()         # shuffle
df4 = df2.groupBy("region").avg("x")          # shuffle
df3.show()  # Action 1 → Job 1
df4.show()  # Action 2 → Job 2
## Without cache df2 is recomputed twice for df3/df4
## With cache, work is reduced inside stages
```

- Joins:
 - **Broadcast Hash Join:** Send the small table to every machine, and join it with chunks of the large table locally. No need to move the large table around. Related config:
spark.sql.autoBroadcastJoinThreshold

How it works (example)

Data:

Small table (users)

id	name
1	A
2	B

Large table (transactions)

id	amount
1	100
2	200
1	300

Execution:

Small table **is** copied to all executors:

Executor 1 → gets full small table

Executor 2 → gets full small table

Each executor processes its chunk of large table:

Executor 1:

(1,100), (2,200)

Executor 2:

(1,300)

For each row **in** large table:

Look up matching id **in** small table

Join immediately

Result:

(1,A,100)

(2,B,200)

(1,A,300)

- **Shuffle Hash Join:** Move both tables so that same keys come together, then: Build a quick lookup (hash map) from smaller data; Then for each row in larger data → find matches. Less used these days.

How it works (example)

Data:

df1 (small)

df2 (large)

id val	id val
1 A	1 X
2 B	2 Y
1 C	3 Z

Step 1: Shuffle both tables by id

Partition 1 (id=1) Partition 2 (id=2,3)

df1: (1,A), (1,C) df1: (2,B)

df2: (1,X) df2: (2,Y), (3,Z)

Now matching keys are together

Step 2: Inside each partition build lookup from smaller table;

HashMap:

1 → [(A), (C)]

Now process larger table rows:

Row: (1,X)

→ find id=1 in hashmap

→ matches (A), (C)

Result:

(1,A,X)

(1,C,X)

...

- **Sort Merge Join:** Move both tables so the same keys come together, then sort both sides; then walk through them together and match rows. Related config: `spark.sql.join.preferSortMergeJoin`

How it works (example)

Data:

df1	df2
id val	id val
1 A	1 X
2 B	2 Y
1 C	3 Z

Step 1: Shuffle both tables by id; Now partitions contain same keys

Step 2: Sort inside each partitions

Partition A:

df1: (1,A), (1,C)

```

df2: (1,X)
Partition B:
df1: (2,B)
df2: (2,Y), (3,Z)
# Already sorted but Spark ensures sorting
Step 3: Walk through both (like merging sorted arrays)
Compare:
1 vs 1 → Say in partition A -> 1A-1X, 1C-1X -> match → output.
Other id's no match.
2 vs 2 → Say in partition B -> 2B-2Y -> match → output. Other
id's no match.
Result: (1,A,X), (1,C,X), (2,B,Y)

```

****Note**:**

- A Spark partition **is** NOT just raw bytes like a file block. A logical chunk of data processed by one task, exposed **as** an iterator of rows.
- Spark sees something like: Partition A: df1_iter = Iterator[(1,A), (1,C)] **and** df2_iter = Iterator[(1,X)
- It **is not** merged into one single contiguous array. It's **two independent streams of data in partition**. Contiguous bytes idea may come from thinking in terms of HDFS/storage, but during execution, Spark converts that into row iterators / UnsafeRows.

- **Bucketing** = pre-shuffling data by a key and storing it that way on disk. Data is divided into a fixed number of buckets using:
`bucket = hash(join_key) % num_buckets.`
Rows with the same key → always go to the same bucket.
 - **It happens only during write**, i.e saved as tables. Syntax: `.bucketBy(...).saveAsTable(...)`. It is not applied on normal dataframe in memory generally.
 - Bucketing is a storage-level optimization, not runtime transformation. Eg:

```

df1.write.bucketBy(4,
"id").sortBy("id").saveAsTable("bucketed_df1")

```



```
df2.write.bucketBy(4,
"id").sortBy("id").saveAsTable("bucketed_df2")
Now: Both tables have 4 buckets; Same hashing logic → id = 1 goes
to the same bucket number in both tables.
```

- If both tables are bucketed on the same key, and we have the same number of buckets, then Spark can do bucket-to-bucket join.
- Bucketing may not always avoid shuffling.
- Skewness in data: When some keys/partitions have much more data than others, causing: Few tasks run very slow (stragglers) + Other tasks finish quickly → resources idle. To solve, different ways like Broadcast join, Increase partitions, Adaptive Skew Join, Preaggregation (apply filters early), etc. We'll discuss more on Salting.
 - **Salting:** Break one heavy key into multiple smaller keys. So the heavy key split into multiple partitions. Load gets distributed.

Eg:

```
# Create a "Salted Key". Instead of '1', we get '1_0', '1_4',
'1_9', etc.
```

```
df_large_salted = df_large.withColumn(
    "salted_dept_id",
    concat(col("dept_id"), lit("_"), floor(rand() * 10))
)
```

```
salt_array = array([lit(i) for i in range(10)])
```

```
# Explode the small table so each dept_id exists 10 times
```

```
df_small_exploded = df_small.withColumn("salt",
explode(salt_array)).withColumn("salted_dept_id",
concat(col("dept_id"), lit("_"), col("salt")))
```

Sidenote on: What explode does:

It converts one row with an array/map into multiple rows.

Input:

id	items
1	[A, B, C]
2	[D, E]

```
from pyspark.sql.functions import explode
df.select("id", explode("items").alias("item"))
```

Output:

id	item
1	A
1	B
1	C
2	D
2	E

Coming back, to salting: Assume data:

Large table (orders)

dept_id	order
1	o1
1	o2
1	o3
1	o4 ← 🔥 heavy key
2	o5

Small table (dept info)

dept_id	name
1	Sales
2	HR

What goes wrong without salting: During join: All dept_id = 1 → go to SAME partition.

One task gets: (1,o1), (1,o2), (1,o3), (1,o4)

Other tasks are almost empty

One slow task → overall job slow (data skew)

Hence we add salting.

Step 1: Add salt to LARGE table: salt ∈ {0,1}.

dept_id	order	salt
1	o1	0
1	o2	1
1	o3	0
1	o4	1

```
2      | o5      | 0
```

Heavy key (1) is now split: (1,0) and (1,1). Load distributed.

Problem now: Small table is still:

```
1 | Sales
```

```
2 | HR
```

It does NOT have salt. So join like: (1,0) ≠ (1) ✗, (1,1) ≠ (1)

✗

Join will FAIL / miss data.

Step 2: Expand SMALL table: Duplicate small table for all salts;

It is intentional.

```
dept_id | name | salt
```

```
1      | Sales | 0
```

```
1      | Sales | 1
```

```
2      | HR    | 0
```

```
2      | HR    | 1
```

Step 3: Join on (dept_id, salt). Now matching works:

```
(1,o1,0) → matches (1,Sales,0)
```

```
(1,o2,1) → matches (1,Sales,1)
```

```
(1,o3,0) → matches (1,Sales,0)
```

```
(1,o4,1) → matches (1,Sales,1)
```

Result:

```
dept_id | order | name
```

```
1      | o1     | Sales
```

```
1      | o2     | Sales
```

```
1      | o3     | Sales
```

```
1      | o4     | Sales
```

```
2      | o5     | HR
```

After join: Remove the salt: drop("salt")

- Executors (Fat/Tiny):
 - **Fat Executors:** Few executors with more cores & memory per executor

- **Tiny Executors:** Many executors with fewer cores & memory each
- **Spark UDFs:** UDF (User-Defined Function) is a custom function written by the user to extend Spark's capabilities beyond the built-in functions provided by Spark. Say you have to cater some complex business logic or nested JSON parsing, etc. It runs row by row on executors hence slow.
 - **Native Spark (No UDF):**
 - Execution: Logical plan → optimized by Catalyst
 - Converted to **compiled JVM code (WholeStageCodegen)**.
Runs fully in JVM
 - Runs as a tight **loop inside the JVM**. Vectorized (batch processing). CPU-cache friendly. Predicate pushdown possible.
 - **Eg:** `df.filter(df.age > 30)`
 - **Spark (With UDF):**
 - Execution: JVM → serialize → Python → execute → serialize → JVM
 - Problems:
 - **Serialization Overhead** as data moves from JVM → Python → JVM. Uses Pickle. Happens repeatedly per batch
 - **Cross language:** JVM and Python = separate processes. Communication via sockets/pipes
 - **No Catalyst Optimization.** Spark cannot see inside UDF. So cannot apply push filters down, reorder operations, etc. Also since it executes on worker nodes, logs are not visible in driver node, but only worker nodes.
 - **No Code Generation:** Native → compiled Java. UDF → interpreted Python
 - **Breaks Vectorization:** Native → batch processing. UDF → often row-wise or less efficient batching
 - Pandas UDF could be a better alternative. As it uses **Apache Arrow**. Columnar data transfer. Vectorized execution. Faster than Python UDF, although slower than native spark.

- Spark always deals with serialization at some level, but DataFrames/Datasets minimize and optimize it, while RDDs rely on generic serialization. RDDs store data as JVM/Python objects requiring heavy serialization, while DataFrames use Tungsten's binary format, reducing serialization overhead and enabling optimized execution.
- Data Warehouse vs Lake vs Lakehouse:
 - **Warehouse:** Structured, curated data for analytics & BI.
 - **Lake:** Store everything (raw data) cheaply.
 - **Lakehouse** (Modern approach): Combine Data Lake + Data Warehouse.
 - Others:
 - **Schema-on-Read vs Schema-on-Write**
 - Schema-on-write → validate before storing (warehouse)
 - Schema-on-read → apply when reading (lake)
 - **ETL vs ELT**
 - ETL → Transform before loading (warehouse)
 - ELT → Load first, transform later (lake/lakehouse)
 - Medallion Architecture (Lakehouse): Bronze → raw data, Silver → cleaned, Gold → aggregated
- Copy-on-Write (COW) and Merge-on-Read (MOR) are two strategies for handling row-level updates (like UPDATE or DELETE) in modern data lake formats like Apache Iceberg, Hudi, and Delta Lake
 - **COW:**
 - Mechanism: Total Replacement. When a row is modified, the system locates the file containing that row and rewrites a brand-new version of the entire file with the changes applied.
 - Write Speed: Slow and expensive. Even a tiny change forces a rewrite of a large file (High Write Amplification).
 - Read Speed: Fast and efficient. The query engine reads clean, finished files with no extra "math" required.
 - **Best For: Read-heavy workloads** where data doesn't change often (e.g., historical archives).
 - **MOR:**

- Mechanism: Log-based Changes. Instead of rewriting the big data file, the system keeps the original file as-is and writes a small "sidecar" file (a log of deletes or updates).
 - Write Speed: Very fast. It only records the delta (the change), not the whole file (Low Write Amplification).
 - Read Speed: Slower. The query engine must read the base data PLUS apply the log of changes at runtime to show the final result.
 - Best For: Write-heavy or streaming workloads (e.g., CDC pipelines where data flows in constantly).
- External vs Delta tables:
 - External:
 - Data stored outside Spark metastore, only metadata is stored.
 - Points to data in: S3 / HDFS / ADLS. Dropping table → data NOT deleted.
 - No ACID guarantees. Works with formats like Parquet, CSV, JSON.
 - Delta:
 - It is a storage format (table format), not a table type, i.e: Parquet files + _delta_log (metadata layer). It does NOT just “virtually apply changes”. Delta is not a row-level mutation system rather a file-level versioning system.
 - spark.read.format("delta").load(...) reads _delta_log and figures out valid files for the current version and reads only them.
 - Stores data as parquet files only internally. _delta_log stores metadata like: File additions/removals, schema, transactions, etc. You can perform DELETE/UPDATE row level operations (plain parquet external tables don't allow that). Eg:

```

-> deltaTable.delete("date < '2024-01-01'")
-> deltaTable.update(
  condition="status = 'pending'",
  set={"status": "'processed'"}
)

```

- Other properties of delta tables:

- **ACID Transactions**: Safe concurrent reads/writes
- **Schema Enforcement & Evolution**: Prevents bad writes, Supports schema changes
- **Time Travel**: Ability to query a table exactly as it existed at a specific point in history. Eg:
`spark.read.format("delta").option("versionAsOf", 5).load(path)`
- **Change Data Feed (CDF)**: Read only changed data. Eg:
`spark.read.format("delta").option("readChangeFeed", true).option("startingVersion", 2).option("endingVersion", 3)`
- **Optimistic Concurrency Control**: Multiple writers allowed, Conflict detection via versions. “Last writer wins” only if there is no conflict.
- **MERGE operation**: Upserts data. Eg: `MERGE INTO target USING source ON target.id = source.id WHEN MATCHED THEN UPDATE WHEN NOT MATCHED THEN INSERT`
- **INSERT OVERWRITE operation**: Replaces entire table. Eg: `INSERT OVERWRITE TABLE user_data SELECT * FROM new_snapshot;`
- **Problems: Small Files Problem** is a very common issue in Delta Lake. When you are doing: micro-batch/stream writes, frequent merge/DML operations, over-partitioning, etc. This causes: Metadata bloat (Delta tracks every file in its transaction log, which becomes huge overtime), Increased IO overhead, Higher costs.
- **Solution**:
 - **OPTIMIZE table**: Does compaction (bin-packing small files → large files)
 - **Z-ORDER**: It is a multi-dimensional clustering technique for data files - for delta lake, apache iceberg, apache hudi. It co-locates related column values together in the same set of files to make filtering and range scans faster. Normal external tables lack a metadata layer (like Delta's Transaction Log or

Iceberg's Manifests) to store the min/max statistics created by the Z-Order process. Without these statistics, the query engine cannot perform "data skipping," making the physical Z-Order arrangement of rows useless. Eg: OPTIMIZE events ZORDER BY (country, date)

Eg:

```
SELECT * FROM transactions WHERE country = 'IN' AND event_time BETWEEN '2025-10-01' AND '2025-10-10';
```

Delta physically rearranges data on disk so that rows with similar country and event_time values end up close together (in the same files).

So when you filter by these columns, Delta needs to read fewer files – because the relevant data is tightly clustered instead of being scattered across many files.

Query: OPTIMIZE transactions ZORDER BY (country, event_time);

Internally, it interleaves the bits and stores, like:

country bits: 0 1 1 0 0 1

event_time bits: 1 1 1 0 1 0 0 1 0 1 0 1

Z-order bits: 0 1 1 1 0 0 1 1 0 0 1 0 1 0 1...

Rows are sorted by this interleaved single composite Z-order bit key.

Z-Ordering > Sorting by One Column

- **VACUUM:** Deletes old files (physically). Affects time travel though. Eg: VACUUM table RETAIN 168 HOURS
- Solved inefficiencies:
 - **Deletion Vectors (DVs):** Traditionally, Delta used **Copy-on-Write**—if you deleted one row, the entire Parquet file containing it had to be rewritten. **Deletion Vectors shift this to a Merge-on-Read paradigm.** Eg: delta.enableDeletionVectors

- **Liquid Clustering** is the successor to Partitioning and Z-Order. It is a flexible, "self-tuning" layout that automatically groups related data. How it works: It uses a tree-based algorithm (often Hilbert Curves) to organize data incrementally. Unlike Z-Order, it only clusters the newly arrived data rather than rewriting the entire table every time.
- For columns with high cardinality (like email, user_id, or ip_address) that don't cluster well, Delta uses Bloom Filter Indexes.
 - **Bloom filter** is a space-efficient, probabilistic data structure used to determine if an element is a member of a set. It is exceptionally fast and memory-efficient but comes with a unique trade-off: it can tell you if an item is "definitely not" in a set or "possibly" in a set.

How it works: "Instead of storing the data itself, we store its footprint in a bit array of size.

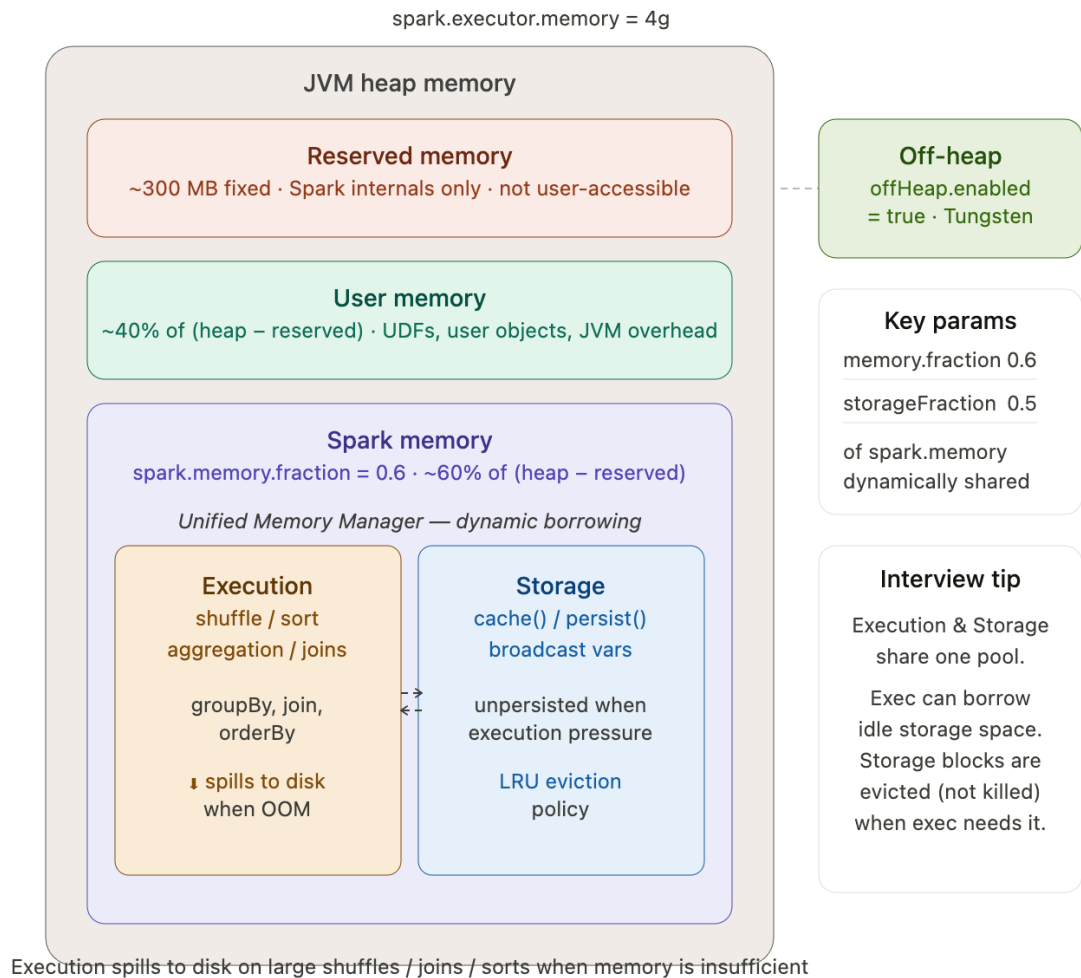
 - Insertion: We run an input through independent hash functions. Each hash outputs an index, and we set those specific bits to 1.
 - Lookup: We hash the query the same way. If we find even a single 0 at any resulting index, we know the item was never added. If all are 1, it's a 'maybe' due to potential bit-overlap from other items (as other items could have similar bits turned on) —which we call a false positive." Say inserting apples, searching for oranges, etc.
 - It performs well at scale:
 - Space: You can track millions of items in just a few megabytes because you aren't storing strings or objects, just bits.

- Speed: Complexity is $O(k)$. Whether you have a thousand items or a billion, the lookup time is the same because it's just 'k' simple math operations.

- **Spark Memory split** in workers: Assume a single 4GB worker:

Executor Heap Memory

- └ Reserved Memory (~300MB)
- └ User Memory
- └ Spark Memory
 - └ Execution Memory
 - └ Storage Memory



- Memory hierarchy — heap splits into 3 parts: Reserved (~300 MB hard floor), User (~40% of remaining), and Spark Memory (~60% of remaining via `spark.memory.fraction`).
- Unified Memory Manager: Execution and Storage are not fixed splits — they share a pool. Execution can borrow idle storage space. When execution pressure is high, cached blocks get LRU-evicted (not killed), freeing space. Storage, however, cannot forcibly reclaim memory held by active execution tasks.
- Spill to disk — happens automatically when execution memory is exhausted during shuffle/sort/join. Performance drops but no OOM.
- Off-heap — optional, outside JVM GC, enabled via `spark.memory.offHeap.enabled=true`. Used by Tungsten's binary processing to reduce GC pauses.
- Other tips:
 - `UNION` removes duplicate rows, `UNION ALL` does not. There is a performance hit when using `UNION` instead of `UNION ALL`, choose based on use case.
 - `foreach` runs function per row (use for debugging, not for I/O as could lead to millions of calls) and `foreachPartition` runs function per partition on executor (ideal for external system writes).
 - `EXISTS/NOT EXISTS` (Stops evaluating as soon as a match is found, faster) > `IN/NOT IN` (Subquery may be fully evaluated, slower on large datasets).
 - UDF and `foreachPartition` process data row by row, but they operate at completely different layers of Spark's architecture — and have very different purposes. UDF transform data inside the Spark SQL/DataFrame engine. Operates on data values (columns, rows) as part of a transformation (select, withColumn, filter, etc.) on executors. The output of a UDF is a new column value, part of the logical plan. Whereas `foreach()` perform actions with side effects, often for external I/O. Also runs on executors, but outside Spark's SQL planner. Operates on raw data (RDD/Row objects) per partition. Used after all transformations are done.

- Spark is faster than Hadoop/MapReduce due to in-memory data processing (in Hadoop every stage writes to disk) so disk I/O reduced, can spill memory to disk if required to handle large datasets, uses Catalyst optimizer, etc.
 - Eg:
 - Hadoop: step → disk → step → disk
 - Spark: step → memory → step → memory
 - Spark + HDFS >> MapReduce + HDFS.
 - Hadoop is better if very big datasets bigger than memory need to be processed.
- **Photon** is Databricks' next-generation vectorized execution engine, written in C++, designed to replace many of Spark's traditional JVM operators. Photon uses: SIMD vectorization, tighter memory layout, CPU instruction-level optimizations (AVX-512, etc.). Photon accelerates: File Scans (Parquet, Delta), Projection, Filter, Aggregations, Joins (hash, sort-merge). Where Photon does NOT work: Python / Scala UDFs, Complex nested types in some cases, MLlib operators, Graph workloads.
- **Use KryoSerializer instead of Java serializer** as when Spark runs distributed jobs: Data is sent between executors (e.g., for shuffles, caching, broadcasts). Each time, Spark must serialize (convert objects into byte form) to transfer them across the network or store them in memory/disk. Hence performance matters. Eg:


```
.config("spark.serializer",
"org.apache.spark.serializer.KryoSerializer")
```
- **Parquet vs Avro vs ORC:**

Feature	Parquet (Columnar)	Avro (Row-based)	ORC (Columnar)
Best For	Spark, general analytics, AWS Athena	Kafka, high-speed ingestion	Apache Hive, ACID transactions

Query Speed	Fastest for reading specific columns	Slow for analytics (must read whole row)	Fast, especially for filtering with indexes
Compression	High (column-specific)	Moderate	Highest (stripe-level optimization)
Schema Evolution	Basic (mostly adding columns)	Superior (renames, deletes, defaults)	Moderate

- Note:

--Note--

Parquet format supports Bloom filters, but they are **not** enabled by default. Standard Parquet metadata stores the minimum **and** maximum values **for** each row group. If you query WHERE id = 'abc', **and** the row group's range is aaa to zzz, the engine must scan that group because abc falls inside the range.

To use Bloom filters, you must enable them specifically during the write process so Spark can build the index and store it in the Parquet file's metadata.

```
Eg: df.write.mode("overwrite") \
    .option("parquet.bloom.filter.enabled", "true") \
    .option("parquet.bloom.filter.columns", "user_id, txn_id") \
    .option("parquet.bloom.filter.expected.ndv#user_id", "100000") \
    .parquet("path/to/output")
```

-

Structured Streaming [High-level basics]

- **Core mental model:** Spark treats streaming as incremental batch processing on an unbounded DataFrame. The engine reads new data on each trigger, builds an incremental logical plan, executes a Spark DAG, optionally updates state, writes to a sink, then commits offsets to

checkpoint. The abstraction you work with is a DataFrame that logically grows forever.

- **Pipeline:** Source → Trigger → Transformations → Output Mode → Sink, with Checkpoint running alongside.
- **Driver role:** coordinates triggers, tracks offsets, plans the query, manages checkpoint commits.
- **Executor role:** runs micro-batch tasks, maintains state store partitions, executes transformations.
- **Micro-batch vs Continuous Processing**

Mode	How it works	Latency	Production status
Micro-batch	Batches executed as Spark jobs	100ms-seconds	Standard, full guarantees
Continuous	Long-running tasks, row-by-row	~1ms	Experimental, limited operators

- Even Structured Streaming is fundamentally micro-batch in all real production systems. Continuous mode is rarely used.
- **What happens in each micro-batch trigger:**
 - Read new offsets from source
 - Build incremental logical plan
 - Execute Spark DAG on executors
 - Update state store (if stateful)
 - Write to sink
 - Commit offsets + checkpoint
- Triggers:
 - **Trigger.ProcessingTime("10 seconds")** – fires on fixed interval, default mode.

- **Trigger.AvailableNow** — (Spark 3.3+) processes all backlog across multiple safe micro-batches then stops; recommended for incremental ETL.
- **Trigger.Once** — single batch then stops; can OOM on large backlog, now superseded. Continuous — experimental, record-by-record, sub-millisecond latency, avoid in prod.

	Trigger.Once	Trigger.AvailableNow
Batch count	Single huge batch	Multiple micro-batches
Large backlog	Can OOM	Safe
Status	Older	Recommended

- Time semantics:
 - **Processing time** — when Spark processed the record.
 - **Event time** — when the event actually occurred at the source. These can differ significantly.
 - Example: user clicked at 10:00, Kafka delayed delivery until 10:15 — event time is still 10:00. Watermarks operate on event time, not processing time. This distinction is critical for correctness of windowed aggregations.
- **Watermarking**
 - withWatermark("event_time", "10 minutes"): Spark tracks the maximum event time seen so far and allows data to arrive up to 10 minutes late relative to that event time. Data older than the watermark is considered too late and may be discarded.
 - Why watermarks matter:
 - Correctness for Append mode on aggregations: Without a watermark, Spark cannot know whether a future late event

might still update an old aggregation/window result.

Therefore the result is never considered final, making Append mode invalid for aggregations whose results may still change.

With a watermark, Spark knows when a window is sufficiently old and can safely emit the final result in Append mode.

(Stateless append queries do not require watermarks because rows are emitted once and never updated.)

- **Bounding state size:** Stateful operations maintain intermediate state in the state store. Without watermarks, old aggregation/join state may accumulate forever, eventually causing excessive memory usage, GC pressure, spills, or OOM errors.
- **State cleanup / eviction:** As the watermark advances, Spark evicts expired window state and old stream-stream join state from the state store.
- **Window types;**
 - **Tumbling window** — fixed size, non-overlapping: Eg: `window(col("ts"), "10 minutes")`
 - **Sliding window** — overlapping, a record can belong to multiple windows: Eg: `window(col("ts"), "10 minutes", "5 minutes")`
 - **Session window** — dynamic, closes after an inactivity gap: Eg: `session_window(col("ts"), "30 minutes")`
- **Operations**
 - **Stateless** — filter, map, select. No state store touched. Rows are emitted once and never revised. Effectively Append behavior regardless of mode declared; Update mode is technically allowed but behaves identically.
 - **Stateful** — aggregations, windowed ops, stream-stream joins. Uses the StateStore API internally: versioned, checkpointed, one state partition per shuffle partition. Backed by HDFS/S3 (default) or RocksDB (Spark 3.2+, `spark.sql.streaming.stateStore.providerClass`). RocksDB is strongly preferred for large or long-lived state — avoids JVM GC pressure.
 - **State store internals:** large cardinality aggregation keys → massive state growth → GC pressure → OOM. Optimization levers:

watermarks, careful key design, RocksDB backend, proper partitioning.

- Stream joins
 - **Stream-Static:** join a stream against a static DataFrame. No watermark needed. The static side behaves like a normal batch DataFrame and is joined against each incoming micro-batch.
 - **Stream-Stream:** both sides unbounded. Watermark is mandatory on both sides to bound the join buffer. Without it, Spark must buffer all rows from both sides indefinitely.
- **Message delivery guarantees:** (how a system handles data as it moves from a source to a destination)
 - **At-Most-Once:** Data may be lost but is never duplicated (fastest, lowest reliability).
 - **At-Least-Once:** Data is never lost but may be duplicated (standard for most pipelines).
 - **Exactly-Once:** Data is neither lost nor duplicated (highest accuracy, requires checkpoints/idempotency).
- Output modes
 - **Append** — new rows only. Valid for: stateless ops, and stateful ops with watermark (once window is closed). Illegal on non-watermarked aggregations — Spark throws at runtime.
 - **Update** — only rows changed this batch. Most memory-efficient for aggregations. Not supported by file sinks.
 - **Complete** — entire result table every batch. Requires an aggregation. Expensive for large result sets.
- Source-side rate limiting
 - The old `spark.streaming.backpressure.enabled=true` is a DStreams config — it does not apply to Structured Streaming. For Structured Streaming, control ingestion rate through source-specific options:
Eg: `.option("maxOffsetsPerTrigger", 100000)` # Kafka;
`.option("maxFilesPerTrigger", 10)` # file source
- Sinks
 - **foreach** — row-level, runs on executors. Not idempotent by default. If a batch fails midway and retries, your writer runs again on already-processed rows. Spark's checkpoint only tracks source

offsets, not what your external writer did. Avoid for production DB writes.

- **foreachBatch** — batch-level callback. Callback executes on the driver; batch_df operations inside run distributed on executors. batchId is provided — use it to make writes idempotent (e.g. deduplicate on batchId before insert). Exactly-once semantics are achievable if and only if your sink write is implemented idempotently using batchId or transactional logic. Spark cannot magically guarantee external DB semantics.
- **Built-in sinks:** Kafka (exactly-once natively), Parquet/JSON (micro-batch file writes), memory (testing only).
- **Delta Lake sink** — ACID transactional writes, exactly-once behavior, supports streaming reads and writes, CDC patterns, and the standard Bronze/Silver/Gold lakehouse architecture. Increasingly the default production sink. Eg:
`df.writeStream.format("delta").option("checkpointLocation", "...").start("path/to/table")`
- **Fault tolerance and exactly-once:** Spark guarantees: exactly-once processing of input offsets (via checkpointed offsets + replayability). End-to-end exactly-once depends entirely on the sink:

Sink	Guarantee
Kafka	Exactly-once (native)
Delta Lake	Exactly-once (transactional)
foreachBatch + dedup	Exactly-once (if implemented correctly)

JDBC / foreach	At-least-once unless you add dedup
----------------	------------------------------------

- **Checkpointing:** Stores three things: source offsets (e.g. Kafka partition offsets), state store snapshots (for stateful ops), job metadata and progress. Stored on HDFS or S3. Without it, Spark cannot resume from failure.
- **Unsupported / restricted operations:** Global sort on an unbounded stream, certain non-time-bounded stream-stream joins, collect() and similar actions, non-deterministic operations. Root cause: streaming datasets are unbounded so these operations have no valid completion point.
- **Common production problems:** Small file problem from frequent micro-batches writing to object storage. State explosion from high-cardinality keys without watermarks. Checkpoint corruption (requires manual recovery or offset reset). Data skew in stateful aggregations. Kafka partition imbalance causing uneven executor load. Late data silently dropped when it exceeds the watermark – no error, just gone. Long GC pauses with the default JVM state store on large state (fix: RocksDB).
- **Kafka & Spark streaming:** Kafka partitions map to Spark tasks. Parallelism heavily depends on Kafka partition count. Spark stores consumed offsets in checkpoint, not Kafka consumer groups (usually). maxOffsetsPerTrigger controls ingestion rate. Too few Kafka partitions limits streaming parallelism.

Hadoop-MapReduce

- Hadoop = HDFS (storage) + MapReduce (compute)
- HDFS (Hadoop Distributed File System – Distributed Storage Layer)
 - Purpose: Reliable, fault-tolerant storage for very large files across a distributed cluster.

- Architecture:
 - **NameNode (Master): Stores metadata** – file names, block locations, permissions. Keeps a transaction log (EditLog) and a checkpoint (FsImage). Doesn't store actual data, only where blocks live. Single point of failure in early Hadoop versions (fixed later via Secondary NameNode / Standby NameNode / High Availability mode).
 - **DataNodes (Slaves): Store actual file blocks (default size: 128 MB or 256 MB)**. Send periodic heartbeats and block reports to the NameNode. Perform replication, deletion, and block creation as instructed by the NameNode.
- Key Concepts:
 - **Replication factor**: Default = 3 → improves fault tolerance and availability.
 - **Rack Awareness**: **NameNode knows rack topology** → places replicas on different racks to balance network efficiency and fault resilience.
 - **Read/Write Path**: Client → NameNode (for metadata) → Read from nearest Datanodes or write blocks of data.
- Benefits: High throughput, scalable storage, fault-tolerant, optimized for large sequential reads/writes.
- MapReduce (Distributed Processing Framework)
 - Purpose: Parallel computation model that processes massive data using Map() and Reduce() functions.
 - Flow:
 - Job Submission: Client submits job to JobTracker (Hadoop v1) or YARN ResourceManager (v2).
 - Split Phase: Input is divided into chunks (InputSplits), typically aligned with HDFS blocks.
 - Map Phase: Mapper processes each split → outputs intermediate key-value pairs.
 - Shuffle & Sort: Framework groups all values by key and transfers them to the appropriate reducer.
 - Reduce Phase: Reducer aggregates or combines values per key → produces final output.

- Key Points:
 - Idempotent: Multiple runs on same data yield same output.
 - Fault Tolerance: If a node fails, tasks are re-run on another node using data replicas.
 - Data Locality Optimization: Tries to schedule Map tasks on nodes where data already resides.
 - Output written back to HDFS.
- Limitation: High latency for iterative or real-time processing — hence, frameworks like Spark were built to overcome this.

Sample Problem:

Count frequency of each word in a large dataset. Step-by-step flow in Hadoop

1. Input Splitting

File stored in HDFS

Split into blocks (e.g., 128 MB each)

Each block → processed by a Mapper

2. Map Phase

Each mapper processes its chunk:

Input: "hello world hello"

Output (key-value pairs):

(hello, 1)

(world, 1)

(hello, 1)

3. Shuffle & Sort (MOST IMPORTANT)

Hadoop groups same keys together across all mappers

(hello, [1,1])

(world, [1])

Happens across network (expensive step)

4. Reduce Phase

Reducer aggregates:

(hello, 2)

(world, 1)

Final Output: Stored back in HDFS

- YARN (Yet Another Resource Negotiator — Cluster Resource Management Layer)

- Purpose: Decouples resource management from job scheduling, making Hadoop cluster multi-tenant and more efficient.

Miscellaneous(s)

Question1: Assume daily data volume to be 1TB/day from Kafka. Discuss about infra requirements if you were to ingest/parse data and store in S3 - in daily basis (batch) or near real-time basis (streaming).

Answer:

Common Assumptions

Parameter	Value	Notes
Daily data volume	1 TB/day (≈ 1000 GB)	Uncompressed JSON input
Average record size	1 KB	≈ 1 billion records/day
Storage layer	HDFS / S3 / Delta Lake	
Compression after parse	Parquet ($\approx 5\times$ smaller)	
Cluster type	Spark on YARN or Kubernetes	
Task	JSON parsing $\rightarrow$ schema mapping $\rightarrow$ write Parquet/Delta	

Batch Job (Run Once per Day):

- Workload Pattern
 - One Spark job runs once a day.
 - Reads 1 TB JSON → parses → writes to Parquet.
 - Target: finish in ~1 hour.
- JSON Parsing Throughput
 - Average: 30 – 50 MB/sec per vCPU.
 - Assume 40 MB/sec/vCPU for estimation.
 - Formula: Total time (sec) = $(1000 \text{ GB} \times 1024 \text{ MB/GB}) / (\text{Throughput} \times \text{vCPUs})$
 - Target time = 3600 sec → vCPUs $\approx (1000 \times 1024) / (40 \times 3600) \approx 7$ cores (ideal)
 - Add ~5× overhead for Spark shuffles, GC, I/O, etc. → $\approx 40\text{--}50$ vCPUs needed realistically.
- Suggested Infra (Batch): Note that if you're fine with 2–3 hr runtime, halve the cores (~20–25 vCPUs).

Resource	Estimate
Executors	10 – 12
Cores / executor	4 – 5
Total vCPUs	40 – 50
Memory / executor	16 – 24 GB
Total memory	200 – 250 GB
Cluster nodes	~5–6 × m5.4xlarge (AWS)

Cost	~ \$3-4 per hour of runtime
------	-----------------------------

Near Real-time Streaming Job:

- Workload Pattern
 - Continuous Kafka ingestion rate: $1 \text{ TB} / 24 \text{ h} = 1000 \text{ GB} / 86400 \text{ s} \approx 11.6 \text{ MB/sec}$
 - Goal: process each micro-batch within seconds or a minute.
- Efficiency
 - Structured Streaming is $\sim 2\text{-}3\times$ less efficient than batch due to micro-batching and checkpointing.
 - Rule of thumb: ~ 2 vCPUs per MB/sec input rate
 - At 11.6 MB/sec input: $11.6 \times 2 \text{ vCPUs} \approx 23 \text{ vCPUs}$
 - Add 50 % headroom $\rightarrow \approx 35\text{-}40 \text{ vCPUs}$ total (continuous).
- Suggested Infra (Streaming): Note that streaming runs 24×7 , unlike batch.

Resource	Estimate
Executors	8 – 10
Cores / executor	4
Total vCPUs	32 – 40
Memory / executor	16 – 24 GB
Total memory	160 – 200 GB
Cluster nodes	$\sim 4\text{-}5 \times \text{m5.4xlarge (AWS)}$

Cost	$\$3\text{--}4/\text{hr} \times 24 \text{ h} = \$75\text{--}100/\text{day}$
------	---

Comparison Summary

Aspect	Batch (Daily)	Streaming (Near Real-time)
Data processed	1 TB once/day	Continuous 1 TB/day
Compute needed	~40–50 vCPUs (1 hr run)	~35–40 vCPUs continuous
Memory	~200–250 GB	~160–200 GB
Cost	~\$3–4/hr (only 1 hr/day)	~\$75–100/day
Latency	Up to 1 day	Seconds – minutes
Complexity	Easier	Higher (checkpointing, watermarking)

Key Takeaways

- Batch = cheaper, simpler if 1-day delay acceptable. Streaming = real-time insights but higher cost (~20–30× more).
- JSON parsing is CPU-intensive — prefer Avro or Parquet ingestion.
- Autoscaling or serverless Spark (Databricks, EMR on EKS) reduces idle cost.
- Can I use less infra for streaming to save cost?
 - Yes — if your streaming job does very little per-record work, one or two vCPUs may keep up with a modest Kafka throughput. But if your streaming job performs CPU-heavy JSON parsing, joins,

windowing, stateful operations, or frequent checkpoints, you need many more cores to avoid lag and operational issues.

- Trade-offs when you reduce infra
 - Increased processing latency — microbatches take longer to finish → higher end-to-end latency (seconds → minutes → hours).
 - Backpressure and Kafka lag — consumer lag will grow if processing rate < input rate; lag must be stored in Kafka (requires sufficient retention).
 - Larger state / checkpoint growth — slower processing increases state growth and checkpoint sizes, slowing recoveries.
 - Spill to disk / GC pressure — insufficient memory leads to disk spill and longer task times; long GC pauses may cause executor failures.
 - Lower fault tolerance and recovery speed — on failure, recovery needs more time if checkpoints are large and cluster is small.
 - Higher operational complexity — must monitor lag, tune partitions, tune microbatch durations, autoscale carefully.
 - Potential for data loss if retention insufficient — if Kafka retention expires before backlog is processed.
- Example scenarios (for 1 TB/day → 11.6 MB/s)
 - Scenario A — Light work (minimal parsing):
 - $\text{per_core_MBps} = 12 \text{ MB/s}$
 - $\text{required_cores} = 11.6 / 12 \approx 1.0$
 - headroom 2× → $\text{allocated_cores} \approx 2$ vCPUs. When appropriate: messages are already compact (AVRO/Protobuf), transforms are trivial, no stateful operations.
 - Scenario B — Medium work (JSON parsing + enrichments)
 - $\text{per_core_MBps} = 4 \text{ MB/s}$
 - $\text{required_cores} = 11.6 / 4 \approx 2.9$

- headroom 2× → allocated_cores ≈ 6 vCPUs
When appropriate: parsing JSON, some UDFs, light joins, small state windows.
- Scenario C — Heavy work (stateful joins, large windows, heavy UDFs)
 - per_core_MBps = 1 MB/s
 - required_cores = 11.6 / 1 ≈ 11.6
 - headroom 2× → allocated_cores ≈ 24 vCPUs
When appropriate: large keyed state, big aggregations, frequent checkpoints, complex UDFs.
- Note: memory per executor should be aligned to avoid spills (e.g., 16–32 GB per executor depending on JVM overhead and state size). Also distribute workload across many Kafka partitions to parallelize.

Question2: Why RAM faster than Disk (HDD/SSD)?

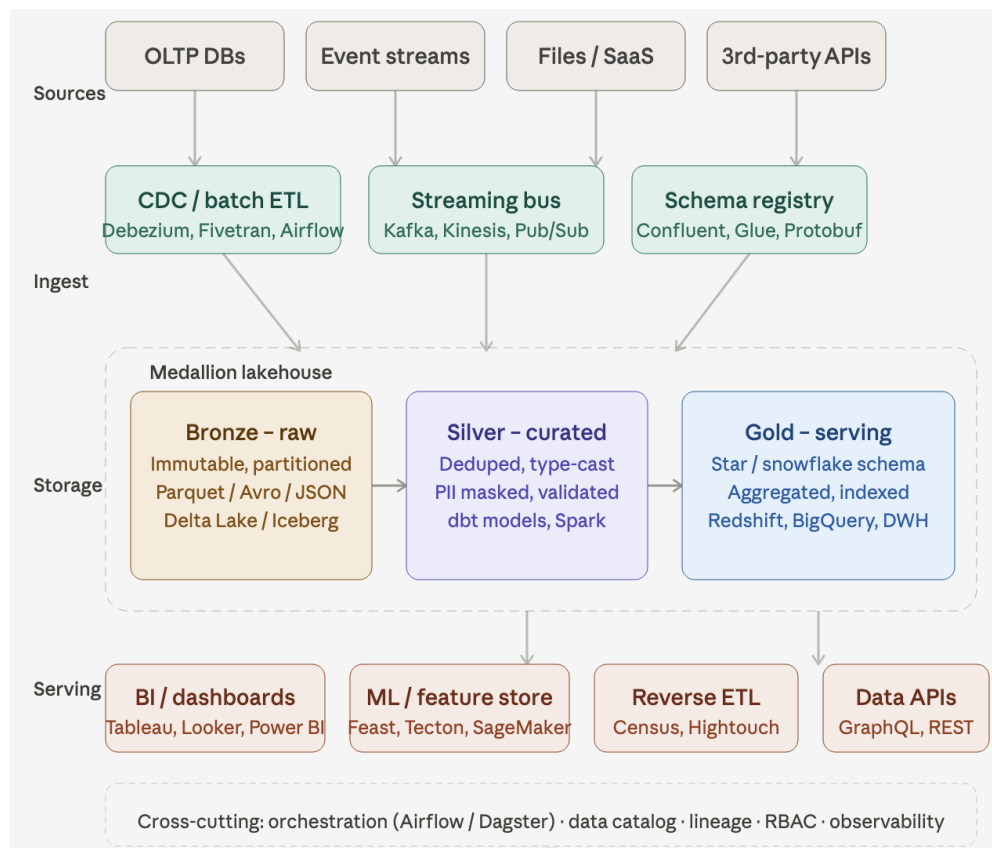
Answer:

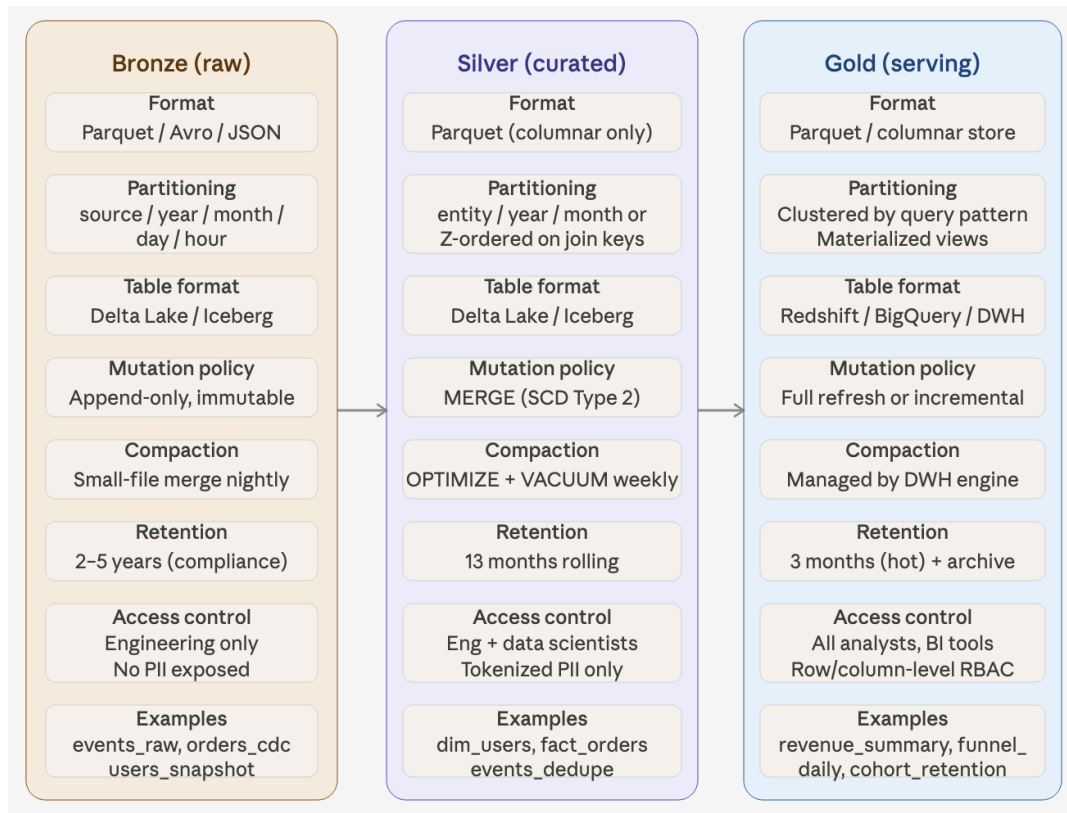
- **RAM (Memory):** Fully electronic (no moving parts). Data stored using electrical charge in memory cells (transistors + capacitors)
 - Each bit: Charged → 1, Not charged → 0
 - Access: Directly connected to CPU via memory bus. CPU can read/write almost instantly
 - Result: Access time: nanoseconds (10^{-9} sec), Very high bandwidth (GB/s)
 - Nature: Volatile → data lost when power off, Built for speed
- **HDD (Hard Disk Drive - Disk):** Structure: Spinning platters (glass/aluminum base), Coated with magnetic material, Read/write head moves over surface. Data stored: Surface divided into tiny magnetic regions (bits).
 - Each region: Magnetized one way → 1, Opposite direction → 0.
 - Access: To read data: Disk must spin, Head must move (seek) to correct track, Wait for correct sector to rotate under head

- Result: This causes: Seek time + rotational latency, Time in milliseconds (10^{-3} sec)
- Nature: Persistent (non-volatile), Optimized for storage capacity & cost, not speed
- **SSD (Solid State Drive - Disk)**
 - No spinning, no mechanical parts
 - Data stored as electrical charge in flash memory cells
 - Faster than HDD, but still slower than RAM

Question3: Design data warehouse system.

Answer:





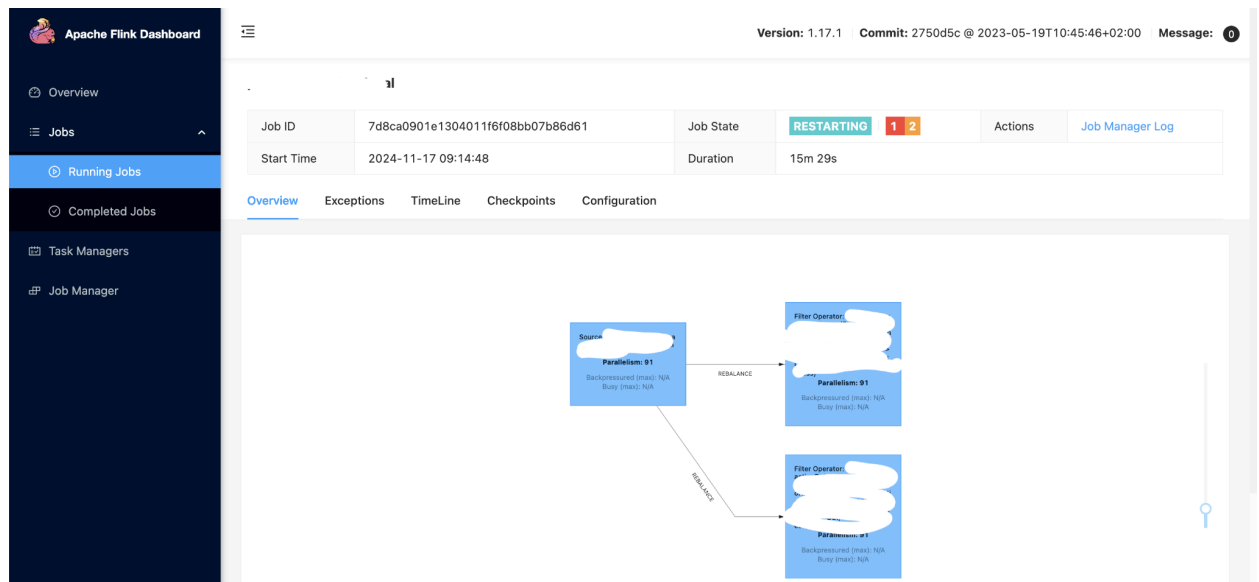
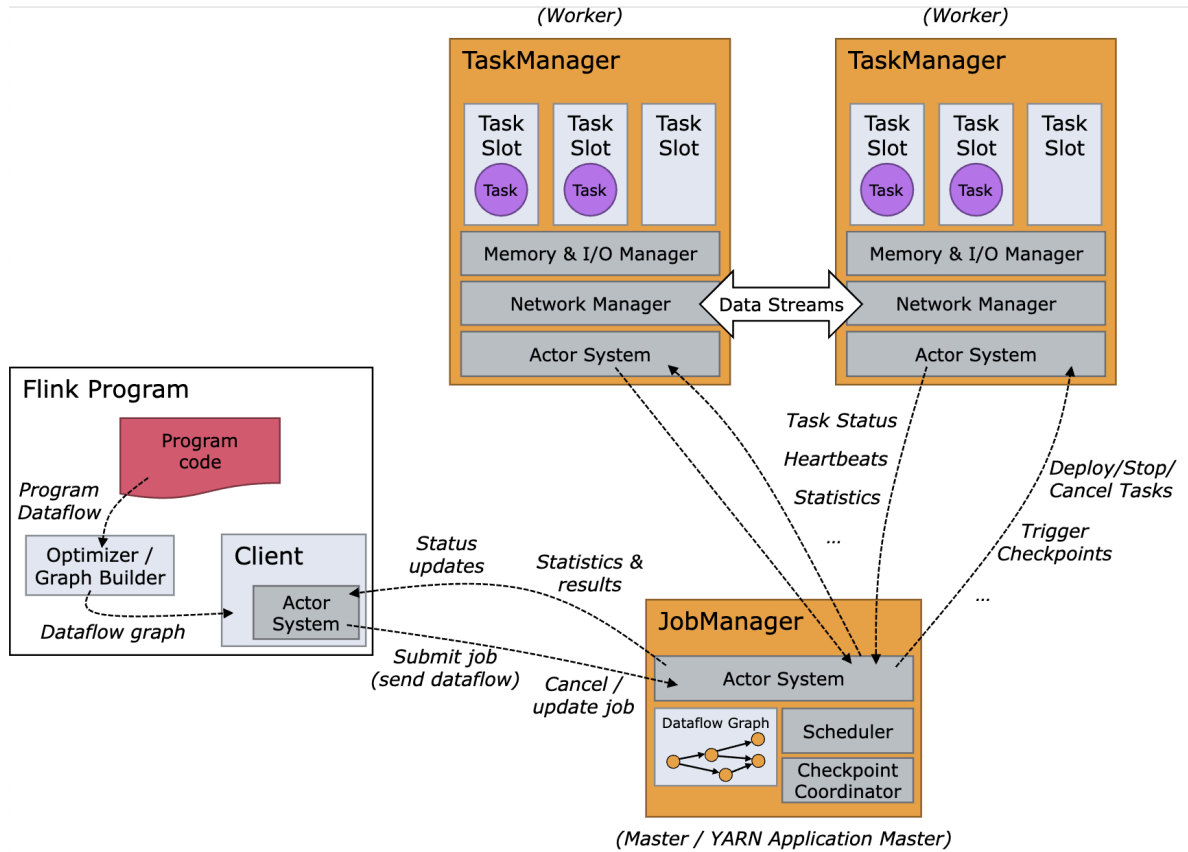
Question: What improvements have been made in the Spark 4.x version?

Answer:

Spark 4.x mainly improved: Structured Streaming state management, low-latency real-time streaming, Spark Connect, SQL capabilities, PySpark performance, and observability/debugging. Major highlights include: transformWithState API, AQE for streaming, Real-Time Mode, VARIANT type support, and production-ready Spark Connect.

Flink

Flink



- True stream processing engine with event-by-event processing, unlike Spark's micro-batch.
- Components:

- Flink uses an internal RPC framework and Netty-based network stack for communication between JobManagers and TaskManagers. Older Flink versions relied heavily on Akka.
- JobManager: Coordinates distributed execution, Schedules tasks, Manages checkpoints, Tracks job status, Handles failures and recovery.
 - JobManager high availability (HA) works by having multiple JobManager instances, with one acting as the leader and others as standbys, to eliminate the single point of failure.
- TaskManager: Worker nodes that execute tasks, Manage task slots, Buffer and exchange data streams, Maintain state backends
 - Task Slots: Logical fraction of resources in TaskManager or a Container for multiple tasks. Memory isolated, CPU shared.
 - A slot is a resource container that can host multiple subtasks through slot sharing.
 - Total available parallelism = number of TaskManagers × slots per TaskManager
 - Task: One or more chained operators running in one thread. All tasks run simultaneously.
 - All tasks run simultaneously (pipelined execution model)
 - Tasks are separated by network shuffles (e.g., keyBy(), rebalance(), broadcast())
 - Compatible operators are chained into a single task to avoid serialization overhead
 - Threads: Typically, each subtask chain runs in one main task thread.
 - A slot may host multiple subtask threads via slot sharing. Chained operators run in the same thread.
 - Total threads = $\Sigma(\text{Parallelism of each Task})$.
Approximate task execution threads $\approx$ total parallel subtasks across chained tasks. Actual JVM thread count is higher due to GC, network, RocksDB, async I/O, etc.
 - Parallelism: Number of parallel instances of an operator/task. It can be set per operator using

.setParallelism(N) or globally. Match parallelism to bottleneck and throughput requirements of each stage.

- Client: Submits jobs to JobManager, Can be CLI, web UI, or programmatic
- ResourceManager: Manages task slots, Provisions containers (YARN, Kubernetes, Mesos).

Eg:

Infra:

Assume 2 TaskManagers, 1 JobManager

Machine: 64 cores, 256GB RAM per TaskManager

taskmanager.numberOfTaskSlots::4

Parallelism.default::8 -> Global parallelism, i.e for all operators

Sample Flink Job:

```
DataStream<SensorEvent> stream = env
    .addSource(new FlinkKafkaConsumer<>(...)) // Read from Kafka
    .name("Kafka Source")
    .filter(event-> event.temperature > 0) // Filter invalid data
    .name("Filter Invalid")
    .map(event-> enrichEvent(event)) // Transform/enrich
    .name("Enrich Data")
    .keyBy(event-> event.sensorId) // Partition by sensor- SHUFFLE
    .process(new ApiCallFunction()) // API call per sensor
    .name("API Call")
    .addSink(new JdbcSink<>(...)) // Write to database
    .name("Database Sink");
...
```

LOGICAL RESOURCE ALLOCATION

Step 1: Job Submission

1. Client submits job to JobManager
2. JobManager receives job and creates ExecutionGraph (logical plan)
3. JobManager decides how to chain operators together
4. JobManager assigns parallelism to each operator
5. ResourceManager allocates task slots from TaskManagers

Step 2: Operator Chaining

Flink chains compatible operators into Tasks to avoid serialization overhead:

Resulting Tasks:

- Task 1: `[Kafka Source → Filter → Map]` (chained together)
- Task 2: `[API Call]` (separated by keyBy shuffle)
- Task 3: `[Database Sink]`

Rules for Chaining: Same parallelism across operators. No shuffle between them (no `keyBy`, `rebalance`, `broadcast`) as shuffle creates task boundary. Connected by forward partitioning (one-to-one). Chaining is enabled (default).

Step 3: Task Distribution

With parallelism = 8:

```

```
Task 1: [Source → Filter → Map] Parallelism: 8 → 8 threads
 ↓ (network shuffle via keyBy)
Task 2: [API Call] Parallelism: 8 → 8 threads
 ↓
Task 3: [Database Sink] Parallelism: 8 → 8 threads
> Total threads = 3 tasks × 8 parallelism = 24 threads
```

## Step 4: Slot Allocation

With `taskmanager.numberOfTaskSlots: 4` and 2 TaskManagers:

Total slots available: 2 TaskManagers × 4 slots = 8 slots

This means 3 threads per slot, 24 threads total across 8 slots

## PHYSICAL RESOURCE ALLOCATION

(image pasted after this content \*\*1)

Each slot gets ~16 cores worth of CPU time

More Slots Than Cores (Oversubscription): Good for: I/O-bound workloads (waiting on network/disk/API calls), Bad for: CPU-bound workloads (heavy computation)

Opposite case with undersubscription.

One slot per core is recommended. Although it depends on the

running task if it's CPU or IO intensive.

Other notes:

Pipelining in 3 threads (Multiple events processing simultaneously at different stages)

(image pasted after this content \*\*2)

Backpressure is created if downstream threads queue is getting filled up.

-----

Other ex:

Job:

```
env.setParallelism(8); // Global default
DataStream<Event> stream = env
 .addSource(kafkaSource)
 .setParallelism(10) // ← Override: 10 Kafka consumers
 .name("Source")
 .filter(e -> e.valid) // Inherits 10
 .map(e -> enrich(e)) // Inherits 10
 .name("Transform")
 .keyBy(e -> e.key) // ← SHUFFLE: Task boundary
 .window(...)
 .aggregate(...)
 .setParallelism(20) // ← Override: 20 window operators
 .name("Windowed Agg")
 .keyBy(e -> e.userId) // ← SHUFFLE: Task boundary
 .process(new ApiCall())
 .setParallelism(4) // ← Override: Only 4 API callers
 .name("API")
 .map(e -> format(e)) // Inherits 4
 .keyBy(e -> e.region) // ← SHUFFLE: Task boundary
 .addSink(jdbcSink)
 .setParallelism(16); // ← Override: 16 DB writers
```

Logical Tasks Created

Task 1: [Source → Filter → Map] Parallelism: 10 → 10 threads  
↓ (keyBy shuffle)

Task 2: [Window → Aggregate]      Parallelism: 20 → 20 threads  
      ↓ (keyBy shuffle)  
Task 3: [Process (API) → Map]      Parallelism: 4 → 4 threads  
      ↓ (keyBy shuffle)  
Task 4: [Sink]                      Parallelism: 16 → 16 threads

Total threads = 10 + 20 + 4 + 16 = 50 threads

Slots needed = max(10, 20, 4, 16) = 20 slots (with slot sharing)

### Physical Slot Distribution

With 20 slots available (5 TaskManagers × 4 slots):

SLOT DISTRIBUTION WITH SLOT SHARING:

|          |          |             |            |             |             |
|----------|----------|-------------|------------|-------------|-------------|
| Slot 1:  | T1-inst0 | + T2-inst0  | + T3-inst0 | + T4-inst0  | = 4 threads |
| Slot 2:  | T1-inst1 | + T2-inst1  | + T3-inst1 | + T4-inst1  | = 4 threads |
| Slot 3:  | T1-inst2 | + T2-inst2  | + T3-inst2 | + T4-inst2  | = 4 threads |
| Slot 4:  | T1-inst3 | + T2-inst3  | + T3-inst3 | + T4-inst3  | = 4 threads |
| Slot 5:  | T1-inst4 | + T2-inst4  | + T3-empty | + T4-inst4  | = 3 threads |
| Slot 6:  | T1-inst5 | + T2-inst5  | + T3-empty | + T4-inst5  | = 3 threads |
| Slot 7:  | T1-inst6 | + T2-inst6  | + T3-empty | + T4-inst6  | = 3 threads |
| Slot 8:  | T1-inst7 | + T2-inst7  | + T3-empty | + T4-inst7  | = 3 threads |
| Slot 9:  | T1-inst8 | + T2-inst8  | + T3-empty | + T4-inst8  | = 3 threads |
| Slot 10: | T1-inst9 | + T2-inst9  | + T3-empty | + T4-inst9  | = 3 threads |
| Slot 11: | T1-empty | + T2-inst10 | + T3-empty | + T4-inst10 | = 2 threads |
| Slot 12: | T1-empty | + T2-inst11 | + T3-empty | + T4-inst11 | = 2 threads |
| Slot 13: | T1-empty | + T2-inst12 | + T3-empty | + T4-inst12 | = 2 threads |
| Slot 14: | T1-empty | + T2-inst13 | + T3-empty | + T4-inst13 | = 2 threads |
| Slot 15: | T1-empty | + T2-inst14 | + T3-empty | + T4-inst14 | = 2 threads |
| Slot 16: | T1-empty | + T2-inst15 | + T3-empty | + T4-inst15 | = 2 threads |
| Slot 17: | T1-empty | + T2-inst16 | + T3-empty | + T4-empty  | = 1 thread  |
| Slot 18: | T1-empty | + T2-inst17 | + T3-empty | + T4-empty  | = 1 thread  |
| Slot 19: | T1-empty | + T2-inst18 | + T3-empty | + T4-empty  | = 1 thread  |
| Slot 20: | T1-empty | + T2-inst19 | + T3-empty | + T4-empty  | = 1 thread  |

Total: 50 threads across 20 slots

Average: 2.5 threads per slot

## Per-TaskManager View

TaskManager 1 (32 cores, 4 slots):

Slot 1: 4 threads }  
Slot 2: 4 threads } 16 threads total  
Slot 3: 4 threads } competing for 32 cores  
Slot 4: 4 threads }

TaskManager 2 (32 cores, 4 slots):

Slot 5: 3 threads }  
Slot 6: 3 threads } 12 threads total  
Slot 7: 3 threads } competing for 32 cores  
Slot 8: 3 threads }

TaskManager 3 (32 cores, 4 slots):

Slot 9: 3 threads }  
Slot 10: 3 threads } 12 threads total  
Slot 11: 2 threads } competing for 32 cores  
Slot 12: 2 threads }

TaskManager 4 (32 cores, 4 slots):

Slot 13: 2 threads }  
Slot 14: 2 threads } 8 threads total  
Slot 15: 2 threads } competing for 32 cores  
Slot 16: 2 threads }

TaskManager 5 (32 cores, 4 slots):

Slot 17: 1 thread }  
Slot 18: 1 thread } 4 threads total  
Slot 19: 1 thread } competing for 32 cores (underutilized!)  
Slot 20: 1 thread }

How Data Flows with Different Parallelism

Network Shuffle Behavior

10 Source instances reading from Kafka



[Rebalance/Hash Partition by key]  
↓  
20 Window instances (each receives from ~0.5 sources on average)  
↓  
[Hash Partition by userId]  
↓  
4 API instances (each receives from ~5 window instances)  
↓  
[Hash Partition by region]  
↓  
16 Sink instances (each receives from ~0.25 API instances)

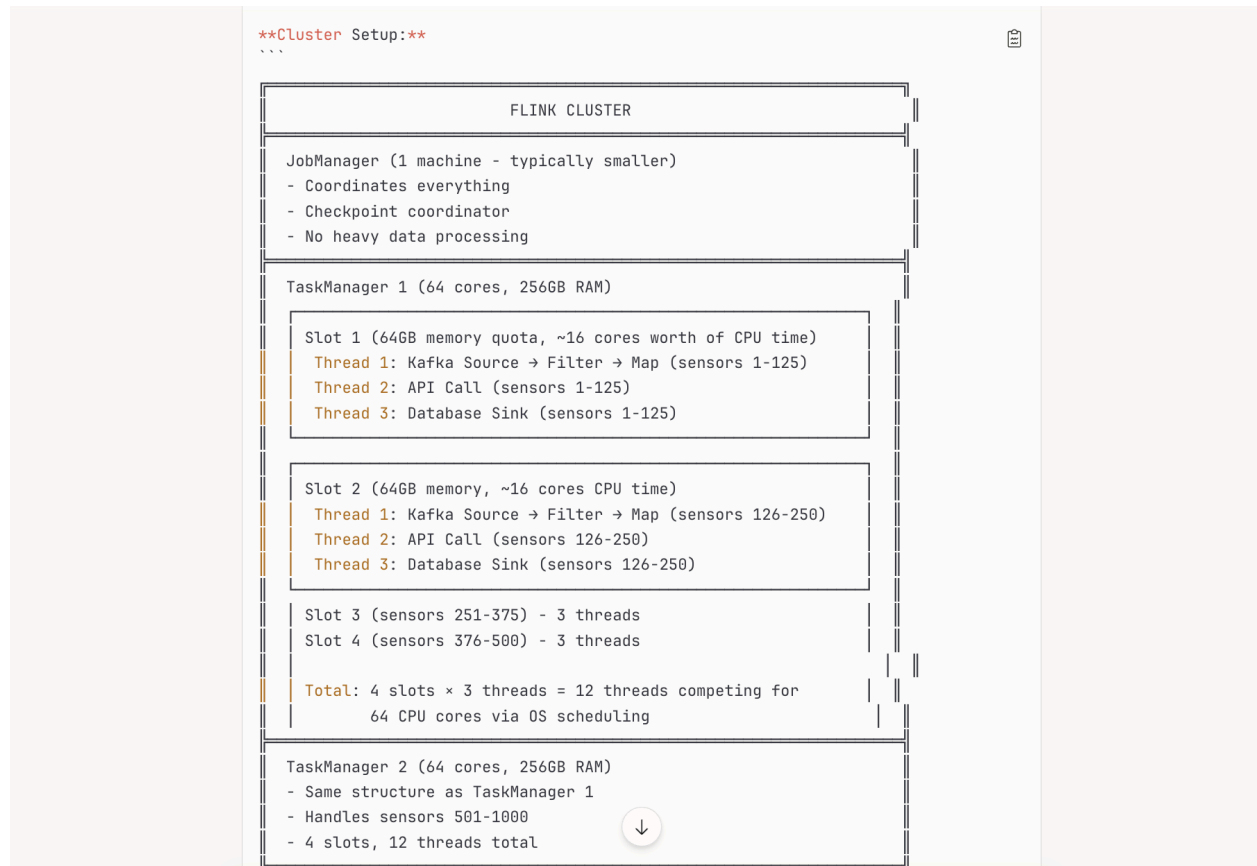
Example event flow:

Event with key="sensor123", userId="user456", region="us-east"

1. Read by Source-instance-7 (one of 10)
2. Filter + Map in Source-instance-7
3. Network shuffle:  $\text{hash}(\text{"sensor123"}) \% 20 = 13$   
→ Sent to Window-instance-13
4. Windowing in Window-instance-13
5. Network shuffle:  $\text{hash}(\text{"user456"}) \% 4 = 2$   
→ Sent to API-instance-2
6. API call in API-instance-2
7. Map in API-instance-2
8. Network shuffle:  $\text{hash}(\text{"us-east"}) \% 16 = 8$   
→ Sent to Sink-instance-8
9. DB write in Sink-instance-8

All threads use OS scheduling to compete for the 32 cores

(Physical resource allocation \*\*1)



(Pipelining in thread \*\*2)

### ### How Events Flow (Pipelined)

**\*\*3 threads are active simultaneously, processing different events:\*\***  
...

| Time  | Thread 1<br>(Source) | Thread 2<br>(API) | Thread 3<br>(Sink) |
|-------|----------------------|-------------------|--------------------|
| 0ms   | Read e1              |                   |                    |
| 1ms   | Filter e1            |                   |                    |
| 3ms   | Read e2              | Start API(e1)     |                    |
| 4ms   | Filter e2            | API(e1) running   |                    |
| 6ms   | Read e3              | API(e1) running   |                    |
| 103ms | Read e35             | API(e1) done      | Start Write(e1)    |
| 104ms | Filter e35           | Start API(e2)     | Write(e1)          |
| 113ms | Read e36             | API(e2) running   | Write(e1) done     |
| 114ms | Filter e36           | API(e2) running   | Start Write(e2)    |
| ...   |                      |                   |                    |

#### **\*\*Key Insights:\*\***

- Thread 1 doesn't wait for Thread 2 or 3
- Events are **\*\*buffered** in queues**\*\*** between operators
- Multiple events are "in flight" at different stages simultaneously
- Maximum throughput achieved through pipelining

- Watermarks:
  - Special markers in stream indicating event time has progressed to certain point. Tell Flink when to trigger window computations. Events older than the current watermark may be considered late depending on window boundaries and allowed lateness.
  - Operators forward minimum watermark from all input streams (input unions, joins, etc.). Ensures correctness across parallel operations.
  - Watermarking if: aggressive - data loss, conservative - data lag.
  - Types:
    - Periodic Watermarks:
      - Generated at fixed intervals (default: 200ms)
      - Based on maximum observed timestamp minus allowed lateness

```
Eg: DataStream<Event> stream = env
 .addSource(new FlinkKafkaConsumer<>("events", new
 EventDeserializationSchema(), props))
```



```

 .assignTimestampsAndWatermarks(
 WatermarkStrategy

.<Event>forBoundedOutOfOrderness(Duration.ofSeconds(5))
 .withTimestampAssigner((event, ts) ->
event.getEventTime())
 .withIdleness(Duration.ofMinutes(1)) // handle idle
Kafka partitions
);
- Emits watermark every 200ms by default.
- forBoundedOutOfOrderness: Allows events that are up to 5s
 late (within watermark delay).
- withIdleness(): prevents watermark stalls if one Kafka
 partition stops producing.
- withTimestampAssigner(...): To extract event time from
 payload. Pattern: ISO-8601 string → epoch milliseconds, as
 Flink uses epoch millis internally for event time

```

#### ■ Punctuated Watermarks:

- Generated based on special markers in stream.
- Event-driven watermark generation

Eg:

```

DataStream<Event> stream = env
 .addSource(new CustomSourceFunction())
 .assignTimestampsAndWatermarks(
 new WatermarkStrategy<Event>() {
 @Override
 public WatermarkGenerator<Event>
createWatermarkGenerator(WatermarkGeneratorSupplier.Context
context) {
 return new WatermarkGenerator<Event>() {
 private long currentMaxTimestamp =
Long.MIN_VALUE;

 @Override
 public void onEvent(Event event, long
eventTimestamp, WatermarkOutput output) {

```

```

 currentMaxTimestamp =
Math.max(currentMaxTimestamp, eventTimestamp);
 // Emit punctuated watermark when special flag is seen
 if (event.isEndOfBatch()) {
 output.emitWatermark(new
Watermark(currentMaxTimestamp - 1));
 }
 }
 @Override
 public void onPeriodicEmit(WatermarkOutput
output) {
 // No-op for punctuated watermark
 }
};
}
@Override
public TimestampAssigner<Event>
createTimestampAssigner(TimestampAssignerSupplier.Context ctx) {
 return (event, recordTimestamp) ->
event.getTime(); });

```

- To handle late arriving data:
  - Use Allowed Lateness: Eg:
 

```
stream.keyBy(...).window(TumblingEventTimeWindows.of(Time
e.minutes(1))).allowedLateness(Time.minutes(5)).sideOutputLa
teData(lateTag.aggregate(...))
```
  - Side Outputs: Redirect late data to separate stream for special
 handling, can reprocess, log, or send to dead letter queue.

```

final OutputTag<Event> lateEventsTag = new
OutputTag<Event>("late-events"){
};
SingleOutputStreamOperator<AggregatedEvent> result = stream
 .keyBy(Event::getKey)
 .window(TumblingEventTimeWindows.of(Time.minutes(1)))
 .allowedLateness(Time.minutes(5))
 .sideOutputLateData(lateEventsTag)
 .aggregate(new MyAggregateFunction());
// Retrieve late data for special handling

```

```
DataStream<Event> lateStream =
result.getSideOutput(lateEventsTag);
lateStream
 .addSink(new DeadLetterQueueSink<>("late-events-dlq"));
```

- Watermark Configuration: Increase allowed out-of-orderness; Tradeoff: Higher latency before window triggers

```
env.getConfig().setAutoWatermarkInterval(1000L); // emit every 1
second instead of 200ms
```

- Window Reassignment: Custom logic to reassign late events to appropriate windows
- Windows: A window defines how data in a stream is grouped for processing over time or count. Since data in streams is unbounded, windows help you create bounded slices of it for aggregation or analysis. Flink decides window boundaries based on
  - > Event time (Timestamp when event actually occurred. Embedded in the event record),
  - > Processing time (Timestamp when event processed by Flink operator,
  - > Non-deterministic (Depends on system speed)), or
  - > Ingestion time (Timestamp when event enters Flink source)
- Types:
  - Tumbling Windows: Fixed-size, non-overlapping. Each event belongs to exactly one window. Eg:
 

```
TumblingEventTimeWindows.of(Time.minutes(5)).
```

    - Use case: 5-minute aggregations, Periodic reports (e.g., sales per 5 minutes).
 

```
|----5m----|----5m----|----5m----|
```
  - Sliding Windows: Fixed size with configurable slide interval. Windows can overlap. Eg:
 

```
SlidingEventTimeWindows.of(Time.minutes(10),
Time.minutes(5))...
```

    - Use case: Moving averages, rolling metrics
 

```
|-----10m-----|
|-----10m-----|
```

|-----10m-----|

- Session Windows: Dynamic size based on inactivity gap. No fixed duration. The windows automatically merge when events arrive within the inactivity gap. Eg:  
`EventTimeSessionWindows.withGap(Time.minutes(10))`
  - Use case: User session analysis, burst detection  
User clicks → gap < 10m → same window  
User idle → gap > 10m → new session starts
- Global Windows: Single window containing all events, Requires custom trigger.
  - Use case: Custom windowing logic
- Count Windows: Based on number of elements. Eg:  
`countWindow(100) // tumbling. countWindow(100, 10) // sliding`
  - Use case: Batch-style aggregation
- Triggers: Control when window is evaluated
  - `EventTimeTrigger`: Fire when watermark passes window end
  - `ProcessingTimeTrigger`: Fire based on processing time
  - `CountTrigger`: Fire after N elements
  - `ContinuousEventTimeTrigger`: Fire periodically within window
  - Custom Triggers: Implement `TriggerResult` (CONTINUE, FIRE, PURGE, FIRE\_AND\_PURGE)
    - FIRE emits results but keeps data for future updates; FIRE\_AND\_PURGE emits and clears state to free memory.
- Evictors: Remove elements from window before/after computation
  - `TimeEvictor`: Keep only elements within time range
  - `CountEvictor`: Keep only last N elements
  - `DeltaEvictor`: Keep elements within delta threshold
  - Custom Evictors: Implement `Evictor` interface

Eg:  
stream

```

.keyBy(...)
.window(GlobalWindows.create())
.trigger(CountTrigger.of(1000))
.evictor(TimeEvictor.of(Time.hours(1)))
.reduce(...)

```

Visual:

|               |                                     |    |    |    |    |
|---------------|-------------------------------------|----|----|----|----|
| Event Stream: | E1                                  | E2 | E3 | E4 | E5 |
| Timestamps:   | 1s                                  | 2s | 4s | 7s | 9s |
| Window:       | [0 ----- 10)                        |    |    |    |    |
| Watermark:    | ↑ (fires trigger at 10s)            |    |    |    |    |
| Trigger:      | CONT → CONT → CONT → CONT → FIRE    |    |    |    |    |
| Evictor:      | (removes old events before compute) |    |    |    |    |

- State Management: “state” means remembering information about past events to influence future processing. Flink stores and manages this state fault-tolerantly across operators and keys.
  - Types:
    - Keyed State: Scoped per key (after a `keyBy()` operation). Only accessible in keyed streams. Each key has its own isolated state partition. Eg: `ValueState<T>` (Single value per key), `ListState<T>` (List of values), `MapState<K,V>`, `ReducingState<T>`, etc.
    - Operator State: Scoped per operator instance, not per key. Used when partitioning isn’t applied (e.g., non-keyed source/sink). Eg: `ListState`, `UnionListState`, `BroadcastState<K,V>`
  - The State Backend determines how and where state is stored:
    - `HashMapStateBackend`: Stores state in-memory (JVM heap), fast, snapshots to external storage, suitable for smaller state workloads but not ideal for very large production state.
    - `EmbeddedRocksDBStateBackend`: RocksDB (disk), slower but scalable, best for prod.

```

StreamExecutionEnvironment env =
StreamExecutionEnvironment.getExecutionEnvironment();

```

```
env.setStateBackend(new EmbeddedRocksDBStateBackend());
env.getCheckpointConfig().setCheckpointStorage("s3://flink-checkpoints");
```

- State TTL (Time-To-Live): TTL prevents unbounded state growth. Automatic expiration of state after configured time. Can be incremental, snapshot based, compaction.

```
StateTtlConfig ttlConfig = StateTtlConfig
 .newBuilder(Time.days(7))
 .setUpdateType(StateTtlConfig.UpdateType.OnCreateAndWrite)
 .setStateVisibility(StateTtlConfig.StateVisibility.NeverReturnExpired)
 .cleanupInRocksdbCompactFilter()
 .build();
```

Types:

OnCreateAndWrite: Reset TTL on creation and writes

OnReadAndWrite: Reset TTL on reads and writes

Disabled: TTL never updated

- State Rescaling: When you change job parallelism, Flink redistributes state. Works accordingly for Keyed and Operator states.
- Checkpoints & Recovery: State is snapshotted periodically into durable storage (e.g., S3, HDFS) On recovery, Flink restores state exactly as before the failure. Steps:
  - Aligned Checkpoints:
    - Triggering:
      - The JobManager initiates a checkpoint periodically (as configured).
      - Checkpoint barriers are injected into the source tasks (Kafka, S3, etc.).
    - Barrier Propagation:
      - These barriers flow through the data stream, tagging all records as belonging to a specific checkpoint (say, #42).
      - Records before the barrier → belong to checkpoint 41.

- Records after the barrier are processed beyond checkpoint #42.
- State Snapshotting:
  - When an operator has received a barrier for checkpoint #42 from all its inputs, it:
    - Operators buffer channels that already received checkpoint barriers until all input barriers arrive.
    - Snapshots its current operator state + keyed state asynchronously.
    - Persists this state to a checkpoint storage backend (e.g., S3, HDFS).
- Barrier Acknowledgement:
  - Once all operators complete their snapshots, ack messages go to the JobManager.
  - When all are acked → checkpoint considered complete.
  - No global stop — processing continues concurrently (async snapshotting), just locally the operator may pause & faster upstream inputs would be buffered, downstream operators would still be running; Once checkpoint complete, then the local operator resumes.
- What causes checkpoint alignment delays?
  - Skewed input rates — one Kafka partition slower than others.
  - Large operator state — snapshot takes long.
  - High backpressure — data flow uneven, barriers delayed.
  - Network congestion — delays barrier propagation.
  - Asynchronous snapshot thread saturation in RocksDB.
- Unaligned checkpoints: Operators don't wait for barrier alignment, In-flight records are included in checkpoint snapshots hence big checkpoint size, Checkpoints complete even under heavy backpressure.
- Others:
  - Checkpoint Components: Operator state, Keyed state, In-flight records (for exactly-once).

- Flink guarantees exactly-once consistency for internal state. External exactly-once depends on sink support for transactional or idempotent writes.
- Checkpoint: Automatic, periodic, lightweight, may be deleted  
AND Savepoint: Manual, for versioning, upgrades, always retained

```
env.enableCheckpointing(60000); // Every 60 seconds
env.getCheckpointConfig().setCheckpointingMode(CheckpointingMode.EXACTLY_ONCE);
env.getCheckpointConfig().setMinPauseBetweenCheckpoints(30000);
env.getCheckpointConfig().setCheckpointTimeout(600000);
env.getCheckpointConfig().setMaxConcurrentCheckpoints(1);
env.getCheckpointConfig().setTolerableCheckpointFailureNumber(3);
```

- Exactly-once in Flink:
  - Exactly-Once Processing: Internal state updated exactly once per event, Achieved through checkpointing, Flink guarantees this for state updates
  - Exactly-Once Delivery: Each event written to sink exactly once, Requires transactional or idempotent sinks, Harder to achieve, depends on external system. Delivery Guarantees:
    - At-Most-Once: Fire and forget, No checkpointing needed, Data loss possible, Lowest overhead
    - At-Least-Once: Checkpoint and replay on failure, Duplicates possible, Most sinks support this
    - Exactly-Once: Transactional sinks or idempotent writes, No duplicates or loss, Highest overhead.
  - Steps:
    - Source:
      - Must be replayable (Kafka offsets, file positions)
      - Checkpoint source positions
      - Reset to checkpoint on recovery
    - Internal Processing:
      - Barrier snapshotting ensures consistent state
      - All operators see same snapshot of inputs



- Sink:
  - Transactional Sinks: Two-phase commit (2PC)
  - Idempotent Sinks: Deterministic IDs, overwrite on replay.  
Eg: `env.enableCheckpointing(60000);`  
`sink.setDeliveryGuarantee(DeliveryGuarantee.AT_LEAST_ONCE)`
- Partitioning Strategy:
  - It defines how records are distributed across parallel subtasks in Flink's dataflow graph. This directly impacts state locality, load balancing, and network shuffle costs.
  - Strategies:
    - KeyBy: `stream.keyBy(event -> event.getUserId())`
      - Hash partition by key
      - keyBy hashes records into key-groups which are assigned to subtasks. The same key always maps to the same subtask.
      - Enables keyed state
      - Can cause data skew if keys not uniformly distributed
    - Rebalance: `stream.rebalance()`
      - Round-robin distribution
      - Even load distribution
      - Causes network shuffle
      - Use when dealing with skewed data
    - Rescale: `stream.rescale()`
      - Local round-robin to subset of downstream tasks
      - Less network overhead than rebalance
      - Use when upstream and downstream parallelism are multiples
    - Forward:
      - One-to-one connection (no shuffle)
      - Preserves partitioning
      - Lowest overhead
    - Broadcast: `stream.broadcast()`
      - Send all elements to all parallel instances

- Expensive, use sparingly
  - Useful for small reference data
- Handle/detect data skew:
  - Flink UI to detect skewness.
  - Key Salting. Eg: `.keyBy(event -> event.getUserId() + "_" + (event.getUserId().hashCode() % 4))`
  - Rebalance Before Stateful Operator: Helps redistribute load before `keyBy` when skewed distribution is known. Eg: `rebalance().keyBy(...)`
  - Dynamic Load Balancing: Implement custom partitioner. Eg: `stream.partitionCustom(new CustomPartitioner(), event -> event.getKey());`
  - Scale Up Parallelism: Could operator parallelism. Eg: `env.setMaxParallelism(128);`
    - `maxParallelism` is different from runtime parallelism. It defines the number of key-groups used for keyed-state partitioning and future rescaling flexibility.
- Process Functions: Process Functions are low-level API for stream processing. They give fine-grained control over: Event-time / processing-time timers, Custom logic beyond windowing / simple transformations, State access (keyed or operator), Side outputs (for late data or special cases). Types:
  - `ProcessFunction<IN, OUT>`: Operates on non-keyed streams. Gives access to processing time, event time, and side outputs. Cannot use keyed state (only operator state).

```
stream.process(new ProcessFunction<Event, Result>() {
 @Override
 public void processElement(Event event, Context ctx,
 Collector<Result> out) {
 out.collect(new Result("Received: " + event));
 }
});
```

- `KeyedProcessFunction<K, IN, OUT>`: Operates on keyed streams (after `.keyBy()`). Can use keyed state (`ValueState`, `ListState`, etc.). Can

register event-time or processing-time timers per key.

- Non-keyed stream: A normal `DataStream` where all events are processed together by each parallel subtask, without partitioning by key. Every operator instance gets a subset of the stream — distributed in parallel — but no key-based grouping is done.
- Keyed Stream: Created by applying `.keyBy(...)` on a stream. Partitions the stream by key value, so all records with the same key go to the same parallel subtask. Usecase: If you need to track information per user, session, merchant, device, etc., like: Counting clicks per user:

Example for Keyed stream:

```
stream
 .keyBy(Event::getUserId)
 .process(new KeyedProcessFunction<String, Event,
UserCount>() {
 private ValueState<Integer> count;
 public void open(Configuration parameters) {
 count = getRuntimeContext().getState(new
ValueStateDescriptor<>("count", Integer.class));
 }
 public void processElement(Event event, Context ctx,
Collector<UserCount> out) throws Exception {
 int newCount = (count.value() == null ? 0 :
count.value()) + 1;
 count.update(newCount);
 out.collect(new UserCount(event.getUserId(),
newCount));
 }
 });
```

Example for Keyed Process function:

```
sstream.keyBy(Event::getUserId)
 .process(new KeyedProcessFunction<String, Event, Result>() {
 ...
 })
```

- `ProcessWindowFunction<IN, OUT, KEY, W extends Window>`: Used inside window operations. Gives access to window metadata (start/end timestamp). Provides all elements of the window for full aggregation.

```
stream.keyBy(Event::getUserId)
 .window(TumblingEventTimeWindows.of(Time.minutes(5)))
 .process(new ProcessWindowFunction<Event, Result, String,
TimeWindow>() {
 @Override
 public void process(String key, Context ctx,
Iterable<Event> events, Collector<Result> out) {
 long count =
StreamSupport.stream(events.spliterator(), false).count();
 out.collect(new Result(key, count,
ctx.window().getEnd()));
 }
 });
```

- `ProcessAllWindowFunction<IN, OUT, W extends Window>`: Same as above, but for non-keyed windows (global).

```
stream.windowAll(TumblingProcessingTimeWindows.of(Time.seconds(10)
))
 .process(new ProcessAllWindowFunction<Event, Result,
TimeWindow>() {
 @Override
 public void process(Context ctx, Iterable<Event> events,
Collector<Result> out) {
 out.collect(new Result("Total count = " +
StreamSupport.stream(events.spliterator(), false).count()));
 }
 });
```

- `CoProcessFunction<IN1, IN2, OUT>`: Processes two connected streams together. Can access state shared between both inputs.

```

DataStream<EventA> streamA = ...
DataStream<EventB> streamB = ...
streamA.connect(streamB)
 .process(new CoProcessFunction<EventA, EventB, Result>() {
 private ValueState<EventA> pendingA;
 ...
 })

```

- Async I/O in Flink: Avoid blocking operators waiting for external service calls. Increase throughput with concurrent requests. Reduce latency. Use in API calls, DB reqs and lookups, etc.
  - `orderedWait()` preserves output ordering but may introduce head-of-line blocking if slower requests delay later completed results.
  - `unorderedWait()`: Allow reordering (higher throughput)

Eg:

```

class AsyncDatabaseRequest extends RichAsyncFunction<String,
Tuple2<String, String>> {
 private transient DatabaseClient client;

 @Override
 public void open(Configuration parameters) {
 client = new DatabaseClient();
 }

 @Override
 public void asyncInvoke(String key,
ResultFuture<Tuple2<String, String>> resultFuture) {
 CompletableFuture<String> future = client.asyncQuery(key);

 future.whenComplete((result, error) -> {
 if (error != null) {
 resultFuture.completeExceptionally(error);
 } else {
 resultFuture.complete(Collections.singleton(new
Tuple2<>(key, result)));
 }
 });
 }
}

```

```

 }
 });
}

@Override
public void timeout(String input, ResultFuture

```

- Broadcast State pattern: Broadcasting rules/config to all parallel instances for enrichment. Each task (operator instance) maintains its own replica of the broadcast state (i.e its a type of operator state replicated across all parallel subtasks), ensuring every event is processed with the latest rules – without network lookups. Broadcast state is fully replicated across all parallel subtasks → memory pressure grows linearly with number of subtasks × state size. To Manage Broadcast State Growth: Evict stale entries, State TTL, Compaction, Shard/split broadcast stream (multiple streams), Use rocksdb backend.

```

// Stream 1: Rules/config (low volume)
BroadcastStream<Rule> broadcastStream = rulesStream
 .broadcast(rulesStateDescriptor);

// Stream 2: Events (high volume)
DataStream<Event> eventStream = ...;

```

```
// Connect and process
eventStream
 .connect(broadcastStream)
 .process(new BroadcastProcessFunction<Event, Rule, Result>() {
 @Override
 public void processElement(Event event, ReadOnlyContext
ctx, Collector<Result> out) {
 // Read broadcast state
 for (Map.Entry<String, Rule> entry :

ctx.getBroadcastState(rulesStateDescriptor).immutableEntries()) {
 Rule rule = entry.getValue();
 if (rule.matches(event)) {
 out.collect(new Result(event, rule));
 }
 }
 }

 @Override
 public void processBroadcastElement(Rule rule, Context
ctx, Collector<Result> out) {
 // Update broadcast state

ctx.getBroadcastState(rulesStateDescriptor).put(rule.getId(),
rule);
 }
 });
```

- Side Output: Additional output streams from operators for specific use cases. Used for late data handling, error streams, etc.

```
final OutputTag<Event> lateDataTag = new
OutputTag<Event>("late-data"){ };
final OutputTag<Event> errorTag = new
OutputTag<Event>("errors"){ };
```

```

SingleOutputStreamOperator<Result> mainStream = stream
 .process(new ProcessFunction<Event, Result>() {
 @Override
 public void processElement(Event event, Context ctx,
Collector<Result> out) {
 try {
 if (event.getTimestamp() <
ctx.timerService().currentWatermark()) {
 // Late data
 ctx.output(lateDataTag, event);
 } else if (!event.isValid()) {
 // Invalid data
 ctx.output(errorTag, event);
 } else {
 // Normal processing
 out.collect(process(event));
 }
 } catch (Exception e) {
 ctx.output(errorTag, event);
 }
 }
 });

// Get side outputs
DataStream<Event> lateData =
mainStream.getSideOutput(lateDataTag);
DataStream<Event> errors = mainStream.getSideOutput(errorTag);

```

- A `SingleOutputStreamOperator<T>` represents a transformation in your Flink job that produces one output stream of type `T`. This operator produces exactly one output stream. For transformations that produce multiple output types (like side outputs), Flink uses: `ProcessFunction + OutputTag` to produce multiple outputs (main + side).
  - `DataStream` → the base abstraction.
  - `SingleOutputStreamOperator` → a `DataStream` that comes from an operation producing one output.
  - `KeyedStream`, `WindowedStream` → specializations for



keyed/windowed data.

- Async Functions:
  - AsyncFunction<IN, OUT>: Lightweight - just asyncInvoke()
  - RichAsyncFunction<IN, OUT>: + lifecycle hooks (open, close, getRuntimeContext). Any "Rich" prefix = Lifecycle + RuntimeContext access Examples: RichMapFunction, RichFlatMapFunction, RichAsyncFunction RuntimeContext: Provides metadata & utilities about running task
- Joins:
  - For event-time Join between two streams, can use intervalJoin.

```
SingleOutputStreamOperator<JoinedEvent> joinedStream =
 leftStream
 .keyBy(LeftEvent::getJoinKey)
 .intervalJoin(rightStream.keyBy(RightEvent::getJoinKey))
 .between(Time.seconds(-60), Time.seconds(60))
 .process(new ProcessJoinFunction<LeftEvent, RightEvent,
JoinedEvent>() {
 @Override
 public void processElement(
 LeftEvent left,
 RightEvent right,
 Context ctx,
 Collector<JoinedEvent> out
) throws Exception {
 JoinedEvent joined = new JoinedEvent();
 joined.setId(UUID.randomUUID().toString());
 joined.setTimestamp(right.getEventTimestamp());
 // Combine/enrich from both sides
 out.collect(joined);
 }
 });
```

How .between() Works

For each LEFT event at time T:

Match RIGHT events where:

$\text{rightTime} \in [\text{leftTime} - 60\text{s}, \text{leftTime} + 60\text{s}]$

Example:

Left event at 12:00:30

Matches right events from 11:59:30 to 12:01:30

Key Points

Requirement: Both streams must be keyed on join key

Time-based: Uses event time (not processing time)

Symmetric window: -60s to +60s means 2-minute total window

ProcessJoinFunction: Defines join logic & output type

- Sinks (S3/Kafka, etc):
  - FileSink API to write in S3. Kafka methods to publish in Kafka.

```
// FileSink API:
OutputFileConfig fileConfig = OutputFileConfig.builder()
 .withPartPrefix("user-data-")
 .withPartSuffix(".json")
 .build();
FileSink<T> sink = FileSink
 .forRowFormat(new Path(s3BasePath), new Encoder<T>() {
 @Override
 public void encode(T element, OutputStream stream) throws
IOException {
 String json =
objectMapper.writeValueAsString(element);
 stream.write(json.getBytes(StandardCharsets.UTF_8));
 stream.write('\n'); // Newline-delimited JSON
 }
 })
 .withBucketAssigner(new
DateTimeBucketAssigner<>("yyyy-MM-dd"))
 .withRollingPolicy(
 DefaultRollingPolicy.builder()
 .withRolloverInterval(5000) // Roll every 5s
 .withInactivityInterval(5000) // Roll after 5s idle
 .withMaxPartSize(1024) // Roll after 1KB
 .build()
```

```
)
.withOutputFileConfig(fileConfig)
.build();
```

Info:

BucketAssigner: Organizes files by date/time

RollingPolicy: When to close & start new files. Like: Rollover

interval: Time-based, Inactivity: Idle period, Max size: File size threshold.

Encoder: Custom serialization logic

-----

// Kafka Sink:

```
KafkaRecordSerializationSchema<Event> schema =
 KafkaRecordSerializationSchema.<Event>builder()
 .setTopic("output-topic")
 .setValueSerializationSchema(new EventSchema())
 .build();
```

```
KafkaSink<Event> kafkaSink = KafkaSink.<Event>builder()
 .setBootstrapServers("broker1:9092,broker2:9092")
 .setRecordSerializer(schema)
 .build();
```

// Usage

```
stream.sinkTo(kafkaSink);
```

Info:

KafkaRecordSerializationSchema: How to serialize + which topic

ValueSerializationSchema: Custom serializer for your type

setRecordSerializer: Wires serialization logic to sink

- Flink's Table API and SQL API are high-level, declarative APIs built on top of the DataStream API. You can register DataStreams as Tables, perform Table or SQL queries, and then convert the results back to streams.

```
StreamTableEnvironment tableEnv =
StreamTableEnvironment.create(env);
```

```
// Register a stream as a table with schema and watermark
```

```

tableEnv.createTemporaryView(
 "Events",
 eventStream,
 Schema.newBuilder()
 .column("userId", DataTypes.STRING())
 .column("amount", DataTypes.DOUBLE())
 .column("eventTime", DataTypes.TIMESTAMP(3))
 .watermark("eventTime", "eventTime - INTERVAL '5' SECOND")
 .build()
);

// Table API query
Table result = tableEnv.from("Events")
 .groupBy($"userId")
 .select($"userId", $"amount").sum().as("totalAmount"));

// Convert back to DataStream
DataStream<Row> resultStream = tableEnv.toDataStream(result);

// Equivalent SQL ex:
Table result = tableEnv.sqlQuery(
 "SELECT userId, SUM(amount) AS totalAmount " +
 "FROM Events GROUP BY userId"
);

```

- CEP (Complex Event Processing): Detect patterns in event streams. Used for fraud detection.

```

Pattern<Event, ?> pattern = Pattern.<Event>begin("start")
 .where(new SimpleCondition<Event>() {
 @Override
 public boolean filter(Event event) {
 return event.getType().equals("login");
 }
 })
 .next("middle")
 .where(event -> event.getType().equals("payment"))

```

```
.followedBy("end")
.where(event -> event.getType().equals("logout"))
.within(Time.minutes(10));
```

- Other notes:
  - Operations in Flink:

- Transformations:
  - map: One-to-one transformation that applies a function to each element. Eg: `dataStream.map(event -> event.toUpperCase());`
  - flatMap: One-to-many transformation that can produce zero, one, or multiple elements for each input. Eg: `dataStream.flatMap((event, out) -> {for (String word : event.split(" ")) { out.collect(word); }});`
  - filter: Keeps elements that satisfy a condition. Eg: `dataStream.filter(event -> event.contains("error"));`
- Keying & Partitioning:
  - keyBy: Groups the stream by a key (creates a `KeyedStream`). Eg: `dataStream.keyBy(event -> event.getUserId());`
  - shuffle: Redistributes elements randomly. Eg: `dataStream.shuffle();`
  - rebalance: Evenly distributes elements across tasks. Eg: `dataStream.rebalance();`
  - rescale: Redistributes elements in a round-robin fashion among downstream tasks. Eg: `dataStream.rescale();`
  - broadcast: Replicates each element to all parallel tasks. Eg: `dataStream.broadcast();`
- Aggregations & Windows:
  - reduce: Combines elements of a `KeyedStream` using a reduce function. Eg: `keyedStream.reduce((a, b) -> new Sum(a.value + b.value));`
  - aggregate: Applies an aggregation function (like sum, min, max). Eg: `keyedStream.sum("amount");`  
`keyedStream.min("latency");` `keyedStream.max("value");`
  - window: Groups elements of a stream into finite sets based on time or count. Eg:

```

keyedStream.window(TumblingEventTimeWindows.of(Time.seconds(5)));
keyedStream.window(SlidingProcessingTimeWindows.of(Time.seconds(10), Time.seconds(5)));
keyedStream.countWindow(100, 10); // sliding count window
- windowAll: Creates a window on a non-keyed stream. Eg:
 dataStream.windowAll(TumblingProcessingTimeWindows.of(Time.minutes(1)));
- Joining & Combining
 - join: Joins two data streams based on a key. Eg:
 stream1.join(stream2).where(e1 ->
 e1.getKey().equalTo(e2 ->
 e2.getKey()).window(TumblingEventTimeWindows.of(Time.seconds(5))).apply((e1, e2) -> new Tuple2<>(e1, e2)));
 - coGroup: Groups and joins two streams in a more flexible way than join. Eg: stream1.coGroup(stream2).where(e1 ->
 e1.getKey().equalTo(e2 ->
 e2.getKey()).window(TumblingEventTimeWindows.of(Time.seconds(5))).apply(new CoGroupFunction<...>() {...}));
 - union: Combines multiple streams of the same type. Eg:
 stream1.union(stream2, stream3);
 - connect: Combines two streams of potentially different types. Eg: stream1.connect(stream2).map(new
 CoMapFunction<Type1, Type2, ResultType>() {...});

```

- Flink Performance Optimization: Parallelism Tuning (setParallelism), State Optimization (ValueState or MapState, Implement state TTL, Incremental Checkpointing, RocksDB Backend state), Checkpoint Optimization, Resource Configs, Network Optimize (Chain operators, reduce shuffle, local aggregations before shuffle), Operator Optimization (Use AggregateFunction instead of ProcessWindowFunction, Batch operations if possible).
- Backpressure reasons: Slow operators, External system latency, Insufficient resources, Data skew (hot keys), GC pressure, Checkpoint alignment delays.

- In eks yaml deployment config file you may use: upgradeMode key. The upgradeMode setting determines how Flink jobs behave during upgrades or restarts:
    - last-state mode: When a job is upgraded or restarted, it will attempt to restore from the most recent successful checkpoint. This preserves the processing state and allows the job to continue from where it left off.
    - stateless mode: The job will start from scratch after an upgrade, without attempting to restore any previous state. This is like a completely fresh deployment.
    - Note that if you have checkpointing related configs in yaml; The checkpointing configuration remains active in both modes - it's just that the stateless mode ignores existing checkpoints when pipeline starting up.
  - Deployment modes in Flink:
    - Session Mode: Long-running cluster, Multiple jobs share same cluster, Resources shared across jobs, Use case: Development, multiple small jobs
    - Application Mode: One cluster per application, Main method runs on cluster, Better resource isolation, Use case: Production applications
    - Per-Job Mode (Deprecated): One cluster per job, Cluster terminated after job finishes, Use case: Batch jobs, isolation
-





# Linux

## Linux

- **sudo:** Sudo is a command in Linux allows users to run commands with privileges that only root user have.
- **grep -i "UNix" geekfile.txt:** Case insensitive search
- **grep -c "unix" geekfile.txt:** Displaying the count of number of matches
- **grep -l "unix" \* (or) grep -l "unix" f1.txt f2.txt f3.txt f4.txt:** Display the file names that matches the pattern
- **grep -w "unix" geekfile.txt:** Checking for the whole words in a file
- **head <file>** Displays the first parts of files, with the syntax, head [options] [file\_name].
- **tail <file>:** Displays the last N number of lines of the given input.
  - **tail -f /var/log/syslog:** It only shows the last part of the logs, where problems usually lie. tail will continue watching the log file and print out the next line written to the file.
  - If you do: `pip install -r /tmp/requirements.txt --ignore-installed | tee /tmp/pip_installation.log` - It will install pip requirements packages as well as output the process in pip\_installation.log file. You can watch this log file using tail command: `tail -f pip_installation.log`
- **cat <file\_name>:** View file content
  - Eg: `cat <file> | head -88 | tail -1`: To display the 88th line of a file
- **fdisk -l:** The fdisk command is used for disk partitioning and disk management on Unix-like systems. The -l option lists information about all available disks and their partitions. This command is typically used to view the current disk layout and partitioning scheme of the system.
- **ps -ef:** The ps command is used to display information about running processes on Unix-like systems. The -ef options stand for "everyone" (-e) and "full-format" (-f), which together instruct ps to display information about all processes in a detailed format. This includes information such as the process ID (PID), the terminal associated with the process, the user running the process, CPU and memory usage, and the command being executed.
  - `ps -ef | grep pip`: Lists down all running processes by the name pip.
- **ls -l <dir path else it lists current path files>:** Lists files and directories in the specified directory (or the current directory if none is

specified) with detailed information, including permissions, ownership, size, and modification date.

- `ls -lrt`: Lists files and directories in the current directory in long format (-l) sorted by modification time in reverse order (-r for reverse, -t for time).
- `pwd`: Prints the current working directory.
- `ls -la`: Lists all files (including hidden files) and directories in the current directory in long format, showing detailed information, including hidden files (-a for all).
- `su`: Allows switching to another user account. If no username is specified, it defaults to the superuser (root) account.
- `su root`: Switches to the root user account.
- `whoami`: Displays the username of the current user.
- `who`: Lists about users who are currently logged in to the system.
- `last`: This command in Linux is used to display the history of last logged in users.
- `which python`: Displays path where executable file of python is located.
- `top`: Displays real-time information about system processes, including CPU and memory usage.
  - In your keyboard if you press button 'c' after running top command - it will display the stats in detailed manner. There are other keyboard buttons as well to press which would display other functionalities with top command.
- `set | grep AWS_KEY`: Displays environment variables (set) and filters the output to show lines containing AWS\_KEY using grep.
- `screen -S vikas`: Starts a new screen session with the name vikas.
  - `screen -ls`: Lists currently running screen sessions. screen command in Linux provides the ability to launch and use multiple shell sessions from a single ssh session.
- `find ~ -name aws_credentials`: Searches for files named aws\_credentials in the user's home directory (~).
  - `find / -name "requirements.txt" 2>/dev/null`: Searches for file from root directory. The /dev/null command suppresses error messages about directories you don't have permission to read.
  - `find . -name "requirements.txt"`: Command will search for the file starting from the current directory (.) and look through all subdirectories.
  - `find / -type d -name "efs"`: Command will search for a directory with name "efs".

- `grep -iR AWS_KEY *`: Searches recursively (-R) for occurrences of `AWS_KEY` in files in the current directory and its subdirectories (\*).
- `exit`: Exits the current shell or session.
- `grep -iR access <path>`: Searches recursively for occurrences of `access` in files within the specified path.
- `echo $ec2`: Prints the value of the environment variable `ec2`.
- `pip install -U s3fs=0.4.0`: Installs or upgrades the `s3fs` Python package to version 0.4.0 using `pip`.
- `cd ./venvs`: Changes the current directory to `venvs`, which is located in the current directory (.).
- `df -h`: The `df` command is used to display disk space usage on Unix-like systems. The `-h` option stands for "human-readable" and formats the output in a more easily understandable way, showing sizes in kilobytes, megabytes, gigabytes, etc., rather than in raw bytes.
- `du psycopg2-2.9.3.tar.gz`: Displays the disk usage of the `psycopg2-2.9.3.tar.gz` file.
- `du -sh psycopg2-2.9.3.tar.gz`: Displays the disk usage of the `psycopg2-2.9.3.tar.gz` file in a human-readable format (-h) and summarizes the total (-s).
- `du -sh *`: Displays the disk usage of all files and directories in the current directory in a human-readable format and summarizes the total.
  - `sudo du -sh /*`: Displays disk usage for all files/directories with root user privilege.
- `du -sh * | sort -h`: Displays the disk usage of all files and directories in the current directory in a human-readable format, summarizes the total, and sorts the output numerically and in a human-readable way (-h).
- `conda list`: Lists installed packages and their versions using Conda, a package manager for Python.
- `echo $PATH`: It prints out the value of the `PATH` environment variable.
- `sudo -su user`: It is short for `sudo -s -u user`. The `-s` option means to run the shell specified in the environment variable `SHELL` if this has been set, or else the user's login shell. The `-u user` option means to run the command as the specified user rather than root.
- `sudo su user`: It will use `sudo` to run the command `su user` as the root user. The `su` command will then invoke the login shell of the specified username. The `su user` command could be run without the use of `sudo`, but by running it as root it will not require the password of the target user.
- `rm /path/to/directory/*`: To remove all non-hidden files\* in a directory

- `rm -r /path/to/directory/*`: To remove all non-hidden files and sub-directories (along with all of their contents) in a directory
- `rm -rf "/path/to the/directory/"*`: Force removal of files
- `history`: Provides a chronological list of previously executed commands
- `bash`: `bash` is a command interpreter, a shell, a program with an interface that interprets the commands that you put into it.
- `pstree`: To know how many sub-shells deep you are.
- `netstat`: The network statistics command is a networking tool used for troubleshooting and configuration, that can also serve as a monitoring tool for connections over the network.
- `ps aux`: Similar to `ps -ef`. Displays information about running processes.
- `ssh -i`: To `ssh` into a machine. `-i` is `identity_file` which selects a file from which the identity (private key) for public key authentication is read.
- `ping -c 1 $(hostname)`: To get IP address of the machine
- `watch`: Used to execute a program periodically, showing output in fullscreen.
  - `watch -n 1 "ps -ef | grep ssh"`: Executes `ps -ef | grep ssh` command every second and displays output.
- `kill -9 <pid>`: To kill a process/pid.
  - In Linux, an executable stored on disk is called a program, and a program loaded into memory and running is called a process. When a process is started, it is given a unique number called process ID (PID) that identifies that process to the system. If you ever need to kill a process, for example, you can refer to it by its PID.
  - In addition to a unique process ID, each process is assigned a parent process ID (PPID) that tells which process started it. The PPID is the PID of the process's parent.
  - Eg case:

```
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17600
root 17600 17599 0 Jul04 pts/2 00:00:00 ssh
ubuntu@10.80.10.5 pip install -r /tmp/requirements.txt
--ignore-installed
ubuntu 47424 42931 0 09:41 pts/3 00:00:00 grep
--color=auto 17600
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17599
root 17599 17598 0 Jul04 pts/2 00:00:06
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
```

```

5392337109202/AnsiballZ_command.py
root 17600 17599 0 Jul04 pts/2 00:00:00 ssh
ubuntu@10.80.10.5 pip install -r /tmp/requirements.txt
--ignore-installed
ubuntu 47422 42931 0 09:41 pts/3 00:00:00 grep
--color=auto 17599
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17598
root 17598 17597 0 Jul04 pts/2 00:00:00 /bin/sh -c
echo BECOME-SUCCESS-ajehwmoeqrbnefnxsyzfglmuebkerlqz ;
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
root 17599 17598 0 Jul04 pts/2 00:00:06
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
ubuntu 47429 42931 0 09:41 pts/3 00:00:00 grep
--color=auto 17598
ubuntu@ip-10-80-10-5:/tmp$ ps -ef | grep 17597
root 17597 17596 0 Jul04 pts/2 00:00:00 sudo -H -S -n
-u root /bin/sh -c echo
BECOME-SUCCESS-ajehwmoeqrbnefnxsyzfglmuebkerlqz ; /usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
root 17598 17597 0 Jul04 pts/2 00:00:00 /bin/sh -c
echo BECOME-SUCCESS-ajehwmoeqrbnefnxsyzfglmuebkerlqz ;
/usr/bin/python3
/home/ubuntu/.ansible/tmp/ansible-tmp-1720107928.4152038-772178-15
5392337109202/AnsiballZ_command.py
ubuntu 47467 42931 0 09:43 pts/3 00:00:00 grep
--color=auto 17597

```

- cd /proc/<your pid>/fd: [To get process details from its PID, we use it.](#)  
Post cd into the directory we can run ls or cat command and get infos on processes. Eg: sudo ls -l /proc/330046/fd/1
  - Cases:

/proc/cpuinfo: Information about the processor, such as its type, make, model, and performance.

/proc/[pid]/cmdline: This holds the complete command line for the process, unless the whole process has been swapped out, or unless the process is a zombie. In either of these later cases, there is nothing in this file: i.e. a read on this file will return 0 characters. The command line arguments appear in this file as a set of null-separated strings, with a further null byte after the last string.

/proc/[pid]/cwd: This is a link to the current working directory of the process.

/proc/[pid]/environ: This file contains the environment for the process. The entries are separated by null characters, and there may be a null character at the end.

/proc/[pid]/exe: The exe file is a symbolic link containing the actual path name of the executed command. The exe symbolic link can be dereferenced normally - attempting to open exe will open the executable. You can even type /proc/[pid]/exe to run another copy of the same process as [pid].

/proc/[pid]/fd: This is a subdirectory containing one entry for each file which the process has opened, named by its file descriptor, and which is a symbolic link to the actual file (as the exe entry does). Thus, 0 is standard input, 1 standard output, 2 standard error, etc.

/proc/[pid]/maps: A file containing the currently mapped memory regions and their access permissions.

/proc/[pid]/mem: The mem file provides a means to access the process memory pages, using open, fseek and read commands.

/proc/[pid]/root: This is a link to the root directory which is

seen by the process. This root directory **is** usually **"/"**, but it can be changed by the **chroot** command.

**/proc/[pid]/stat**: This file provides status information about the process. This **is** used by the Process Show utility. It **is** defined **in** **fs/proc/array.c** source file **and** may differ **from** one distribution to another.

**/proc/devices**: List of device drivers configured into the currently running kernel.

**/proc/dma**: Shows which DMA channels are being used at the moment. etc etc...

- **cmp <source path file> <destination path file>**: It is used to compare the two files byte by byte and helps you to find out whether the two files are identical or not.
- **cp <source path file> <destination path file>**: Command creates a copy of the source\_file at the specified destination.
- **sh test\_file.sh** : To run a shell script. sh is the bourne shell. There are several shells, of which bourne is the old standard, installed on all unix systems, and generally the one you can guarantee will exist. The shell is the command interpreter that takes your input, provides output back to the screen, to the correct files, etc, and provides all the basic built-in commands you need to manage jobs, kill, test expressions, etc. Your command above is saying to run that shell-script using the bourne shell. Different shells use different syntax, so using the correct shell is a requirement. The first line of the shell should also define which to use: **#!/bin/sh** says use **/bin/sh**.
  - The **-c** option in the **sh** command is used to execute a command or a series of commands provided as an argument to the **-c** option. Eg: **sh -c "command1; command2; command3"**
- **chmod a+x test\_file.sh**: The **chmod a+x** command in Linux adds the execute permission to a file for all users (owner, group, and others). All [chmod commands](#) [al](#).
  - [File permissions control](#) [al](#) which actions can be performed by which users. Read, Write, and Execute are the three actions possible for every file. Three important commands you'll use when managing file



permissions: chmod (Change mode), chown (Change ownership), chgrp (Change group).

- Users are classified under three broad categories: Normal users, Groups, and Others. Linux allows users to set permissions at a very granular level. You can secure your file or directory in every possible location of a file system.
- chmod is a command that lets you change the permissions of a file or directory to all types of users.
  - user level permissions: u – Grant permission to a user; g – Grant permission to a group (A Group of users); o – Grant permission to others
  - file level permissions: r – Grants read permission; w – Grant write permission; x – Grant execute permission
- We can execute .sh files by just running them like this, eg: `./install.sh`. But it is possible file may not execute, for such case, we need to change the relevant permissions, eg: `chmod +x install.sh` and then run. To remove permissions, one could run `chmod -x install.sh`. Similarly, say to remove read permissions: `chmod -r install.sh`.
- sudo dmesg: All the messages received from the kernel ring buffer is displayed when we execute the command “dmesg”, here only the latest messages will be shown.
  - The kernel ring buffer can be thought of as a log file, but for the kernel itself. However, unlike other log files, it's stored in memory rather than in a disk file.
- sudo apt-get install iftop: apt-get (Advanced Packaging Tool) is a command-line tool that helps in handling packages in Linux.
  - Similarly to uninstall:
    - apt-get remove iftop: will remove the binaries, but not the configuration or data files of the package packagename. It will also leave dependencies installed with it on installation time untouched.
    - apt-get purge iftop: will remove about everything regarding the package packagename, but not the dependencies installed with it on installation. Both commands are equivalent. Particularly useful when you want to 'start all over' with an application because you messed up the configuration. However, it does not remove configuration or data files residing in users home

directories, usually in hidden folders there. There is no easy way to get those removed as well.

- `apt-get autoremove`: removes orphaned packages, i.e. installed packages that used to be installed as a dependency, but aren't any longer. Use this after removing a package which had installed dependencies you're no longer interested in.
- `aptitude remove iftop`: will also attempt to remove other packages which were required by `packagename` on but are not required by any remaining packages. Note that `aptitude` only remembers dependency information for packages that it has installed.
- [etc, etc... al](#)

- `clear`: Clear the terminal screen in Linux
- `date`: Displays date in UTC in linux
- `scp <file/host> <file/host>`: `scp` (secure copy) command in Linux system is used to copy file(s) between servers in a secure way. The `SCP` command or secure copy allows the secure transferring of files between the local host and the remote host or between two remote hosts. It uses the same authentication and security as it is used in the Secure Shell (SSH) protocol. `SCP` is known for its simplicity, security, and pre-installed availability.
  - Eg: `sp -r circleci@13.120.15.80:/home/repo/file/values.yml migrated_values.vml`
  - `scp user@remotehost:/home/user/file_name`: Securely copy a file from remote machine to our local machine
  - `scp [file_name] remoteuser@remotehost:/remote/directory`: Securely copy a file from a local machine to a remote machine
- `cat /etc/passwd`: To list all users in a Linux system you can read the file (as seen). Each line in this file represents a user account, and the first field in each line is the username.
- `cat /etc/group`: The `/etc/group` file is a configuration file on Unix-like systems that stores the group information for the system. It contains a list of groups, each with a unique group name and a list of users who are members of that group.
- `cat /etc/sudoers`: The `/etc/sudoers` file is a configuration file on Unix-like systems that specifies which users or groups are allowed to run commands with superuser (root) privileges using the `sudo` command.
- `tcpdump`: It is used to capture, filter, and analyze network traffic such as TCP/IP packets going through your system.

- In terminal, if you type a long command, and want to move the cursor to first alphabet/position, press: `Ctrl + a` in keyboard.
- In terminal, if you want to retrace and use a command previously/historically used, click `Ctrl + r` in keyboard and search the command in terminal, for further historical instance, click `Ctrl + r` again and again.
- `cat date_2024_08_04_data.csv | wc -l`: 'wc' stands for word count. In given command we count the no. of lines in the file.
- Run `gunzip` `zip/unzip` command(s) and `tar` file commands:
  - `gunzip originalGeoIP2-City.tar.gz` (unzip file) -> `tar -xvf originalGeoIP2-City.tar` (decompress tar file) -> `tar -cvf gunzipTarGeoIP2-City.tar GeoIP2-City_20230512/` (compress folder to tar file) -> `gzip gunzipTarGeoIP2-City.tar` (zip back)
- `md5sum file.yml`: The MD5 sum of a file is a 32-character hexadecimal number that verifies a file's integrity. It's commonly used to check if a file has changed due to a disk error, faulty transfer, or other reasons.
- `aws cloudwatch describe-alarms --query "MetricAlarms[][AlarmName, AlarmActions]" | jq -r '.[1] | select(.[1] != null) | "\(.[])"'`: The command you provided is a combination of AWS CLI and the `jq` tool, which is used for processing JSON data. The entire command retrieves a list of CloudWatch alarms, specifically their names and actions. It then filters this list to include only those alarms that have defined actions (i.e., their action field is not null) and outputs each alarm name and its associated actions as plain text strings. The pipe symbol takes the output from the previous command and feeds it into `jq`, just note.

---

## Vim commands:

- Exiting Vim
  - `:q!`: Exit Vim terminal without saving
  - `:wq!`: Save and exit Vim terminal
- Editing Files
  - `:%d`: Clear all contents of a file
  - `:set nu`: Add line numbers to the file
- Navigating Files
  - `gg`: Move to the first line of the file
  - `G`: Move to the last line of the file
  - `gg=G`: Reindent the whole file

- gv: Reselect the last visual selection
- ^: Move to the first non-blank character of the line
- g\_: Move to the last non-blank character of the line
- gf: Jump to the file name under the cursor
- Manipulating Text
  - xp: Swap characters forward
  - Xp: Swap characters backward
  - yyp: Duplicate the current line
  - yapP: Duplicate the current paragraph
  - dat: Delete around an HTML tag, including the tag
  - dit: Delete inside an HTML tag, excluding the tag
  - w: Move one word to the right
  - b: Move one word to the left
  - dd: Delete the current line
  - <<: Outdent the current line
  - >>: Indent the current line
  - ~: Toggle the case of the current character
  - gUw: Uppercase until the end of the word
  - gUiw: Uppercase the entire word
  - gUU: Uppercase the entire line
  - gu\$: Lowercase until the end of the line
  - da": Delete the next double-quoted string
  - +: Move to the first non-whitespace character of the next line
  - S: Delete the current line and go into insert mode
  - I: Insert at the beginning of the line
  - ci": Change what's inside the next double-quoted string
  - ca{: Change inside the curly braces (try [, (, etc.)
  - vaw: Visually select a word
  - dap: Delete the whole paragraph
  - r: Replace a character
- Navigating Changes
  - [: Jump to the beginning of the last yanked text
  - ]: Jump to the end of the last yanked text
  - g;: Jump to the last change you made
  - g,: Jump back forward through the change list
- Repeating Commands
  - &: Repeat the last substitution on the current line

- g&: Repeat the last substitution on all lines
  - Additional Commands
    - :%s/old/new/g: Replace all occurrences of "old" with "new" in the file
    - u: Undo the last action
    - Ctrl+r: Redo the last undone action
    - %: Move to the matching parenthesis, bracket, or brace
    - \*: Search forward for the word under the cursor
    - #: Search backward for the word under the cursor
    - f{char}: Move forward to the next occurrence of {char}
    - F{char}: Move backward to the previous occurrence of {char}
    - t{char}: Move forward until before the next occurrence of {char}
    - T{char}: Move backward until before the previous occurrence of {char}
    - /{pattern}: Search forward for the {pattern}
    - ?{pattern}: Search backward for the {pattern}
    - n: Repeat the last search in the same direction
    - N: Repeat the last search in the opposite direction
    - Shift + v: Visual editor mode
- 

#### Misc Notes:

- /dev/null is a like a virtual device, which has a special property: Any data written to /dev/null vanishes or disappears. Because of this characteristic, it is also called bitbucket or blackhole.
  - Eg: echo "helloworld" > /dev/null; Then if you do: cat /dev/null; You won't get any output.
  - It is mainly used to discard standard output and standard error from an output.
- [Linux Directory Structure \\_al](#)
  - / – The root directory: Everything, all the files and directories, in Linux are located under 'root' represented by '/'.
    - /bin – Binaries: The '/bin' directly contains the executable files of many basic shell commands like ls, cp, cd etc. Mostly the programs are in binary format here and accessible by all the users in the Linux system.
    - /dev – Device files: This directory only contains special files, including those relating to the devices. These are virtual files, not physically on the disk. Some interesting examples of these files are:
      - /dev/null: can be sent to destroy any file or string

- /dev/zero: contains an infinite sequence of 0
  - /dev/random: contains an infinite sequence of random values
- /etc – Configuration files: The /etc directory contains the core configuration files of the system, use primarily by the administrator and services, such as the password file and networking files. If you need to make changes in system configuration (for example, changing the hostname), the etc folder is where you'll find the respective files.
- /usr – User binaries and program data: In '/usr' go all the executable files, libraries, source of most of the system programs. For this reason, most of the files contained therein is readonly (for the normal user)
- /home – User personal data: Home directory contains personal directories for the users. The home directory contains the user data and user-specific configuration files. As a user, you'll put your personal files, notes, programs etc in your home directory. When you create a user on your Linux system, it's a general practice to create a home directory for the user. Suppose your Linux system has two users, Alice and Bob. They'll have a home directory of their own at locations /home/alice and /home/bob. Do note that Bob won't have access to /home/alice and vice versa.
- /lib – Shared libraries: Libraries are basically codes that can be used by the executable binaries. The /lib directory holds the libraries needed by the binaries in /bin and /sbin directories. Libraries needed by the binaries in the /usr/bin and /usr/sbin are located in the directory /usr/lib.
- /sbin – System binaries: This is similar to the /bin directory. The only difference is that it contains the binaries that can only be run by root or a sudo user. You can think of the 's' in 'sbin' as super or sudo.
- /tmp – Temporary files: As the name suggests, this directory holds temporary files. Many applications use this directory to store temporary files. Even you can use directory to store temporary files. But do note that the contents of the /tmp directories are deleted when your system restarts. Some Linux system also delete files old files automatically so don't store anything important here.
- /var – Variable data files: Var, short for variable, is where programs store runtime information like system logging, user tracking, caches, and other files that system programs create and manage. The files stored here are NOT cleaned automatically and hence it provides a good place for system administrators to look for information about their system behavior. For example, if you want to check the login history in your Linux system, just check the content of the file in /var/log/wtmp.

- /boot – Boot files: The '/boot' directory contains the files of the kernel and boot image, in addition to LILO and Grub. It is often advisable that the directory resides in a partition at the beginning of the disc.
- /proc – Process and kernel files: The '/proc' directory contains the information about currently running processes and kernel parameters. The content of the proc directory is used by a number of tools to get runtime system information. For example, if you want to check processor information in Linux, you can simply refer to the file /proc/cpuinfo. You want to check memory usage of your Linux system, just look at the content of /proc/meminfo file.
- /opt – Optional software: Traditionally, the /opt directory is used for installing/storing the files of third-party applications that are not available from the distribution's repository. The normal practice is to keep the software code in opt and then link the binary file in the /bin directory so that all the users can run it.
- /root – The home directory of the root: There is /root directory as well and it works as the home directory of the root user. So instead of /home/root, the home of root is located at /root. Do not confuse it with the root directory (/).
- /media – Mount point for removable media: When you connect a removable media such as USB disk, SD card or DVD, a directory is automatically created under the /media directory for them. You can access the content of the removable media from this directory.
- /mnt – Mount directory: This is similar to the /media directory but instead of automatically mounting the removable media, mnt is used by system administrators to manually mount a filesystem.
- /srv – Service data: The /srv directory contains data for services provided by the system. For example, if you run a HTTP server, it's a good practice to store the website data in the /srv directory.

---

Other ref: <https://github.com/chassing/linux-sysadmin-interview-questions>

# FastAPI-SpringBoot



# FastAPI & Spring Boot Notes

## FastAPI Notes

### Introduction

FastAPI is a modern high-performance Python framework used for building APIs. It is built on top of Starlette for the web layer and Pydantic for validation and serialization. FastAPI is designed for high concurrency, async support, automatic validation, and automatic API documentation generation.

Common use cases include microservices, AI/ML APIs, realtime systems, and backend APIs.

---

### Architecture

Typical request flow:

Client

↓

Load Balancer

↓

Uvicorn / Gunicorn Workers

↓

ASGI Server

↓

Middleware

↓

Routing

↓

Dependency Injection

↓

Validation

↓

Business Logic

↓

Database / External Services

↓

Serialization

↓

JSON Response

---

## ASGI

ASGI (Asynchronous Server Gateway Interface) is the asynchronous successor to WSGI. Unlike WSGI, ASGI supports non-blocking request handling, WebSockets, and streaming.

WSGI frameworks such as Flask follow a synchronous thread/process-based model, whereas ASGI frameworks like FastAPI use an event-loop-based concurrency model that scales efficiently for I/O-heavy workloads.

---

## Async Programming

FastAPI heavily relies on Python's `async` and `await` syntax for asynchronous programming.

```
@app.get("/")
async def get_users():
 users = await db.fetch()
 return users
```

When a coroutine waits for I/O, control is returned to the event loop, which schedules another coroutine. This allows a single thread to efficiently handle many concurrent requests.

Async programming is highly beneficial for:

- Database calls
- External API calls
- Network I/O

It is not useful for CPU-intensive tasks such as image processing or heavy computations because of Python's GIL.

---

## Event Loop

The event loop is responsible for scheduling and managing coroutines. When a coroutine pauses during I/O, the event loop switches execution to another task instead of blocking the thread.

Execution flow:

Request arrives

↓

Coroutine starts

↓

Coroutine waits on I/O

↓

Event loop schedules another coroutine

↓

Original coroutine resumes after I/O completion

This architecture significantly improves concurrency while reducing thread overhead.

---

## Dependency Injection

FastAPI provides lightweight dependency injection using `Depends()`.

```
from fastapi import Depends

def get_db():
 return Session()

@app.get("/")
def users(db=Depends(get_db)):
 pass
```

Dependency injection improves modularity, loose coupling, and testability. Common use cases include database sessions, authentication, shared services, Redis clients, and configuration management.

---

## Pydantic

Pydantic is used for request validation, parsing, serialization, and schema generation.

```
from pydantic import BaseModel

class User(BaseModel):
 id: int
 name: str
```

Pydantic automatically validates incoming payloads based on Python type hints. Invalid payloads return HTTP 422 responses automatically. Pydantic v2 internally uses a Rust-based validation engine, making serialization and validation extremely fast.

---

## Middleware

Middleware executes before and after request processing. It is commonly used for authentication, logging, metrics collection, tracing, CORS handling, and rate limiting.

```
@app.middleware("http")
async def middleware(request, call_next):
 response = await call_next(request)
 return response
```

---

## Database Integration

FastAPI commonly uses SQLAlchemy with PostgreSQL or MySQL. Async database access is typically implemented using SQLAlchemy async engines. Connection pooling is important because creating database connections is expensive. Proper session handling prevents connection leaks and pool exhaustion.

---

## Scaling and Performance

FastAPI scales well because of its async non-blocking architecture. Common production scaling strategies include:

- Multiple worker processes
- Horizontal scaling
- Stateless services
- Redis caching
- Load balancing

A common mistake is performing blocking synchronous I/O inside async endpoints, which blocks the event loop and reduces concurrency benefits.

---

## Spring Boot Notes

### Introduction

Spring is a modular Java framework providing dependency injection, IoC container management, transaction management, AOP, and web framework support. Spring Boot simplifies Spring development through auto-configuration, embedded servers, starter dependencies, and production-ready tooling.

---

## **Architecture**

Typical request flow:

Client



Tomcat



DispatcherServlet



Controller



Service



Repository



Hibernate/JPA



Database



Response

---

## **IoC and Dependency Injection**

Inversion of Control (IoC) means the Spring container manages object creation and lifecycle instead of the application manually creating dependencies.

Dependency Injection (DI) is the mechanism used to implement IoC.

Dependencies are injected into classes rather than instantiated directly.

Constructor injection is the preferred approach because it improves immutability and testability.

---

## **Bean Lifecycle**

A bean is an object managed by the Spring container.

Lifecycle flow:

Bean Instantiation

↓

Dependency Injection

↓

@PostConstruct

↓

Bean Ready

↓

Bean Usage

↓

@PreDestroy

↓

Bean Destroyed

Singleton is the default bean scope. However, singleton beans are not automatically thread-safe because multiple threads can access the same instance concurrently.

Best practice is to keep singleton services stateless.

---

## **Spring Boot Startup Flow**

Application startup flow:

main()

↓

SpringApplication.run()

↓

Create ApplicationContext

↓

Component Scan

↓

Auto Configuration

↓

Bean Creation

↓

Embedded Server Startup

↓

Application Ready

Spring Boot auto-configuration works using `@Conditional` annotations and classpath detection.

Priority order:

Explicit user bean > Auto-configured bean

---

## DispatcherServlet

DispatcherServlet is the front controller of Spring MVC. It receives incoming requests, determines the correct controller, invokes business logic, and constructs the response.

Flow:

Request

↓

DispatcherServlet

↓

Handler Mapping

↓

Controller

↓

Service

↓

Repository

↓

Database

↓

Response

---

## AOP and Proxies

Spring uses Aspect-Oriented Programming (AOP) for transactions, logging, security, metrics, and other cross-cutting concerns.

Internally, Spring commonly uses:

- JDK Dynamic Proxies
- CGLIB Proxies

The proxy intercepts method calls and applies additional behavior such as transaction handling.

---

## Transaction Management

`@Transactional` enables declarative transaction management.

Execution flow:

Method Call

↓

Spring Proxy Intercepts

↓

Transaction Starts

↓

Business Logic Executes

↓

Commit / Rollback

An important interview topic is the self-invocation problem. Internal method calls bypass the Spring proxy, causing transactional annotations to not work in certain scenarios.

---

## Connection Pooling

Spring Boot uses HikariCP as the default connection pool.

Connection pooling:

- Reuses database connections
- Reduces latency
- Improves scalability
- Prevents expensive connection creation overhead

Improper pool sizing can cause throughput bottlenecks under high traffic.

---

## Spring MVC vs WebFlux

Spring MVC follows a blocking thread-per-request model. Each request occupies a worker thread until completion.

WebFlux is Spring's reactive non-blocking framework built on Reactor using `Mono` and `Flux`. It uses an event-loop-based concurrency model similar to FastAPI's async architecture.

WebFlux is useful for highly concurrent I/O-heavy workloads.



---

## Hibernate and JPA

JPA is a specification for ORM, while Hibernate is the most common JPA implementation.

Hibernate responsibilities include:

- SQL generation
- Entity mapping
- Dirty checking
- Caching

A major interview topic is the N+1 query problem, where one parent query triggers many additional child queries. Solutions include:

- JOIN FETCH
- EntityGraph
- Batch fetching

LAZY fetching is generally preferred over EAGER fetching to avoid unnecessary database queries.

---

## Caching

Spring supports caching using annotations such as:

- `@Cacheable`
- `@CacheEvict`
- `@CachePut`

Common providers include Redis, Caffeine, and EhCache.

Caching reduces database load and improves latency significantly.

---

## Messaging and Async Processing

Spring integrates well with Kafka and RabbitMQ for event-driven architectures.

Common annotations:

- `@KafkaListener`
- `@RabbitListener`
- `@Async`

Async processing is commonly used for background tasks, notifications, and decoupled communication between services.

---

## Production Concerns

Important production topics: Structured logging, Metrics collection, Distributed tracing, Correlation IDs, Monitoring, Horizontal scaling, Circuit breakers, Retry mechanisms.

Common tools: Micrometer, Prometheus, Grafana, OpenTelemetry, Resilience4j.

-----

AWS

## 1. AWS Fundamentals

| Question                          | Answer                                                                                      |
|-----------------------------------|---------------------------------------------------------------------------------------------|
| What is AWS?                      | Cloud platform offering on-demand compute, storage, and services via pay-as-you-go pricing. |
| Benefits of AWS?                  | Scalability, flexibility, cost efficiency, global reach, and managed services.              |
| Shared Responsibility Model?      | AWS secures the cloud infrastructure; you secure data, applications, and access.            |
| IaaS vs PaaS vs SaaS?             | IaaS: EC2, PaaS: Elastic Beanstalk, SaaS: WorkMail.                                         |
| Region vs AZ vs Edge Location?    | Region = geographic area; AZ = isolated data centers; Edge = CDN caching points.            |
| Global Accelerator vs CloudFront? | Accelerator = TCP/UDP optimization; CloudFront = HTTP caching.                              |
| AWS Local Zones?                  | Extension of regions for ultra-low latency applications near users.                         |

---

## 2. EC2 (Elastic Compute Cloud)

- An Amazon Machine Image (AMI) serves as a template for virtual servers, known as EC2 instances, enabling rapid and consistent provisioning.

Features:

- Root Volume: The AMI captures a state, including the operating system and pre-installed applications, and stores it as a template. This template is employed to establish the root volume of the EC2 instance.
- Customizations and Snapshots: Beyond the root volume, the AMI can include additional storage volumes. These might have specific data sets or applications. When the AMI is utilized to launch an EC2 instance, it ensures that these configured volumes are also established and linked to the new EC2 instance.

- In AWS, every EC2 instance automatically gets a Public IP and can optionally be assigned an Elastic IP for more flexibility.

| Question                                        | Answer                                                                                                               |
|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| What is EC2?                                    | Virtual servers in AWS cloud.                                                                                        |
| On-Demand vs Reserved vs Spot vs Savings Plans? | On-Demand = flexible; Reserved = 1-3yr commit; Spot = cheapest (interruptible); Savings Plans = flexible commitment. |
| What is an AMI?                                 | Pre-configured OS template for launching instances.                                                                  |
| Security Group vs NACL?                         | SG = stateful (instance-level); NACL = stateless (subnet-level).                                                     |
| Stop vs Terminate vs Hibernate?                 | Stop = pause + retain EBS; Terminate = delete; Hibernate = save RAM state.                                           |
| Instance Store vs EBS?                          | Instance Store = ephemeral (lost on stop); EBS = persistent block storage.                                           |
| EC2 Placement Groups?                           | Cluster = low latency; Spread = HA across hardware; Partition = distributed apps.                                    |
| Enhanced Networking?                            | SR-IOV for higher PPS, lower latency (ENA, Intel VF).                                                                |
| Nitro System?                                   | Hypervisor replacement providing better performance and security.                                                    |
| How to troubleshoot failed instance launch?     | Check instance limits, subnet capacity, AMI availability, EBS snapshot status.                                       |

---

### 3. S3 (Simple Storage Service)

| Question | Answer |
|----------|--------|
|----------|--------|

|                                                                  |                                                                                     |
|------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| What is S3?                                                      | Object storage for unstructured data (unlimited scalability).                       |
| S3 Storage Classes?                                              | Standard, IA, One Zone-IA, Intelligent-Tiering, Glacier Instant/Flexible/Deep.      |
| S3 Durability vs Availability?                                   | Durability = 11 nines (data loss protection); Availability = uptime (99.9%-99.99%). |
| S3 Versioning?                                                   | Keeps multiple versions of objects (protects from deletions).                       |
| Lifecycle Policies?                                              | Automates transitions between storage classes and expiration.                       |
| Cross-Region Replication (CRR) vs Same-Region Replication (SRR)? | CRR = DR/compliance; SRR = log aggregation/compliance.                              |
| S3 Event Notification?                                           | Triggers Lambda, SQS, SNS on object changes (PUT/DELETE).                           |
| Pre-Signed URL?                                                  | Grants temporary access to private objects via signed URL.                          |
| S3 Consistency Model?                                            | Strong read-after-write consistency for all operations.                             |
| S3 Transfer Acceleration?                                        | Uses CloudFront edge locations to speed up uploads.                                 |
| Multipart Upload?                                                | Uploads large files in parts (required for >5GB objects).                           |
| S3 vs EBS vs EFS?                                                | S3 = object; EBS = block (single instance); EFS = shared file system.               |

|                                       |                                                                               |
|---------------------------------------|-------------------------------------------------------------------------------|
| S3 Intelligent-Tiering?               | Auto-moves objects between access tiers based on usage.                       |
| S3 Object Lock vs Glacier Vault Lock? | Object Lock = WORM (immutable); Vault Lock = compliance policy lock.          |
| How to optimize S3 costs?             | Lifecycle policies, Intelligent-Tiering, delete incomplete multipart uploads. |

---

#### 4. Networking (VPC, Subnets, Gateways)

| Question                             | Answer                                                                  |
|--------------------------------------|-------------------------------------------------------------------------|
| What is VPC?                         | Logically isolated virtual network in AWS.                              |
| Public vs Private Subnet?            | Public has IGW route (internet access); private does not.               |
| Internet Gateway vs NAT Gateway?     | IGW = bidirectional internet; NAT = outbound only from private subnets. |
| NAT Gateway vs NAT Instance?         | NAT Gateway = managed, HA; NAT Instance = EC2-based, manual HA.         |
| Route Table?                         | Defines routing rules for traffic within VPC.                           |
| Security Group vs NACL?              | SG = stateful, instance-level; NACL = stateless, subnet-level.          |
| VPC Peering?                         | Private connection between two VPCs (no transitive routing).            |
| Transit Gateway?                     | Hub connecting multiple VPCs, VPNs, Direct Connect.                     |
| VPC Endpoint (Gateway vs Interface)? | Gateway = S3/DynamoDB; Interface = PrivateLink for other services.      |

|                                  |                                                                                 |
|----------------------------------|---------------------------------------------------------------------------------|
| PrivateLink?                     | Securely expose services to other VPCs without internet.                        |
| Direct Connect?                  | Dedicated network connection from on-prem to AWS.                               |
| VPN vs Direct Connect?           | VPN = encrypted over internet; Direct Connect = dedicated, higher bandwidth.    |
| Bastion Host vs Session Manager? | Bastion = jump server via SSH; Session Manager = secure shell without SSH keys. |
| VPC Flow Logs?                   | Captures traffic metadata for debugging (accepted/rejected flows).              |
| Elastic IP?                      | Static public IPv4 address for instances.                                       |
| CIDR Block Sizing?               | /16 = largest VPC (65,536 IPs); /28 = smallest subnet (16 IPs).                 |

---

## 5. IAM (Identity & Access Management)

| Question                                 | Answer                                                                    |
|------------------------------------------|---------------------------------------------------------------------------|
| What is IAM?                             | Manages users, roles, and permissions in AWS.                             |
| User vs Group vs Role?                   | User = person; Group = collection of users; Role = temporary credentials. |
| IAM Policy Structure?                    | JSON with Effect, Action, Resource, Condition.                            |
| Least Privilege Principle?               | Grant minimum permissions required for tasks.                             |
| Identity-based vs Resource-based Policy? | Identity = who can act; Resource = what allows access.                    |
| STS AssumeRole?                          | Temporary credentials for cross-account or service access.                |



|                                       |                                                                                |
|---------------------------------------|--------------------------------------------------------------------------------|
| IAM Roles vs Resource-based Policies? | Roles = assume credentials; Resource = direct access (e.g., S3 bucket policy). |
| MFA?                                  | Two-factor authentication for enhanced security.                               |
| AWS Organizations?                    | Centralized multi-account management with SCPs.                                |
| SCP (Service Control Policy)?         | Restricts actions across organization accounts (guardrails).                   |
| Permission Boundaries?                | Max permissions a user/role can have (even with explicit allow).               |
| IAM Access Analyzer?                  | Identifies resources shared with external entities.                            |
| Cross-Account Access?                 | Use IAM roles with trust policies.                                             |
| IAM Policy Evaluation Logic?          | Explicit Deny > Explicit Allow > Default Deny.                                 |

---

## 6. Storage & Databases

| Question               | Answer                                                                                     |
|------------------------|--------------------------------------------------------------------------------------------|
| EBS vs Instance Store? | EBS = persistent; Instance Store = ephemeral (lost on stop).                               |
| EBS Volume Types?      | gp3/gp2 = SSD (general); io2/io1 = provisioned IOPS; st1 = HDD throughput; sc1 = cold HDD. |
| EBS Snapshots?         | Incremental backups stored in S3 (restore to any AZ).                                      |
| EBS vs EFS?            | EBS = block (single instance); EFS = shared NFS file system.                               |

|                                    |                                                                                 |
|------------------------------------|---------------------------------------------------------------------------------|
| EFS vs FSx?                        | EFS = Linux NFS; FSx = Windows (FSx for Windows) or high-perf (FSx for Lustre). |
| What is RDS?                       | Managed relational database (MySQL, PostgreSQL, Oracle, SQL Server, MariaDB).   |
| Multi-AZ vs Read Replica?          | Multi-AZ = HA (sync failover); Read Replica = read scaling (async).             |
| RDS Backup vs Snapshot?            | Backup = automated daily; Snapshot = manual point-in-time.                      |
| Aurora vs RDS?                     | Aurora = 5x MySQL, 3x PostgreSQL performance, auto-scaling storage.             |
| Aurora Global Database?            | Cross-region read replicas with <1s replication lag.                            |
| Aurora Serverless?                 | Auto-scaling database with pay-per-request pricing.                             |
| DynamoDB?                          | Fully managed NoSQL database with single-digit ms latency.                      |
| Partition Key vs Sort Key?         | Partition Key = distribution; Sort Key = ordering within partition.             |
| DynamoDB Streams?                  | Captures item changes for event-driven processing.                              |
| DAX (DynamoDB Accelerator)?        | In-memory cache for microsecond latency.                                        |
| DynamoDB On-Demand vs Provisioned? | On-Demand = auto-scaling; Provisioned = pre-defined capacity.                   |
| Global Tables?                     | Multi-region, active-active DynamoDB replication.                               |

|                                   |                                                                        |
|-----------------------------------|------------------------------------------------------------------------|
| DynamoDB vs RDS?                  | DynamoDB = NoSQL, horizontal scaling; RDS = SQL, vertical scaling.     |
| Redshift?                         | Data warehouse for OLAP analytics (petabyte-scale).                    |
| ElastiCache (Redis vs Memcached)? | Redis = persistence, replication, pub/sub; Memcached = simple caching. |

## 7. Serverless (Lambda, SQS, SNS, API Gateway)

| Question                   | Answer                                                                   |
|----------------------------|--------------------------------------------------------------------------|
| What is Lambda?            | Run code without managing servers (event-driven, auto-scaling).          |
| Cold Start?                | Initial delay when Lambda execution environment is created.              |
| How to reduce cold starts? | Provisioned concurrency, smaller packages, compiled languages (Go/Java). |
| Lambda Timeout Limit?      | 15 minutes maximum.                                                      |
| Lambda Memory & CPU?       | Memory allocates proportional CPU (128MB-10GB).                          |
| Lambda Layers?             | Shared libraries, dependencies, or custom runtimes.                      |
| Lambda Triggers?           | S3, SQS, SNS, API Gateway, EventBridge, DynamoDB Streams, Kinesis.       |
| SQS vs SNS?                | SQS = queue (pull, decoupling); SNS = pub/sub (push, fanout).            |
| SQS Standard vs FIFO?      | Standard = at-least-once, high throughput; FIFO = exactly-once, ordered. |

|                           |                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------|
| Dead Letter Queue (DLQ)?  | Stores failed messages for debugging.                                                |
| Visibility Timeout (SQS)? | Time a message is hidden after being received (prevents duplicate processing).       |
| SNS Fanout?               | Publish once, deliver to multiple subscribers (SQS, Lambda, HTTP).                   |
| API Gateway?              | Managed service to create, publish, and secure APIs.                                 |
| API Gateway Throttling?   | Default 10,000 RPS with burst; configurable per stage/method.                        |
| EventBridge?              | Serverless event bus for app integration (successor to CloudWatch Events).           |
| Step Functions?           | Orchestrates workflows with visual state machines (retry, parallel, error handling). |
| Lambda vs Fargate?        | Lambda = stateless, <15min; Fargate = containerized, long-running.                   |

---

## 8. Containers & DevOps

| Question             | Answer                                                            |
|----------------------|-------------------------------------------------------------------|
| ECS vs EKS?          | ECS = AWS-native containers; EKS = managed Kubernetes.            |
| Fargate?             | Serverless compute for ECS/EKS (no EC2 management).               |
| ECS Task vs Service? | Task = running container; Service = maintains desired task count. |
| Task Definition?     | Blueprint defining container image, CPU, memory, IAM role.        |

|                                   |                                                                               |
|-----------------------------------|-------------------------------------------------------------------------------|
| ALB vs NLB vs GWLB?               | ALB = HTTP/S (L7); NLB = TCP/UDP (L4); GWLB = firewall/IDS (L3).              |
| Target Groups?                    | Routes traffic to registered targets (instances, IPs, Lambda).                |
| Health Checks?                    | Determines target availability (HTTP/TCP probes).                             |
| CodePipeline?                     | Automates CI/CD workflows.                                                    |
| CodeBuild?                        | Compiles, tests, and packages code.                                           |
| CodeDeploy?                       | Automates deployments (EC2, Lambda, ECS) with rollback.                       |
| Deployment Strategies?            | Blue/Green, Rolling, Canary, All-at-once.                                     |
| CloudFormation?                   | Infrastructure-as-Code (IaC) using JSON/YAML templates.                       |
| CloudFormation Stack vs StackSet? | Stack = single region; StackSet = multi-region/account.                       |
| CDK vs CloudFormation?            | CDK = code-based (TypeScript/Python); CloudFormation = declarative templates. |
| Terraform vs CloudFormation?      | Terraform = multi-cloud; CloudFormation = AWS-native.                         |
| Systems Manager (SSM)?            | Manages EC2/on-prem servers (patching, configuration, Session Manager).       |

---

## 9. Security & Monitoring

| Question    | Answer                                      |
|-------------|---------------------------------------------|
| CloudWatch? | Monitors metrics, logs, and creates alarms. |

|                                     |                                                                         |
|-------------------------------------|-------------------------------------------------------------------------|
| CloudWatch vs CloudTrail?           | CloudWatch = performance monitoring; CloudTrail = API audit logs.       |
| CloudTrail?                         | Records all API calls for auditing and compliance.                      |
| AWS Config?                         | Tracks resource configuration changes and compliance.                   |
| AWS Inspector?                      | Automated security assessment for EC2, containers, Lambda.              |
| GuardDuty?                          | Threat detection using ML (monitors VPC Flow Logs, DNS, CloudTrail).    |
| Security Hub?                       | Centralizes security findings across AWS services.                      |
| Macie?                              | Discovers and protects sensitive data in S3 (PII, credentials).         |
| KMS (Key Management Service)?       | Manages encryption keys (symmetric/asymmetric).                         |
| KMS vs CloudHSM?                    | KMS = managed multi-tenant; CloudHSM = dedicated single-tenant HSM.     |
| Encryption at Rest vs Transit?      | Rest = storage encryption (KMS); Transit = TLS/SSL in motion.           |
| Secrets Manager vs Parameter Store? | Secrets = auto-rotation; Parameter Store = free config storage.         |
| IAM Access Analyzer?                | Identifies resources accessible from outside your account.              |
| WAF (Web Application Firewall)?     | Protects against SQL injection, XSS, rate-limiting.                     |
| Shield Standard vs Advanced?        | Standard = free DDoS protection; Advanced = enhanced + cost protection. |

|                  |                                                                               |
|------------------|-------------------------------------------------------------------------------|
| Trusted Advisor? | Best practice recommendations (cost, security, performance, fault tolerance). |
|------------------|-------------------------------------------------------------------------------|

## 10. Architecture & Design

| Question                            | Answer                                                                                                    |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------|
| High Availability?                  | Multi-AZ, Load Balancers, Auto Scaling, health checks.                                                    |
| Fault Tolerance?                    | System continues operating despite component failures (self-healing).                                     |
| Disaster Recovery Strategies?       | Backup/Restore < Pilot Light < Warm Standby < Multi-Site (increasing RTO/RPO).                            |
| RTO vs RPO?                         | RTO = recovery time; RPO = data loss tolerance.                                                           |
| Multi-AZ vs Multi-Region?           | AZ = within region HA; Region = geographic DR.                                                            |
| Auto Scaling Group?                 | Adjusts EC2 instance count based on demand (target tracking, step scaling).                               |
| Well-Architected Framework Pillars? | Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, Sustainability. |
| Elastic Beanstalk?                  | PaaS for deploying apps without managing infrastructure.                                                  |
| Decoupling Microservices?           | Use SQS, SNS, EventBridge, API Gateway.                                                                   |
| Caching Strategies?                 | CloudFront (edge), ElastiCache (app), DAX (DynamoDB), API Gateway cache.                                  |
| CDN in AWS?                         | CloudFront for low-latency content delivery.                                                              |

|                            |                                                                        |
|----------------------------|------------------------------------------------------------------------|
| Event-Driven Architecture? | Lambda + S3/SNS/SQS/EventBridge for asynchronous processing.           |
| Saga Pattern?              | Distributed transactions using Step Functions or compensating actions. |
| Strangler Fig Pattern?     | Gradually replace legacy systems by routing traffic to new services.   |

## 11. Advanced / Tricky

| Question                   | Answer                                                                                  |
|----------------------------|-----------------------------------------------------------------------------------------|
| S3 Consistency?            | Strong read-after-write consistency for all operations.                                 |
| Lambda Concurrency Limits? | 1,000 concurrent executions per region (soft limit, can increase).                      |
| Cross-Account Access?      | IAM roles with trust policies or resource-based policies.                               |
| VPC Flow Logs Use Cases?   | Troubleshoot connectivity, detect security threats, audit traffic.                      |
| Throttling in DynamoDB?    | Exceeding provisioned throughput (use on-demand or exponential backoff).                |
| Handling traffic spikes?   | ALB + Auto Scaling + CloudFront + caching.                                              |
| Data migration to AWS?     | Snowball (TB-PB), DataSync (online sync), Transfer Family (SFTP).                       |
| RDS performance issues?    | Slow queries (Performance Insights), read replicas, parameter tuning, upgrade instance. |



|                                   |                                                                     |
|-----------------------------------|---------------------------------------------------------------------|
| Global latency optimization?      | CloudFront, Global Accelerator, Route 53 latency routing.           |
| Lambda vs Step Functions?         | Lambda = single function; Step Functions = orchestrate workflows.   |
| How to debug Lambda failures?     | CloudWatch Logs, X-Ray tracing, DLQs.                               |
| DynamoDB hot partition?           | Use composite partition keys, burst capacity, or on-demand mode.    |
| S3 eventual consistency (legacy)? | Old model (before Dec 2020) had eventual for overwrites—now strong. |
| ECS vs EKS trade-offs?            | ECS = simpler, AWS-native; EKS = Kubernetes portability.            |
| CloudFormation Drift Detection?   | Identifies manual changes to stack resources.                       |

---

## 12. Scenario-Based (Senior-Level)

| Scenario                        | Answer                                               |
|---------------------------------|------------------------------------------------------|
| Sudden traffic spike?           | ALB + Auto Scaling + CloudFront caching.             |
| Large file uploads (GB-TB)?     | S3 Transfer Acceleration + multipart upload.         |
| Real-time analytics?            | Kinesis Data Streams → Lambda → DynamoDB/Redshift.   |
| Process millions of messages?   | SQS + Lambda with batch processing or ECS workers.   |
| Cost optimization for dev/test? | Stop/start schedules, Spot Instances, Savings Plans. |
| 1TB on-prem migration?          | Snowball (offline) or DataSync (online).             |

|                                      |                                                                                              |
|--------------------------------------|----------------------------------------------------------------------------------------------|
| Accidental EC2 deletion?             | Restore from AMI or termination protection.                                                  |
| API rate limiting?                   | API Gateway throttling + usage plans.                                                        |
| Detect unusual IAM activity?         | CloudTrail + GuardDuty + EventBridge alerts.                                                 |
| Multi-account governance?            | AWS Organizations + SCPs + CloudFormation StackSets.                                         |
| Lambda running out of memory?        | Increase memory (also increases CPU), optimize code, use Lambda layers.                      |
| RDS running out of storage?          | Enable storage auto-scaling or upgrade instance.                                             |
| S3 costs too high?                   | Lifecycle policies, Intelligent-Tiering, delete incomplete uploads, analyze S3 Storage Lens. |
| Application slow during peak hours?  | Add read replicas, ElastiCache, CloudFront, optimize queries.                                |
| Need zero downtime deployment?       | Blue/Green with ALB target groups or Canary with CodeDeploy.                                 |
| Compliance requires audit logs?      | CloudTrail + S3 bucket with Object Lock.                                                     |
| Database connection pool exhaustion? | Use RDS Proxy for connection pooling.                                                        |
| Lambda cold starts affecting UX?     | Provisioned concurrency or use Fargate for long-running.                                     |
| Need to process data across regions? | S3 CRR + Lambda + EventBridge or Step Functions.                                             |

---

### 13. Cost Optimization (Critical for Senior Role)

| Question                             | Answer                                                                                          |
|--------------------------------------|-------------------------------------------------------------------------------------------------|
| Top cost optimization strategies?    | Reserved/Spot/Savings Plans, right-sizing, S3 lifecycle, auto-scaling, delete unused resources. |
| Cost Allocation Tags?                | Track costs by project, team, environment using tags.                                           |
| AWS Cost Explorer?                   | Visualizes spending patterns and forecasts.                                                     |
| AWS Budgets?                         | Sets spending alerts and thresholds.                                                            |
| Savings Plans vs Reserved Instances? | Savings Plans = flexible (compute/EC2/Fargate); Reserved = specific instance type.              |
| Spot Instances best practices?       | Use for stateless, fault-tolerant workloads with Spot Fleet or ASG.                             |
| S3 Storage Lens?                     | Organization-wide visibility into S3 usage and cost optimization.                               |
| Compute Optimizer?                   | Recommends optimal EC2/Lambda/EBS configurations.                                               |
| Right-sizing?                        | Match instance type to workload needs (CPU/memory/network).                                     |

---

#### 14. Troubleshooting (Must-Know for Senior Role)

| Problem                   | Solution                                                                            |
|---------------------------|-------------------------------------------------------------------------------------|
| EC2 instance won't start? | Check instance limits, subnet capacity, AMI availability, EBS snapshot status.      |
| Cannot connect to RDS?    | Verify security groups, NACLs, route tables, DB subnet group, public accessibility. |
| S3 403 Access Denied?     | Check bucket policy, IAM policy, ACLs, Block Public Access settings.                |

|                                      |                                                                            |
|--------------------------------------|----------------------------------------------------------------------------|
| Lambda timing out?                   | Increase timeout, optimize code, check VPC NAT Gateway for outbound calls. |
| High DynamoDB read/write throttling? | Enable auto-scaling, use on-demand mode, or refactor partition key.        |
| ALB returning 503 errors?            | Check target health, security groups, ASG capacity, connection draining.   |
| VPC peering not working?             | Verify route tables, no CIDR overlap, security groups allow traffic.       |
| CloudFormation stack stuck?          | Check resource dependencies, IAM permissions, service limits.              |

-----

React/Node

## NodeJS

- Node.js is an open-source, cross-platform JavaScript runtime environment that executes code outside of a web browser. It is built on V8, the same JavaScript engine within Chrome, and optimized for high performance.
  - Node.js employs event-driven architecture and non-blocking I/O for efficiency. While it operates off a single main thread, it can harness the full power of multi-core systems by launching child threads for specific tasks, such as file compression or image processing. To manage these child threads, Node.js uses a combination of:
    - A thread pool, powered by the libuv library.
    - Worker threads for dedicated, offloaded computation.
  - Node.js Event Loop: When a task in Node.js is designated to operate on a child thread, the main event loop hands it over to the thread pool. This setup allows Node.js to stay responsive to incoming requests, benefiting from asynchronous I/O. The main event loop regains control once the task on the child thread is completed, and results are ready.
  - <https://github.com/Devinterview-io/node-interview-questions>
- 

## ReactJS

- Component-based architecture: UI is broken into reusable, self-contained components
  - Declarative: You describe what the UI should look like, React handles the DOM updates
  - Unidirectional data flow: Data flows down from parent to child components via props
  - Virtual DOM: React maintains a lightweight copy of the actual DOM for efficient updates.
    - It is a programming concept where a virtual representation of the UI is kept in memory and synced with the real DOM through a process called reconciliation. When state changes:

- React creates a new Virtual DOM tree
- Compares it with the previous tree (diffing algorithm)
- Calculates the minimum number of changes needed
- Updates only those parts in the real DOM (batching updates)
- This makes updates more efficient than directly manipulating the DOM.
- JSX is a syntax extension for JavaScript that looks similar to HTML. It allows you to write markup inside JavaScript: Eg: JSX is a syntax extension for JavaScript that looks similar to HTML. It allows you to write markup inside JavaScript.
- A React Element is a plain JavaScript object that describes what you want to see on the UI. It represents a DOM node or a component at a specific point in time. Elements are immutable: once created, you cannot change their properties. Instead, you create new elements to reflect updates.
- A Component is a function or class that returns an element (or a tree of elements) to describe part of the UI. Components can accept inputs (called props) and manage their own state (in case of class or function components with hooks). Components allow you to split the UI into independent, reusable pieces, each isolated and composable. You can define a component using a function or a class
- When several components need to share the same changing data then it is recommended to lift the shared state up to their closest common ancestor.
- State of a component is an object that holds some information that may change over the lifetime of the component. The important point is whenever the state object changes, the component re-renders. It is always recommended to make our state as simple as possible and minimize the number of stateful components.
- Props are inputs to components. Eg: `<Element reactProp={"1"} />`
- <https://github.com/sudheerj/reactjs-interview-questions>

---

# Languages



## Languages

- A language is:
  - Category 1: Static vs Dynamic:
    - Statically typed if the type of a variable is known at compile time, eg: `int a = 5`. Means that you must specify what type each variable is. Eg: C, C++, Java, Rust, Go, Scala. Faster.
    - Dynamically typed if the type is known during run-time. Eg: Perl, Ruby, Python, PHP, JavaScript, Erlang. Slower.
  - Category 2: Compiled vs Interpreted:
    - Your CPU runs instructions in 0s and 1s. So every instruction defined via a language must eventually become machine code. The difference is WHEN and HOW that translation happens.
    - Compiled languages Flow: Source code → Compiler → Machine code executable → Run the executable. Compiler: Reads the entire program; Does static analysis; Optimizes globally. Interpreted languages Flow: Source code → Interpreter → Execute line by line.
    - Interpreter: Reads one statement, Executes it immediately, Moves to the next. Compiled languages came first. Note that today almost all modern “interpreted” languages compile internally.
    - What did interpreted languages solve?: Early computers were painful; Compilation took minutes to hours hence debugging meant: Write code, Compile, Run, Crash, Repeat. Interpreters solved this: Immediate feedback, Interactive programming (REPL), Dynamic behavior.
- Python:
  - Python code goes through both compilation and interpretation.
    - Bytecode Compilation: High-level Python code is transformed into low-level bytecode by the Python interpreter with the help of a compiler. Bytecode is a set of instructions that Python's virtual machine (PVM) can understand and execute.

- On-the-fly Interpretation: The PVM reads and executes bytecode instructions in a step-by-step manner.
- While some programming languages compile directly to machine code, Python compiles to bytecode. This bytecode is then executed by the Python virtual machine. This extra step of bytecode execution can make Python slower in certain use-cases when compared to languages that compile directly to machine code. The advantage, however, is that bytecode is platform-independent. A Python program can be run on any machine with a compatible PVM, ensuring cross-platform support.
- Source Code to Bytecode steps: Lexical Analysis & Token generation, Syntax Parsing, Semantic Analysis, Bytecode Generation
- While Python typically uses a combination of interpretation and compilation, JIT (Just-In-Time) boosts efficiency by selectively compiling parts of the program that are frequently used or could benefit from optimization. JIT compiles sections of the program to machine code on-the-fly. This direct machine code generation for frequently executed parts can significantly speed up those segments, blurring the line between traditional interpreters and compilers.
- PEP 8 (Python Enhancement Proposal 8) is a style guide for Python code that promotes code consistency, readability, and maintainability.
- Memory Allocation
  - The "heap" is the pool of memory for storing objects. The Python memory manager allocates and deallocates this space as needed.
  - In latest Python versions, the obmalloc system is responsible for small object allocations. This system preallocates small and medium-sized memory blocks to manage frequently created small objects.

- The allocator abstracts the system-level memory management, employing memory management libraries like Glibc to interact with the operating system.
- Larger blocks of memory are primarily obtained directly from the operating system.
- Stack and Heap separation is joined by "Pool Allocator" for internal use.
- Garbage Collection
  - Python employs a method called reference counting along with a cycle-detecting garbage collector.
  - Reference Counting
    - Every object has a reference count. When an object's count drops to zero, it is immediately deallocated.
    - This mechanism is swift, often releasing objects instantly without the need for garbage collection.
    - However, it can be insufficient in handling circular references.
  - Cycle-Detecting Garbage Collector
    - Python has a separate garbage collector that periodically identifies and deals with circular references.
    - This is, however, a more time-consuming process and is invoked less frequently than reference counting.
- Python handles memory management quite differently from languages like C or C++:
  - In Python, the developer isn't directly responsible for memory allocations or deallocations, reducing the likelihood of memory-related bugs.
  - The memory manager in Python is what's known as a "general-purpose memory manager" that can be slower than the dedicated memory managers of C or C++ in certain contexts.
  - Python, especially due to the existence of a garbage collector, might have memory overhead compared to C or C++ where manual memory management often results in minimal

overhead is one of the factors that might contribute to Python's sometimes slower performance.

- The level of memory efficiency isn't as high as that of C or C++. This is because Python is designed to be convenient and easy to use, often at the expense of some performance optimization.

- Datatypes: (Apart from int, float, bool, etc).

| Type              | Mutable | Ordered         | Allows Duplicates | Example          |
|-------------------|---------|-----------------|-------------------|------------------|
| List              | ✓ Yes   | ✓ Yes           | ✓ Yes             | [1, 2, 3]        |
| Tuple             | ✗ No    | ✓ Yes           | ✓ Yes             | (1, 2, 3)        |
| Set               | ✓ Yes   | ✗ No            | ✗ No              | {1, 2, 3}        |
| Dictionary (dict) | ✓ Yes   | ✓ (Python 3.7+) | ✗ (unique keys)   | {"a": 1, "b": 2} |

- A frozenset is an immutable version of a set.
- Additional Data Structures (from `collections` module): deque, Counter, defaultdict, OrderedDict, namedtuple
- List comprehension vs Generator: List builds memory; generator is lazy: `[x*x for x in range(3)]` ; `(x*x for x in range(3))`
- `@staticmethod` no self; `@classmethod` gets cls.
- Inheritance & `super()`: MRO defines lookup order; `super()` calls parent.
  - MRO = the order in which Python looks for a method or attribute when it's called on an object. MRO exists to resolve ambiguity in multiple inheritance. Prevents “diamond problem” (method being inherited twice). Python uses the C3 linearization to compute MRO. Rule summary:
    - Child appears before its parents.
    - Maintain the order of base classes as declared.
    - Each parent appears only once.

```
class A:
```

```

 def show(self): print("A")
class B(A):
 def show(self): print("B")
class C(A):
 def show(self): print("C")
class D(B, C):
 pass

d = D()
d.show()
print(D.__mro__)

Output
B
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>,
<class '__main__.A'>, <class 'object'>)
Python searched for show() in this order: D → B → C → A →
object
ClassName.__mro__

```

- Exception handling:

```

try:
 risky_operation()
except IndexError: # Handle specific exception types first.
 handle_index_error()
except Exception as e: # More general exception must come last.
 handle_generic_error()
finally:
 cleanup()
raise Exception("Some generic error occurred")

```

- The with keyword provides a more efficient and clean way to handle resources, like files, ensuring their proper closure when operations are complete or in case of any exceptions. The resource should implement a context manager, typically by having `__enter__` and `__exit__` methods.

```

with open("example.txt", "r") as file:
 data = file.read()

```

```
File is automatically closed when the block is exited.
```

- There are times when not raising an exception is appropriate. You can use pass or continue in an exception block when you want to essentially ignore an exception and proceed with the rest of your code.
- zip() Function: This creates a dictionary by zipping two lists, where the first list corresponds to the keys, and the second to the values.

```
keys = ["a", "b", "c"]
values = [1, 2, 3]
zipped = zip(keys, values)
dict_from_zip = dict(zipped) # Result: {"a": 1, "b": 2, "c": 3}
```

- Both the == and is operators in Python are used for comparison, but they function differently.
  - The == operator checks for value equality.
  - The is operator, on the other hand, validates object identity,

```
Example 1: same values but different objects
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b) # True -- same values
print(a is b) # False -- different objects (different memory locations)
Example 2: same object reference
c = a
print(a == c) # True -- same values
print(a is c) # True -- both refer to the same object
```

- \*args collects a variable number of positional arguments into a tuple, while \*\*kwargs does the same for keyword arguments into a dictionary.

```
Eg:
Ex1: args
def sum_all(*args):
 result = 0
 for num in args:
 result += num
```

```
 return result
print(sum_all(1, 2, 3, 4)) # Output: 10
```

Ex2: kwargs

```
def print_values(**kwargs):
 for key, value in kwargs.items():
 print(f"{key}: {value}")
Keyword arguments are captured as a dictionary
print_values(name="John", age=30, city="New York")
Output:
name: John
age: 30
city: New York
```

- In Python, a decorator is a design pattern and a feature that allows you to modify functions and methods dynamically. Decorators wrap a target function, allowing you to execute custom code before and after that function. They are typically higher-order functions that take a function as an argument and return a new function. This paradigm of "functions that modify functions" is often referred to as metaprogramming. Use cases: logging, caching, counting/profiling, etc.

```
def greet_decorator(func):
 def wrapper():
 print("Before fun")
 func()
 print("After fun")
 return wrapper
```

```
@greet_decorator
def say_hello():
 print("Hii")
```

```
say_hello()
Output:
Before fun
Hii
```

## # After fun

- Create module in python adding `__init__.py` file.
- `self` represents the current instance of a class which is used to access the properties and methods of a class.
- Copy in Py:
  - Shallow Copy: Creates a new container object (like a new list or dict). But elements inside are references to the same inner objects. So changes to inner (nested) objects affect both copies.
  - Deep Copy: Creates a completely independent clone — both outer and inner objects are copied recursively. So modifying one does not affect the other.
- GIL = Global Interpreter Lock
  - A mutex (mutual exclusion lock) that allows only one thread to execute Python bytecode at a time within one process — even on multi-core CPUs.
  - CPython (the main Python implementation) manages memory dynamically.
    - Python objects are stored in heap memory.
    - CPython uses reference counting to track when objects are no longer needed.
    - If multiple threads modify the reference count of the same object at once, that can cause race conditions and memory corruption.
    - To prevent that, the GIL ensures that only one thread manipulates Python objects at a time.
  - If we removed GIL, CPython would need to:
    - Add fine-grained locks around every object.
    - That would make all operations slower, even for single-threaded programs.
    - So, GIL is a performance tradeoff — simplicity + safety at the cost of true parallelism.



- Languages like Java, C++, Go, or Rust don't need a GIL because:
  - They have thread-safe memory management (e.g., GC with atomic operations).
  - Or they don't use reference counting (e.g., Java's mark-and-sweep GC).
  - Or they use compile-time safety guarantees (like Rust's borrow checker).
  - That's why they can run multiple threads truly in parallel on multiple cores.
- We can't say, Python is a dynamically-typed lang, so it's handled like this, but rather about CPython's memory model and implementation choice (reference counting).

Some ex., related to Cores/Process/Threads:

Lets say you have 4 cores.

Case 1: One Process, 4 Threads (under GIL)

Threads run one at a time, taking turns (time slicing). Even on 4 cores, only one executes Python code. However, I/O operations (file, network, DB) release the GIL, so I/O-bound tasks benefit.

✓ Good for: I/O-bound tasks (web requests, API calls).

✗ Bad for: CPU-bound tasks (math, computation).

-----  
Case 2: 4 Processes (Each Single Thread)

Each process has its own interpreter and GIL. So, true parallelism across cores. Memory is separate, so need inter-process communication (IPC) to share data.

✓ Good for: CPU-bound tasks (ML, computation).

✗ Consumes more memory; communication overhead.

- Java:
  - Java is a high-level, object-oriented programming language designed to be platform-independent. Its core philosophy is encapsulated in the concept of "Write Once, Run Anywhere" (WORA). This is achieved through several key components:

- Java Virtual Machine (JVM): The JVM acts as an abstraction layer between Java code and the underlying hardware or operating system. It interprets and executes Java bytecode, ensuring consistent behavior across different platforms.
- Bytecode: Java source code is compiled into platform-independent bytecode, which can be executed by any JVM, regardless of the underlying system architecture.
- Standard Library: Java provides a comprehensive standard library that offers cross-platform capabilities for common tasks like file handling and networking.
- Java Achieves WORA via:
  - Platform Independence: Java bytecode can run on any device with a compatible JVM, from smartphones to supercomputers.
  - JVM Customization: Each operating system has a tailored JVM version, ensuring WORA functionality across diverse environments.
  - Garbage Collection: Automatic memory management reduces the risk of memory leaks and simplifies development across platforms.
  - Security: Java's design excludes direct memory manipulation through pointers, enhancing security across different systems.
- Static members belong to the class rather than instances, allowing for utility functions and shared data.
- (OOPS related stuff - Java - covered in LLD section of this doc)
- JVM (Java Virtual Machine) is an abstract computing machine that provides a runtime environment in which Java bytecode can be executed. JRE (Java Runtime Environment) is the minimum environment required to execute a Java application. It consists of the JVM, core libraries, and other supporting files. JDK (Java Development Kit) is a superset of the JRE, providing everything needed for Java application development.

- ClassLoader is a crucial component in Java's runtime environment, responsible for loading class files into memory. It has different types.
- Classpath vs Path: Classpath is a parameter that tells the Java Virtual Machine (JVM) where to find compiled Java classes (.class files) and packages during runtime. Path is a system environment variable that specifies directories where executable programs are located. It's used by the operating system to find executables when you run commands in the terminal or command prompt.
- Wrapper classes in Java allow you to work with primitive data types as objects. They are particularly useful when working with generic collections or when using features that require objects, such as Java Bean properties. Eg: Primitive: boolean; Wrapper Class: Boolean; Conversion Methods: .valueOf(), .parseBoolean(), .booleanValue(); Primitive Example: true; Wrapper Example: Boolean.TRUE
- Garbage Collection:
  - In Java, the Virtual Machine (JVM) manages memory through automatic garbage collection (GC). This process identifies and reclaims objects that are no longer in use. Objects are considered "alive" if they are reachable from the root object, which can be a Thread, Stack, or Static reference. Unreachable objects are eligible for garbage collection. There are different reference types that play a role in determining an object's reachability and GC eligibility.
    - Strong Reference: Default reference — object not eligible for GC until explicitly set to null. No default collection until no references exist. Eg: MyClass obj = new MyClass();
    - Weak Reference: Object eligible for GC once no strong refs exist; useful for caches. Collection eagerly on next GC. Eg: WeakReference<MyClass> ref = new WeakReference<>(obj);
    - Soft Reference: Object kept until JVM needs memory; used for memory-sensitive caches. Collection only in

low-memory situations. Eg: `SoftReference<MyClass> ref = new SoftReference<>(obj);`

- Phantom Reference: Doesn't prevent GC; object enqueued in `ReferenceQueue` after finalization. Used for cleanup tracking. Collection happens after `finalize()` before actual memory reclaim. Eg: `new PhantomReference<>(obj, refQueue);`
- GC Algos: Java uses different garbage collection algorithms, like Mark and Sweep algorithm. In Mark phase it detects all the unreachable objects and Sweep phase it reclaim the heap space used by the garbage objects and make the space available again to the program. Types:
  - Serial GC: Single-threaded collector. Suitable for small applications with limited memory
  - Parallel GC: Uses multiple threads for minor garbage collection. Default for most applications.
  - Concurrent Mark Sweep (CMS) GC: Minimizes pauses by doing most of its work concurrently with the application threads.
  - G1 (Garbage First) GC: Designed for applications with large amounts of memory. Divides the heap into regions for more efficient collection.
- Java divides heap into Young, Old, and (pre-8) PermGen to optimize GC — most short-lived objects are collected quickly, reducing pause times. GC can pause application threads (called Stop-The-World pauses), because objects may reference each other in complex ways, the JVM must ensure no thread is modifying memory while GC identifies and removes unused objects — otherwise, memory corruption could occur.
  - Young Generation: Newly created objects start here. Most objects die young → frequent Minor GCs. Divided into:
    - Eden Space – new objects allocated here.

- Survivor Spaces (S0, S1) – objects that survive GC move between these.
  - If an object survives multiple Minor GCs → promoted to Old Gen.
- Old (Tenured) Generation: Holds long-lived objects that survived multiple GCs. Collected less often via Major/Full GC. Slower but happens less frequently.
- Permanent Generation (PermGen): (Java 7 and earlier) Stores metadata, class info, static variables, method definitions, etc. Fixed size – can throw `OutOfMemoryError: PermGen space`. \
  - Replaced in Java 8+ by Metaspace, which grows dynamically in native memory.
- The `this` keyword in Java is a reference to the current instance of a class.

```
public class Person {
 private String name;

 public Person(String name) {
 this.name = name; // 'this.name' refers to the instance
 variable
 }
}
```

- In Java, the `final` keyword offers restrictions and benefits. It primarily maintains the immutability of different entities.
- The Just-In-Time (JIT) compiler is a component of the runtime environment that improves the performance of Java applications by compiling bytecodes to native machine code at run time.
- Class loading:
  - Static Class Loading: Creating objects and instance using `new` keyword is known as static class loading. The retrieval of class definition and instantiation of the object is done at compile time.

- Dynamic Class Loading: Loading classes use `Class.forName()` method. Dynamic class loading is done when the name of the class is not known at compile time.
- Transient: The transient modifier tells the Java object serialization subsystem to exclude the field when serializing an instance of the class. When the object is then deserialized, the field will be initialized to the default value; i.e. null for a reference type, and zero or false for a primitive type.
  - Serialization is a mechanism of converting the state of an object into a byte stream. Deserialization is the reverse process where the byte stream is used to recreate the actual Java object in memory.

```
public transient int limit = 55; // will not persist
public int b; // will persist
```

- Volatile: The volatile modifier tells the JVM that writes to the field should always be synchronously flushed to memory, and that reads of the field should always read from memory. This means that fields marked as volatile can be safely accessed and updated in a multi-thread application without using native or standard library-based synchronization.

```
public class MyRunnable implements Runnable {
 private volatile boolean active;
 public void run() {
 active = true;
 while (active) {
 }
 }
 public void stop() {
 active = false;
 }
}
```

- Types of Memory areas allocated by the JVM:

- 1. Classloader: Classloader is a subsystem of JVM that is used to load class files.
- 2. Class(Method) Area: Class(Method) Area stores per-class structures such as the runtime constant pool, field and method data, the code for methods.
- 3. Heap: It is the runtime data area in which objects are allocated.
- 4. Stack: Java Stack stores frames. It holds local variables and partial results, and plays a part in method invocation and return. Each thread has a private JVM stack, created at the same time as thread.
- 5. Program Counter Register: PC (program counter) register. It contains the address of the Java virtual machine instruction currently being executed.
- 6. Native Method Stack: It contains all the native methods used in the application.
- Java constructor chaining is a method of calling one constructor with the help of another while considering the present object. It can be done in 2 ways –
  - Within same class: It can be done using this() keyword for constructors in the same class.
  - From base class: By using super() keyword to call a constructor from the base class.

```
class Temp
{
 // default constructor 1
 // default constructor will call another constructor
 // using this keyword from same class
 Temp() {
 // calls constructor 2
 this(5);
 System.out.println("The Default constructor");
 }

 // parameterized constructor 2
```

```

Temp(int x) {
 // calls constructor 3
 this(10, 20);
 System.out.println(x);
}

// parameterized constructor 3
Temp(int x, int y) {
 System.out.println(10 + 20);
}

public static void main(String args[]) {
 // invokes default constructor first
 new Temp();
}
}

```

- Ruby/Rails:
  - A gem is just ruby code. It is installed on a machine and it's available for all ruby applications running on that machine. Rails, rake, json, rspec – are all examples of gems. Plugin is also ruby code but it is installed in the application folder and only available for that specific application.
  - Active Record: Simplifies data handling from databases, treating tables as classes and rows as objects.
  - Action View: Manages user interface elements like forms and templates.
  - Action Controller: Handles requests, processes data, and manages flow between models and views.

# MVC Code in Ruby:

```

Controller
class ArticlesController < ApplicationController
 def index
 @articles = Article.all
 end
end

```



```

 end
 end

 # Model
 class Article < ApplicationRecord
 end

 # View (ERB)
 <% @articles.each do |article| %>
 <%= article.title %>
 <% end %>

```

- The Gemfile in a Ruby on Rails application acts as a manifest of all required gems for the project. It serves as the foundation for the Bundler tool, which automates the process of managing and versioning gem dependencies.
  - Convention over Configuration is a foundational principle in Ruby on Rails.
  - Routes in Ruby: Eg: `match '/articles/:id', to: 'articles#show', via: 'get'`
- JavaScript:
    - `var/let/const`:

| Behavior       | <b>var</b>         | <b>let</b> | <b>const</b> |
|----------------|--------------------|------------|--------------|
| Scope          | Function or Global | Block      | Block        |
| Initialization | Optional           | Optional   | Required     |
| Redeclaration  | Yes                | No         | No           |

|                              |           |                |                |
|------------------------------|-----------|----------------|----------------|
| Reassignment                 | Yes       | Yes            | No             |
| Accessing before declaration | undefined | ReferenceError | ReferenceError |

- == is the abstract equality operator while === is the strict equality operator.
- Event Loop in JavaScript enables non-blocking asynchronous execution in a single-threaded environment. How It Works
  - Call Stack → Executes synchronous code line by line.
  - Web APIs / Node APIs → Handle async tasks (like setTimeout, fetch, I/O).
  - Queues → When async tasks finish, callbacks go to:
    - Microtask Queue → Promises (then, catch, finally), queueMicrotask()
    - Macrotask Queue → setTimeout, setInterval, fetch, DOM events
  - Event Loop → Keeps checking:
    - If Call Stack is empty → run all Microtasks first
    - Then run one Macrotask, and repeat
- Event delegation is a technique in JavaScript where a single event listener is attached to a parent element instead of attaching event listeners to multiple child elements. When an event occurs on a child element, the event bubbles up the DOM tree, and the parent element's event listener handles the event based on the target element.
- cookie, sessionStorage and localStorage in browsers: All of the following are mechanisms of storing data on the client, the user's browser in this case. localStorage and sessionStorage both implement the Web Storage API interface.
  - Cookies: Suitable for server-client communication, small storage capacity, can be persistent or session-based, domain-specific. Sent to the server on every request.

- localStorage: Suitable for long-term storage, data persists even after the browser is closed, accessible across all tabs and windows of the same origin, highest storage capacity among the three.
- sessionStorage: Suitable for temporary data within a single page session, data is cleared when the tab or window is closed, has a higher storage capacity compared to cookies.
- Garbage collection in JavaScript is an automatic memory management mechanism that reclaims memory occupied by objects and variables that are no longer in use by the program. The two most common algorithms are mark-and-sweep and generational garbage collection.
  - Mark-and-sweep: Algorithm effectively identifies and removes objects that have become unreachable, freeing up memory for new allocations.
  - Generational garbage collection: Leveraged by modern JavaScript engines, objects are divided into different generations based on their age and usage patterns. Frequently accessed objects are moved to younger generations, while less frequently used objects are promoted to older generations. This optimization reduces the overhead of garbage collection by focusing on the younger generations, where most objects are short-lived.
- The splice() method in JavaScript is used to add, remove, or replace elements within an array. Modifies original array.
- Promises are used to handle asynchronous operations, especially in languages like JavaScript, which often work with non-blocking operations such as network requests, file I/O, and timers. When an operation is asynchronous, it doesn't immediately return a result; instead, it works in the background and provides the result later. Handling this in a clean, organized way can be difficult without a structured approach.
- Prototypal inheritance = Objects inherit from other objects.
- A callback function is a function passed into another function as an argument - higher-order function. The callbacks are needed

because javascript is an event driven language. That means instead of waiting for a response, javascript will keep executing while listening for other events.

```
function firstFunction() {
 // Simulate a code delay
 setTimeout(function () {
 console.log("First function called");
 }, 1000);
}
function secondFunction() {
 console.log("Second function called");
}
firstFunction();
secondFunction();
// Output:
// Second function called
// First function called
```

## As observed from the output, javascript didn't wait for the response of the first function and the remaining code block got executed. So callbacks are used in a way to make sure that certain code doesn't execute until the other code finishes execution.

#####

## Function as argument eg:

```
function greet(name) {
 return "Hello, " + name;
}
function processUserInput(callback) {
 const name = "Alice";
 console.log(callback(name));
}

processUserInput(greet);
```

- GoLang:
  - I have written a detailed article in my [personal blog](#).

- Go (Golang) is a statically typed, compiled programming language designed at Google for simplicity, concurrency, scalability, and high-performance backend systems.
- Packages, Imports, and Project Structure: Every Go file belongs to a package. Packages provide modularity and namespace separation. The main package represents the executable entry point. Imports are explicit, and unused imports fail compilation to prevent dead code accumulation. Go modules (go.mod) manage dependencies and versioning with deterministic builds. Production projects commonly separate handlers/controllers, services/business logic, repositories/data layer, models, configs, middleware, and shared/internal packages. Go communities generally prefer relatively simple and flatter package structures compared to deeply nested enterprise Java layouts.
- Variables, Constants, and Types:
  - var defines mutable variables with automatic zero initialization. := is shorthand declaration syntax used inside functions.
  - const defines immutable compile-time constants.
  - type defines structs, interfaces, aliases, and custom primitive types.
  - Go avoids classical inheritance; composition and interfaces are preferred.
  - Struct embedding enables composition and inheritance-like behavior without deep class hierarchies.
- Functions and Methods:
  - Functions are first-class citizens and can be passed, returned, and stored. Go encourages small focused functions. Multiple return values are heavily used, especially for error handling: `func getUser() (User, error)`
  - Methods are functions attached to structs.
  - Value receivers copy structs; pointer receivers avoid copies and allow mutation.
  - Pointer receivers are preferred for large structs or mutable state.

- Interfaces: Go interfaces are satisfied implicitly; no explicit implements keyword exists. A type implements an interface automatically by defining required methods. Interfaces promote loose coupling, mocking, testing, and composition-oriented design. Small focused interfaces are preferred. Common interfaces: io.Reader, io.Writer, http.Handler. Common Go philosophy: “accept interfaces, return structs.”
- Control Flow:
  - Go intentionally keeps control flow minimalistic. if supports inline initialization: `if err := process(); err != nil { return err }`.
  - Go only has one looping construct: `for`.
  - `switch` statements automatically break unless explicit `fallthrough` is used.
  - `select` is critical for concurrent programming and waits on multiple channel operations simultaneously.
- Error Handling:
  - Go avoids exceptions for normal application errors. Errors are explicit return values:

```
result, err := do()
if err != nil { return err }
```

- Errors are expected states; panics are exceptional states. Modern error handling commonly uses: `fmt.Errorf`, `errors.Is`, `errors.As`. These support wrapping, propagation, and classification of errors.
- `panic`, `defer`, `recover`:
  - `panic` triggers stack unwinding and abnormal control flow within the current goroutine. Used for unrecoverable programmer mistakes, corrupted state, impossible invariants, and startup failures.
  - `panic` should NOT replace normal error handling.
  - `defer` delays execution until function return: `defer file.Close()`. Deferred calls execute in LIFO order. Common `defer` use cases: file closing, mutex unlocking, DB transaction rollback,

tracing/metrics cleanup, recover catches panics inside deferred functions and prevents process crashes.

- Goroutines:
  - Goroutines are lightweight concurrent execution units managed by the Go runtime. Eg: go process()
  - Goroutines are much cheaper than OS threads because of: small initial stacks (~2KB), dynamically growing stacks, user-space scheduling, cheaper context switching.
  - Production systems may run hundreds of thousands of goroutines simultaneously.
  - Goroutines are not directly mapped 1:1 to OS threads.
- M:N Scheduler and GMP Model:
  - Go uses an M:N scheduler where many goroutines are multiplexed onto fewer OS threads.
  - Core components:
    - $G \rightarrow$  Goroutine
    - $M \rightarrow$  Machine (OS thread)
    - $P \rightarrow$  Processor (logical scheduler)
  - Goroutines are placed into runnable queues. Processors schedule runnable goroutines onto worker threads.
  - Scheduler responsibilities: work stealing, asynchronous preemption, syscall handling, runtime-managed scheduling
  - Blocking syscalls may temporarily tie up OS threads, after which the scheduler remaps goroutines efficiently.
  - The runtime integrates scheduling with a network poller (epoll/kqueue/IOCP), enabling massive concurrent network connections without thread-per-request architectures.
- GOMAXPROCS:
  - Controls how many OS threads can execute Go code simultaneously. Defaults to the number of CPU cores.
  - Important distinction: goroutines provide concurrency; GOMAXPROCS controls parallelism.
  - A system may have 1 million goroutines but only 8 actively executing on 8 CPU cores.

○

- Context Switching:
  - OS thread context switching is expensive because the kernel must save registers, switch memory contexts, and transition kernel/user mode.
  - Goroutine switching is cheaper because scheduling mostly occurs in user space and stacks are smaller.
  - Go runtime significantly reduces concurrency overhead compared to thread-per-request architectures.
- Channels:
  - Channels are Go's core communication primitive implementing CSP (Communicating Sequential Processes). Go philosophy: "Do not communicate by sharing memory; share memory by communicating."
  - Channels provide synchronization, communication, and coordination between goroutines.
  - Unbuffered channels synchronize sender and receiver directly.
  - Buffered channels temporarily decouple producers and consumers using internal queues. Uses: worker pools, batching, rate limiting, event queues. Improper usage may cause: deadlocks, goroutine leaks, blocked pipelines. Only senders should close channels.
- select Statement:
  - select waits on multiple channel operations simultaneously.
  - Critical for: cancellation, timeouts, fan-in/fan-out systems, graceful shutdown, async orchestration.
  - default makes select non-blocking.
  - select + context.Context form the backbone of cancellation-aware systems.
- WaitGroup:
  - sync.WaitGroup coordinates goroutine completion.
  - Core methods: Add(), Done(), Wait().
  - Common use cases: parallel API calls, worker synchronization, graceful shutdown coordination



- WaitGroup only waits; it does not propagate errors or cancellations.
- errgroup + context are commonly used for advanced coordination.
- Mutexes and Synchronization:
  - sync.Mutex provides exclusive locking.
  - sync.RWMutex supports multiple readers and one writer. RWMutex helps in read-heavy workloads but increases complexity and contention overhead.
  - Mutexes protect shared memory and establish happens-before guarantees.
- Go Memory Model:
  - Defines visibility guarantees between goroutines.
  - Without synchronization, compiler/runtime/CPU reordering may make writes invisible across goroutines.
  - Synchronization primitives like mutexes, channels, atomics, and coordination mechanisms like WaitGroups establish ordering guarantees.
  - Race conditions occur when multiple goroutines access shared memory concurrently and at least one operation is a write.
- Race Detector: Built-in race detector: Eg: go test -race.
- Slices:
  - Slices are lightweight descriptors over arrays.
  - Internally consist of: pointer to backing array, length, capacity
  - Appending beyond capacity allocates a new backing array and copies elements.
  - Passing slices copies only the slice descriptor, not underlying data.
  - Multiple slices may share the same backing array.
- Maps:
  - Go maps are runtime-managed hash tables.
  - Maps automatically resize and rehash as entries grow.
  - Maps are NOT safe for concurrent writes.
  - Concurrent map writes may cause runtime panics.

- Synchronization options:
    - mutexes
    - sync.Map
    - Sharding techniques
- new vs make:
  - New: allocates memory, zero initializes, returns pointer.
  - Make: initializes runtime-backed structures, used only for slices, maps, and channels.
  - Important distinction: new(map) → pointer to nil map; make(map) → usable initialized map
- Escape Analysis:
  - The compiler decides whether variables are stack allocated or heap allocated. Variables escaping function scope are allocated on the heap. Heap allocations increase GC pressure. Escape analysis significantly impacts performance. Tooling: go build -gcflags="-m"
- Garbage Collection:
  - Go uses a concurrent tri-color mark-and-sweep garbage collector optimized for low pause times.
  - Goals: low latency, predictable pauses, minimal tuning
  - GC is mostly concurrent with very small stop-the-world phases.
  - Major phases: mark reachable objects, sweep unreachable objects, reclaim memory
  - Go prioritizes latency and operational simplicity over maximum throughput.
- Go GC vs Java GC vs Python GC:
  - Go GC:
    - concurrent tri-color mark-and-sweep
    - low latency
    - predictable pauses
    - minimal tuning
    - optimized for scalable backend systems
  - Java GC:

- advanced generational collectors (G1GC, ZGC, Shenandoah)
  - heap generations (young/old)
  - highly optimized for throughput and large heaps
  - significantly more tuning complexity
- Python GC:
  - primarily reference counting
  - cyclic GC for circular references
  - GIL limits multicore concurrency
  - higher runtime overhead compared to Go
- Overall:
  - Go prioritizes simplicity and scalable concurrency.
  - Java prioritizes throughput and GC flexibility.
  - Python prioritizes developer productivity.
- sync.Pool:
  - Used for temporary object reuse to reduce allocations and GC pressure.
  - Common use cases: buffers, serialization objects, reusable request objects, Objects inside sync.Pool may disappear anytime during GC. sync.Pool should not be treated as a permanent cache.
- unsafe Package: unsafe bypasses Go's type safety and memory guarantees. Used for: runtime internals, zero-copy optimizations, Interoperability, advanced performance tuning. Risks: memory corruption, invalid pointers, portability issues, Most business applications should avoid unsafe.
- Context Package: Used for: cancellation propagation, deadlines/timeouts, request-scoped metadata. Eg: HTTP servers, gRPC, DB calls, etc.
- Benchmarking and Profiling:
  - Benchmarking: go test -bench=.
  - Profiling uses pprof.
- gRPC and Protocol Buffers:
  - gRPC is a high-performance RPC framework built on HTTP/2 and Protocol Buffers.

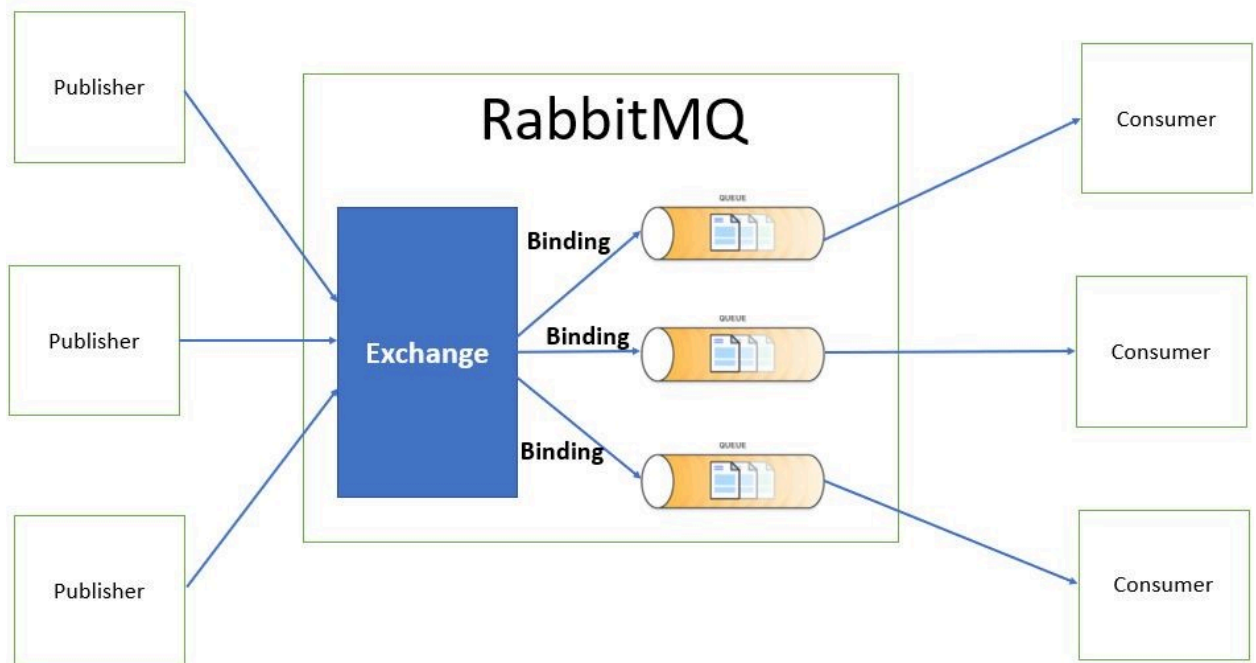
- Advantages over REST:
    - binary serialization
    - smaller payloads
    - faster communication
    - bidirectional streaming
    - strong typing
    - schema contracts
  - Protocol Buffers define schemas in .proto files and generate strongly typed Go code automatically.
  - Widely used in: microservices, distributed systems, internal infrastructure.
  - Production Concurrency Patterns:
    - Worker Pool Pattern: Jobs → Channel → Worker Goroutines → Results
    - Fan-out distributes work across workers.
    - Fan-in aggregates results back into a single stream.
    - Backpressure becomes important when producers generate data faster than consumers process data.
    - Common overload-control mechanisms:
      - buffered channels
      - batching
      - queue sizing
      - rate limiting
    - Goroutine leaks are major production issues and commonly occur because of:
      - blocked channels
      - forgotten receivers
      - missing context cancellation
      - infinite waits
    - Proper cancellation and cleanup are critical in long-running systems.
-

\_Misc\_Tech

## Misc Tech:

### RabbitMQ

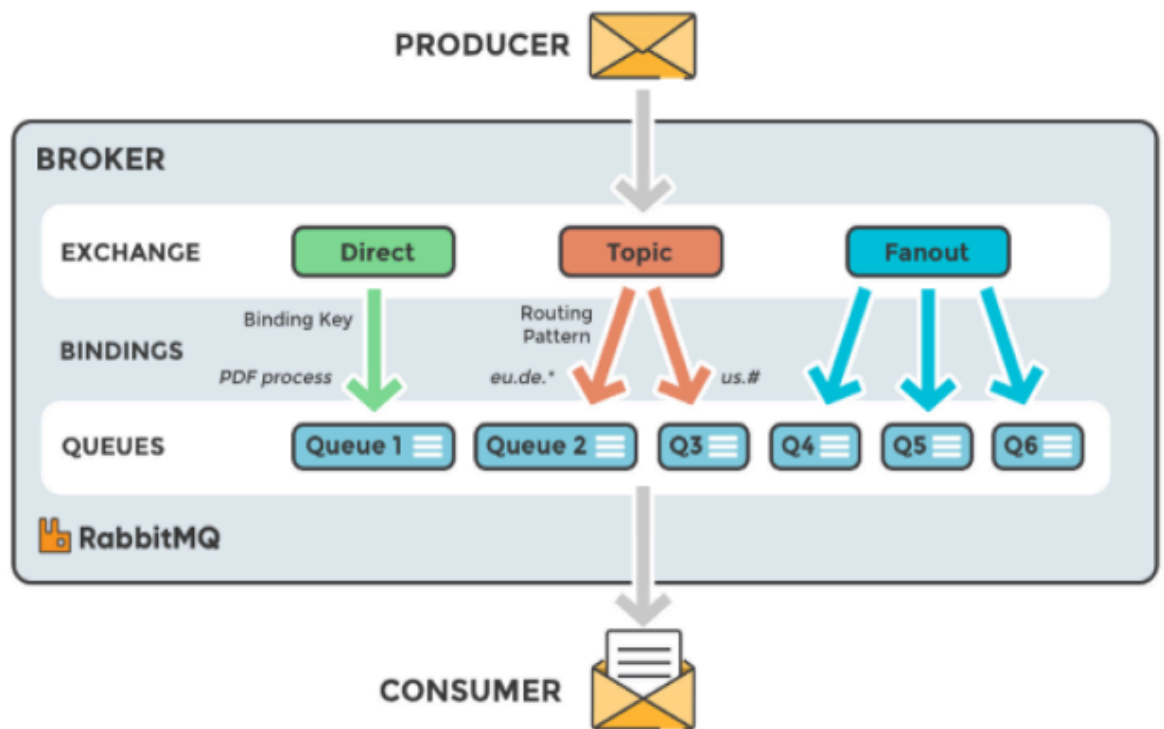
- RabbitMQ is a message broker or queue manager that implements the AMQP (Advanced Message Queuing) protocol. It takes messages from a queue and sends them to consumers.
  - Kafka uses the binary protocol over TCP to stream messages across real-time data pipelines
  - Protocol: A standardised way for systems to talk to each other.



- RabbitMQ does not publish the messages directly to a queue but via an exchange. Exchange ensures that the message is published to the right queue, it receives message and routing info. Each exchange is linked to a queue via binding. There are 4 types of exchanges:
  - Direct exchange delivers messages to queues based on a message routing key. The routing key is a message attribute added to the message header by the producer. Think of the routing key as an "address" that the exchange is using to decide how to route the message. A message goes to the

queue(s) with the binding key that exactly matches the routing key of the message.

- Fanout exchange copies and routes a received message to all queues that are bound to it regardless of routing keys or pattern matching as with direct and topic exchanges. The keys provided will simply be ignored. Fanout exchanges can be useful when the same message needs to be sent to one or more queues with consumers who may process the same message in different ways
- Topic exchanges route messages to queues based on wildcard matches between the routing key and the routing pattern, which is specified by the queue binding. Messages are routed to one or many queues based on a matching between a message routing key and this pattern. Eg: The routing patterns may contain an asterisk ("") or ("#") symbol like agreements.ru.berlin.#.
- Headers exchange routes messages based on arguments containing headers and optional values, similar to Topic exchange.



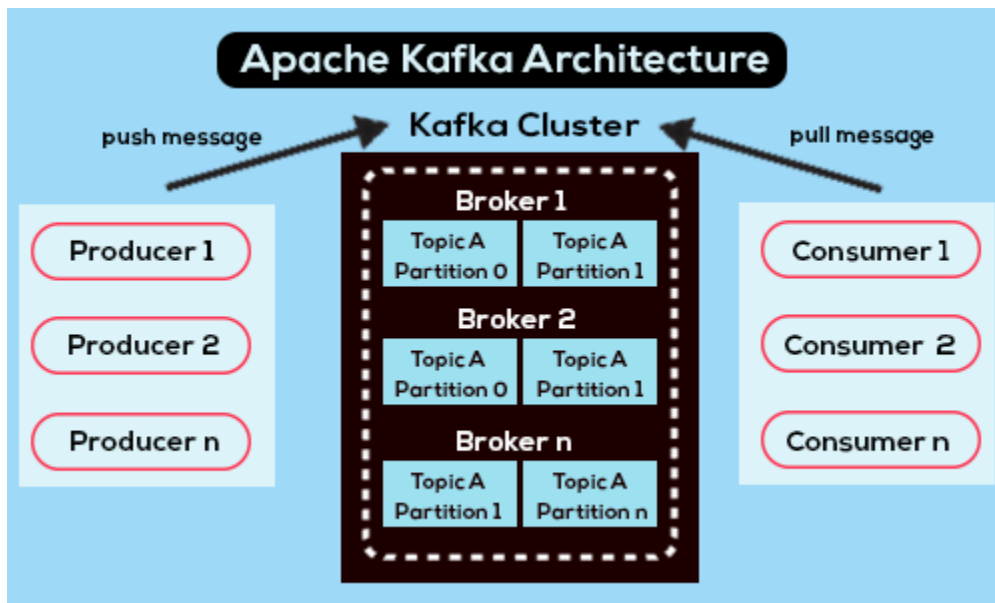
- We can define the queue's priority range at the time the queue is created.
- Dead-letter queue (DLQ), sometimes which is also known as an undelivered-message queue, is a holding queue for messages that cannot be delivered to their destinations due to some reason or other. Few failure criteria:
  - Message that is sent to a queue that does not exist.
  - Queue length limit exceeded.
  - Message length limit exceeded.
  - Message is rejected by another queue exchange.
  - Message reaches a threshold read counter number, because it is not consumed. Sometimes this is called a "back out queue".
- RabbitMQ default behavior:
  - If you didn't define the time-to-live argument, RabbitMQ will try to re-queue your message forever.
  - If you defined a TTL but didn't define a dlx, after TTL, RabbitMQ will remove your message to the queue and you will lose the message.
  - If you defined a TTL and a dlx, after the TTL, RabbitMQ will send the message to the exchange defined in the dlx.
- Although you can have a strategy to retry messages in the RMQ.
- Pika is a Python client library that implements the AMQP (Advanced Message Queuing Protocol) to communicate with RabbitMQ.

## Data Ecosystem

- Ingestion (Kafka, NiFi) - Topics, partitions, processors, data flow patterns
- Storage (S3, Hudi, Iceberg, Delta Lake) - Storage classes, table formats, ACID operations
- Compute (Spark, Flink) - Execution models, transformations, state management
- Query (Trino, Hive) - MPP architecture, query optimization, execution engines
- Catalog (Hive Metastore, AWS Glue) - Metadata management, crawlers, schema registry
- Orchestration (Airflow, Dagster) - DAGs, operators, asset-based orchestration
- Governance (Atlas, Ranger) - Lineage, classification, access control



## Kafka



- Core:
  - Topic: Logical category/feed where messages are published.
  - Partition: Unit of parallelism within a topic. Each partition is an ordered, immutable log of records.
  - Message order is guaranteed only within a single partition.
  - Broker: A Kafka server that stores partitions and handles client requests. A cluster usually has multiple brokers (e.g., broker1, broker2, ...).
    - The partition leader on that broker will be re-elected from followers (if in-sync replicas exist).
  - Consumer group:
    - A set of consumers working together to consume a topic's partitions without overlap.
    - One partition can be consumed by only one consumer (within the same group). But one consumer can consume multiple partitions.
      - If more consumers than partitions → some consumers stay idle

- If fewer consumers than partitions → consumers take multiple partitions
- Multiple Consumer Groups → Each group gets its own copy of the topic stream. Kafka topic does not care how many consumer groups exist. Kafka enforces ordering & exclusivity only within a consumer group, not across groups. Adding more consumer groups does NOT affect each other. Across groups: The same partition can be consumed by many different groups independently.
- If new consumer joins: Rebalance occurs – partitions redistributed among consumers.
- Rebalances can cause temporary unavailability, so production systems often use static membership or incremental rebalancing.
  - Incremental Rebalancing: Instead of revoking all partitions from every consumer during a rebalance, Kafka only revokes or assigns partitions that actually need to move, i.e some partitions from existing consumer to new consumer, rest flow runs intact.
  - Static Membership: Each consumer gets a stable member ID (group.instance.id) that persists across restarts. So if a consumer disconnects and reconnects quickly, Kafka knows it's the same member, not a new one.
- Offsets: Sequential IDs assigned to messages within a partition. In case of failure, consumer re-reads from last committed offset.

Types of offset commit:

- Auto Commit: Commits the latest offset returned by poll() every auto.commit.interval.ms (default: 5000 ms). Con: Can cause message loss/duplicate processing.
- Manual Synchronous Commit: Explicitly commit offsets after successful processing. Con: Can reduce throughput.

- Manual Asynchronous Commit: Sends commit request and returns immediately. Con: No automatic retry, possible duplicates.
- Under the hood, Kafka topics = folders, and partitions = append-only log files stored on disk. Each message is written sequentially at the end of a partition with a unique offset (like a line number). Once written, messages are immutable — never updated or deleted (until retention expires).
  - The producer simply appends data to the end of a partition. No routing logic, no locks. This makes Kafka extremely fast (thanks to sequential disk writes and OS page cache).
  - Consumers explicitly ask for “messages from offset X onward.” Kafka just serves bytes from disk, like a range query in a database.
  - The commit operation only records, *separately*, the offset up to which the consumer has processed data safely. This is stored in a special Kafka internal topic: `__consumer_offsets`. So when your consumer calls `commitSync()` or `commitAsync()`, Kafka does not touch your topic’s data — instead, it writes a metadata record to `__consumer_offsets` like: `(group.id = analytics-consumer, topic = orders, partition = 0, offset = 1000)`
- Each partition has one leader and multiple followers, i.e: partition is replicated across multiple brokers for fault tolerance. Producers send data only to the leader. Consumers read data only from the leader. Followers just copy (replicate) the data in the background. How do I decide the number of partitions?:  $\text{Number of partitions} = \text{desired parallelism} = \text{target throughput} \div \text{per-partition throughput}$ .
  - Throughput-based: Example: You need 50 MB/s ingestion. One partition gives ~5 MB/s. → You need ~10 partitions.
  - Consumer parallelism: If a consumer group has 20 instances → Topic should have  $\geq 20$  partitions.

- Key-based ordering: If you want ordering by a key (say `userId`), that key must land in one partition. More partitions = better spread = better parallelism.
- Replication Process: Producer writes to the leader. Leader writes data to its local log. Followers fetch data from the leader and append it to their logs. Once a message is replicated to the required number of followers (defined by `acks`), Kafka acknowledges success to the producer. ISR (In-Sync Replicas) = Leader + followers that are fully caught up with the leader. If leader fails, one of the followers becomes leader.
- Schema Registry: Ensures data consistency between producers and consumers. Validates message schemas (Avro, Protobuf, JSON Schema). If producer sends incompatible schema the registry rejects it.
- Kafka used Zookeeper for metadata info - which topics in partitions, broker leader for which partition, configurations, etc. Now a days, KRaft used.
- Kafka - At-least-once delivery semantics by default (others, have to handle).
  - At-most-once: Messages may be lost, but never redelivered
  - At-least-once (default): No data loss, but possible duplicates\
  - Exactly-once: No data loss and no duplicates
- What makes Kafka high throughput?
  - Sequential disk writes (append-only log): Kafka writes to disk sequentially, not randomly. Sequential disk writes are as fast as RAM due to OS page cache.
  - Zero-copy architecture: Kafka uses: `sendfile()`, OS page cache, Bypasses user-space copying. This avoids: CPU overhead, Multiple memory copies, Syscalls.
  - Partitioned parallelism: Every partition is: independent, append-only, consumed independently. More partitions → more consumers → more throughput.
  - Dumb broker, smart consumer: Kafka brokers do almost no processing, do not re-encode messages, simply store bytes and serve them. Consumers handle: offset tracking, ordering,

semantics. This "log-based" architecture makes Kafka extremely scalable.

- Schema registry in Kafka: It is an application that resides outside of your Kafka cluster and handles the distribution of schemas to the producer and consumer by storing a copy of schema in its local cache. With the schema registry in place, the producer, before sending the data to Kafka, talks to the schema registry first and checks if the schema is available. If it doesn't find the schema then it registers and caches it in the schema registry. Once the producer gets the schema, it will serialize the data with the schema and send it to Kafka in binary format prepended with a unique schema ID. When the consumer processes this message, it will communicate with the schema registry using the schema ID it got from the producer and deserialize it using the same schema. If there is a schema mismatch, the schema registry will throw an error letting the producer know that it's breaking the schema agreement.

## Airflow

- Workflow orchestration tool. Workflows are defined as DAGs.
- Terms:
  - Components: Scheduler, Executor, Webserver for UI, Metadata DB. When you run a PythonOperator task:
    - When you deploy a DAG file (example\_dag.py) to Airflow's dags/ folder: The Scheduler scans and parses it. It records this info (DAG ID, task ID, schedule interval, dependencies, etc.) in the Metadata Database.
    - At runtime (say if scheduled @daily), the Scheduler: updates the data in metadata db - marking dag run to scheduled, creating/updating task instance state, etc. Scheduler delegates the task to Executor, doesn't directly give to workers.
    - Assume using CeleryExecutor / KubernetesExecutor → pushes task messages to a queue (like Redis/RabbitMQ) for distributed workers.

- Redis is normally known as an in-memory cache, but it can also act as a message broker / queue (FIFO). Celery is a Python task queue framework — the orchestrator that uses Redis (or RabbitMQ) underneath as the message broker. Workers are processes that listen to this queue, take tasks from it, execute them, and report results back. Workers can run on same machine or different cluster of machines.
- The Executor sends the task message (“run `example_dag.run_my_script` for execution date X”) to an available Worker. The Worker picks it up from the queue.
- Worker Executes the PythonOperator script task. Post that it updates state in Metadata db. Scheduler polls data from this db to check if tasks failed/complete to trigger downstream tasks. Webserver UI reads from the DB and displays appropriate status.
- Task: A single unit of work, defined via Operators.
- Operator: Template for defining tasks (e.g., PythonOperator, BashOperator).
- XCom: Used to pass small data between tasks.
- Sensors, etc.

## Dask

- Dask is a parallel computing framework for Python that scales from a single machine to a cluster.
- Components: Client, Scheduler, Workers, Task graph. Dask builds a task graph of operations. Scheduler assigns tasks to workers. Workers execute tasks in parallel, report results. Results are aggregated and returned to client.
- Dask DataFrame: It is a collection of many smaller Pandas DataFrames, partitioned along an index.
- Dask Array: Parallel, chunked numpy array for numerical computations.
- Dask Bag: Handles unstructured/semi-structured data (JSON, logs, text files). Similar to PySpark RDD. Can convert to DataFrame using `.to_dataframe()`.

- Dask builds a task graph first, then executes when `.compute()` or `.persist()` is called. (Lazy evaluation)

```
A Dask graph is a dictionary mapping keys to computations:
{'x': 1,
 'y': 2,
 'z': (add, 'x', 'y'), // Computation function
 'w': (sum, ['x', 'y', 'z']), // Some computation being performed
 'v': [(sum, ['w', 'z']), 2]}
```

## Ansible

- It's an automation engine for configuration management, application deployment, and orchestration.
- Consider command: `ansible-playbook -vv -i inventories/${INSTALL_TO_ENV}/hosts.py airflow.yml --vault-password-file=~/.vault_pass_no_prompt`
  - Ansible playbooks are a fundamental component of Ansible, serving as the primary means to automate tasks across multiple systems. They are written in YAML format.
  - When you execute a playbook using the command `ansible-playbook <playbook.yml>`, Ansible parses the YAML file and prepares to execute the defined plays. Ansible connects to each host specified in the inventory file and executes the tasks in order. If a task fails on one host, Ansible continues executing tasks on other hosts unless configured otherwise. During execution, Ansible provides real-time feedback on the status of each task, including successes and failures. After completing all tasks, Ansible summarizes the execution results, detailing which tasks succeeded or failed.
- Dissecting command:
  - `ansible-playbook`: This is the command used to run Ansible playbooks. It initiates the execution of the specified playbook file.
  - `-vv`: This option increases the verbosity level of the output. The `-v` flag can be repeated (up to four times) to provide more detailed logging information during execution. This is useful for debugging and understanding what Ansible is doing behind the scenes.

- `-i inventories/prod/hosts.py`: This specifies the inventory file that contains the list of hosts on which the playbook will run. In this case, `hosts.py` is likely a dynamic inventory script, which means it generates the list of hosts dynamically rather than being a static file.
- `airflow.yml`: This is the name of the playbook file that will be executed. The playbook contains tasks that define what actions should be performed on the specified hosts.
- `--vault-password-file=~/.vault_pass_no_prompt`: This option specifies a file that contains the password used to decrypt any sensitive data stored in Ansible Vault within the playbook or any included files.
- Ansible Vault is a feature within Ansible that provides a secure mechanism for managing sensitive data, such as passwords, API keys, and other confidential information. It allows users to encrypt values and data structures within Ansible projects, ensuring that sensitive information is not stored in plaintext, which could pose a security risk.
  - Encryption and Decryption: Ansible Vault enables users to encrypt files containing sensitive data. This encryption uses symmetric encryption with the AES256 algorithm, meaning the same password is used for both encryption and decryption. Users can encrypt existing files or create new encrypted files directly.
  - File-Level Granularity: Ansible Vault operates at the file level, meaning that entire files can be encrypted or left unencrypted. This allows for flexibility in managing sensitive information alongside regular configuration data.
  - Integration with Playbooks: Encrypted content can be seamlessly integrated into Ansible playbooks. When executing a playbook, Ansible automatically decrypts any vault-encrypted content at runtime if the correct password is provided.
  - Secure Storage: By using Ansible Vault, users can securely store sensitive data within their playbooks and configuration files, preventing unauthorized access even if someone gains access to the files themselves.



- In above case we provide this flag in command:  
`--vault-password-file=~/.vault_pass_no_prompt`. The vault password is crucial when working with Ansible Vault because it:
  - Decrypts Encrypted Data: If your playbook or any referenced files (like variables or configuration files) contain encrypted content (e.g., passwords, API keys), Ansible needs to decrypt this data before executing tasks that require it. The `--vault-password-file` option tells Ansible where to find the password needed for decryption.
  - Automates Password Handling: By using a password file (in this case, `~/.vault_pass_no_prompt`), you can automate the process of providing the vault password without being prompted during execution. This is particularly useful for automated deployments or CI/CD pipelines where manual input would be impractical.

## Debezium

- Open-source distributed platform for Change Data Capture (CDC). Captures row-level changes in databases and streams them as events.
- Arch: Database → Transaction Log → Debezium Connector → Kafka → Consumers

## Docker

- Components:
  - Dockerfile: A text document containing commands that assemble a container image.
  - Image: A standalone, executable package containing everything required to run a piece of software.
  - Container: A runtime instance of an image.
- Virtual Machines virtualize the hardware layer, meaning each VM runs its own operating system on top of a hypervisor. This gives strong isolation but causes heavy resource consumption and slow startup times.
- Docker containers, on the other hand, virtualize at the OS level, sharing the host kernel. This makes them extremely lightweight, fast to start, and ideal for microservices or CI/CD environments. However, container isolation isn't as strict as VMs since they share the underlying OS.

- Images are stored in Docker image registries like Docker Hub, which provides a central location for image management and sharing. You can download existing images, modify them, and upload the modified versions also.
- COPY vs ADD in Docker

| Aspect         | COPY                                   | ADD                                                               |
|----------------|----------------------------------------|-------------------------------------------------------------------|
| Purpose        | Simple, direct file and directory copy | Includes advanced features like URL fetch and TAR extraction      |
| URL Support    | ✗ Not supported                        | ✓ Can copy from remote URLs (e.g., <code>ADD https://...</code> ) |
| TAR Extraction | ✗ No extraction                        | ✓ Automatically extracts local <code>.tar</code> archives         |
| Caching        | ✓ Deterministic, respects cache        | ⚠ May bypass cache if source differs slightly                     |

- A Docker volume is a directory or file within a Docker Host's writable layer that isn't tied to a specific container. This decoupling allows data persistence, even after containers have been stopped or removed. Containers are ephemeral — data is lost when they stop. Volumes persist data outside containers. Volume Types:
  - Host-Mounted Volumes: These link a directory on the host machine to the container.
  - Named Volumes: They have a specific name and are managed by Docker.

- Anonymous Volumes: These are generated by Docker and not tied to a specific container or its data.
- Docker provides isolated virtual networks for containers to communicate: bridge (default), host (shares host network), none (no network), overlay (for Swarm multi-host), macvlan (assign MAC address)
- Docker compose, docker commands, etc...

## Nginx

- Nginx = High-performance web server, reverse proxy, load balancer, and cache.
- Event-driven & asynchronous → handles thousands of connections with low memory.
- Process model:
  - Master process starts and creates worker processes.
  - Workers wait for events (client requests) via event loop.
  - When a request comes in:
    - OS (via epoll) notifies an available worker.
    - Worker handles the full request (read → process → send response).
  - Master doesn't handle traffic — it only supervises.
  - On reload, the master spawns new workers and gracefully shuts down old ones (no dropped connections).
- Common roles: Serve static files, Reverse proxy to app servers (e.g., Gunicorn, Node, Spring Boot), Load balance traffic, Terminate SSL/TLS, API Gateway. Eg:

```
/etc/nginx/nginx.conf
events {
 worker_connections 1024;
}
http {
 upstream backend {
 server 127.0.0.1:8080;
 server 127.0.0.1:8081;
 }
 server {
```

```

listen 80;
server_name example.com;
location / {
 proxy_pass http://backend;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
}
location /static/ {
 root /var/www/html;
}
}

```

## Redis

- Execution model
  - Single-threaded event loop using epoll (Linux) / kqueue (BSD/macOS). Handles thousands of clients using non-blocking I/O and I/O multiplexing. No context switching → predictable low latency.
  - Redis 6+ introduces optional I/O threads (for read/write only, not command execution).
  - Simplicity is the performance secret — no threads, no locks, no coordination overhead. Each command runs atomically because only one executes at a time.
- Memory Model
  - Entire dataset lives in RAM → no disk seek latency. Uses jemalloc / mimalloc for efficient memory allocation.
  - Keys and values stored as SDS (Simple Dynamic Strings) — safer and resizable C strings. Values use specialized data structures (Lists, Sets, Hashes, ZSets, Bitmaps, HyperLogLogs, Streams).
  - When memory hits limit (maxmemory), eviction policy applies based on TTL, etc.
- Persistence: Redis is not purely volatile — you choose durability level. Like:
  - RDB (Snapshotting): Saves full in-memory data to .rdb file periodically.

- AOF (Append Only File): Logs every write command.
- Hybrid Mode: Combines RDB + AOF (since Redis 4.0).
- Replication & HA: Primary-Replica model, asynchronous replication.

## PostgreSQL

- Multi-Version Concurrency Control (MVCC) is PostgreSQL's approach to handling concurrent transactions without locking reads.  
How it works: Each transaction sees a snapshot of the database at a specific point in time -> Updates create new row versions rather than overwriting existing data -> Old versions are kept until no transaction needs them (handled by VACUUM) -> Readers don't block writers, and writers don't block readers -> Each row has hidden system columns: xmin (transaction ID that created it) and xmax (transaction ID that deleted/updated it)
  - Benefits: High concurrency with minimal locking, Consistent reads without blocking, Point-in-time recovery capabilities
  - Drawbacks: Requires VACUUM to clean up old row versions, Can lead to table bloat if not managed properly, Increased disk I/O for updates
- Components in Postgres:
  - Postmaster: Main daemon process that handles client connections
  - Backend Processes: One per client connection, handles queries
  - Background Processes:
    - Background Writer: Writes dirty buffers to disk periodically
    - WAL Writer: Writes WAL (Write-Ahead Log) records to disk
    - Checkpointer: Performs checkpoints to ensure data durability
    - Autovacuum Launcher/Workers: Automatic VACUUM and ANALYZE
    - Stats Collector: Collects usage statistics
    - Logical Replication Workers: Handle logical replication
    - Archiver: Archives WAL files for PITR
- By default, PostgreSQL is a single-node OLTP database with one primary (writable) instance and no built-in read replicas. In production, it's often extended using streaming replication or managed services like Amazon RDS / Aurora to support readers and writers.

- `shared_buffers` is the amount of memory PostgreSQL uses for caching data.
- PostgreSQL supports multiple index types: BTree, Hash, Generalized Search Tree, Block Range Index, etc.
- Execution plan of query in psql: `EXPLAIN (ANALYZE, BUFFERS, VERBOSE) SELECT * FROM orders WHERE user_id = 123;`
  - Find slow queries: `SELECT query, mean_exec_time, calls FROM pg_stat_statements ORDER BY mean_exec_time DESC LIMIT 10;`
- EXPLAIN vs EXPLAIN ANALYZE:
  - Explain: Shows the query plan PostgreSQL will use, Estimated costs and row counts, Does NOT execute the query, Safe to run on production
  - Explain Analyze: Actually executes the query, Shows actual execution time and row counts, Shows actual vs estimated comparisons, Includes I/O statistics with BUFFERS option, Can impact production systems
- N+1 query problems in PostgreSQL: N+1 occurs when you fetch a list and then query for related data for each item.

Eg:

```
-- Bad: N+1 in application code
SELECT * FROM users;
-- Then for each user: SELECT * FROM orders WHERE user_id = ?
-- Good: Single query
SELECT u.*, o.*
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
```

- A transaction = a group of SQL operations that run as one logical unit. Either all succeed (COMMIT). Or none take effect (ROLLBACK-restores the database to its previous committed state). A transaction is committed when its changes are permanently written to the database and become visible to other transactions. Until you COMMIT, your changes are private to your session. Isolation defines how much one transaction can "see" of another transaction's changes while both run concurrently.

- Read Uncommitted (acts like Read Committed in Postgres): Reads only committed data (no dirty reads).
- Read Committed (Default): Each SQL statement sees data committed before that statement starts. If another transaction commits after your query, your next query can see the new data.
- Repeatable Read: Sees a snapshot of the database from the moment the transaction started. You will see consistent results across queries, even if others commit changes.
- Serializable (Strictest): Transactions behave as if they were run one after another (fully isolated). PostgreSQL enforces this using predicate locking. Safest, but may cause more rollbacks due to conflicts.
- A deadlock occurs when two or more transactions hold locks that the other needs, causing all of them to wait forever. PostgreSQL detects deadlocks automatically. It kills one of the transactions (throws an error) so the other can continue. The killed transaction must be retried. Avoid deadlock: Lock ordering, Keep transactions short, Use appropriate isolation levels, Use SELECT ... FOR UPDATE or NOWAIT/SKIP LOCKED carefully, Tune deadlock\_timeout, Monitor logs / stats.
  - Advisory locks are application-level locks not tied to database objects. Types:
    - Session-level: Last until explicitly released or session ends
    - Transaction-level: Released at transaction end
- Reduce write latency and increase throughput by optimizing:
  - WAL (Write-Ahead Logging): WAL ensures durability – but too frequent writes slow performance.
  - Checkpoints: Checkpoints write dirty buffers to disk – too frequent = I/O spikes.
  - Background writes: Keeps cache (shared\_buffers) clean proactively.
  - Application-level behavior: Batch Inserts for less WAL overhead, Disable triggers, COPY for bulk loads etc.
- Table Bloat: When rows are updated or deleted, PostgreSQL doesn't immediately remove old row versions – it marks them as dead tuples. Over time, these accumulate, causing table bloat (larger disk size without increase in live rows).

- Internal Mechanism (MVCC):
  - UPDATE → creates a new tuple, marks old one as dead.
  - DELETE → marks tuple as dead, not physically removed.
  - Dead tuples still consume space until vacuumed.
- Impact:
  - Slower sequential scans and index lookups
  - Increased I/O due to bloated pages
  - Noticeable performance degradation on frequently updated tables
- Auto-Vacuum: It is PostgreSQL's background process that automatically removes dead tuples to control bloat and maintain visibility maps. Usually takes an ACCESS SHARE lock (doesn't block reads/writes). On large, busy tables, it can compete for CPU/I/O. Poor tuning may cause lag → excessive bloat or transaction wraparound risk. How It Works:
  - Tracks row updates/deletes per table.
  - When thresholds are crossed (autovacuum\_vacuum\_threshold + scale factor), it triggers vacuuming.
  - Removes dead tuples invisible to active transactions.
  - Updates the Free Space Map (FSM) and Visibility Map (VM).
  - Performs aggressive freezing near vacuum\_freeze\_max\_age to prevent transaction ID wraparound.
- Manual Vacuum:
  - VACUUM → Reclaims dead tuples' space; lightweight and non-blocking.
  - VACUUM ANALYZE → Cleans up and refreshes optimizer statistics.
  - VACUUM FULL → Fully rewrites the table to reclaim disk space (blocks access during operation).
- TOAST (The Oversized-Attribute Storage Technique): TOAST manages large column values (e.g., text, byte, jsonb) that exceed ~2 KB. Internal Behavior:
  - Compresses large values when possible.
  - If still too big, stores them in a per-table TOAST table in 2 KB chunks.
  - The main table only holds a pointer to the TOAST data.
  - Cleanup Considerations:



- VACUUM and autovacuum must also maintain TOAST tables.
  - Large updates/deletes can leave dead TOAST tuples → hidden bloat if not cleaned.
- pg\_repack: It reclaims bloat without blocking most operations, but requires a brief lock during the final table swap. Process Overview:
  - Creates a new compacted copy of the table.
  - Uses triggers to capture ongoing DML changes.
  - Applies captured changes to the new copy.
  - Atomically swaps old and new tables.
  - Not ideal for high-write tables due to the short final lock and trigger overhead.
- pg\_partman: It automates time-based or serial-based partitioning in PostgreSQL.
  - It can auto-create future partitions and drop old ones as per retention policy.
  - Use cron or pg\_cron to schedule periodic maintenance via: `SELECT partman.run_maintenance();`
- Replication means copying data from primary master node → standby slave node in real time.
  - Asynchronous: Primary doesn't wait for standby → faster, may lose last few transactions if crash.
  - Synchronous: Primary waits for standby confirmation → no data loss, but slower.
  - Standby node can be used for read-only queries (hot standby). Tools like Patroni or repmgr handle automatic failover.
- Physical replication copies: Binary WAL (whole DB); But Logical rep., copies data changes (rows) - may cause partial sync, migrations, cross-version.
- Failovers (Automatically switching from a primary master db to a standby replica when the primary fails)
  - Manual: run `pg_ctl promote` on standby.
  - Automatic: tools like Patroni, repmgr, `pg_auto_failover` detect failure and promote standby.
- Partitioning: Range, Hash, List Partitioning.
- JSON vs JSONB

- JSON: Stores exact input text, Faster to insert, Preserves whitespace and key order, No processing overhead on insert, Slower to query
- JSONB (Binary JSON): Stores decomposed binary format, Slower to insert (decomposition cost), Removes whitespace, no key ordering, Much faster to query and process, Supports indexing
- RLS (row-level-security) restricts which rows users can access. Can enable RLS & add policy via a query.
- Backup Strategies:
  - Logical Backup (`pg_dump` / `pg_dumpall`): Exports SQL statements to recreate data. Use for migrations, small-medium DBs. Supports parallel dumps for speed. Restore with `pg_restore`.
  - Physical Backup (`pg_basebackup`): Copies data files + WALs — exact binary snapshot. Use for large databases, replication setup, or PITR. Faster restore, but must match PG version.
  - Continuous Archiving (PITR): Archives WAL logs continuously. Allows Point-In-Time Recovery (restore to any moment). Needs: Base backup + archived WALs.
    - Point-in-Time Recovery (PITR): Take base backup (`pg_basebackup`). Restore base backup to data directory. Set recovery configs in `postgresql.auto.conf`. Create `recovery.signal` file. Start PostgreSQL → replays WALs until target.
- Connection Pooling:
  - PostgreSQL uses a new OS process per connection → expensive setup & memory cost.
  - Default limit: `max_connections = 100`. Frequent connects/disconnects cause connection storms and latency spikes.
  - Pooling reuses connections, reducing overhead and improving throughput.
  - PgBouncer is a lightweight connection pooler for PostgreSQL. It sits between your application and the database, managing and reusing PostgreSQL connections efficiently. Pooling modes:

| Mode                                                                                                  | Behavior                                   | Use Case                      |
|-------------------------------------------------------------------------------------------------------|--------------------------------------------|-------------------------------|
| Session Pooling                                                                                       | Connection held for the entire session     | Compatible but limited reuse  |
| Transaction Pooling  | Connection released after each transaction | Best balance for performance  |
| Statement Pooling                                                                                     | Released after every statement             | Very restrictive, rarely used |

- Use `pg_stat_*` views to monitor runtime activity and performance, and `pg_*` catalogs to inspect metadata and configuration inside PostgreSQL. Eg: `pg_stat_activity`, `pg_stat_database`, `pg_stat_replication`, `pg_lock`, `pg_database`, etc.

## Timescale (Postgres)

- TimescaleDB is a PostgreSQL extension optimized for time-series data. Since it is built on top of PostgreSQL, it supports full SQL, joins, indexes, transactions, ACID guarantees, extensions, and the broader PostgreSQL ecosystem. On top of PostgreSQL capabilities, TimescaleDB adds hypertables, automatic chunking, compression, continuous aggregates (CAGGs), retention policies, and time-series optimizations. Common use cases include metrics storage, IoT telemetry, observability platforms, monitoring systems, event streams, sensor data, and financial tick data.
- Core Idea of TimescaleDB: Instead of storing all time-series data in one massive table, TimescaleDB automatically partitions data into smaller chunks based on time intervals. This improves insert performance, query pruning, retention management, compression efficiency, and overall scalability for large datasets.
- Hypertables: A hypertable is a logical abstraction over multiple chunk tables. Applications interact with it like a normal PostgreSQL table, while TimescaleDB manages partitioning internally.

```
Example:
CREATE TABLE metrics(
```

```

time TIMESTAMPTZ,
device_id INT,
cpu DOUBLE PRECISION
);

SELECT create_hypertable('metrics', 'time');

```

- **Chunks:** Chunks are physical partitions created automatically by TimescaleDB. They are typically partitioned by time and optionally by an additional space partition such as device\_id or region. Benefits include partition pruning, improved parallelism, better compression handling, easier retention management, and faster query execution on time-bounded datasets.
- **Chunk Interval:** Chunk interval defines the time range covered by each chunk, for example 1 day or 7 days. Smaller chunks improve pruning efficiency and query locality but increase metadata and planning overhead. Larger chunks reduce partition count but may hurt pruning and increase scan sizes. A common rule is to size chunks such that actively queried chunks comfortably fit into memory.
- **Compression:** TimescaleDB supports automatic compression of older chunks. Compression provides major storage savings and can sometimes improve analytical query performance because fewer blocks are scanned. Compression is columnar internally, making it efficient for repetitive time-series data. Typically, recent hot data remains uncompressed for fast writes and updates, while older cold historical data is compressed.
- **Retention Policies:** Retention policies automatically drop old chunks after a configured duration, for example deleting data older than 90 days. This is especially useful for monitoring and observability systems where raw metrics lose value after some retention window.
- **Continuous Aggregates (CAGGs):** CAGGS are one of the most important TimescaleDB features. The problem with raw time-series data is that repeatedly aggregating billions of rows is expensive.

Example:

```

SELECT time_bucket('1 hour', time),

```

```
 AVG(cpu)
FROM metrics
GROUP BY 1;
```

- As data volume grows, such queries become increasingly expensive. Continuous Aggregates solve this by maintaining incrementally refreshed materialized aggregations over time-series data. Unlike traditional materialized views, CAGGs refresh incrementally instead of recomputing everything.

```
Example:
CREATE MATERIALIZED VIEW cpu_hourly
WITH (timescaledb.continuous)
AS
SELECT time_bucket('1 hour', time) AS bucket,
device_id, AVG(cpu) avg_cpu
FROM metrics
GROUP BY bucket, device_id;
```

- How CAGGs Work: Raw data remains stored in hypertables, aggregate results are stored separately, and background jobs refresh only modified time buckets instead of recomputing the entire dataset. This provides major performance improvements for dashboards and analytical workloads.
- Real-time Aggregates: CAGGs can combine materialized historical aggregates with fresh recent raw data, allowing near real-time query results while still avoiding expensive full-table aggregations.
- Refresh Policies: Background jobs automatically refresh aggregates at configured intervals, for example every 5 minutes. Refresh windows are important because late-arriving data may require recomputation of recent buckets. A common strategy is continuously refreshing the last few hours of data.
- Time Buckets: `time_bucket()` is TimescaleDB's core time-grouping function. Example: `time_bucket('5 minutes', time)`

- It is conceptually similar to DATE\_TRUNC but optimized specifically for time-series aggregation and heavily used with CAGGs.
- Late Arriving Data: Late-arriving events are common in IoT and distributed systems, for example an event generated at 10:00 arriving at 10:10. Continuous aggregates handle this using refresh windows that periodically recompute recent buckets. The refresh window must be large enough to account for expected lateness.
- Data Retention with CAGGs: A common architecture is retaining raw data for a short duration while keeping aggregated data much longer. For example, raw metrics may be retained for 7 days while hourly aggregates are retained for 1 year.
- Compression + CAGGs Pattern: A very common production setup is: recent data remains raw and uncompressed for fast ingestion, older historical data is compressed for storage efficiency, and analytical queries primarily hit continuous aggregates. This significantly reduces storage cost and query latency.
- Why TimescaleDB over Plain PostgreSQL Partitioning?: TimescaleDB provides automatic chunk management, optimized query planning, built-in compression, continuous aggregates, automated retention handling, and simpler operational management compared to manually managing PostgreSQL partitions.
- Why not Cassandra or InfluxDB?: TimescaleDB benefits from the PostgreSQL ecosystem, meaning full SQL support, joins, relational modeling, strong consistency, mature tooling, and transactional guarantees. However, extremely large-scale write-heavy workloads may scale better horizontally in systems like Cassandra.
- Important Internals: During inserts, rows are automatically routed into the correct chunks. During query execution, the planner prunes irrelevant chunks using time filters. Without time predicates, queries may scan many chunks and become slow.
- Optimization Rule: Always query with a time filter. Eg: Bad: `SELECT * FROM metrics WHERE device_id = 1;`  
Better: `SELECT * FROM metrics WHERE time > now() - interval '1 day' AND device_id = 1;`

- Indexes in TimescaleDB: Common indexes are: (time DESC) or (device\_id, time DESC). This is because most workloads are recent-first and time-range bounded. Compression on old chunks may reduce the usefulness of indexes for historical analytical queries since compressed scans are often more efficient than indexed lookups.

## Kubernetes

- K8s is an open-source container orchestration platform used to deploy, manage, scale, and automate containerized applications. It abstracts infrastructure management and provides self-healing, autoscaling, service discovery, rolling updates, load balancing, and declarative infrastructure management. Originally developed by Google and now maintained by CNCF.
- Kubernetes follows a Control Plane + Worker Node architecture.
  - Control Plane components include API Server, etcd, Scheduler, Controller Manager, and Cloud Controller Manager. API Server is the central entry point for all operations and all components communicate through it. etcd is the distributed key-value store storing cluster state and metadata. Scheduler decides where Pods should run based on CPU, memory, affinity, taints, and resource availability. Controller Manager continuously reconciles desired state vs actual state using controllers like Deployment controller and ReplicaSet controller. Cloud Controller Manager integrates Kubernetes with AWS/GCP/Azure services.
  - Worker Nodes run actual applications. Main components are Kubelet, Container Runtime, and Kube Proxy. Kubelet ensures containers are running correctly on the node, container runtime executes containers using containerd/CRI-O, and kube-proxy handles networking and Service routing.
- Pod is the smallest deployable unit in Kubernetes. A Pod can contain one or more tightly coupled containers sharing the same network namespace and storage volumes. Containers inside the same Pod communicate via localhost. Pods are ephemeral, meaning they can die and be recreated anytime. Applications should not store permanent data inside Pods. Kubernetes generally does not manage Pods directly; instead, higher-level

controllers like Deployments and StatefulSets manage them. Pod lifecycle phases are Pending, Running, Succeeded, Failed, and Unknown. Container states include Waiting, Running, and Terminated.

- Deployment is used for stateless applications and provides declarative deployment management, rolling updates, rollback, autoscaling support, and self-healing. Internally, Deployments create ReplicaSets which maintain the required number of Pods. Rolling updates gradually replace old Pods with new ones without downtime. Rollback allows reverting to a previous working version if deployment fails.
- ReplicaSet ensures a fixed number of Pod replicas are always running. If a Pod crashes, ReplicaSet automatically creates a replacement Pod. Usually ReplicaSets are managed indirectly through Deployments rather than manually.
- StatefulSet is used for stateful applications requiring stable identity and persistent storage. Unlike Deployments, Pods in StatefulSets maintain stable names like pod-0, pod-1 even after restarts.
- Deployment vs StatefulSet: Deployment is for stateless applications where Pods are interchangeable. StatefulSet is for stateful applications requiring stable identity and persistent storage.
- DaemonSet ensures one Pod runs on every node. Mostly used for infrastructure workloads such as logging agents, monitoring agents, security tools, and node exporters. Examples include Fluentd and Prometheus Node Exporter.
- Job runs a task until completion, useful for batch processing, ETL jobs, and migrations. CronJob schedules Jobs using cron syntax, useful for backups and scheduled maintenance tasks.
- Namespaces provide logical isolation within a cluster and are commonly used to separate dev, QA, and production environments. Default namespaces include default, kube-system, and kube-public.
- Labels are key-value metadata attached to Kubernetes objects like app=payment or env=prod. Selectors are used to identify matching resources. Services and Deployments use selectors to identify Pods.
- Pods are ephemeral and their IPs change frequently, so Services provide stable networking endpoints. ClusterIP exposes service internally within the cluster and is the default Service type. NodePort exposes service



externally using a port on each node. LoadBalancer creates a cloud load balancer for external access. Headless Service uses clusterIP=None and is mainly used with StatefulSets for direct Pod discovery.

- Kubernetes provides built-in DNS-based service discovery. Pods can communicate using service names directly.
- Ingress manages external HTTP/HTTPS traffic routing into the cluster. It supports path-based routing, host-based routing, SSL termination, and load balancing. Ingress requires an Ingress Controller like NGINX Ingress or Traefik. Example: api.company.com can route to backend service while web.company.com routes to frontend service.
- ConfigMap stores non-sensitive configuration like application configs and feature flags. Secret stores sensitive information like passwords, tokens, and certificates. ConfigMaps and Secrets can be injected into Pods as environment variables or mounted files.
- Volumes provide persistent or shared storage to containers. emptyDir provides temporary storage tied to Pod lifecycle. PersistentVolume (PV) represents actual storage resources, while PersistentVolumeClaim (PVC) is a request for storage by applications. StorageClass enables dynamic provisioning of storage volumes. Stateful applications typically use PVCs with StatefulSets.
- Requests define guaranteed minimum CPU/memory required by a container, while limits define the maximum allowed usage. Scheduler uses resource requests while placing Pods on nodes. Exceeding memory limits causes OOMKilled errors, while exceeding CPU limits causes throttling.
- Liveness Probe checks whether application is alive; failure restarts the container. Readiness Probe checks whether application is ready to receive traffic; failure removes Pod from Service endpoints but does not restart container. Startup Probe is useful for slow-starting applications.
- Every Pod receives a unique IP address and Pods can communicate directly without NAT. Kubernetes networking is implemented using CNI plugins such as Calico, Flannel, and Cilium.
- Network Policies act as firewall rules controlling ingress and egress traffic between Pods.
- Scheduling Concepts: Node Selector is a simple mechanism for placing Pods on specific nodes. Affinity and Anti-Affinity provide advanced

scheduling rules for placing Pods together or separately. Taints repel Pods from nodes while tolerations allow Pods to run on tainted nodes. Example: dedicate GPU nodes only for ML workloads.

- Autoscaling: Horizontal Pod Autoscaler (HPA) automatically scales Pods based on CPU, memory, or custom metrics. Cluster Autoscaler adds or removes nodes depending on workload demand.
- Rolling Updates, Blue-Green, Canary: Rolling updates gradually replace old Pods while maintaining availability.
  - Blue-Green deployment maintains two environments and traffic switches from old to new after validation.
  - Canary deployment gradually shifts traffic to the new version, for example 5% → 20% → 100%, reducing production risk.
- Helm is the package manager for Kubernetes. Helm packages are called Charts and allow reusable templated deployments using values.yaml. Helm simplifies multi-environment deployments and configuration management.
- RBAC (Role Based Access Control) controls permissions in Kubernetes using Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings.
- Common kubectl Commands: kubectl get pods, kubectl get svc, kubectl get deploy are used to view resources. kubectl describe provides detailed resource information. kubectl logs checks container logs. kubectl exec -it allows shell access into a container. kubectl apply -f applies YAML configurations. kubectl scale changes replica count.
- Kubernetes YAML Structure: Most Kubernetes manifests contain apiVersion, kind, metadata, spec, and status. spec defines desired state while status represents actual runtime state maintained by Kubernetes.
- Self-healing means failed Pods are recreated automatically. Kubernetes treats infrastructure as immutable, meaning workloads are replaced rather than modified directly.
- Common Production Issues: CrashLoopBackOff means containers repeatedly crash. ImagePullBackOff indicates image pull failure. Pending Pods usually indicate insufficient resources or scheduling constraints. OOMKilled occurs when containers exceed memory limits. Node NotReady usually indicates kubelet, networking, or infrastructure problems.

- Production Best Practices: Use readiness and liveness probes, define requests and limits, avoid latest image tag, enable RBAC, monitor cluster health, use namespaces properly, avoid running containers as root, backup etcd regularly, and use rolling or canary deployments for safer releases.
- AWS provides EKS, Google provides GKE, and Azure provides AKS. Managed Kubernetes services reduce operational overhead by managing control plane upgrades, scaling, and infrastructure integration.

## Clickhouse

- ClickHouse is an open-source, high-performance columnar OLAP database designed for real-time analytics on massive datasets. It is optimized for read-heavy analytical workloads rather than transactional systems.
- ClickHouse uses a columnar storage model, where data is stored column-by-column instead of row-by-row. This enables highly efficient compression, faster scans, and reading only required columns during query execution. It is designed for OLAP workloads, meaning it excels at aggregation-heavy queries such as GROUP BY, COUNT, SUM, and time-series analytics over billions of rows.
- A key concept in ClickHouse is the MergeTree engine family, which is the foundation of most production tables. Data inserted into ClickHouse is written as immutable “parts”, and background processes continuously merge these parts into larger optimized segments. This is similar to LSM-tree compaction in concept. Because of this design, batch inserts are highly recommended, while frequent small inserts degrade performance due to excessive part creation and merge overhead.
- ClickHouse uses a sparse primary index, meaning it does not index every row like B-tree databases. Instead, it stores index marks at intervals (granules), typically every 8192 rows. This allows ClickHouse to quickly skip large portions of data during scans while keeping index size very small. A granule is the smallest unit of data that can be read during query execution.
- Query execution in ClickHouse follows a highly parallel and vectorized model. Data is read in columnar batches, processed using SIMD-style

operations, and executed across multiple CPU cores, partitions, and shards simultaneously. This results in very high throughput for aggregation-heavy workloads.

- ClickHouse achieves high performance through multiple factors: columnar storage, compression (LZ4, ZSTD, delta encoding), sparse indexing, partition pruning, vectorized execution, and parallel query execution. These optimizations together enable ClickHouse to process billions of rows per second on commodity hardware.
- Replication in ClickHouse is handled using ReplicatedMergeTree with coordination via ZooKeeper or ClickHouse Keeper. Replication is asynchronous and ensures high availability across nodes. Sharding is used for horizontal scaling, where data is distributed across multiple nodes and queries are executed in parallel across shards using Distributed tables.
- Materialized Views are a major optimization feature in ClickHouse. They allow pre-aggregation or transformation of data during ingestion itself. For example, raw event data can be automatically aggregated into hourly or daily summaries, significantly improving dashboard query performance.
- ClickHouse supports TTL (Time-To-Live) policies for automatic data expiration or movement to cheaper storage. This is heavily used in observability and log systems where data retention is limited to a specific time window.
- Instead of traditional indexes, ClickHouse uses data-skipping indexes such as minmax, bloom filters, and set indexes. These indexes help skip irrelevant data blocks instead of pinpoint row-level access, making them efficient for analytical queries.
- ClickHouse is especially powerful for event analytics, logs, metrics, security analytics, and real-time dashboards. It is commonly integrated with Kafka, where data flows from Kafka into ClickHouse via Kafka engine tables and materialized views into MergeTree tables.
- However, ClickHouse is not suitable for heavy transactional systems. Updates and deletes are expensive because data is immutable, and mutations require rewriting data parts asynchronously. While updates are supported (e.g., via ReplacingMergeTree), they are not instantaneous and should not be used for frequent OLTP-style operations.

- A key design best practice is choosing the correct ORDER BY key. It should be based on common filter conditions and time-series patterns to maximize data skipping. Poor ORDER BY design leads to full scans and degraded performance.
- Memory management is important in ClickHouse, especially for aggregations and joins. Large GROUP BY operations may spill to disk if memory limits are exceeded. Although joins are supported, ClickHouse performs best with denormalized data models or pre-aggregated structures.
- Common table engines include MergeTree (base engine), ReplacingMergeTree (deduplication), SummingMergeTree (aggregation), AggregatingMergeTree (state-based aggregation), and ReplicatedMergeTree (replication-enabled storage). Each engine is optimized for specific analytical patterns.

## NATS/Jetstream

- NATS (Neural Autonomic Transport System) is a messaging system that allows different parts of your software to communicate with each other. It delivers messages between services without those services needing to know each other's location or availability.
- At the heart of NATS is subject-based messaging. In NATS, you don't create queues or topics; instead, you publish messages to subjects, which are simple strings describing what the message is about, for example "orders.created" or "users.login.success". These subjects don't need to be created beforehand; they exist simply because you use them. Think of subjects like email addresses. You don't register them before sending an email; you just send to them, and if someone is listening they receive it, otherwise the message goes nowhere.
- Subjects can also be hierarchical, such as "orders.us.california.store1.created" or "orders.europe.uk.london.updated". This hierarchy is not just organizational; it enables powerful pattern matching.
- NATS supports two wildcards. The asterisk (\*) *wildcard matches exactly one token in the subject hierarchy*. So "orders..created" matches "orders.us.created" or "orders.europe.created", but not

“orders.us.california.created” because it has more tokens. The greater-than (>) wildcard matches one or more tokens at the end, so “orders.us.>” matches “orders.us.created”, “orders.us.california.store1.created”, and anything starting with “orders.us.”.

- (Publish-Subscribe Model): The core publish-subscribe model works like this: a publisher sends a message to a subject, and subscribers receive messages that match that subject. In core NATS, messages exist only at the moment of publication. If subscribers are present, they receive copies immediately; if not, the message is dropped. This fire-and-forget nature makes NATS extremely fast, but it also means there is no persistence.
- Queue groups allow load balancing across multiple instances of the same service. When multiple subscribers join the same queue group for a subject, NATS delivers each message to only one member of the group, chosen randomly. This makes horizontal scaling of services very simple.
- A NATS cluster is a group of NATS servers working together as a unified system. From the client’s perspective, it behaves like a single server. The client connects to one node, and if it fails, it automatically reconnects to another server in the cluster. Inside the cluster, servers communicate using a gossip protocol. When a message is published, the server forwards it to other servers that have subscribers interested in that subject, ensuring efficient distribution across the system.
- Core NATS provides at-most-once delivery because it does not store messages. Once a message is sent, it is not retained.
- JetStream is NATS’s distributed persistence layer. It adds storage, replay, and guaranteed delivery while remaining integrated into the same NATS server, so no separate system is required.
- A stream in JetStream is an append-only log of messages. It is configured to capture messages published to specific subjects, and every matching message is stored according to defined retention policies. Streams can store data in memory or on disk depending on configuration.
- Consumers define how messages are read from streams.
  - Push consumers behave like traditional pub-sub, where JetStream actively pushes messages to the application as they arrive.

- Pull consumers work differently: the application explicitly requests messages when ready, for example asking for the next batch, which gives precise control over flow and backpressure.
  - Ordered consumers are a special case designed for strict ordering guarantees without requiring acknowledgments, simplifying use cases where ordering is critical.
- JetStream supports acknowledgments to track message processing. While it helps reliability, exactly-once processing is not automatic and must be handled carefully at the application level.
- Message deduplication is supported using message IDs. If a message with the same ID is published within a deduplication window, JetStream discards it, preventing accidental duplicates.
- Consumers can also filter messages based on subject or headers, allowing fine-grained control over what each consumer receives.
- Durable consumers maintain state, including acknowledged messages. If a consumer disconnects and reconnects, it can resume from where it left off using the same durable name.
- Ephemeral consumers exist only as long as there's an active subscription to them. When the last subscription is closed, the consumer is automatically deleted along with all its state. Ephemeral consumers are useful for temporary operations like debugging or for real-time processors that don't need to maintain state across restarts.
- When you create a stream, you specify how many replicas you want. A replication factor of one means the stream is stored on a single server with no replication. A replication factor of three means the stream is stored on three different servers. JetStream uses the Raft consensus algorithm for replication. For each stream, one node acts as leader and handles all writes, which are then replicated to followers. A write is confirmed only when a majority of replicas acknowledge it.
- A key-value store in JetStream is implemented on top of streams. Each key operation is stored as a message, and the latest message for a key represents its current value. This gives full history, auditability, and the ability to watch changes in real time.
- Similarly, the object store is built on streams and is used for large data such as files or machine learning models. Large objects are automatically

chunked into messages and reassembled when retrieved, making storage and retrieval transparent while still benefiting from JetStream's durability and replication guarantees.

## CSE Fundamentals

- A Web Server receives HTTP requests, serves static content, and forwards dynamic ones to application frameworks (FastAPI, Spring, etc) through interfaces like WSGI (Web Server Gateway Interface, synchronous) or ASGI (Asynchronous Server Gateway Interface, async). In Python, Gunicorn + UvicornWorker runs ASGI apps like FastAPI, similar to how Tomcat hosts Spring apps via the Servlet API in Java.
  - WSGI apps block per request (single thread per request).
  - ASGI apps are event-driven — ideal for concurrent tasks like WebSockets, real-time APIs, or heavy I/O workloads.

Consider environment:

You have FastAPI app (async-based).

Running under Gunicorn, with `--worker-class uvicorn.workers.UvicornWorker` (ASGI worker).

You configured 4 Gunicorn workers.

Each worker handles requests to 3 DynamoDB tables, via aioboto3 async clients.

Machine: 4 cores, 16 GB RAM.

Deployed as 2 ECS tasks, behind a load balancer.

How the System Handles a Request (Step by Step)

A client sends an HTTP request → LB (like ALB) forwards it to one of the ECS tasks.

You have 2 ECS tasks, each running 4 Gunicorn workers.

→ total of 8 worker processes across your cluster.

Each worker is a separate process, each having its own GIL and own asyncio event loop.

Gunicorn manages process-level parallelism.

Each Gunicorn worker:

Runs UvicornWorker, which spins up an ASGI server and event loop (asyncio).



Each worker runs in its own process (→ independent GIL per worker).

Each worker can handle many concurrent async requests (not just one).

Concurrency model is multi-process (Gunicorn) + async inside each process (Uvicorn).

When a request hits FastAPI, it's handled by an async endpoint.

Python is not truly multi-threaded for CPU-bound tasks (due to GIL),

but async I/O is cooperatively multitasked within the same thread.

That means: Only one coroutine runs on the CPU at a time,

But when it hits an I/O wait (like network call to DynamoDB),

→ it awaits the operation,

→ releases control back to the event loop,

→ and allows another coroutine (request handler) to run.

So while your coroutine is waiting for Dynamo to respond, the same thread can process other requests.

The Global Interpreter Lock (GIL) only affects Python bytecode execution (CPU work).

It does not block I/O waits.

Since your workload is I/O-bound (network to DynamoDB), GIL doesn't degrade performance much.

So total concurrency capacity =

ECS tasks \* Gunicorn workers \* concurrent coroutines per worker.

E.g. if each worker handles 500 concurrent connections efficiently,

→ you can handle  $2 \times 4 \times 500 = 4000$  concurrent connections.

More about these connections:

When a client (e.g., browser, mobile app, or another backend) makes an HTTP request:

It opens or reuses a TCP connection (possibly HTTP/1.1 keep-alive or HTTP/2 stream).

The ASGI server (Uvicorn running inside each Gunicorn worker) accepts this socket.

It passes control of that connection to your FastAPI app's coroutine.

In async mode:

Each connection corresponds to one coroutine (async task).

The event loop can hold hundreds or thousands of coroutines concurrently, because each coroutine is paused most of the time (waiting for I/O like DynamoDB responses).

Unlike traditional synchronous servers (where one thread handles one request),

async servers don't block threads during I/O waits.

Even though it's all on one thread, you can have hundreds of outstanding requests.

What limits concurrency?

File descriptors (ulimit -n): Max open sockets

ASGI server max\_connections: Max concurrent connections per worker

Memory per connection: Each request has context data & response buffer, Eg: 100KB per req.

Network I/O speed: Bandwidth, latency to DynamoDB

Event loop scheduling overhead: Async tasks per loop

If multiple clients create more simultaneous requests and if all workers are full, Gunicorn starts queueing or rejecting new connections (HTTP 503 or timeout).

A single client (like a browser) usually reuses a few persistent TCP connections (keep-alive).

Also, we have two layers of async in your setup:

Gunicorn + UvicornWorker → runs FastAPI as an ASGI async web server

aioboto3 → makes async network I/O calls to DynamoDB

Both are async, but they're not "separate async systems" – they're running inside the same event loop.  
If inner logic been sync - it would block thread during API call, impacting latency.

- Core is an individual physical processing unit, while threads are logical/virtual sequences of instructions. If a CPU has more threads than cores, say 4 cores, 8 threads so 2 threads per core, in such a case, the core would be context switching between each thread to process the task. 1 thread per core will get you the best performance. Adding more threads usually helps, but after some point, they cause some performance degradation.
- CPU-intensive vs IO-intensive tasks:
  - CPU-intensive tasks involve complex calculations, mathematical operations, data processing, or simulations that consume a substantial amount of CPU time.
  - Thread-Intensive (I/O-Bound) processes are characterized by tasks that spend a significant amount of time waiting for I/O operations to complete, such as reading/writing to files, making network requests, or interacting with databases. Many I/O operations are inherently blocking, which means they can cause a thread to wait for external resources (e.g., network data, disk I/O) to become available. During this waiting period, the thread is effectively idle, and the system may benefit from switching to another thread to perform useful work.
- CPU vs GPU:
  - CPU: Has few cores (say 4–16) but each is very powerful and complex. It focus on low latency per task – doing one task really fast. Each core has:
    - Large control logic and branch prediction units (for "if/else", loops).
    - Large caches (L1, L2, L3) for low-latency memory access.
    - Out-of-order execution and speculative execution for efficiency.

- GPU: Has hundreds or thousands of simpler cores (Streaming Multiprocessors in NVIDIA). Each “core” can run thousands of threads simultaneously. Very small control logic — most transistor area is used for arithmetic units (ALUs). High memory bandwidth (GDDR / HBM) but less cache per core. Executes code in SIMD / SIMT (Single Instruction, Multiple Data / Thread) fashion: All threads in a “warp” (group of ~32) execute the same instruction in parallel on different data.
- Concourse:
  - A pipeline defines your workflow — how code moves from build → test → deploy.
  - Each job represents a unit of work — e.g., build, test, deploy. Has steps: get, task, put.
  - Resources are external inputs/outputs that Concourse interacts with. Eg: Git repo, S3, Docker, etc.
  - Task: The actual work done — e.g., run tests, build code. A task runs inside a container, defined by a task.yml.
  - Workers are machines (VMs or containers) that run tasks. Each task in a pipeline runs in a container on a worker.
  - Web UI → visual view of pipelines, jobs, and status. Fly CLI → used to interact with Concourse (login, set pipelines, trigger builds).
  - Internally: Web Node (UI), Worker Node (Runs tasks), Postgres DB (Pipeline/resource details), ATC (Air Traffic Controller - Concourse scheduler), Garden / Containerd (Container runtime engine to isolate tasks).
- Tables:
  - Use **external tables** when data lives in **S3 / external systems** (so you don't lose it).
    - MSCK REPAIR TABLE my\_table: Automatically discovers new partitions by scanning the S3 directory structure of a Hive/Glue table and adds them to the metastore. Adds new/missing partitions. Slow for large datasets, scans entire S3 folder tree. Works only for Hive-style partitioning (e.g., key=value directories).

- ALTER TABLE my\_table ADD PARTITION (date='2025-10-24', country='IN') LOCATION: Adds the exact partitions you mention & not full scan, faster than msck command
  - Use **internal tables** when the warehouse manages lifecycle fully.
- Concurrency vs Parallelism:
  - Concurrency: Doing many things at once (logically) – tasks start, run, and finish in overlapping time, but not necessarily simultaneously. Eg: A single-core CPU switching between tasks A and B so quickly that both seem to run together.
  - Parallelism: Doing many things literally at the same time, using multiple cores or processors. Eg: Task A runs on Core 1, Task B runs on Core 2, truly at the same instant.
- Explode vs Normalize: JSON explode creates more rows by expanding a column with lists, while JSON normalize creates more columns by flattening nested JSON objects within a column.

Consider the df:

|   | id | name  | scores       |
|---|----|-------|--------------|
| 0 | 1  | Alice | [90, 95, 88] |
| 1 | 2  | Bob   | [85, 92]     |

```
df_exploded = df.explode('scores')
```

The df\_exploded DataFrame after applying JSON explode:

|   | id | name  | scores |
|---|----|-------|--------|
| 0 | 1  | Alice | 90     |
| 0 | 1  | Alice | 95     |
| 0 | 1  | Alice | 88     |
| 1 | 2  | Bob   | 85     |
| 1 | 2  | Bob   | 92     |

```
df_normalized = pd.json_normalize(df, 'scores', ['id', 'name'], sep='_')
```

The df\_normalized DataFrame after applying JSON normalize:

|   | id | name  | scores_0 | scores_1 | scores_2 |
|---|----|-------|----------|----------|----------|
| 0 | 1  | Alice | 90       | 95       | 88       |
| 1 | 2  | Bob   | 85       | 92       | NaN      |

- Database:

- Schemas:
  - Database Schema: It refers to the overall blueprint or logical structure of a database including tables, views, relationships, constraints, indexes, and other database objects
  - Table Schema: Refers to the structure of an individual table within the database. It defines the columns (fields), data types, constraints, and other attributes.
- Processing:
  - Online analytical processing (OLAP) is to analyze aggregated data.
    - Data Storage: Column-by-column (each column stored separately). Eg: SELECT AVG(balance) FROM customer;
      - Database reads only the Balance column, which is stored contiguously on disk and compressed efficiently.
  - Online transaction processing (OLTP) is to process database transactions.
    - Data Storage: Row-by-row (all fields of a record together)
- Normalization: It is to avoid complexities, eliminate duplicates, and organize data in a consistent way. 1NF - no duplicated rows or columns, 2NF - should be 1NF + no partial dependency, i.e, all non-key attributes are fully dependent on a primary key, 3NF - be in 2NF + no transitive partial dependency, etc.
- Denormalization attempts to improve read performance at the expense of some write performance. Redundant copies of the data are written in multiple tables to avoid expensive joins.
- Cardinality: Uniqueness of data contained in a column. Eg: id - high cardinality.
- ACID is a set of properties of relational database transactions.
  - Atomicity - Each transaction is all or nothing
  - Consistency - Any transaction will bring the database from one valid state to another
  - Isolation - Executing transactions concurrently has the same results as if the transactions were executed serially

- Durability - Once a transaction has been committed, it will remain so
- Federation sharding: Federation (or functional partitioning) splits up databases by function. For example, instead of a single, monolithic database, you could have three databases: forums, users, and products, resulting in less read and write traffic to each database and therefore less replication lag.
- Reasons for choosing:
  - SQL db: Structured data; Strict schema; Relational data; Need for complex joins; Transactions; Clear patterns for scaling; More established: developers, community, code, tools, etc; Lookups by index are very fast
  - NoSQL db: Semi-structured data; Dynamic or flexible schema; Non-relational data; No need for complex joins; Store many TB (or PB) of data; Very data intensive workload; Very high throughput for IOPS
- Database Indexing: Index is just a data structure that makes the searching faster for a specific column in a database. Usually a b-tree or a hash table structure. Creating an index on a field in a table creates another data structure which holds the field value, and a pointer to the record it relates to. This index structure is then sorted, allowing Binary Searches to be performed on it. Eg: Books - Pages - Table of Contents, etc.
  - Data on Disk: Data is stored in blocks (pages) — the smallest unit of disk I/O. Each block holds multiple records and includes pointers to other blocks (like a linked list). Blocks need not be contiguous in storage.
  - Without Index: Searching requires scanning blocks sequentially (linear search). Average cost:  $(N+1)/2$  block accesses (where  $N$  = number of blocks). For non-key fields (duplicates allowed), entire table may need to be scanned ( $\approx N$  accesses).
  - With Sorted Field / Index: Enables binary search, reducing cost to  $\log_2 N$  block accesses. Indexes store sorted mappings

of key → block pointer. Allows direct lookup instead of full-table scan.

- Trade-off: Indexes speed up reads/searches, but slow down writes (INSERT/UPDATE/DELETE must also update the index). They also consume extra disk space.
- View is a virtual table whose contents are defined by a query. You can think of a view as a "saved select statement" that you can repeat.
  - Materialized VIEW: It is a special type of view that stores the result of a query in a physical table. Unlike a regular view, a materialized view does not update automatically when the underlying tables or views change. Instead, you have to refresh it manually or on a schedule using the REFRESH MATERIALIZED VIEW statement.
- Query Federation in Databricks: Query federation allows Databricks to execute queries against data served by other Databricks metastores as well as many third-party database management systems (DBMS) such as PostgreSQL, MySQL, AWS Redshift and Snowflake. To query data from another system you must: Create a foreign connection., etc. Simply put, from databricks UI itself you can run queries over external/third-party db post necessary connections are set up.
- Thread/Process-Core Model Py
  - Thread-per-core model: Single process contains multiple threads, with one thread assigned to each CPU core. Shared address space means all threads can access the same memory. Lightweight synchronization through mutexes, semaphores, etc. Good for: I/O-bound workloads with shared state, Applications where memory efficiency is critical, Languages with efficient threading models (Java, C++)
  - Process-per-core model: Multiple independent processes, with one process assigned to each CPU core. Separate memory spaces with no direct access between processes. More robust isolation prevents cascading failures. Good for: CPU-bound workloads, Languages with threading limitations (Python with GIL), Applications requiring



strong fault isolation, Avoiding complex thread synchronization issues.

- If I run 1 process with 4 threads for CPU-heavy work, due to GIL only one thread runs/executes python bytecode at a time on a core (time-sliced), remaining cores are idle. But if I run 4 processes, each on one core, they can run in parallel and be faster. Note that each process has its own Python interpreter and GIL.
- A thread is the unit of execution within a process. A process has at least one thread. It is called the main thread. A process can have many threads. Each thread has its own stack memory, registers, program counters, etc.
- Threads within a process share a memory address space. It is possible to communicate between threads using that shared memory space. However, one misbehaving thread could bring down the entire process. The operating system run a thread or process on a CPU by context switching.
- Context switching is expensive. It involves saving and loading of registers, switching out memory pages, and updating various kernel data structures. It is generally faster to switch context between threads than between processes. There are fewer states to track, and more importantly, since threads share the same memory address space, there is no need to switch out virtual memory pages, which is very expensive.
- In case of Python:
  - Python 3.13+ introduced an experimental "free-threaded" mode. In this mode, the Global Interpreter Lock is removed, allowing multiple threads within a single process to execute Python bytecode in true parallel across multiple CPU cores.
    - Past (GIL-bound Python): Even on a 4-core machine, Python behaved like a single-lane road on a four-lane highway. Only one thread could execute Python bytecode at a time due to the GIL, so one core was busy while the other three cores sat idle, even within the same process.

- Present (Python 3.13 – Free-threaded mode option):  
The highway is fully open. In a single process, 4 threads can truly run in parallel, each executing on its own core, allowing all 4 cores to be utilized simultaneously.
- You must explicitly run `--disable-gil` / free-threaded build.
- True parallelism applies to CPU-bound Python code. Many C extensions are not yet thread-safe - Hence as a note while the interpreter handles internal safety, remember that your own code still needs to be "thread-safe" using mutexes if multiple threads are modifying the same global variable at once.
- Generators vs Co-routines in Py
  - Generators:
    - Lazy iteration – produce a sequence of values on demand, without storing them in memory.
    - Each call to `next()` resumes from the last yield point. Execution pauses and resumes automatically.
    - Defined using: `def + yield`. Use: Streaming large datasets, pipelines, infinite sequences, reading large files, etc.

A generator is just a special kind of function that can pause and resume. Normally, when you call a function: It runs from start to end; Returns a value once (via `return`); After that, it's done -- memory is freed. But with a generator, when it hits a `yield`, it pauses instead of finishing. It remembers its position, local variables, etc. The next time you ask for a value, it resumes from where it left off.

Eg1:

```
def countdown(n):
 while n > 0:
 yield n # pause here and give back n
 n -= 1 # resume from here next time

gen = countdown(3)
print(next(gen)) # prints 3
print(next(gen)) # prints 2
```

```

print(next(gen)) # prints 1
print(next(gen)) # raises StopIteration

We use it as it doesn't store all values in memory.

Eg2:
with open("huge_file.txt") as f:
 lines = f.readlines() # ❌ Loads entire file into memory
 for line in lines:
 process(line)

```

Instead (below): The generator reads one line from disk at a time. After processing it, the memory is freed. Your app can handle huge files with constant memory usage (a few KBs).

```

def read_large_file(file_path):
 with open(file_path) as f:
 for line in f:
 yield line.strip() # ⚡ yield one line at a time
for log_line in read_large_file("huge_file.txt"):
 process(log_line)

```

- Coroutines:
  - Asynchronous I/O — allow tasks to pause while waiting for I/O, freeing the event loop to run others.
  - await marks points where coroutine yields control to the event loop. The event loop schedules and resumes them when I/O completes. They can run concurrently, though not in parallel (within one thread).
  - Defined using: `async def` + `await`. Use: Network calls, DB queries, file I/O — any I/O-bound workload.
- HTTP:
  - HTTP/1.1 (1999 — Traditional Web)
    - Transport: TCP
    - Text-based protocol — requests and responses are human-readable.
    - Each request-response pair uses a separate TCP connection (in HTTP/1.0).

- HTTP/1.1 added persistent connections (keep-alive), so multiple requests reuse one connection sequentially.
- Still head-of-line (HOL) blocking at the HTTP layer — only one request in flight per connection; others must wait.
  - Head of Line blocking in HTTP terms is often referring to the fact that each browser/client has a limited number of connections to a server and doing a new request over one of those connections has to wait for the ones to complete before it can fire it off. The head of line requests block the subsequent ones. HTTP/2 solves this by introducing multiplexing so that you can issue new requests over the same connection without having to wait for the previous ones to complete. In theory, the pipelining of HTTP/1.1 also offered a way around HOL, but it was tricky and very error-prone to implement in practice. That has made it not get widely enabled on the web even up till today.
- Workarounds: browsers opened 6–8 parallel TCP connections per domain to simulate concurrency.
- Problem: Latency from TCP slow start + blocking + redundant headers (same headers repeated each time).
- HTTP/2 (2015 — Modern Web)
  - Transport: Still TCP
  - Binary protocol — efficient, compact framing (not text-based).
  - Multiplexing: multiple requests and responses can share a single TCP connection concurrently — removes HTTP-level HOL blocking.
  - Header compression (HPACK): reduces redundancy in repeated headers (e.g., cookies).
  - Server Push: server can proactively send resources client will likely need.
  - Remaining limitation: Still suffers from TCP-level head-of-line blocking — if one packet is lost, all streams in that connection stall until it's retransmitted.

- HTTP/3 (2022 – Future of Web)
  - Transport: QUIC (built on UDP)
  - QUIC provides built-in encryption (TLS 1.3) and multiplexing at transport layer – no TCP involved.
  - No TCP-level HOL blocking: each stream runs independently, so packet loss on one stream doesn't stall others.
  - Faster connection setup: 0-RTT or 1-RTT handshake vs 3-way TCP + TLS handshake in HTTP/1/2.
  - Better mobility: connection identified by connection ID (not IP), so if you switch networks (e.g., Wi-Fi → mobile), connection stays alive.
  - Result: HTTP/3 is faster for real-world web browsing, especially over unreliable or mobile networks.
- Race condition: Occurs when system attempts to perform two or more operations at the same time, could lead to issues if due to nature of the system, the operations must be done in the proper sequence.
- [Order of the loops affect performance when iterating over a 2D array](#):

Version 1 (slow)

```
main () {
 int i, j;
 static int x[4000][4000];
 for (j = 0; j < 4000; j++) {
 for (i = 0; i < 4000; i++) {
 x[i][j] = i + j; }
 }
 }
```

Version 2 (fast)

```
main () {
 int i, j;
 static int x[4000][4000];
 for (i = 0; i < 4000; i++) {
 for (j = 0; j < 4000; j++) {
```

```

 x[i][j] = i + j; }
 }
}

```

The issue is the store to the memory location in the array: `x[i][j]`. You have a 2-dimensional array, but memory in the computer is inherently 1-dimensional. So while you imagine your array like this:

```

0,0 | 0,1 | 0,2 | 0,3
----+----+----+----
1,0 | 1,1 | 1,2 | 1,3
----+----+----+----
2,0 | 2,1 | 2,2 | 2,3

```

Your computer stores it in memory as a single line:

```

0,0 | 0,1 | 0,2 | 0,3 | 1,0 | 1,1 | 1,2 | 1,3 | 2,0 | 2,1 | 2,2...

```

In the 2nd version (fast), you access the array by looping over the 2nd number first, i.e.: `x[0][0]` `x[0][1]` `x[0][2]` `x[0][3]` `x[1][0]` etc... ...Meaning that you're hitting them all in order.

In first case, it's jumping here and there.

Data from your memory gets brought over to the CPU in little chunks (called 'cache lines'), typically 64 bytes. If you have 4-byte integers, that means you're getting 16 consecutive integers in a neat little bundle. It's actually fairly slow to fetch these chunks of memory; your CPU can do a lot of work in the time it takes for a single cache line to load.

Now look back at the order of accesses:

- The second "fast" version is (1) grabbing a chunk of 16 ints, (2) modifying all of them, (3) repeat  $4000 \times 4000 / 16$  times. That's nice and fast, and the CPU always has something to work on.
- The first "slow" version is (1) grab a chunk of 16 ints, (2)

modify only one of them, (3) repeat 4000\*4000 times. That's going to require 16 times the number of "fetches" from memory. Your CPU will actually have to spend time sitting around waiting for that memory to show up, and while it's sitting around you're wasting valuable time.

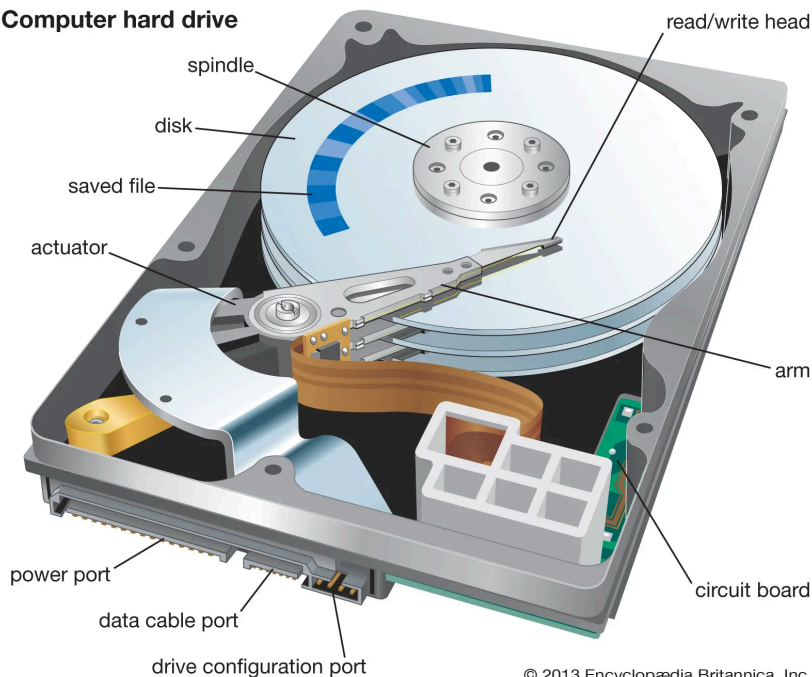
- Medallion Architecture is a data design pattern used to logically organize data in a lakehouse, with the goal of incrementally and progressively improving the structure and quality of data as it flows through each layer of the architecture (from Bronze  $\Rightarrow$  Silver  $\Rightarrow$  Gold layer tables).
- Namespace: It is a declarative region that provides a scope to the identifiers (the names of types, functions, variables, etc) inside it.
- [Why is processing a sorted array faster than processing an unsorted array](#)  
[\\_al](#): It involves concept of Branch prediction. Branch prediction attempts to guess whether a conditional jump will be taken or not. Branch target prediction attempts to guess the target of a taken conditional or unconditional jump before it is computed by decoding and executing the instruction itself. It is a technique to predict the outcome of a conditional operation.
- Ephemeral storage is a temporary, non-persistent storage space for a compute instance that is erased when the instance is terminated.
- A bastion host is a publicly facing server that acts as an entry-point to the system which is protected from the high-end firewall or located in a private server.
- C++ vs Java Code execution differences:

| Aspect      | C/C++                                                                                       | Java                                                                                     |
|-------------|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Compilation | Source code $\rightarrow$ compiled by a native compiler $\rightarrow$ machine code (binary) | Source code $\rightarrow$ compiled by <code>javac</code> $\rightarrow$ bytecode (.class) |
| Execution   | Executed directly by CPU (hardware-dependent).                                              | Executed by JVM (Java Virtual Machine). "Write Once, Run                                 |

|        |                                                                                                                  |                                                                                                                                                                                                    |
|--------|------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|        | Compiled binary is not portable across architectures or OSes.                                                    | Anywhere” – JVM handles platform differences.                                                                                                                                                      |
| Target | Architecture-specific (x86, ARM, etc.)                                                                           | Platform-independent (JVM provides abstraction)                                                                                                                                                    |
| Other  | Precompiled machine code → near-native performance.<br>Manual memory management using malloc/free or new/delete. | JVM uses JIT (Just-In-Time) Compilation → dynamically compiles frequently executed (“hot”) code to native machine code. Can perform runtime optimizations. Garbage collector takes care of memory. |

- P99 latency: 99th percentile latency; It tells you that 99% of requests are faster than this number, and only the slowest 1% take longer. In distributed systems, tail latencies dominate user experience.
- Storage Evolution: From CHS to LBA to ZNS

#### Computer hard drive



© 2013 Encyclopædia Britannica, Inc.

- Disk components:



- Platters: Several circular disks stacked vertically. Each coated with magnetic material
  - Spindle: Rotates all platters at high speed
  - Tracks: Concentric circles on platter
  - Head = Which platter surface
  - Sectors: Each track divided into 512B / 4KB sectors
  - Cylinder: Stack of matching tracks across platters
- Early Era — CHS (Cylinder, Head, Sector) Addressing:
  - OS accessed drives via physical geometry: (cylinder, head, sector) → exact physical location.
  - Problem: Tight coupling between OS and hardware. Any change in drive geometry required OS and driver updates.
- Abstraction Layer — LBA (Logical Block Addressing)
  - Replaced CHS with a logical view: drive = flat array of blocks (LBA 0..N). The drive firmware (controller) internally maps LBAs → physical locations.
  - Pros: Platform independence; Cons - Added indirection overhead in mapping table, etc.
- Modern Revisit — ZNS (Zoned Namespace) NVMe
  - Idea: Reintroduce partial geometry awareness back to the OS. Drive divided into Zones — groups of LBAs guaranteed to be physically contiguous.
  - Rules: Writes must be sequential within a zone. A zone must be erased entirely before reuse. Drive still does mapping, but only zone-level, not block-level (smaller mapping table).
- Eg: RocksDB, LevelDB (LSM-based stores) benefit because their design is append-only / sequential-write heavy, matching ZNS semantics.
- Docker Image Optimization: Faster CI/CD pipelines, reduced network transfer, quicker scaling on K8s, lower storage costs across registries (ECR), better security, etc. Some tips:
  - Base Image Choice — Minimal Alpine: Eg: Switch from python:3.9 (~900 MB) → python:3.9-alpine (~40 MB). ~95% size reduction.

- Layer Minimization: Every RUN, COPY, ADD → new filesystem layer.  
Group related commands: RUN apk update && apk add --no-cache git && rm -rf /var/cache/apk/\*... etc.
- Use .dockerignore: Avoid copying unnecessary files (like .git/, venv/, \_\_pycache\_\_/).
- Place static layers (like requirements.txt) before dynamic ones (COPY .) to reuse cache efficiently. Eg:

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
This way, dependency install layer is reused unless
requirements.txt changes.
```

- Single-stage images include build tools (gcc, gfortran, build-base, musl-dev, etc.) → bloat image size. Hence do multi-stage builds.
  - Stage 1: Builder: Installs heavy build dependencies → Builds and installs Python deps (pip install) → Optionally removes large libs after wheels are built.
  - Stage 2: Production: Copies only built artifacts and runtime files → No compiler or headers → ultra-lean runtime.
- Decorators vs Annotations:
  - Decorators: Used to modify or enhance the behavior of code elements (functions, methods, classes) by wrapping them with additional functionality. Eg:

```
def my_decorator(func):
 def wrapper(*args, **kwargs):
 print("Something before the function.")
 result = func(*args, **kwargs)
 print("Something after the function.")
 return result
 return wrapper

@my_decorator
def say_hello():
 print("Hello!")
```

```
say_hello()
In this example, @my_decorator adds "Something before..." and
"Something after..." messages to the say_hello function's
execution without directly modifying say_hello itself.
```

- Annotations: Primarily used to add metadata to code elements (classes, methods, fields, parameters) without altering their core logic or behavior. Can be imagined as passive markers or tags. Eg:

```
@Override
public String toString() {
 return "MyObject";
}

// Other ex:
@RestController
class UserController {
 @GetMapping("/users")
 public List<User> getUsers() { ... }
}
```

DSA

## DSA Patterns:

<https://leetcode.com/discuss/post/5886397/dsa-patterns-you-need-to-know-by-anubhav-x7og/>

### 1. Fast and Slow Pointer

**Description:** This technique uses two pointers moving at different speeds to solve problems involving cycles, such as finding the middle of a list, detecting loops, or checking for palindromes.

- [Linked List Cycle II](#)
- [Remove nth Node from the End of List](#)
- [Find the Duplicate Number](#)
- [Palindrome Linked List](#)
- [\[Self Added v2\] Intersection of two Linkedlists](#)

---

### 2. Overlapping Intervals

**Description:** Intervals are often manipulated through sorting and merging based on their start and end times.

- [Basic Merge: Merge Intervals](#)
- [Interval Insertion: Insert Interval](#)
- [My Calendar ii](#)
- [Minimum Number of Arrows to Burst Balloons](#)
- [Non-overlapping Intervals](#)
- [\[Self Added v2\] Merge Sorted Array](#)
- [\[Self Added v2\] Meeting Rooms II](#)

---

### 3. Prefix Sum

**Description:** Prefix Sums/Products are techniques that store cumulative sums or products up to each index, allowing for quick subarray range queries.

- [Find the middle index in array](#)
- [Product of array except self](#)
- [Maximum product subarray](#)
- [Number of ways to split array](#)
- [Range Sum Query 2D](#)

---

## 4. Sliding Window

**Description:** A sliding window is a subarray or substring that moves over data to solve problems efficiently in linear time.

### Fixed Size

- [Maximum Sum Subarray of Size K](#)
- [Number of Subarrays having Average Greater or Equal to Threshold](#)
- [Repeated DNA sequences](#)
- [Permutation in String](#)
- [Sliding Subarray Beauty](#)
- [Sliding Window Maximum](#)

### Variable Size

- [Longest Substring Without Repeating Characters](#)
- [Minimum Size Subarray Sum](#)
- [Subarray Product Less Than K](#)
- [Max Consecutive Ones](#)
- [Fruits Into Baskets](#)
- [Count Number of Nice Subarrays](#)
- [Minimum Window Substring: Minimum Window Substring](#)

---

## 5. Two Pointers

**Description:** The two pointers technique involves having two different indices move through the input at different speeds to solve various array or linked list problems.

- [Two Sum II - Input Array is Sorted](#)
- [Dutch National Flag: Sort Colors](#)
- [Next Permutation](#)
- [Bag of Tokens](#)
- [Container with most water](#)
- [Trapping Rain Water](#)

---

## 6. Cyclic Sort (Index-Based)

**Description:** Cyclic sort is an efficient approach to solve problems where numbers are consecutively ordered and must be placed in the correct index.

- [Missing Number](#)
- [Set Mismatch](#)
- [Find Missing Numbers](#)
- [First Missing Positive](#)

---

## 7. Reversal & Manipulation of Linked List (In-place)

**Description:** Reversing a linked list in place without using extra space is key for problems that require in-place list manipulations.

- [Reverse Linked List](#)
- [Reverse Nodes in k-Group](#)
- [Swap Nodes in Pairs](#)
- [\[Self added v2\] Flatten LinkedList](#)

---

## 8. Matrix Manipulation

**Description:** Problems involving 2D arrays (matrices) are often solved using row-column traversal or manipulation based on matrix properties.

- [Rotate Image](#)
- [Spiral Matrix](#)
- [Set Matrix Zeroes](#)
- [Game of Life](#)

---

## 9. Breadth First Search (BFS)

**Description:** BFS explores nodes level by level using a queue. It is particularly useful for shortest path problems.

- [Shortest Path in Binary Matrix](#)
- [Rotten Oranges](#)
- [Word Ladder: Word Ladder](#)
- [As Far From Land as Possible](#)

---

## 10. Depth First Search (DFS)

**Description:** DFS explores as far as possible along a branch before backtracking. It's useful for graph traversal, pathfinding, and connected components.

- [Number of Closed Islands](#)
- [Coloring a Border](#)
- [DFS from boundary: Number of Enclaves](#)
- [Shortest time: Time Needed to Inform all Employees](#)
- [Cyclic Find: Find Eventual Safe States](#)

---

## 11. Backtracking

**Description:** Backtracking helps in problems where you need to explore all potential solutions, such as solving puzzles, generating combinations, or finding paths.

- [Permutation ii](#)
- [Combination Sum](#)
- [Generate Parenthesis](#)
- [N-Queens](#)
- [Sudoku Solver](#)
- [Palindrome Partitioning](#)
- [Word Search: Word Search](#)

---

## 12. Modified Binary Search

**Description:** A modified version of binary search that applies to rotated arrays, unsorted arrays, or specialized conditions.

- [Search in Rotated Sorted Array](#)
- [Find Minimum in Rotated Sorted Array](#)
- [Find Peak Element](#)
- [Single element in a sorted array](#)
- [Minimum Time to Arrive on Time](#)
- [Capacity to Ship Packages within 'd' Days](#)
- [Koko Eating Bananas](#)



- [Find in Mountain Array](#)
  - [Median of Two Sorted Arrays](#)
  - [\[Self added v2\] Magnetic force between two balls](#)
  - [\[Self added v2\] Split Array Largest Sum](#)
- 

### 13. Bitwise XOR

**Description:** XOR is a powerful bitwise operator that can solve problems like finding single numbers or efficiently pairing elements.

- [Single Number II](#)
  - [Single Number III](#)
  - [Find the Original array of Prefix XOR](#)
  - [XOR Queries of a Subarray](#)
- 

### 14. Top 'K' Elements

**Description:** This pattern uses heaps or quickselect to efficiently find the top 'K' largest/smallest elements from a dataset.

- [Top K Frequent Elements](#)
  - [Kth Largest Element](#)
  - [Ugly Number II](#)
  - [K Closest Points to Origin](#)
- 

### 15. K-way Merge

**Description:** The K-way merge technique uses a heap to efficiently merge multiple sorted lists or arrays.

- [Find K Pairs with Smallest Sums](#)
  - [Kth Smallest Element in a Sorted Matrix](#)
  - [Merge K Sorted Lists](#)
  - [Smallest Range: Smallest Range Covering Elements from K Lists](#)
- 

### 16. Two Heaps

**Description:** This pattern uses two heaps (max heap and min heap) to solve problems involving tracking medians and efficiently managing dynamic data.

- [Find Median from Data Stream](#)
- [Sliding Window Median](#)
- [IPO](#)

---

## 17. Monotonic Stack

**Description:** A monotonic stack helps solve range queries by maintaining a stack of elements in increasing or decreasing order.

- [\[Self Added\] Next Greater Element I](#)
- [Next Greater Element II](#)
- [Next Greater Node in Linked List](#)
- [Daily Temperatures](#)
- [Online Stock Span](#)
- [Maximum Width Ramp](#)
- [Largest Rectangle in Histogram](#)
- [\[Self added v2\] Find Celebrity](#)

---

## 18. Trees

Level Order Traversal (BFS in Binary Tree)

- [Level order Traversal](#)
- [Zigzag Level order Traversal](#)
- [Even Odd Tree](#)
- [Reverse odd Levels](#)
- [Deepest Leaves Sum](#)
- [Add one row to Tree](#)
- [Maximum width of Binary Tree](#)
- [All Nodes Distance K in Binary tree](#)

[Self added] Tree Traversals

- [\[Self added v2\] Left, Right, Top View of Binary Tree](#)

## Tree Construction

- [Construct BT from Preorder and Inorder](#)
- [Construct BT from Postorder and Inorder](#)
- [Maximum Binary Tree](#)
- [Construct BST from Preorder](#)

## Height related Problems

- [Maximum Depth of BT](#)
- [Balanced Binary Tree](#)
- [Diameter of Binary Tree](#)
- [Minimum Depth of BT](#)

## Root to leaf path problems

- [Binary Tree Paths](#)
- [Path Sum ii](#)
- [Sum Root to Leaf numbers](#)
- [Smallest string starting from Leaf](#)
- [Insufficient nodes in root to Leaf](#)
- [Pseudo-Palindromic Paths in a Binary Tree](#)
- [Binary Tree Maximum Path Sum](#)

## Ancestor problem

- [LCA of Binary Tree](#)
- [Maximum difference between node and ancestor](#)
- [LCA of deepest leaves](#)
- [Kth Ancestor of a Tree Node](#)

## Binary Search Tree

- [Validate BST](#)
- [Range Sum of BST](#)
- [Minimum Absolute Difference in BST](#)
- [Insert into a BST](#)
- [LCA of BST](#)

---

## 19. DYNAMIC PROGRAMMING

### Take / Not take (DP)

**Description:** Solve optimization problems like selecting items with the max/min value under certain constraints.

- [\[Self Added\] House Robber i](#)
- [House Robber ii](#)
- [Target Sum](#)
- [Partition Equal Subset Sum](#)
- [Ones and Zeroes](#)
- [Last Stone Weight ii](#)

### Infinite Supply (DP)

**Description:** Similar to the 0/1 knapsack, but items can be chosen multiple times.

- [Coin Change](#)
- [Coin Change II](#)
- [Perfect Squares](#)
- [Minimum Cost For Tickets](#)
- [\[Self added v2\] Rod Cutting Problem](#)

### Longest Increasing subsequence

**Description:** It involves finding the longest subsequence of a given sequence where the elements are in ascending order

- [Longest Increasing Subsequence](#)
- [Largest Divisible Subset](#)
- [Maximum Length of Pair Chain](#)
- [Number of LIS](#)
- [Longest String Chain](#)

### DP on Grids

**Description:** Dynamic Programming on matrices involves solving problems that can be broken down into smaller overlapping subproblems within a matrix.

- [\[Self Added\] Unique Paths i](#)
- [Unique Paths ii](#)
- [Minimum Path Sum](#)

- [Triangle](#)
- [Minimum Falling Path Sum](#)
- [Maximal Square](#)
- [Cherry Pickup](#)
- [Dungeon Game: Dungeon Game](#)

## DP on Strings

**Description:** It Involves 2 strings, whenever you are considering two substrings/subsequence from given two strings, concentrate on what happens when the last characters of the two substrings are same, i.e, matching.

- [Longest Common Subsequence](#)
- [Longest Palindromic Subsequence](#)
- [Palindromic Substrings](#)
- [Longest Palindromic Substrings](#)
- [Edit Distance](#)
- [Minimum ASCII Delete Sum for Two Strings](#)
- [Distinct Subsequences](#)
- [Shortest Common Supersequence](#)
- [Wildcard Matching](#)

## DP on Stocks

**Description:** It focuses on maximizing profit from buying and selling stocks over time while considering constraints.

- [\[Self Added\] Buy and Sell Stocks i](#)
- [Buy and Sell Stocks ii](#)
- [Buy and Sell Stocks iii](#)
- [Buy and Sell Stocks iv](#)
- [Buy and Sell Stocks with Cooldown](#)
- [Buy and Sell Stocks with Transaction fee](#)

## Partition DP (MCM)

**Description:** It Involves a sequence that needs to be divided into partitions in an optimal way. The goal is often to minimize or maximize a cost function, such as computation time, multiplications, or some other metric, by exploring all possible partitions and combining results from subproblems.

- [Partition array for Maximum Sum](#)
  - [Minimum Cost to Cut a Stick](#)
  - [Burst Balloons](#)
  - [Palindrome Partitioning ii](#)
  - [\[Self added v2\] Egg Dropping](#)
- 

## 20. Graphs

[Self added] Basic Questions

- [\[Self added v2\] Clone graph](#)
- [\[Self added v2\] Number of islands](#)

Topological Sort

**Description:** Topological sorting is useful for tasks that require dependency resolution (InDegree) in directed acyclic graphs (DAGs).

- [Course Schedule](#)
- [Course Schedule II](#)
- [Strange Printer ii](#)
- [Sequence Reconstruction](#)
- [Alien Dictionary](#)
- [\[Self added v2\] Cycle detection in Directed & Undirected graph](#)

Union Find (Disjoint Set)

**Description:** Union-Find (or Disjoint Set) is used to solve problems involving connectivity or grouping, often in graphs.

- [Number of Operations to Make Network Connected](#)
- [Redundant Connection](#)
- [Accounts Merge](#)
- [Satisfiability of Equality Equations](#)

Graph Algorithms

**Description:** Advanced graph algorithms are used to solve complex problems involving shortest paths, minimum spanning trees, and graph cycles.

- [Kruskal's Algorithm: Minimum Cost to connect all Points](#)
- [Dijkstra's Algorithm: Cheapest Flights Within K Stops](#)
- [Floyd-Warshall: Find the City with Smallest Number of Neighbours at a Threshold Distance](#)
- [Bellman Ford: Network Delay time](#)

---

## 21. Greedy

**Description:** Greedy algorithms make local optimal choices at each step, which lead to a global optimal solution for problems like scheduling and resource allocation.

- [Jump Game ii](#)
- [Gas Station](#)
- [Bag of Tokens](#)
- [Boats to Save People](#)
- [Wiggle Subsequence](#)
- [Car Pooling](#)
- [Candy](#)

---

## 22. Design Data Structure

**Description:** It involves building custom data structures to efficiently handle specific operations, like managing data access, updates, and memory usage. Focusing on optimizing performance and resource management.

- [Design Twitter](#)
- [Design Browser History](#)
- [Design Circular Deque](#)
- [Snapshot Array](#)
- [LRU Cache](#)
- [LFU Cache](#)

---

Some Useful Articles on LeetCode for Better Understanding!

## Two Pointers

- [Solved all Two Pointers problems in 100 days](#)

## Sliding Window

- [Sliding Window Technique and Question Bank](#)
- [C++ Maximum Sliding Window Cheatsheet Template!](#)

## Greedy

- [Greedy for Beginners: Problems & Sample Solutions](#)
- [Top Greedy Questions](#)

## Linked List

- [Become Master In Linked List](#)
- [Must-Do LinkedList Problems on LeetCode](#)

## Trees

- [Tree Question Pattern | 2021 Placement](#)
- [Master Tree Patterns](#)

## Binary Search

- [5 Variations of Binary Search](#)
- [Binary Search for Beginners: Problems & Patterns](#)

## Dynamic Programming (DP)

- [Dynamic Programming Patterns](#)
- [DP for Beginners: Problems & Patterns](#)

## Graphs

- [Graph For Beginners](#)
- [Become Master In Graph](#)
- [Graph algorithms + problems to practice](#)

## Bit Manipulation

- [Bit Manipulation Problem solving](#)
- [All Types of Patterns for Bits Manipulations and How to use it](#)



\_\_Patterns1

-----  
DS

**Vector** - pb, pop\_back, insert/erase(v.begin()+i, x), size(), empty, front->index0, back->indexN-1, sort/reverse-vBegin|vEnd.., clear

**String** - pb/pop\_back, insert/erase/replace, s.find("x")  
sort/reverse, stoi(x)/to\_string(x)

**Set** - insert(x)/ erase, find -> .end() to compare, size, empty, count(x) -> 1 or 0

**Map** - insert/ erase, find -> .end(), size, empty

**Queue** - push(x), pop, front, back, empty, size

**Stack** - push(x), pop, top, empty, size

**Heap** - priority\_queue<int> pq / priority\_queue<int, vector<int>, greater<int>>, push, pop, top, empty, size

**Deque** (double ended) - push-pop\_front-back, erase, empty, size

**List** - similar functions as deque + list1.splice(pos, list2, first, last) - moves range  
-----

## Pointers

| Context                             | Meaning                       |
|-------------------------------------|-------------------------------|
| Declaration ( <code>int* p</code> ) | "p is a pointer"              |
| Usage ( <code>*p</code> )           | "go to address and get value" |

(\*head).next OR using head->next; Both are exactly the same.  
-----

## General bfs

Shortest path in matrix

queue<pair<int,int>> q, push 0,0 starting node, have a dirs[8][2] for directions, while() then for(), pair<int,int> p check in while loop, dist=grid[nr][nc] store the distance in cell - which you use in for loop for 8 directions to push new pair in queue if condition met and store the dist+1 value in the grid cell.

Rotten oranges

Same queue pair, increment fresh++ else store the rotten oranges, then while() then while(size--) here i also do minutes++, then for() in 4 dirs

```
int orangesRotting(vector<vector<int>>& grid) {
 int m = grid.size(), n = grid[0].size();
 queue<pair<int,int>> q; int fresh = 0;
 for (int i=0;i<m;i++) {
 for (int j=0;j<n;j++) {
 if (grid[i][j] == 2) q.push(make_pair(i,j));
 else if (grid[i][j] == 1) fresh++;
 }
 }
 if (fresh == 0) return 0;
 int minutes = -1;
 int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
 while (!q.empty()) {
 int size = q.size();
 minutes++;
 while (size-->0) {
 pair<int,int> p = q.front();
 q.pop();
 int r = p.first, c = p.second;
 for (int i=0;i<4;i++) {
 int nr = r + dirs[i][0];
 int nc = c + dirs[i][1];
 if (nr<0||nr<m||nc<0||nc<n||grid[nr][nc] == 1) {
 grid[nr][nc] = 2;
 fresh--;
 q.push(make_pair(nr,nc));
 }
 }
 }
 }
 return (fresh == 0 ? minutes : -1);
}
```

Word ladder

Use set string to store the wordlist begin/end, queue string int pair, string has the word, int has the counter, while() then double for() -> to iterate over char 'a' to 'z' char++, if the word i get from queue front(), match then erase() from set and push in queue<tempword, steps+1>.

Far from land

Same story, store pairs of i,j in queue where value in grid=1, then while() dist++ -> Inner while() -> for () for directions, and making pairs if(nr range [0,m) and nc range [0,n) and gridij==0...

-----

General dfs

Number of closed islands

for()->for() to iterate over grid, and if gridij==0 then we do a dfs - if dfs true then ++res else skip, coming in dfs, i do bool check in all 4 directions dfs(grid, i+-1, j+-1) and d1&&d2&&d3&&d4, base conditions are if i,j<0 or i,j>grid.size, then return false, or if gridij visited with 2 then false, and for every visit i modify cell value of grid to 2.

```
int closedIsland(vector<vector<int>>& grid) {
 int res = 0;
 for (int i = 0; i < grid.size(); i++){
 for (int j = 0; j < grid[0].size(); j++){
 if (grid[i][j] == 0){
 res += dfs(grid, i, j) ? 1 : 0;
 }
 }
 }
 return res;
}

bool dfs(vector<vector<int>>& g, int i, int j){
 if (i < 0 || j < 0 || i >= g.size() || j >= g[0].size()){
 return false;
 }
 if (g[i][j] == 1 || g[i][j] == 2){
 return true;
 }
 g[i][j] = 2;
```

```

 bool down = dfs(g, i+1, j);
 bool right = dfs(g, i, j+1);
 bool up = dfs(g, i-1, j);
 bool left = dfs(g, i, j-1);
 return up && down && left && right;
}

```

Coloring border - skip

Number of enclaves

Same like above, let dfs keep coloring the graph, in main(), call dfs for boundaries first to color them -> j[0, m) for dfs(0,j,grid) & dfs(n-1, j, grid) later i[0,n) for dfs(i,0,grid) & dfs(i,m-1,grid), later for()->for() and if gridij==1 ans++.

Time needed to inform employees

Create the graph from [manager[i],i], return dfs which will iterate over adj in for() to get maxtime=max(maxtime, dfs()), and return informtime[u] + maxtime, maxtime is initialized as 0 before the for().

Find eventual safe states

Same as finding cycle in graph, if(!vis[i]) call dfs, in dfs, if no cycle then safenode[node]=1, which will be ans.

-----

Design DS

Twitter - skip

Browser history - skip

Circular Deque - skip

Snapshot array - skip

LFU cache - skip

LRU cache

Have list<pii> dq, u\_mp<int, pii:iterator> mp, set cap in constructor, getFun -> check key in mp[key]->second & move dq.splice(dq.begin(), dq, mp[key]), a condition for mp.end() -1, putFun -> first same logic to getFun to move keys, else if dq.size()==cap then mp.erase(dq.back().first) and dq.pop\_back(), else normal dq.push\_front({key,val}) & mp[key]=dq.begin()

-----

Tree-LevelOrderTraversal

Level order traversal

[while(!q.empty())->for(q.size)] queue<TreeNode\*> q, push root, while(!q.empty()) - run for loop based on q size, q.front()->pop(), inside loop, push x->left/right in queue and x->val in temp\_ans, outside for() push temp\_ans in ans

```
vector<vector<int>> levelOrder(TreeNode* root) {
 vector<vector<int>> ans; queue<TreeNode*> q;
 if(root==NULL) return ans;
 q.push(root);
 while(!q.empty()){
 int size = q.size(); vector<int> l;
 for(int i=0; i<size; i++){
 TreeNode* x = q.front();
 q.pop();
 l.push_back(x->val);
 if(x->left) q.push(x->left);
 if(x->right) q.push(x->right);
 }
 ans.push_back(l);
 }
 return ans;
}
```

Zigzag level order traversal

[while(!q.empty())->for(q.size)] Same as above, keep bool direction flag after for loop, l[i]=x->val else l[size-i-1]=x->val

Even-odd tree

[while(!q.empty())->for(q.size)] Same story, just v=cur->val, if level%2 even -> if v%2 even|| v<=prev then false, same for odd,

false cases take, and then `prev=v`, if `curr->left/right` then push in queue - all inside for loop, `level++` after for loop

Reverse odd levels

[`while(!q.empty())->for(q.size)`] Same story, after `for()` if(`level%2 odd`), then `l=0,r=ans.size()-1`, while(`l<r`)  
`swap(ans[l]->value,ans[r]->value)` `l++,r--` to swap at odd levels.  
**Swapping in extreme ends, reverses things**

Deepest leaves sum

[`while(!q.empty())->for(q.size)`] Same story, just initialize sum before `while()` and let `sum+=cur->val` in `for()`, in last level, it will anyways reset to 0 and store last level sum

Add one row tree

[`while(!q.empty())->for(q.size)`] Same story, only when `d==depth-1` (`depth++` after `for()`), `cur=q.front`, `l` is `cur->left`/`r` is `right` temporarily, new `cur->left/right=new Node(val)` & `cur->left->left=l` & `right->right=r`

Max width of binary tree

[`while(!q.empty())->for(q.size)`], here `queue<pair<Node*, int>>`, before `while()` get `leftInd = q.front().second`, then in `for()` get the `pair<Node*,int> p.first/second`, make `rightInd` or `tempInd=p.second`, if(`cur->left/right`) push {`cur->left`, `tempInd*2` or `(tempInd*2)+1` depending on node left/right child}, after `for()`  
`ans=max(ans,rightInd-leftInd+1)`

All nodes distance K in BT

Have `unordered_map<Node*,Node*> parent`, `unordered_set<Node*> vis`, build parent map in separate `fun()` will do level order traversal in single `while()` no inner `for()` using `queue<Node*>` and insert `parent[curr->left/right]=curr`, coming back, new `queue<Node*>`, push node there and even in `vis` set to mark visited, then `while()` has a `dist` variable to mark distance, if `dist==k` then break then and `dist++` -> nested `for()` -> inside have `arr[3]=curr->left/right`, `parent[cur]`, 3 nodes, and another nested `for(j,3)->` take the `Node*` in `arr` in `nxt` var, if(`nxt && !vis.count(nxt)`) means `nxt` exists and

its not visited, then q.push(nxt) & vis.insert(nxt). So loop runs, checks current dist breaks if it has reached k, and below code then iterates via BFS

-----

Tree traversals

Left view

Dfs call to cur->left then right, level+1 for both, and pass &maxLevel by reference, if level>maxLevel, [ans.pb\(data\)](#), maxLevel=level

Right view

Same as above, sequence of dfs call would be right, then left

Top view

All rests on this: if (mp.count(dist)==0 || mp[dist].second>level) then mp[dist] = {curr->data, level}, send level+1,dist+-1 to recursive calls

```
void topViewDFS(Node* curr, int dist, int level,
 map<int, pair<int,int>> &mp) {
 if (curr == NULL) return;
 if (mp.count(dist) == 0 || mp[dist].second > level) {
 mp[dist] = {curr->data, level};
 }
 topViewDFS(curr->left, dist - 1, level + 1, mp);
 topViewDFS(curr->right, dist + 1, level + 1, mp);
}
```

-----

Tree construction

Construct BT from Preorder and Inorder - skip

Construct BT from Postorder and Inorder - skip

Construct BST from Preorder - skip

Maximum Binary Tree

Get l,r (0,size), find index of max nums from l to r and using value in that create tree, in recursive calls call from l to index-1 & index+1 to r



```

TreeNode* build(vector<int>& nums, int l, int r) {
 if(l > r) return NULL;
 int idx = l;
 for(int i = l; i <= r; i++)
 if(nums[i] > nums[idx]) idx = i;
 TreeNode* root = new TreeNode(nums[idx]);
 root->left = build(nums, l, idx - 1);
 root->right = build(nums, idx + 1, r);
 return root;
}

```

-----  
Tree height: Return-Based DFS (Bottom-Up), Parent needs computed value from children

Maximum Depth of BT

root==null - return 0, lh=fun(node->l), rh=fun(node->r), return 1+max(lh,rh)

Balanced Binary Tree

Similar as above, extra condition - if(abs(lh - rh) > 1) return -1

Diameter of Binary Tree

Same as depth of BT, just a global ans, ans = max(ans, lh + rh);

Minimum Depth of BT

Similar to max depth of BT, but return 1 + min(lh, rh) and also, if tree one half not available handle cases, i.e add before return  
if(root->left == NULL) return 1 + right - as we want min height, if 1 half null then will give null height which is not true

Binary Tree Maximum Path Sum

Same as max depth of BT, just that, left=max(0, dfs(root->left), similarly right, and then global ans stores max() of ans & root->val+left+right, return root->val + max(left, right);

LCA of Binary Tree

Same story for left/right, base condition - if (!root || root == p || root == q) return root, if left&&right return root, else return left?left:right

Maximum difference between node and ancestor

Same left, right story, but base condition - if (!node) return mx - mn, where mn is min & mx is max in each iteration, return max(left, right)

```
int dfs(TreeNode* node, int mn, int mx) {
 if (!node) return mx - mn;
 mn = min(mn, node->val);
 mx = max(mx, node->val);
 int left = dfs(node->left, mn, mx);
 int right = dfs(node->right, mn, mx);
 return max(left, right);
}
int maxAncestorDiff(TreeNode* root) {
 return dfs(root, root->val, root->val);
}
```

LCA of deepest leaves - skip

-----

Tree root: Leaf path problems

Section: Backtracking DFS (Top-Down, push-recursive\_call-pop)

Binary Tree Paths

Simple recursive call to left/right node, before them, if path str empty then push node->value as to\_string, else add '-' and then value, only case is if both left&right null then push\_back path you constructed into the ans string vector

Path Sum ii

Similar like above, but here apart from push back we also pop back if sum not matches after 2 recursive calls, if sum matches then push in another res vector

```
vector<vector<int>> res;
vector<int> curr;
vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
 dfs(root, targetSum);
 return res;
}
void dfs(TreeNode* root, int sum) {
```

```

 if(root == NULL) return;
 curr.push_back(root->val);
 if(root->left==NULL && root->right==NULL && sum==root->val)
 res.push_back(curr);
 dfs(root->left, sum - root->val);
 dfs(root->right, sum - root->val);
 curr.pop_back();
}

```

Smallest string starting from Leaf

Same as binary tree paths, just that, `char c = 'a' + root->val` (to increase the chars, like `'a'+2 = 'c'`), `curr = c + curr` to prepend value, then same if `node->left/right` null and `ans=min(ans,curr)`, `ans` could initially be `'~'` as its lexicographically high

Insufficient nodes in root to Leaf

```

sum += root->val;
// store original leaf status
bool isLeaf = (!root->left && !root->right);
root->left = dfs(root->left, sum, limit);
root->right = dfs(root->right, sum, limit);
if (isLeaf) {
 return (sum >= limit) ? root : NULL;
}
// internal node: keep only if at least one child survives
return (root->left || root->right) ? root : NULL;

```

Pseudo-Palindromic Paths in a Binary Tree

Section: DFS with State Passing (No pop needed)

Sum Root to Leaf numbers

-----

Kth Ancestor of a Tree Node

-----

## Validate BST

## Range Sum of BST

## Minimum Absolute Difference in BST

## Insert into a BST

## LCA of BST

This image shows a full page of white paper with horizontal dashed lines, typical of primary-ruled notebook paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings present.

\_DS-Cpp-SyntaxBasics

## Basic STL DS

---

**Vector:** Dynamic array – contiguous memory, supports random access.

| Operation / Usage   | Syntax                                        | Complexity       | Notes                       |
|---------------------|-----------------------------------------------|------------------|-----------------------------|
| Empty               | <code>vector&lt;int&gt; v;</code>             | $O(1)$           | Empty vector                |
| Fixed size          | <code>vector&lt;int&gt; v(5);</code>          | $O(n)$           | 5 elements initialized to 0 |
| Fixed size + value  | <code>vector&lt;int&gt; v(5, 1);</code>       | $O(n)$           | {1,1,1,1,1}                 |
| From array          | <code>vector&lt;int&gt; v(arr, arr+n);</code> | $O(n)$           | Copies elements             |
| Copy another vector | <code>vector&lt;int&gt; v2(v1);</code>        | $O(n)$           |                             |
| Add element         | <code>v.push_back(x);</code>                  | Amortized $O(1)$ | Append                      |
| Remove last         | <code>v.pop_back();</code>                    | $O(1)$           | Remove last                 |
| Insert              | <code>v.insert(v.begin()+i, x);</code>        | $O(n)$           | Shift elements              |
| Erase               | <code>v.erase(v.begin()+i);</code>            | $O(n)$           | Shift elements              |
| Erase range         | <code>v.erase(v.begin(), v.begin()+k);</code> | $O(n)$           | Remove range                |
| Access              | <code>v[i] / v.at(i)</code>                   | $O(1)$           | Random access               |
| Size                | <code>v.size();</code>                        | $O(1)$           | Count                       |
| Empty               | <code>v.empty();</code>                       | $O(1)$           | Check empty                 |

|              |                                               |               |                      |
|--------------|-----------------------------------------------|---------------|----------------------|
| Front / Back | <code>v.front(); v.back();</code>             | $O(1)$        | Ends                 |
| Iterate      | <code>for (auto &amp;x : v)</code>            | $O(n)$        | Range-based          |
| Sort         | <code>sort(v.begin(), v.end());</code>        | $O(n \log n)$ | Needs<br><algorithm> |
| Reverse      | <code>reverse(v.begin(),<br/>v.end());</code> | $O(n)$        |                      |
| Clear        | <code>v.clear();</code>                       | $O(n)$        | Remove all           |

**Note:**

- 2D Vector: `vector<vector<int>> v;`
  - `vector<vector<int>> v(n, vector<int>(m, 0));`
- 1D Pair Vector: `vector<pair<int,int>> vp;`
  - `vp.push_back({x, y});`
  - `vp[i].first, vp[i].second`

-----

**Set:** Ordered, unique elements – balanced BST (red-black tree). For `std::map` and `std::set`:

- Insertion =  $O(\log n)$ , NOT  $O(n \log n)$
- Because each insertion involves finding one place to insert (using a balanced tree lookup) – not re-sorting or rebalancing all elements.

**Unordered\_set:** Hash set – unique, unordered.

| Operation / Usage | Syntax                                       | Complexity    | Notes                 |
|-------------------|----------------------------------------------|---------------|-----------------------|
| Empty             | <code>set&lt;int&gt; s;</code>               | $O(1)$        |                       |
| Init list         | <code>set&lt;int&gt; s =<br/>{1,2,3};</code> | $O(n \log n)$ |                       |
| Insert            | <code>s.insert(x);</code>                    | $O(\log n)$   | Ignores<br>duplicates |

|             |                   |          |              |
|-------------|-------------------|----------|--------------|
| Erase value | s.erase(x);       | O(log n) |              |
| Find        | s.find(x);        | O(log n) | Iterator     |
| Count       | s.count(x);       | O(log n) | 1 or 0       |
| Size        | s.size();         | O(1)     |              |
| Empty       | s.empty();        | O(1)     |              |
| Iterate     | for (auto &x : s) | O(n)     | Sorted order |
| Lower bound | s.lower_bound(x); | O(log n) | $\geq x$     |
| Upper bound | s.upper_bound(x); | O(log n) | $> x$        |
| Clear       | s.clear();        | O(n)     |              |

**Note:**

- Pair Set: set<pair<int,int>> s;
  - s.insert({x,y}), s.find({x,y});
  - unordered\_set<pair<int,int>> ❌ not directly supported
- To find in set:

```
auto it = s.find("abc");
if (it != s.end()) {
 cout << "Found: " << *it;
}
```

-----

**Map: Sorted key-value pairs (unique keys, ordered by key).**

**unordered\_map: Hash map – key-value pairs, no order.**

| Operation     | Syntax                            | Complexity | Notes         |
|---------------|-----------------------------------|------------|---------------|
| Create        | map<int,int> mp;                  | O(1)       |               |
| Init list     | map<int,int> mp = {{1,2}, {3,4}}; | O(n log n) |               |
| Insert/update | mp[key] = val;                    | O(log n)   | Add or update |



|             |                                                                                           |             |              |
|-------------|-------------------------------------------------------------------------------------------|-------------|--------------|
| Insert pair | <code>mp.insert({k, v});</code>                                                           | $O(\log n)$ |              |
| Find        | <code>mp.find(key);</code>                                                                | $O(\log n)$ | Iterator     |
| Erase       | <code>mp.erase(key);</code>                                                               | $O(\log n)$ |              |
| Size        | <code>mp.size();</code>                                                                   | $O(1)$      |              |
| Empty       | <code>mp.empty();</code>                                                                  | $O(1)$      |              |
| Iterate     | <code>for (auto &amp;p : mp) cout &lt;&lt; p.first &lt;&lt; " " &lt;&lt; p.second;</code> | $O(n)$      | Sorted order |

**Note:**

```

- Using pair:
 map<int,int> = OK. map<int, pair<int,int>> mp = OK.
- To search:
 auto it = mp.find(key);
 if (it != mp.end()) {
 cout << it->second << endl; // Safe access
 }

```

-----

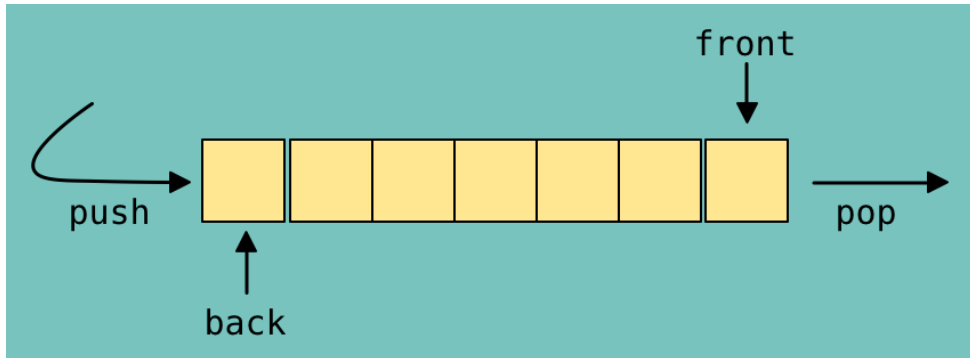
**Queue: FIFO – first in, first out.**

| Operation | Syntax                           | Complexity | Notes        |
|-----------|----------------------------------|------------|--------------|
| Create    | <code>queue&lt;int&gt; q;</code> | $O(1)$     |              |
| Push      | <code>q.push(x);</code>          | $O(1)$     | Enqueue      |
| Pop       | <code>q.pop();</code>            | $O(1)$     | Dequeue      |
| Front     | <code>q.front();</code>          | $O(1)$     | Peek         |
| Back      | <code>q.back();</code>           | $O(1)$     | Last element |

|       |                         |        |  |
|-------|-------------------------|--------|--|
| Empty | <code>q.empty();</code> | $O(1)$ |  |
| Size  | <code>q.size();</code>  | $O(1)$ |  |

**Note:**

- `queue<pair<int,int>> q; q.push({x,y});`



- Also - Queue pops from the front element, vector pops\_back from the back.

-----

**Stack: LIFO – last in, first out.**

| Operation | Syntax                            | Complexity | Notes |
|-----------|-----------------------------------|------------|-------|
| Create    | <code>stack&lt;int&gt; st;</code> | $O(1)$     |       |
| Push      | <code>st.push(x);</code>          | $O(1)$     |       |
| Pop       | <code>st.pop();</code>            | $O(1)$     |       |
| Top       | <code>st.top();</code>            | $O(1)$     | Peek  |
| Empty     | <code>st.empty();</code>          | $O(1)$     |       |
| Size      | <code>st.size();</code>           | $O(1)$     |       |

**Note:**

- `stack<pair<int,int>> st; st.push({x,y});`
-

**Priority\_queue: Max-heap by default.**

| Operation | Syntax                                                                                | Complexity  | Notes                        |
|-----------|---------------------------------------------------------------------------------------|-------------|------------------------------|
| Create    | <code>priority_queue&lt;int&gt; pq;</code>                                            | $O(1)$      | Max-heap                     |
| Min-heap  | <code>priority_queue&lt;int,<br/>vector&lt;int&gt;, greater&lt;int&gt;&gt; pq;</code> | $O(1)$      | Min-heap                     |
| Push      | <code>pq.push(x);</code>                                                              | $O(\log n)$ |                              |
| Pop       | <code>pq.pop();</code>                                                                | $O(\log n)$ |                              |
| Top       | <code>pq.top();</code>                                                                | $O(1)$      | Peek<br>largest/<br>smallest |
| Size      | <code>pq.size();</code>                                                               | $O(1)$      |                              |
| Empty     | <code>pq.empty();</code>                                                              | $O(1)$      |                              |

**Note:**

- Pair Heaps:
  - **MaxHeap:** `priority_queue<pair<int,int>> pq`
  - **MinHeap:** `priority_queue<pair<int,int>,  
vector<pair<int,int>>, greater<pair<int,int>>> pq`

-----

**Deque: Double-ended queue – push/pop from both ends.**

| Operation          | Syntax                                                          | Complexity | Notes           |
|--------------------|-----------------------------------------------------------------|------------|-----------------|
| Create             | <code>deque&lt;int&gt; dq;</code>                               | $O(1)$     |                 |
| Init               | <code>deque&lt;int&gt; dq(5, 1);</code>                         | $O(n)$     | {1,1,1,<br>1,1} |
| Push<br>front/back | <code>dq.push_front(x);</code><br><code>dq.push_back(x);</code> | $O(1)$     |                 |
| Pop<br>front/back  | <code>dq.pop_front();</code><br><code>dq.pop_back();</code>     | $O(1)$     |                 |

|         |                                                |        |  |
|---------|------------------------------------------------|--------|--|
| Erase   | <code>dq.erase(dq.begin()+i)</code>            | $O(n)$ |  |
| Access  | <code>dq[i]; dq.front();<br/>dq.back();</code> | $O(1)$ |  |
| Size    | <code>dq.size();</code>                        | $O(1)$ |  |
| Empty   | <code>dq.empty();</code>                       | $O(1)$ |  |
| Iterate | <code>for (auto &amp;x : dq)</code>            | $O(n)$ |  |

**Note:**

- `deque<pair<int,int>> dq; dq.push_back({x,y});  
dq.push_front({x,y});`

-----

**pair & tuple: Used to store multiple values together.**

- `map<int,int>` and `pair<int,int>` DO share the same access syntax: `.first` and `.second`. But they are not the same DS.

```
pair<int, string> p = {1, "one"};
cout << p.first << " " << p.second;

tuple<int, string, double> t = {1, "A", 3.14};
int a; string b; double c;
tie(a, b, c) = t;
```

-----

**Common Operations Across STL Containers**

| Operation             | Syntax Example                                                      | Used In                     | Meaning           |
|-----------------------|---------------------------------------------------------------------|-----------------------------|-------------------|
| <code>.push(x)</code> | <code>st.push(x); q.push(x);<br/>pq.push(x); v.push_back(x);</code> | stack, queue,<br>pq, vector | Add<br>element    |
| <code>.pop()</code>   | <code>st.pop(); q.pop(); pq.pop();<br/>v.pop_back();</code>         | stack, queue,<br>pq, vector | Remove<br>element |

|                                |                                                |                              |                |
|--------------------------------|------------------------------------------------|------------------------------|----------------|
| <code>.top()</code>            | <code>st.top(); pq.top();</code>               | stack, priority_queue        | Peek top       |
| <code>.front()</code>          | <code>q.front(); dq.front(); v.front();</code> | queue, deque, vector         | Peek first     |
| <code>.back()</code>           | <code>v.back(); dq.back();</code>              | vector, deque                | Peek last      |
| <code>.size()</code>           | <code>v.size(); s.size(); mp.size();</code>    | All                          | Count elements |
| <code>.empty()</code>          | <code>v.empty(); q.empty();</code>             | All                          | Check if empty |
| <code>.clear()</code>          | <code>v.clear(); s.clear(); mp.clear();</code> | Most                         | Remove all     |
| <code>.begin() / .end()</code> | <code>v.begin(); s.end();</code>               | All iterable                 | Iteration      |
| <code>.find(x)</code>          | <code>s.find(x); mp.find(key);</code>          | set, map, unordered variants | Search element |
| <code>.insert(x)</code>        | <code>s.insert(x); v.insert(it,x);</code>      | Many                         | Insert element |
| <code>.erase(x)</code>         | <code>s.erase(x); mp.erase(key);</code>        | Many                         | Remove element |

-----

### Quick Big-O Summary

| Container | Insert           | Erase  | Find   | Access | Ordered | Notes              |
|-----------|------------------|--------|--------|--------|---------|--------------------|
| vector    | Amortized $O(1)$ | $O(n)$ | $O(n)$ | $O(1)$ | ✗       | Fast random access |

|                |             |             |             |        |      |                  |
|----------------|-------------|-------------|-------------|--------|------|------------------|
| set            | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | –      | ✓    | Sorted unique    |
| unordered_set  | $O(1)$      | $O(1)$      | $O(1)$      | –      | ✗    | Hash-based       |
| map            | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ | –      | ✓    | Key-value sorted |
| unordered_map  | $O(1)$      | $O(1)$      | $O(1)$      | –      | ✗    | Key-value hash   |
| queue          | $O(1)$      | $O(1)$      | –           | –      | FIFO |                  |
| stack          | $O(1)$      | $O(1)$      | –           | –      | LIFO |                  |
| priority_queue | $O(\log n)$ | $O(\log n)$ | –           | $O(1)$ | Heap | Max/min element  |

-----

## String

| Operation    | Syntax                                      | Complexity       | Notes                          |
|--------------|---------------------------------------------|------------------|--------------------------------|
| From literal | <code>string s="hello";</code>              | $O(n)$           | Copy from literal              |
| Repeat char  | <code>string s(5, 'a');</code>              | $O(n)$           | "aaaaa"                        |
| Concatenate  | <code>s1+s2;</code> or <code>s1+=s2;</code> | $O(n)$           | Creates new string             |
| Append       | <code>s.append("abc");</code>               | $O(k)$           | Append another string          |
| Push back    | <code>s.push_back('x');</code>              | Amortized $O(1)$ | Add char at end                |
| Pop back     | <code>s.pop_back();</code>                  | $O(1)$           | Remove last char               |
| Front/Back   | <code>s.front()/.back()</code>              | $O(1)$           | Eg: In 'hello';<br>front: 'h', |

|                        |                                 |          |                           |
|------------------------|---------------------------------|----------|---------------------------|
|                        |                                 |          | back: 'o'.                |
| Access                 | s[i] or s.at(i)                 | O(1)     | Random access             |
| Size                   | s.size(); /<br>s.length();      | O(1)     | Same function             |
| Substring              | s.substr(pos,<br>len);          | O(len)   | Returns substring         |
| Find substring         | s.find("abc");                  | O(n)     | Returns index or<br>npos. |
| Replace                | s.replace(pos,<br>len, "xyz");  | O(n)     | Modify in place           |
| Insert                 | s.insert(pos,<br>"abc");        | O(n)     | Insert substring          |
| Erase                  | s.erase(pos, len);              | O(n)     | Remove substring          |
| Clear                  | s.clear();                      | O(n)     | Empty string              |
| Compare                | s1==s2, s1<s2                   | O(n)     | Lexicographic             |
| Iterate                | for(auto &c:s)                  | O(n)     | Char-wise                 |
| Convert → int          | stoi(s)                         | O(n)     | Throws if invalid         |
| Convert → long<br>long | stoll(s)                        | O(n)     |                           |
| Convert →<br>double    | stod(s)                         | O(n)     |                           |
| Convert ← int          | to_string(x)                    | O(n)     |                           |
| Reverse                | reverse(s.begin(),<br>s.end()); | O(n)     | From <algorithm>          |
| Sort                   | sort(s.begin(),<br>s.end());    | O(nlogn) | Alphabetical order        |

**Note:**

```
- To search string: Eg:
auto pos = s.find("abc"); //OK
if(pos != string::npos){ // OK
 cout << pos;
}
```

---

## Other DS

- `list<T>` (Doubly Linked List - has prev/next nodes)
    - `push_back`, `push_front`, `pop_back`, `pop_front`, `erase(it)`, `remove(value)`, `splice()`.
    - For `splice`:
      - `list1.splice(pos, list2, it)`: Moves element at `it` in `list2` to `list1` before `pos`
      - `list1.splice(pos, list2, first, last)`: Moves range `[first, last)` from `list2` into `list1`
  - `forward_list<T>` (Singly Linked List)
    - `push_front`, `pop_front`, `erase_after(it)`, `splice_after()`
  - `multiset<T>` / `unordered_multiset<T>` (Set which allows duplicates)
    - `insert`, `erase(value or it)`, `find`, `count`, `equal_range`
  - `multimap<K,V>` / `unordered_multimap<K,V>` (One key → many values)
    - `insert`, `erase(key or it)`, `find`, `equal_range`
  - `bitset<N>` (Fixed size bit array)
    - `set`, `reset`, `flip`, `test`, `count`
-



\_Construct-DS-Algos

## Linkedlist

```
class ListNode {
public:
 int val; ListNode* next;
 ListNode(int x) {
 val = x;
 next = nullptr;
 }
};

// Insert at end
ListNode* insertEnd(ListNode* head, int val) {
 ListNode* node = new ListNode(val);
 if (!head) return node;
 ListNode* temp = head;
 while (temp->next != nullptr) temp = temp->next;
 temp->next = node;
 return head;
}

// Insert at beginning
ListNode* insertBegin(ListNode* head, int val) {
 ListNode* node = new ListNode(val);
 node->next = head;
 return node;
}

void printList(ListNode* head) {
 while (head != nullptr) {
 cout << head->val << " -> ";
 head = head->next;
 }
}

void linkedListExample() {
 ListNode* head = nullptr;
 for (int i = 1; i <= 5; i++) head = insertEnd(head, i);
 head = insertBegin(head, 0);
 printList(head);
}
```

---

## Graph - Unweighted

```
// Add edge (undirected by default)
void addEdge(int u, int v, vector<vector<int>>& adj, bool directed =
false) {
 adj[u].push_back(v);
 if (!directed) adj[v].push_back(u);
}

// DFS traversal
void dfs(int u, vector<vector<int>>& adj, vector<int>& vis,
vector<int>& ans) {
 vis[u] = 1;
 ans.push_back(u);
 for (int v : adj[u]) {
 if (!vis[v]) {
 dfs(v, adj, vis, ans);
 }
 }
}

// BFS traversal
void bfs(int start, vector<vector<int>>& adj, vector<int>& vis,
vector<int>& ans) {
 queue<int> q;
 q.push(start);
 vis[start] = 1;
 while (!q.empty()) {
 int u = q.front();
 q.pop();
 ans.push_back(u);
 for (int v : adj[u]) {
 if (!vis[v]) {
 vis[v] = 1;
 q.push(v);
 }
 }
 }
}
```

```
// Example usage
void graphExample() {
 int n = 6;
 vector<vector<int>> adj(n); // standard adjacency list
 addEdge(0, 1, adj);
 addEdge(0, 2, adj);
 addEdge(1, 3, adj);
 addEdge(1, 4, adj);
 addEdge(2, 5, adj);
 vector<int> vis(n, 0), ans;
 dfs(0, adj, vis, ans);
 cout << "DFS: ";
 for (int x : ans) cout << x << " ";
 cout << "\n";
 ans.clear();
 fill(vis.begin(), vis.end(), 0);
 bfs(0, adj, vis, ans);
 cout << "BFS: ";
 for (int x : ans) cout << x << " ";
 cout << "\n";
}

```

## Graph - Weighted

```
// Add weighted edge (directed by default)
void addWeightedEdge(
 int u, int v, int w,
 vector<vector<pair<int,int>>>& adj,
 bool directed = true
) {
 adj[u].push_back({v, w});
 if (!directed) adj[v].push_back({u, w});
}
```

```

// DFS traversal (weights ignored for traversal order)
void dfsWeighted(
 int u,
 vector<vector<pair<int,int>>>& adj,
 vector<int>& vis, vector<int>& ans
) {
 vis[u] = 1;
 ans.push_back(u);
 for (auto& [v, w] : adj[u]) {
 if (!vis[v]) {
 dfsWeighted(v, adj, vis, ans);
 }
 }
}

// BFS traversal (weights ignored)
void bfsWeighted(
 int start,
 vector<vector<pair<int,int>>>& adj,
 vector<int>& vis, vector<int>& ans
) {
 queue<int> q;
 q.push(start);
 vis[start] = 1;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 ans.push_back(u);
 for (auto& [v, w] : adj[u]) {
 if (!vis[v]) {
 vis[v] = 1;
 q.push(v);
 }
 }
 }
}

// Example usage
void weightedGraphExample() {
 int n = 6;
 vector<vector<pair<int,int>>> adj(n); // standard adj list

```

```

addWeightedEdge(0, 1, 10, adj);
addWeightedEdge(0, 2, 5, adj);
addWeightedEdge(1, 3, 1, adj);
addWeightedEdge(1, 4, 2, adj);
addWeightedEdge(2, 5, 9, adj);
vector<int> vis(n, 0), ans;
dfsWeighted(0, adj, vis, ans);
cout << "DFS (weighted, ignores weights): ";
for (int x : ans) cout << x << " \n";
ans.clear();
fill(vis.begin(), vis.end(), 0);
bfsWeighted(0, adj, vis, ans);
cout << "BFS (weighted, ignores weights): ";
for (int x : ans) cout << x << " \n";
cout << "Adjacency List:\n";
for (int i = 0; i < n; i++) {
 cout << i << ": ";
 for (auto& [v, w] : adj[i]) {
 cout << "(" << v << ", " << w << ") ";
 }
 cout << "\n";
}
}

```

---

## Stack & Queue

```

void stackQueueExample() {
 stack<int> st;
 st.push(1); st.push(2); st.push(3);
 cout << "Stack top: " << st.top() << "\n";
 st.pop();
 cout << "Stack top after pop: " << st.top() << "\n";
 queue<int> q;
 q.push(10); q.push(20); q.push(30);
 cout << "Queue front: " << q.front() << "\n";
 q.pop();
 cout << "Queue front after pop: " << q.front() << "\n";
}

```

-----

## Tree

```
class TreeNode {
public:
 int data; TreeNode* left; TreeNode* right;
 TreeNode(int val) {
 data = val;
 left = nullptr; right = nullptr;
 }
};

TreeNode* addNode(int val) { // Insert node manually or level-order
 TreeNode* node = new TreeNode(val);
 return node;
}

void inorder(TreeNode* root, vector<int>& ans) {
 if (!root) return; // Inorder traversal
 inorder(root->left, ans);
 ans.push_back(root->data);
 inorder(root->right, ans);
}

void preorder(TreeNode* root, vector<int>& ans) {
 if (!root) return; // Preorder traversal
 ans.push_back(root->data);
 preorder(root->left, ans);
 preorder(root->right, ans);
}

void levelOrder(TreeNode* root, vector<int>& ans) {
 if (!root) return; // Level order traversal
 queue<TreeNode*> q;
 q.push(root);
 while (!q.empty()) {
 TreeNode* node = q.front(); q.pop();
 ans.push_back(node->data);
 if (node->left) q.push(node->left);
 if (node->right) q.push(node->right);
 }
}
```

```
// Example usage
void treeExample() {
 TreeNode* root = addNode(10);
 root->left = addNode(2);
 root->right = addNode(3);
 root->left->left = addNode(28);
 root->left->right = addNode(25);
 root->right->left = addNode(33);
 root->right->right = addNode(31);
 vector<int> ans;
 inorder(root, ans);
 cout << "Inorder: ";
 for (int x : ans) cout << x << " \n";
 ans.clear();
 preorder(root, ans);
 cout << "Preorder: ";
 for (int x : ans) cout << x << " \n";
 ans.clear();
 levelOrder(root, ans);
 cout << "Level order: ";
 for (int x : ans) cout << x << " ";
 cout << "\n";
}

```

## Others:

### Graph Traversal Algos

#### Kruskal's Algorithm (Minimum Spanning Tree)

Use when: Graph is undirected + weighted, need minimum spanning tree. Idea: Sort edges → apply DSU to pick edges that connect different components. Input graph edges: (u, v, w):

(0,1,1), (0,2,4), (1,2,2), (1,3,7), (2,3,3)

Sorted edges (by weight):

1: (0,1), 2: (1,2), 3: (2,3), 4: (0,2), 7: (1,3)

Pick without forming cycles → MST cost = 1 + 2 + 3 = 6

```
struct DSU {
```



```

vector<int> parent, rankv;
DSU(int n) {
 parent.resize(n);
 rankv.assign(n, 0);
 for (int i = 0; i < n; i++) parent[i] = i;
}
int find(int x) {
 if (parent[x] != x) parent[x] = find(parent[x]);
 return parent[x];
}
bool unite(int a, int b) {
 int pa = find(a), pb = find(b);
 if (pa == pb) return false;
 if (rankv[pa] < rankv[pb]) swap(pa, pb);
 parent[pb] = pa;
 if (rankv[pa] == rankv[pb]) rankv[pa]++;
 return true;
}
};
int kruskalMST(int n, vector<vector<int>>& edges) {
 // edges[i] = {weight, u, v}
 sort(edges.begin(), edges.end());
 DSU dsu(n);
 int mstCost = 0;
 for (auto &e : edges) {
 int w = e[0], u = e[1], v = e[2];
 if (dsu.unite(u, v))
 mstCost += w;
 }
 return mstCost;
}

```

## Dijkstra's Algorithm (Shortest Path in Weighted Graph)

Use when: Graph has positive weights only.

Idea: Priority queue → always expand smallest distance.

Graph:

0 → 1 (4)

0 → 2 (1)

2 → 1 (2)

1 → 3 (1)

2 → 3 (5)

Running Dijkstra from 0:

Shortest distances:

0 → 0 = 0

0 → 2 = 1

0 → 1 = 1 + 2 = 3

0 → 3 = 3 + 1 = 4

```
int dijkstra(int n, vector<vector<pair<int,int>>>& adj, int src, int
dst) {
 vector<int> dist(n, INT_MAX);
 dist[src] = 0;
 priority_queue<pair<int,int>,
 vector<pair<int,int>>,
 greater<pair<int,int>>> pq;
 pq.push({0, src}); // {distance, node}
 while (!pq.empty()) {
 auto cur = pq.top(); pq.pop();
 int d = cur.first, u = cur.second;
 if (u == dst) return d;
 if (d > dist[u]) continue;
 for (auto &p : adj[u]) {
 int v = p.first, w = p.second;
 if (d + w < dist[v]) {
 dist[v] = d + w;
 pq.push({dist[v], v});
 }
 }
 }
 return dist[dst];
}
```

### Floyd-Warshall (All-Pairs Shortest Path)

Use when: Need distance between every pair of nodes, graph max size ~200.

Works with: positive & negative weights (but no negative cycles)

Initial matrix (INF = 99999):

```
0 3 INF
2 0 5
INF INF 0
```

After Floyd-Warshall:

```
0 3 8
2 0 5
INF INF 0
```

```
void floydWarshall(vector<vector<int>>& dist) {
 int n = dist.size();
 for (int k = 0; k < n; k++) {
 for (int i = 0; i < n; i++) {
 for (int j = 0; j < n; j++) {
 if (dist[i][k] == INT_MAX || dist[k][j] == INT_MAX)
 continue;
 dist[i][j] = min(dist[i][j], dist[i][k] +
dist[k][j]);
 }
 }
 }
}
```

### Bellman-Ford (Shortest Path with Negative Weights)

Use when: Graph may have negative edges.

Also detects: Negative cycles.

Graph edges:

```
0 → 1 (4)
0 → 2 (5)
1 → 2 (-3)
2 → 3 (2)
```

Bellman-Ford from 0:

dist = [0, 4, 1, 3]

```

int bellmanFord(int n, vector<vector<int>>& edges, int src) {
 // edges[i] = {u, v, w}
 const int INF = 1e9;
 vector<int> dist(n, INF);
 dist[src] = 0;
 // Relax edges (n-1) times
 for (int i = 1; i < n; i++) {
 bool changed = false;
 for (auto &e : edges) {
 int u = e[0], v = e[1], w = e[2];
 if (dist[u] != INF && dist[u] + w < dist[v]) {
 dist[v] = dist[u] + w;
 changed = true;
 }
 }
 if (!changed) break;
 }
 // Check negative cycle
 for (auto &e : edges) {
 int u = e[0], v = e[1], w = e[2];
 if (dist[u] != INF && dist[u] + w < dist[v]) {
 return -1; // negative cycle detected
 }
 }
 return dist[src]; // or return dist[] array for all nodes
}

```

-----

## Binary Search Algo

```
int binarySearch(vector<int> &arr, int x) {
 int low = 0;
 int high = arr.size() - 1;
 while (low <= high) {
 int mid = low + (high - low) / 2;
 // Check if x is present at mid
 if (arr[mid] == x)
 return mid;
 // If x greater, ignore left half
 if (arr[mid] < x)
 low = mid + 1;
 // If x is smaller, ignore right half
 else
 high = mid - 1;
 }
 // If we reach here, then element was not present
 return -1;
}
```

---

GenDS-FastSlowPointer

## General DS - Fast and Slow Pointer

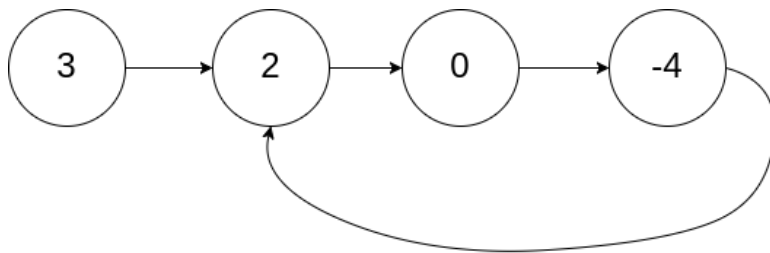
**Description:** This technique uses two pointers moving at different speeds to solve problems involving cycles, such as finding the middle of a list, detecting loops, or checking for palindromes.

- [Linked List Cycle](#)
- [Linked List Cycle II](#)
- [Remove nth Node from the End of List](#)
- [Palindrome Linked List](#)
- [Find the Duplicate Number](#)
- [\[Self Added v2\] Intersection of two Linkedlists](#)

Given head, the head of a linked list, determine if the linked list has a cycle in it.

Input: head = [3,2,0,-4], pos = 1. Output: true

Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).



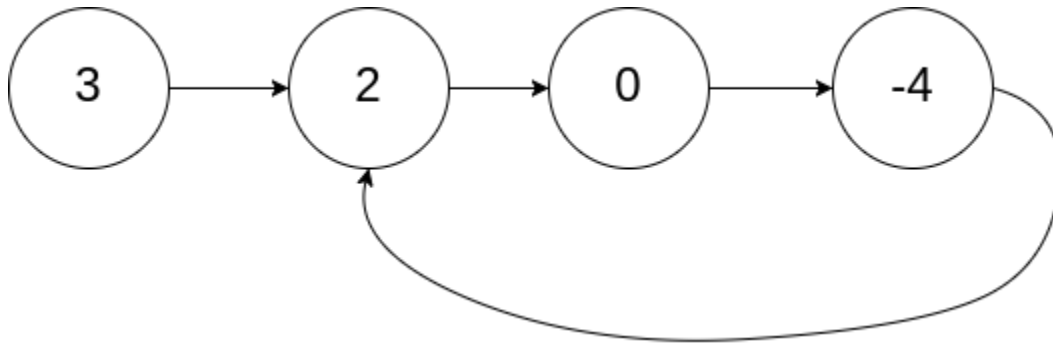
```
bool hasCycle(ListNode *head) {
 if (!head || !head->next) return false;
 ListNode *slow = head;
 ListNode *fast = head;
 while (fast && fast->next) { // doing fast & fast->next, so that
 fast->next->next don't crash. As fast->next->next is nullptr defined
 in constructor, so it's still accessible
 slow = slow->next;
 fast = fast->next->next;
 if (slow == fast)
 return true;
 }
 return false;
}
```

Given the **head** of a linked list, return the node where the cycle begins. If there is no cycle, return **null**.

Input: head = [3,2,0,-4], pos = 1

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.



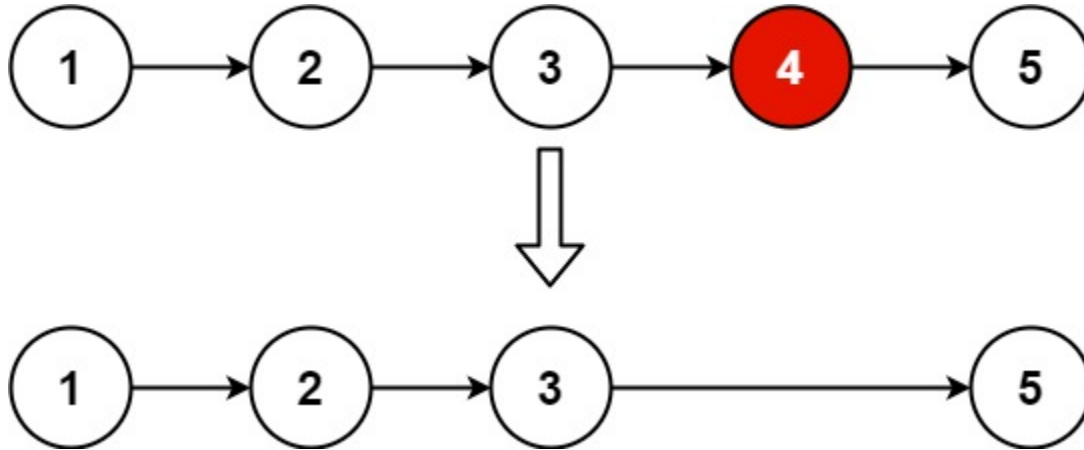
```
ListNode *detectCycle(ListNode *head) {
if (!head || !head->next) return nullptr;
ListNode *slow = head;
ListNode *fast = head;
while (fast && fast->next) { // Phase 1: Detect cycle
 slow = slow->next;
 fast = fast->next->next;
 if (slow == fast) {
 // Cycle found → move to phase 2
 slow = head;
 // Phase 2: find cycle start
 while (slow != fast) {
 slow = slow->next;
 fast = fast->next;
 }
 return slow; // meeting point = cycle start
 }
}
return nullptr; // no cycle
}
```



Given the head of a linked list, remove the nth node from the end of the list and return its head.

Input: head = [1,2,3,4,5], n = 2

Output: [1,2,3,5]

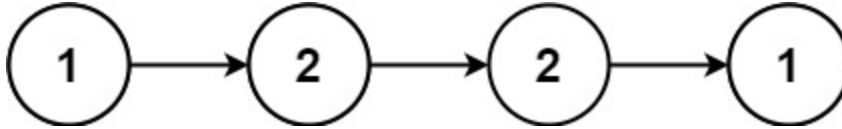


```
ListNode* removeNthFromEnd(ListNode* head, int gap) {
 ListNode dummy(0);
 dummy.next = head;
 ListNode* fast = &dummy;
 ListNode* slow = &dummy;
 // Move fast gap+1 steps ahead
 for (int i = 0; i <= gap; i++) {
 fast = fast->next;
 }
 // Move both until fast reaches end
 while (fast != nullptr) {
 fast = fast->next;
 slow = slow->next;
 }
 // Remove the nth node from end
 slow->next = slow->next->next;
 return dummy.next;
}
```

Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Input: head = [1,2,2,1]

Output: true



```

bool isPalindrome(ListNode* head) {
 vector<int> listVals;
 while (head) {
 listVals.push_back(head->val);
 head = head->next;
 }
 int left = 0, right = listVals.size() - 1;
 while (left < right && listVals[left] == listVals[right]) {
 left++;
 right--;
 }
 return left >= right;
}
// other approach could be to reverse linkedlist and compare
}

```

**Given an array of integers `nums` containing  $n + 1$  integers where each integer is in the range  $[1, n]$  inclusive. There is only one repeated number in `nums`, return this repeated number. You must solve the problem without modifying the array `nums` and using only constant extra space.**

**Input: `nums = [1,3,4,2,2]`. Output: 2**

```

int findDuplicate(vector<int>& nums) {
 int slow = nums[0];
 int fast = nums[0];
 do { // Step 1: Detect cycle - Floyd Cycle Detection
 slow = nums[slow];
 fast = nums[nums[fast]];
 } while (slow != fast);
 slow = nums[0]; // Step 2: Find entry point (duplicate)
 while (slow != fast) {
 slow = nums[slow];
 fast = nums[fast];
 }
 return slow;
}
}

```

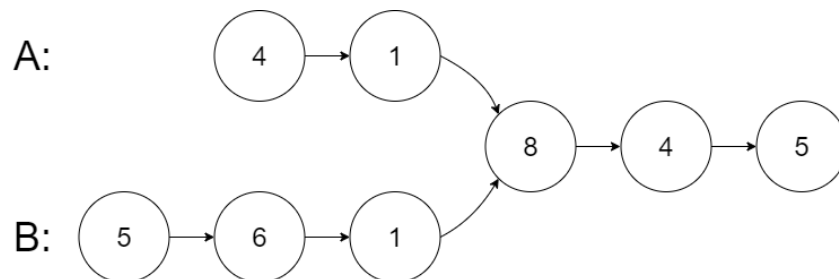
Given the heads of two singly linked-lists headA and headB, return the node at which the two lists intersect. If the two linked lists have no intersection at all, return null.

For example, the following two linked lists begin to intersect at node c1: The test cases are generated such that there are no cycles anywhere in the entire linked structure.

Note that the linked lists must retain their original structure after the function returns.

Input: intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3. Output: Intersected at '8'

Explanation: The intersected node's value is 8 (note that this must not be 0 if the two lists intersect). From the head of A, it reads as [4,1,8,4,5]. From the head of B, it reads as [5,6,1,8,4,5]. There are 2 nodes before the intersected node in A; There are 3 nodes before the intersected node in B. - Note that the intersected node's value is not 1 because the nodes with value 1 in A and B (2nd node in A and 3rd node in B) are different node references. In other words, they point to two different locations in memory, while the nodes with value 8 in A and B (3rd node in A and 4th node in B) point to the same location in memory.



```
ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
 if (!headA || !headB) return nullptr;
 ListNode *pA = headA;
 ListNode *pB = headB;
 // One pointer finishes its list, redirect to other list head.
 while (pA != pB) {
 pA = pA ? pA->next : headB;
 pB = pB ? pB->next : headA;
 }
 return pA; // either intersection node or nullptr
}
```

# GenDS-OverlappingIntervals

## General DS - Overlapping Intervals

Description: Intervals are often manipulated through sorting and merging based on their start and end times.

- [Basic Merge: Merge Intervals](#)
- [Interval Insertion: Insert Interval](#)
- [My Calendar ii](#)
- [Minimum Number of Arrows to Burst Balloons](#)
- [Non-overlapping Intervals](#)
- [\[Self Added v2\] Merge Sorted Array](#)
- [\[Self Added v2\] Meeting Rooms II](#)

Given an array of intervals where  $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$ , merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.  
Input:  $\text{intervals} = [[1,3],[2,6],[8,10],[15,18]]$   
Output:  $[[1,6],[8,10],[15,18]]$ . Explanation: Since intervals  $[1,3]$  and  $[2,6]$  overlap, merge them into  $[1,6]$ .

```
vector<vector<int>> merge(vector<vector<int>>& intervals) {
 if (intervals.empty()) return {};
 // Step 1: Sort by start time, Step 2: Iterate and merge
 sort(intervals.begin(), intervals.end());
 vector<vector<int>> merged;
 merged.push_back(intervals[0]);
 for (int i = 1; i < intervals.size(); ++i) {
 auto &last = merged.back(); // last interval in result
 // overlap check: next.start <= last.end
 if (intervals[i][0] <= last[1]) {
 // merge by updating end
 last[1] = max(last[1], intervals[i][1]);
 } else {
 // no overlap → push new interval
 merged.push_back(intervals[i]);
 }
 }
 return merged;
}
```

You are given an array of non-overlapping intervals `intervals` where `intervals[i] = [starti, endi]` represent the start and the end of the *i*th interval and `intervals` is sorted in ascending order by `starti`. You are also given an interval `newInterval = [start, end]` that represents the start and end of another interval.

Insert `newInterval` into `intervals` such that `intervals` is still sorted in ascending order by `starti` and `intervals` still does not have any overlapping intervals (merge overlapping intervals if necessary). Return `intervals` after the insertion. Note that you don't need to modify `intervals` in-place. You can make a new array and return it.

Input: `intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]]`, `newInterval = [4,8]`. Output: `[[1,2],[3,10],[12,16]]`

Explanation: Because the new interval `[4,8]` overlaps with `[3,5]`, `[6,7]`, `[8,10]`.

```
vector<vector<int>> insert(vector<vector<int>>& intervals,
vector<int>& newInterval) {
 vector<vector<int>> res;
 int i = 0, n = intervals.size();
 // Add all intervals before newInterval
 while (i < n && intervals[i][1] < newInterval[0]) {
 res.push_back(intervals[i]);
 i++;
 }
 // Merge all overlapping intervals
 while (i < n && intervals[i][0] <= newInterval[1]) {
 newInterval[0] = min(newInterval[0], intervals[i][0]);
 newInterval[1] = max(newInterval[1], intervals[i][1]);
 i++;
 }
 res.push_back(newInterval);
 while (i < n) { // Add remaining intervals
 res.push_back(intervals[i]);
 i++;
 }
 return res;
}
```

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a triple booking.

A triple booking happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers `startTime` and `endTime` that represents a booking on the half-open interval  $[startTime, endTime)$ , the range of real numbers  $x$  such that  $startTime \leq x < endTime$ . Implement the `MyCalendarTwo` class:

`MyCalendarTwo()` Initializes the calendar object.

`boolean book(int startTime, int endTime)` Returns true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.

Input

```
["MyCalendarTwo", "book", "book", "book", "book", "book", "book"]
[[], [10, 20], [50, 60], [10, 40], [5, 15], [5, 10], [25, 55]]
```

Output

```
[null, true, true, true, false, true, true]
```

Explanation

```
MyCalendarTwo myCalendarTwo = new MyCalendarTwo();
```

```
myCalendarTwo.book(10, 20); // return True, The event can be booked.
```

```
myCalendarTwo.book(50, 60); // return True, The event can be booked.
```

```
myCalendarTwo.book(10, 40); // return True, The event can be double
booked.
```

```
myCalendarTwo.book(5, 15); // return False, The event cannot be
booked, because it would result in a triple booking.
```

```
myCalendarTwo.book(5, 10); // return True, The event can be booked,
as it does not use time 10 which is already double booked.
```

```
myCalendarTwo.book(25, 55); // return True, The event can be booked,
as the time in [25, 40) will be double booked with the third event,
the time [40, 50) will be single booked, and the time [50, 55) will
be double booked with the second event.
```

```
vector<pair<int, int>> bookings; // all events
```

```
vector<pair<int, int>> overlaps; // double-booked regions
```

```

MyCalendarTwo() {}
bool book(int start, int end) {
 // Check against all double-booked intervals first (to prevent
 triple booking)
 for (int i = 0; i < overlaps.size(); i++) {
 int os = overlaps[i].first;
 int oe = overlaps[i].second;
 if (max(start, os) < min(end, oe)) {
 // If overlap exists with a double-booked range, triple
 booking would occur
 return false;
 }
 }
 // Add new overlaps (where this event overlaps existing single
 bookings)
 for (int i = 0; i < bookings.size(); i++) {
 int bs = bookings[i].first;
 int be = bookings[i].second;
 if (max(start, bs) < min(end, be)) {
 int overlapStart = max(start, bs);
 int overlapEnd = min(end, be);
 overlaps.push_back({overlapStart, overlapEnd});
 } // Check if overlap exists
 }
 // Finally add this booking
 bookings.push_back({start, end});
 return true;
}

```

/\*

Imagine:

Case1:

```

*-----

```

Case2:

```

*-----

```

Dry run:



| Step     | Input   | Result | bookings                                 |
|----------|---------|--------|------------------------------------------|
| overlaps |         |        |                                          |
| 1        | [10,20] | ✓      | [[10,20]]                                |
| 2        | [50,60] | ✓      | [[10,20],[50,60]]                        |
| 3        | [10,40] | ✓      | [[10,20],[50,60],[10,40]]                |
| 4        | [5,15]  | ✗      | same                                     |
| 5        | [5,10]  | ✓      | [[10,20],[50,60],[10,40],[5,10]]         |
| 6        | [25,55] | ✓      | [[10,20],[50,60],[10,40],[5,10],[25,55]] |

\*/

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [xstart, xend]` denotes a balloon whose horizontal diameter stretches between `xstart` and `xend`. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with `xstart` and `xend` is burst by an arrow shot at `x` if `xstart <= x <= xend`. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path. Given the array `points`, return the minimum number of arrows that must be shot to burst all balloons.

Input: `points = [[10,16],[2,8],[1,6],[7,12]]`. Output: 2

Explanation: The balloons can be burst by 2 arrows:

- Shoot an arrow at `x = 6`, bursting the balloons `[2,8]` and `[1,6]`.
- Shoot an arrow at `x = 11`, bursting the balloons `[10,16]` and `[7,12]`.

```
int findMinArrowShots(vector<vector<int>>& points) {
 if (points.empty()) return 0;
```

```

// Step 1: Sort by end coordinate
sort(points.begin(), points.end(), [](auto &a, auto &b) {
 return a[1] < b[1];
});
int arrows = 1;
int end = points[0][1]; // arrow at the end of first balloon
// Step 2: Traverse balloons
for (auto &p : points) {
 if (p[0] > end) { // no overlap
 arrows++;
 end = p[1]; // new arrow at new balloon's end
 }
}
return arrows;
}

```

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

// It's conceptually similar to the Minimum Number of Arrows to Burst Balloons, but instead of counting arrows, we count how many intervals we must remove to eliminate overlaps.

`removals = total intervals - non-overlapping kept intervals`. All code remains same, just we return `removals`.

You are given two integer arrays `nums1` and `nums2`, sorted in non-decreasing order, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array `nums1`. To accommodate this, `nums1` has a length of  $m + n$ , where the first  $m$  elements denote the elements that should be merged, and the last  $n$  elements are set to 0 and should be ignored. `nums2` has a length of  $n$ .

Input: `nums1 = [1,2,3,0,0,0]`,  $m = 3$ , `nums2 = [2,5,6]`,  $n = 3$

Output: `[1,2,2,3,5,6]`

Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.

The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

```
void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
 int i = m - 1; // last index of nums1 actual values
 int j = n - 1; // last index of nums2
 int k = m + n - 1; // last index of nums1 total size
 while (j >= 0) {
 if (i >= 0 && nums1[i] > nums2[j]) {
 nums1[k--] = nums1[i--];
 } else {
 nums1[k--] = nums2[j--];
 }
 }
}
```

Given an array of meeting time intervals `intervals` where `intervals[i] = [start_i, end_i]`, return the minimum number of conference rooms required.

input: `intervals = [[0, 30], [5, 10], [15, 20]]`

output: 2

```
int minMeetingRooms(vector<vector<int>>& intervals) {
 int n = intervals.size();
 vector<int> starts, ends;
 for (auto &it : intervals) {
 starts.push_back(it[0]);
 }
}
```

```
 ends.push_back(it[1]);
 }
 sort(starts.begin(), starts.end());
 sort(ends.begin(), ends.end());
 int rooms = 0, endPtr = 0;
 for (int i = 0; i < n; i++) {
 if (starts[i] < ends[endPtr]) {
 rooms++; // need new room
 } else {
 endPtr++; // meeting ended → free a room
 }
 }
 return rooms;
}
```

GenDS-PrefixSum

## General DS - Prefix Sum

Description: Prefix Sums/Products are techniques that store cumulative sums or products up to each index, allowing for quick subarray range queries.

- [Find the middle index in array](#)
- [Product of array except self](#)
- [Maximum product subarray](#)
- [Number of ways to split array](#)
- [Range Sum Query 2D](#)

Given a 0-indexed integer array `nums`, find the leftmost `middleIndex` (i.e., the smallest amongst all the possible ones).

A `middleIndex` is an index where  $\text{nums}[0] + \text{nums}[1] + \dots + \text{nums}[\text{middleIndex}-1] == \text{nums}[\text{middleIndex}+1] + \text{nums}[\text{middleIndex}+2] + \dots + \text{nums}[\text{nums.length}-1]$ .

If `middleIndex == 0`, the left side sum is considered to be 0.

Similarly, if `middleIndex == nums.length - 1`, the right side sum is considered to be 0.

Return the leftmost `middleIndex` that satisfies the condition, or -1 if there is no such index.

Input: `nums = [2,3,-1,8,4]`

Output: 3

Explanation: The sum of the numbers before index 3 is:  $2+3+-1 = 4$

The sum of the numbers after index 3 is:  $4 = 4$

```
int findMiddleIndex(vector<int>& nums) {
 int totalSum = accumulate(nums.begin(), nums.end(), 0);
 int leftSum = 0;
 for (int i = 0; i < nums.size(); i++) {
 int rightSum = totalSum - leftSum - nums[i];
 if (leftSum == rightSum)
 return i; // Found the middle index
 leftSum += nums[i];
 }
 return -1; // No valid index found
}
```

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of `nums` except `nums[i]`.

The product of any prefix or suffix of `nums` is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in  $O(n)$  time and without using the division operation.

Input: `nums = [1,2,3,4]`

Output: `[24,12,8,6]`

```
vector<int> productExceptSelf(vector<int>& nums) {
 int n = nums.size();
 vector<int> answer(n, 1);
 int prefix = 1;
 for (int i = 0; i < n; i++) {
 answer[i] = prefix; // product of everything to the
left of i
 prefix *= nums[i]; // update prefix for next index
 }
 int suffix = 1;
 for (int i = n - 1; i >= 0; i--) {
 answer[i] *= suffix; // multiply by product of
everything to the right of i
 suffix *= nums[i]; // update suffix for next index
 }
 return answer;
}
```

Given an integer array `nums`, find a subarray that has the largest product, and return the product. The test cases are generated so that the answer will fit in a 32-bit integer.

Note that the product of an array with a single element is the value of that element.

Input: `nums = [2,3,-2,4]`

Output: 6

Explanation: `[2,3]` has the largest product 6.

```

int maxProduct(int A[], int n) {
 // store the result that is the max we have found so far
 int r = A[0];
 // imax/imin stores the max/min product of
 // subarray that ends with the current number A[i]
 for (int i = 1, imax = r, imin = r; i < n; i++) {
 // multiplied by a negative makes big number smaller, small
number bigger
 // so we redefine the extremums by swapping them
 if (A[i] < 0)
 swap(imax, imin);
 // max/min product for the current number is either the
current number itself or the max/min by the previous number times
the current one
 imax = max(A[i], imax * A[i]);
 imin = min(A[i], imin * A[i]);
 // the newly computed max value is a candidate for our
global result
 r = max(r, imax);
 }
 return r;
}

```

You are given a 0-indexed integer array `nums` of length `n`.

`nums` contains a valid split at index `i` if the following are true:

The sum of the first `i + 1` elements is greater than or equal to the sum of the last `n - i - 1` elements.

There is at least one element to the right of `i`. That is,  $0 \leq i < n - 1$ .

Return the number of valid splits in `nums`.

Input: `nums = [10,4,-8,7]`

Output: 2

Explanation:

There are three ways of splitting `nums` into two non-empty parts:

- Split `nums` at index 0. Then, the first part is `[10]`, and its sum is 10. The second part is `[4,-8,7]`, and its sum is 3. Since  $10 \geq 3$ ,  $i = 0$  is a valid split.



- Split nums at index 1. Then, the first part is [10,4], and its sum is 14. The second part is [-8,7], and its sum is -1. Since  $14 \geq -1$ ,  $i = 1$  is a valid split.
- Split nums at index 2. Then, the first part is [10,4,-8], and its sum is 6. The second part is [7], and its sum is 7. Since  $6 < 7$ ,  $i = 2$  is not a valid split.

Thus, the number of valid splits in nums is 2.

```
int waysToSplitArray(vector<int>& nums) {
 long long total = 0;
 for (int x : nums) total += x;
 long long prefix = 0;
 int count = 0;
 for (int i = 0; i < nums.size() - 1; i++) {
 prefix += nums[i];
 if (prefix * 2 >= total) count++;
 // prefixSum[i] >= totalSum - prefixSum[i] -> Equivalent to:
 2 * prefixSum[i] >= totalSum
 }
 return count;
}
```

Given a 2D matrix matrix, handle multiple queries of the following type:

Calculate the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

Implement the NumMatrix class:

NumMatrix(int[][] matrix) Initializes the object with the integer matrix matrix.

int sumRegion(int row1, int col1, int row2, int col2) Returns the sum of the elements of matrix inside the rectangle defined by its upper left corner (row1, col1) and lower right corner (row2, col2).

You must design an algorithm where sumRegion works on  $O(1)$  time complexity.

Input

```
["NumMatrix", "sumRegion", "sumRegion", "sumRegion"]
[[[[[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]], [2, 1, 4, 3], [1, 1, 2, 2], [1, 2, 2, 4]]]
```

## Output

[null, 8, 11, 12]

Explanation: NumMatrix numMatrix = new NumMatrix([[3, 0, 1, 4, 2],  
[5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]]);

numMatrix.sumRegion(2, 1, 4, 3); // return 8 (i.e sum of the red rectangle)

numMatrix.sumRegion(1, 1, 2, 2); // return 11 (i.e sum of the green rectangle)

numMatrix.sumRegion(1, 2, 2, 4); // return 12 (i.e sum of the blue rectangle)

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 0 | 1 | 4 | 2 |
| 5 | 6 | 3 | 2 | 1 |
| 1 | 2 | 0 | 1 | 5 |
| 4 | 1 | 0 | 1 | 7 |
| 1 | 0 | 3 | 0 | 5 |

```
NumMatrix(vector<vector<int>>& matrix) {
 int m = matrix.size();
 int n = matrix[0].size();
 prefix = vector<vector<int>>(m + 1, vector<int>(n + 1, 0));
 for (int i = 1; i <= m; i++) {
 for (int j = 1; j <= n; j++) {
 prefix[i][j] = matrix[i-1][j-1]
 + prefix[i-1][j]
 + prefix[i][j-1]
 - prefix[i-1][j-1];
 }
 }
}

int sumRegion(int row1, int col1, int row2, int col2) {
 return prefix[row2+1][col2+1]
 - prefix[row1][col2+1]
 - prefix[row2+1][col1]
 + prefix[row1][col1];
}
```

GenDS-SlidingWindowFixSize

## General DS - Sliding Window

Description: A sliding window is a subarray or substring that moves over data to solve problems efficiently in linear time.

### Fixed Size

- [Maximum Sum Subarray of Size K](#)
- [Number of Subarrays having Average Greater or Equal to Threshold](#)
- [Repeated DNA sequences](#)
- [Permutation in String](#)
- [Sliding Subarray Beauty](#)
- [Sliding Window Maximum](#)

You are given an integer array `nums` and an integer `k`. Find the maximum subarray sum of all the subarrays of `nums` that meet the following conditions:

The length of the subarray is `k`, and

All the elements of the subarray are distinct.

Return the maximum subarray sum of all the subarrays that meet the conditions. If no subarray meets the conditions, return 0.

A subarray is a contiguous non-empty sequence of elements within an array.

Input: `nums = [1,5,4,2,9,9,9]`, `k = 3`

Output: 15

Explanation: The subarrays of `nums` with length 3 are:

- `[1,5,4]` which meets the requirements and has a sum of 10.
- `[5,4,2]` which meets the requirements and has a sum of 11.
- `[4,2,9]` which meets the requirements and has a sum of 15.
- `[2,9,9]` which does not meet the requirements because the element 9 is repeated.
- `[9,9,9]` which does not meet the requirements because the element 9 is repeated.

We return 15 because it is the maximum subarray sum of all the subarrays that meet the conditions

```

int maximumSubarraySum(vector<int>& nums, int k) {
 unordered_map<int, int> freq; // element -> frequency
 long long curSum = 0, maxSum = 0;
 int left = 0;
 for (int right = 0; right < nums.size(); right++) {
 freq[nums[right]]++;
 curSum += nums[right];
 // shrink window if duplicate or size > k
 while (freq[nums[right]] > 1 || right - left + 1 > k) {
 freq[nums[left]]--;
 curSum -= nums[left];
 left++;
 if (freq[nums[left]] == 0)
 freq.erase(nums[left]);
 }
 // valid window of size k with distinct elements
 if (right - left + 1 == k)
 maxSum = max(maxSum, curSum);
 }
 return (int)maxSum;
}

```

Given an array of integers arr and two integers k and threshold, return the number of sub-arrays of size k and average greater than or equal to threshold.

Input: arr = [2,2,2,2,5,5,5,8], k = 3, threshold = 4

Output: 3

Explanation: Sub-arrays [2,5,5],[5,5,5] and [5,5,8] have averages 4, 5 and 6 respectively. All other sub-arrays of size 3 have averages less than 4 (the threshold).

```

int numOfSubarrays(vector<int>& arr, int k, int threshold) {
 int n = arr.size();
 int target = k * threshold; // required minimum sum
 int count = 0;
 int windowSum = 0;
 // Step 1: initialize first window
 for (int i = 0; i < k; i++) {
 windowSum += arr[i];
 }
 if (windowSum >= target) count++;
 // Step 2: slide the window
 for (int i = k; i < n; i++) {
 windowSum += arr[i]; // add rightmost element
 windowSum -= arr[i - k]; // remove leftmost element
 if (windowSum >= target)
 count++;
 }
 return count;
}

```

The DNA sequence is composed of a series of nucleotides abbreviated as 'A', 'C', 'G', and 'T'.

For example, "ACGAATTCCG" is a DNA sequence.

When studying DNA, it is useful to identify repeated sequences within the DNA.

Given a string *s* that represents a DNA sequence, return all the 10-letter-long sequences (substrings) that occur more than once in a DNA molecule. You may return the answer in any order.

Input: *s* = "AAAAACCCCCAAAAACCCCCCAAAAAGGGTTT"

Output: ["AAAAACCCCC", "CCCCCAAAAA"]

```

vector<string> findRepeatedDnaSequences(string s) {
 int n = s.size(); // could also use a map<int,int> to store
sequence & count and get seqs with count >=2
 unordered_set<string> seen; // stores all seen substrings
 unordered_set<string> repeated; // substrings seen more times
 for (int i = 0; i + 10 <= n; i++) {
 string sub = s.substr(i, 10);
 if (seen.count(sub))
 repeated.insert(sub); // found again
 else
 seen.insert(sub); // first occurrence
 }
 return vector<string>(repeated.begin(), repeated.end());
}

```

**Given two strings s1 and s2, return true if s2 contains a permutation of s1, or false otherwise.**

**In other words, return true if one of s1's permutations is the substring of s2.**

**Input: s1 = "ab", s2 = "eidbaooo"**

**Output: true**

**Explanation: s2 contains one permutation of s1 ("ba").**

```

bool checkInclusion(string s1, string s2) {
 if (s1.size() > s2.size()) return false;
 vector<int> count1(26, 0), count2(26, 0);
 // build initial count for s1
 for (char c : s1) count1[c - 'a']++;
 int k = s1.size();
 // sliding window
 for (int i = 0; i < s2.size(); i++) {
 count2[s2[i] - 'a']++; // include current char
 // remove char left of window if window too large
 if (i >= k) count2[s2[i - k] - 'a']--;
 // check if match
 if (count1 == count2) return true;
 }
 return false;
}

```

Given an integer array `nums` containing `n` integers, find the beauty of each subarray of size `k`. The beauty of a subarray is the `x`th smallest integer in the subarray if it is negative, or `0` if there are fewer than `x` negative integers.

Return an integer array containing `n - k + 1` integers, which denote the beauty of the subarrays in order from the first index in the array.

A subarray is a contiguous non-empty sequence of elements within an array.

Input: `nums = [1,-1,-3,-2,3]`, `k = 3`, `x = 2`

Output: `[-1,-2,-2]`

Explanation: There are 3 subarrays with size `k = 3`.

The first subarray is `[1, -1, -3]` and the 2nd smallest negative integer is `-1`.

The second subarray is `[-1, -3, -2]` and the 2nd smallest negative integer is `-2`.

The third subarray is `[-3, -2, 3]` and the 2nd smallest negative integer is `-2`.

// If a positive integer, then beauty would've been `0`.



```

vector<int> getSubarrayBeauty(vector<int>& nums, int k, int x) {
 int n = nums.size();
 vector<int> res;
 vector<int> freq(51, 0); // store freq of -ve's from -50 to -1
 for (int i = 0; i < n; i++) {
 if (nums[i] < 0) // Add current number to frequency
 freq[nums[i] + 50]++;
 // Remove element that goes out of the window
 if (i >= k && nums[i - k] < 0)
 freq[nums[i - k] + 50]--;
 // When window size becomes k, calculate the beauty
 if (i >= k - 1) {
 int count = 0;
 int val = 0;
 // find the x-th smallest negative number
 for (int j = 0; j < 51; j++) {
 count += freq[j];
 if (count >= x) {
 val = j - 50;
 // convert index back to original number
 break;
 }
 }
 res.push_back(val);
 }
 }
 return res;
}

```

**[hard]**

You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

Output: `[3,3,5,5,6,7]`

Explanation:

| Window position     | Max   |
|---------------------|-------|
| -----               | ----- |
| [1 3 -1] -3 5 3 6 7 | 3     |
| 1 [3 -1 -3] 5 3 6 7 | 3     |
| 1 3 [-1 -3 5] 3 6 7 | 5     |
| 1 3 -1 [-3 5 3] 6 7 | 5     |
| 1 3 -1 -3 [5 3 6] 7 | 6     |
| 1 3 -1 -3 5 [3 6 7] | 7     |

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
 deque<int> dq; // stores indices of useful elements
 vector<int> res;
 int n = nums.size();
 for (int i = 0; i < n; ++i) {
 // Remove indices that are out of this window
 if (!dq.empty() && dq.front() <= i - k)
 dq.pop_front();
 // Remove smaller elements from back
 while (!dq.empty() && nums[dq.back()] <= nums[i])
 dq.pop_back();
 // Add current index
 dq.push_back(i);
 // Add current window's max to result (once window size ≥ k)
 if (i >= k - 1)
 res.push_back(nums[dq.front()]);
 }
 return res;
}
/*
```

We maintain a deque such that:

Indices are stored in decreasing order of their values ( $\text{nums}[i]$ )

The front of deque always holds index of current window's maximum

When window moves:

Remove elements from front if they go out of window ( $\text{index} < i - k + 1$ ). Remove elements from back if their value  $\leq$  current element ( $\text{nums}[i]$ ). Push current index  $i$  to deque. If window size  $\geq k$ , add  $\text{nums}[\text{dq.front()}]$  to result

\*/

GenDS-SlidingWindowVariableSize

## General DS - Sliding Window

Description: A sliding window is a subarray or substring that moves over data to solve problems efficiently in linear time.

### Variable Size

- [Longest Substring Without Repeating Characters](#)
- [Minimum Size Subarray Sum](#)
- [Subarray Product Less Than K](#)
- [Max Consecutive Ones](#)
- [Fruits Into Baskets](#)
- [Count Number of Nice Subarrays](#)
- [Minimum Window Substring: Minimum Window Substring](#)

Given a string *s*, find the length of the longest substring without duplicate characters.

Input: *s* = "pwwkew"

Output: 3.

Explanation: The answer is "wke", with the length of 3. Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

```
int lengthOfLongestSubstring(string s) {
 unordered_set<char> seen;
 int left = 0, right = 0, maxLen = 0;
 while (right < s.size()) {
 // if character not seen → expand window
 if (seen.find(s[right]) == seen.end()) {
 seen.insert(s[right]);
 maxLen = max(maxLen, right - left + 1);
 right++;
 }
 else { // if repeated → shrink window
 seen.erase(s[left]);
 left++;
 }
 }
 return maxLen;
}
```

Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

Input: `target = 7, nums = [2,3,1,2,4,3]`

Output: 2

Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

```
int minSubArrayLen(int target, vector<int>& nums) {
 int n = nums.size();
 int left = 0, curSum = 0, minLen = INT_MAX;
 for (int right = 0; right < n; right++) {
 curSum += nums[right];
 while (curSum >= target) {
 minLen = min(minLen, right - left + 1);
 curSum -= nums[left];
 left++;
 }
 }
 return (minLen == INT_MAX) ? 0 : minLen;
}
```

Given an array of integers `nums` and an integer `k`, return the number of contiguous subarrays where the product of all the elements in the subarray is strictly less than `k`.

Input: `nums = [10,5,2,6], k = 100`

Output: 8

Explanation: The 8 subarrays that have product less than 100 are: `[10]`, `[5]`, `[2]`, `[6]`, `[10, 5]`, `[5, 2]`, `[2, 6]`, `[5, 2, 6]`

Note that `[10, 5, 2]` is not included as the product of 100 is not strictly less than `k`.

```

int numSubarrayProductLessThanK(vector<int>& nums, int k) {
 if (k <= 1) return 0; // since product of positives ≥1, no valid
subarray possible
 int left = 0;
 int count = 0;
 long long prod = 1;
 for (int right = 0; right < nums.size(); right++) {
 prod *= nums[right];
 while (prod >= k) { // shrink window if product ≥ k
 prod /= nums[left];
 left++;
 } // all subarrays ending at right and starting from
[left..right] are valid
 count += (right - left + 1);
 }
 return count;
}

```

Given a binary array **nums** and an integer **k**, return the maximum number of consecutive 1's in the array if you can flip at most **k** 0's.

**Input:** **nums** = [1,1,1,0,0,0,1,1,1,1,0], **k** = 2. **Output:** 6

**Explanation:** [1,1,1,0,0,1,1,1,1,1,1]. Bolded numbers were flipped from 0 to 1. The longest subarray is underlined.

```

int longestOnes(vector<int>& nums, int k) {
 int left = 0, zeroCount = 0, maxLen = 0;
 for (int right = 0; right < nums.size(); ++right) {
 if (nums[right] == 0) zeroCount++;
 // shrink window until valid (≤ k flips)
 while (zeroCount > k) {
 if (nums[left] == 0) zeroCount--;
 left++;
 }
 maxLen = max(maxLen, right - left + 1);
 }
 return maxLen;
}

```

You are visiting a farm that has a single row of fruit trees arranged from left to right. The trees are represented by an integer array `fruits` where `fruits[i]` is the type of fruit the *i*th tree produces.

You want to collect as much fruit as possible. However, the owner has some strict rules that you must follow:

You only have two baskets, and each basket can only hold a single type of fruit. There is no limit on the amount of fruit each basket can hold.

Starting from any tree of your choice, you must pick exactly one fruit from every tree (including the start tree) while moving to the right. The picked fruits must fit in one of your baskets.

Once you reach a tree with fruit that cannot fit in your baskets, you must stop.

Given the integer array `fruits`, return the maximum number of fruits you can pick.

Input: `fruits = [1,2,3,2,2]`

Output: 4

Explanation: We can pick from trees `[2,3,2,2]`.

If we had started at the first tree, we would only pick from trees `[1,2]`.

```
int totalFruit(vector<int>& fruits) {
 unordered_map<int, int> basket;
 int left = 0, maxFruits = 0;
 for (int right = 0; right < fruits.size(); ++right) {
 basket[fruits[right]]++; // pick fruit at right
 // if more than 2 fruit types, move left to shrink window
 while (basket.size() > 2) {
 basket[fruits[left]]--;
 if (basket[fruits[left]] == 0)
 basket.erase(fruits[left]);
 left++;
 }
 // update max window size
 maxFruits = max(maxFruits, right - left + 1);
 }
 return maxFruits;
}
```

Given an array of integers `nums` and an integer `k`. A continuous subarray is called nice if there are `k` odd numbers on it.

Return the number of nice sub-arrays.

Input: `nums = [1,1,2,1,1]`, `k = 3`

Output: 2

Explanation: The only sub-arrays with 3 odd numbers are `[1,1,2,1]` and `[1,2,1,1]`.

// Kind of hard to understand - logic dependent on `-> atMost(nums,`

`k) - atMost(nums, k-1)`

```
int numberOfSubarrays(vector<int>& nums, int k) {
 return atMost(nums, k) - atMost(nums, k-1);
}

int atMost(vector<int>& nums, int k) {
 int left = 0, count = 0, odd = 0;
 for (int right = 0; right < nums.size(); right++) {
 if (nums[right] % 2 == 1) odd++;
 while (odd > k) {
 if (nums[left] % 2 == 1) odd--;
 left++;
 }
 count += right - left + 1;
 }
 return count;
}
```

[hard]

Given two strings `s` and `t` of lengths `m` and `n` respectively, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string `""`.

The testcases will be generated such that the answer is unique.

Input: `s = "ADOBECODEBANC"`, `t = "ABC"`

Output: "BANC"

Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string `t`.



```

string minWindow(string s, string t) {
 if (s.size() == 0 || t.size() == 0) return "";
 vector<int> need(128, 0);
 for (char c : t) need[c]++; // required counts
 int required = 0;
 for (int x : need) if (x > 0) required++;
 vector<int> have(128, 0);
 int formed = 0;
 int left = 0, right = 0;
 int bestLen = INT_MAX;
 int bestStart = 0;
 while (right < s.size()) {
 char c = s[right];
 have[c]++;
 if (need[c] > 0 && have[c] == need[c])
 formed++;
 // When valid, start shrinking
 while (formed == required) {
 if (right - left + 1 < bestLen) {
 bestLen = right - left + 1;
 bestStart = left;
 }
 char d = s[left];
 have[d]--;
 if (need[d] > 0 && have[d] < need[d])
 formed--;
 left++;
 }
 right++;
 }
 if (bestLen == INT_MAX) return "";
 return s.substr(bestStart, bestLen);
}

```

# GenDS-TwoPointers

## General DS - Two Pointers

**Description:** The two pointers technique involves having two different indices move through the input at different speeds to solve various array or linked list problems.

- [Two Sum II - Input Array is Sorted](#)
- [Dutch National Flag: Sort Colors](#)
- [Next Permutation](#)
- [Bag of Tokens](#)
- [Container with most water](#)
- [Trapping Rain Water](#)

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where  $1 \leq \text{index1} < \text{index2} \leq \text{numbers.length}$ . Return the indices of the two numbers, index1 and index2, added by one as an integer array [index1, index2] of length 2. The tests are generated such that there is exactly one solution. You may not use the same element twice. Use only constant extra space.

Input: numbers = [2,7,11,15], target = 9

Output: [1,2]

Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].

```
vector<int> twoSum(vector<int>& numbers, int target) {
 int left = 0, right = numbers.size() - 1;
 while (left < right) {
 int sum = numbers[left] + numbers[right];
 if (sum == target)
 return {left + 1, right + 1}; // +1 because 1-indexed
 else if (sum < target)
 left++; // need bigger sum
 else
 right--; // need smaller sum
 }
 return {}; // will never happen, as one solution is guaranteed
}
```

Given an array `nums` with `n` objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Input: `nums = [2,0,2,1,1,0]`

Output: `[0,0,1,1,2,2]`

```
void sortColors(vector<int>& nums) {
 int start = 0;
 int mid = 0;
 int end = nums.size()-1;
 while(mid<=end){
 if(nums[mid]==0){
 swap(nums[mid], nums[start]);
 start++; mid++;
 }
 else if(nums[mid]==1){
 mid++;
 }
 else{
 swap(nums[mid], nums[end]);
 end--;
 }
 }
}
```

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

For example, for arr = [1,2,3], the following are all the permutations of arr: [1,2,3], [1,3,2], [2, 1, 3], [2, 3, 1], [3,1,2], [3,2,1]. The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order). For example, the next permutation of arr = [1,2,3] is [1,3,2].

Similarly, the next permutation of arr = [2,3,1] is [3,1,2].

While the next permutation of arr = [3,2,1] is [1,2,3] because [3,2,1] does not have a lexicographical larger rearrangement.

Given an array of integers nums, find the next permutation of nums. The replacement must be in place and use only constant extra memory.

Input: nums = [1,2,3]

Output: [1,3,2]

// Not quite intuitive, you may/maynot know ans, don't worry.

```
void nextPermutation(vector<int>& nums) {
 int i = nums.size() - 1;
 while (i > 0 && nums[i-1] >= nums[i]) {
 i--;
 }
 if (i == 0) {
 reverse(nums.begin(), nums.end());
 return;
 }
 int j = nums.size() - 1;
 while (j >= i && nums[j] <= nums[i-1]) {
 j--;
 }
 swap(nums[i-1], nums[j]);
 reverse(nums.begin() + i, nums.end());
}
```

You start with an initial power of `power`, an initial score of 0, and a bag of tokens given as an integer array `tokens`, where each `tokens[i]` denotes the value of token `i`. Your goal is to maximize the total score by strategically playing these tokens. In one move, you can play an unplayed token in one of the two ways (but not both for the same token):

- Face-up: If your current power is at least `tokens[i]`, you may play token `i`, losing `tokens[i]` power and gaining 1 score.
- Face-down: If your current score is at least 1, you may play token `i`, gaining `tokens[i]` power and losing 1 score.

Return the maximum possible score you can achieve after playing any number of tokens. Input: `tokens = [100,200,300,400]`, `power = 200`. Output: 2. Explanation: Play the tokens in this order to get a score of 2: Play token0 (100) face-up, reducing power to 100 and increasing score to 1. Play token3 (400) face-down, increasing power to 500 and reducing score to 0. Play token1 (200) face-up, reducing power to 300 and increasing score to 1. Play token2 (300) face-up, reducing power to 0 and increasing score to 2. The maximum score achievable is 2.

```
int bagOfTokensScore(vector<int>& tokens, int power) {
 sort(tokens.begin(), tokens.end());
 int left = 0, right = tokens.size() - 1;
 int score = 0, maxScore = 0;
 while (left <= right) {
 if (power >= tokens[left]) {
 power -= tokens[left];
 score++; left++;
 maxScore = max(maxScore, score);
 } // If we can afford the smallest token → play face-up
 else if (score > 0) {
 power += tokens[right];
 score--; right--;
 } // Else, if possible, trade score for power (face-down)
 else {
 break; // If neither move possible → break
 }
 }
 return maxScore;
}
```

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the `i`th line are  $(i, 0)$  and  $(i, \text{height}[i])$ .

Find two lines that together with the x-axis form a container, such that the container contains the most water.

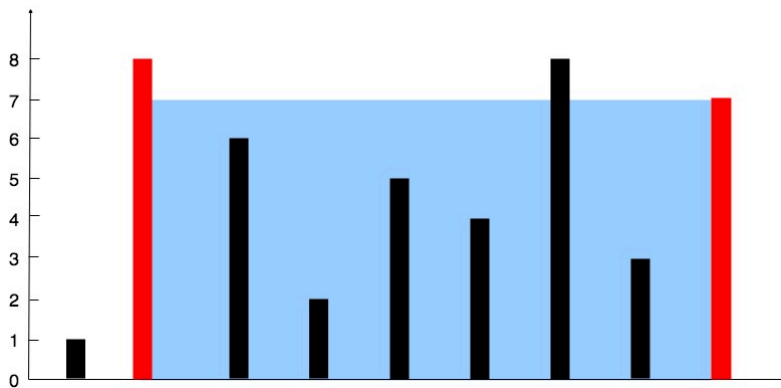
Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Input: `height = [1,8,6,2,5,4,8,3,7]`

Output: 49

Explanation: The above vertical lines are represented by array `[1,8,6,2,5,4,8,3,7]`. In this case, the max area of water (blue section) the container can contain is 49.



```
int maxArea(vector<int>& height) {
 int left = 0, right = height.size() - 1;
 int maxArea = 0;
 while (left < right) {
 int h = min(height[left], height[right]);
 int width = right - left;
 maxArea = max(maxArea, h * width);
 // Move the smaller height inward
 if (height[left] < height[right])
 left++;
 else
 right--;
 }
 return maxArea;
}
```

[hard]\*\*

Given  $n$  non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]

Output: 6

Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.



```
int trap(vector<int>& height) {
 int left = 0, right = height.size() - 1;
 int leftMax = 0, rightMax = 0;
 int water = 0;
 while (left < right) {
 if (height[left] < height[right]) {
 if (height[left] >= leftMax)
 leftMax = height[left];
 else
 water += leftMax - height[left];
 left++;
 } else {
 if (height[right] >= rightMax)
 rightMax = height[right];
 else
 water += rightMax - height[right];
 right--;
 }
 }
 return water;
}
```



# GenDS-CyclicSort

## General DS - Cyclic Sort (Index-Based)

**Description:** Cyclic sort is an efficient approach to solve problems where numbers are consecutively ordered and must be placed in the correct index.

- [Missing Number](#)
- [Set Mismatch](#)
- [Find Missing Numbers](#)
- [First Missing Positive](#)

Given an array `nums` containing `n` distinct numbers in the range `[0, n]`, return the only number in the range that is missing from the array.

Input: `nums = [3,0,1]`

Output: 2

Explanation:

`n = 3` since there are 3 numbers, so all numbers are in the range `[0,3]`. 2 is the missing number in the range since it does not appear in `nums`.

Ans -> Could also use: `Missing number = totalSum - actualSum`.

`totalSum = n(n+1)/2`. Also Bitwise XOR method there. Eg:

```
for(int i=0; i<nums.length; i++){
 res ^= i;
 res ^= nums[i];
}
return res...
```

You have a set of integers `s`, which originally contains all the numbers from 1 to `n`. Unfortunately, due to some error, one of the numbers in `s` got duplicated to another number in the set, which results in repetition of one number and loss of another number. You are given an integer array `nums` representing the data status of this set after the error.

Find the number that occurs twice and the number that is missing and return them in the form of an array.

Input: `nums = [1,2,2,4]`

Output: `[2,3]`

```

vector<int> findErrorNums(vector<int>& nums) {
 int duplicate = -1, missing = -1;
 for (int i = 0; i < nums.size(); i++) {
 int idx = abs(nums[i]) - 1; // Step 1: find the duplicate
 if (nums[idx] < 0) { // already visited → duplicate
 duplicate = abs(nums[i]);
 } else {
 nums[idx] = -nums[idx]; // mark visited
 }
 }
 for (int i = 0; i < nums.size(); i++) {
 if (nums[i] > 0) {
 missing = i + 1;
 break;
 } // Step 2: find the missing (index still positive)
 }
 return {duplicate, missing};
}

```

**Given an array `nums` of `n` integers where `nums[i]` is in the range `[1, n]`, return an array of all the integers in the range `[1, n]` that do not appear in `nums`. Input: `nums = [4,3,2,7,8,2,3,1]`. Output: `[5,6]`**

```

vector<int> findDisappearedNumbers(vector<int>& nums) {
 int n = nums.size();
 // Mark presence by negating nums[x-1]
 for (int i = 0; i < n; i++) {
 int idx = abs(nums[i]) - 1; // target index
 if (nums[idx] > 0)
 nums[idx] = -nums[idx]; // mark as visited
 }
 vector<int> result;
 for (int i = 0; i < n; i++) {
 if (nums[i] > 0) {
 result.push_back(i + 1); // positive means missing
 }
 }
 return result;
}

```

[hard]

Given an unsorted integer array `nums`. Return the smallest positive integer that is not present in `nums`.

You must implement an algorithm that runs in  $O(n)$  time and uses  $O(1)$  auxiliary space.

Input: `nums = [3,4,-1,1]`

Output: 2

Explanation: 1 is in the array but 2 is missing.

```
int firstMissingPositive(vector<int>& nums) {
 int n = nums.size();
 for (int i = 0; i < n; i++) {
 while (nums[i] > 0 && nums[i] <= n && nums[nums[i] - 1] !=
nums[i]) {
 // place nums[i] at its correct position
 swap(nums[i], nums[nums[i] - 1]);
 }
 }
 // now find the first index where nums[i] != i+1
 for (int i = 0; i < n; i++) {
 if (nums[i] != i + 1)
 return i + 1;
 }
 return n + 1;
}
```

# LList-ReversalManipulation

## LinkedList - Reversal & Manipulation of Linked List (In-place)

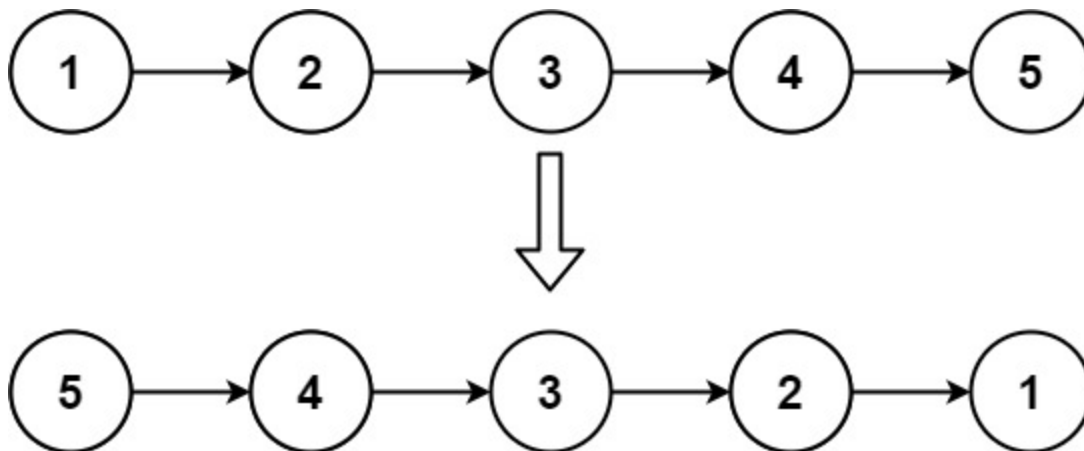
Description: Reversing a linked list in place without using extra space is key for problems that require in-place list manipulations.

- [Reverse Linked List](#)
- [Reverse Nodes in k-Group](#)
- [Swap Nodes in Pairs](#)
- [\[Self added v2\] Flatten LinkedList](#)

Given the head of a singly linked list, reverse the list, and return the reversed list.

Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

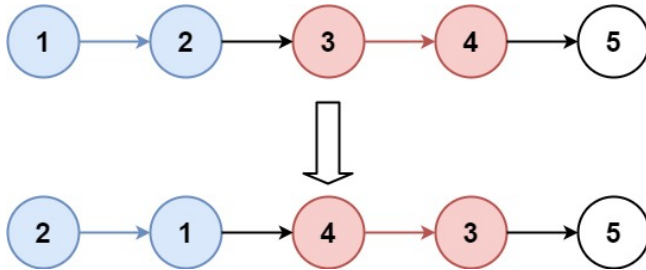


```
ListNode* reverseList(ListNode* head) {
 ListNode* prev = nullptr;
 ListNode* curr = head;
 while (curr) {
 ListNode* next = curr->next; // store next
 curr->next = prev; // reverse pointer
 prev = curr; // move prev
 curr = next; // move curr
 }
 return prev; // new head
}
```

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

You may not alter the values in the list's nodes, only nodes themselves may be changed. Input: head = [1,2,3,4,5], k = 2. Output: [2,1,4,3,5]

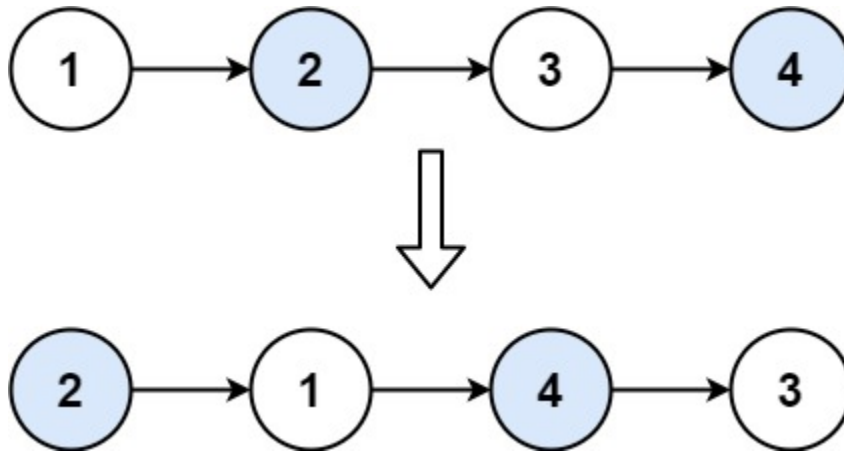


```
ListNode* reverseKGroup(ListNode* head, int k) {
 if (!head || k == 1) return head;
 ListNode dummy(0); dummy.next = head;
 ListNode* prevGroupEnd = &dummy;
 while (true) {
 ListNode* kth = prevGroupEnd;
 for (int i = 0; i < k; i++) {
 kth = kth->next;
 if (!kth) return dummy.next; // Not enough nodes left
 }
 ListNode* groupStart = prevGroupEnd->next;
 ListNode* nextGroupStart = kth->next;
 ListNode* prev = nextGroupStart;
 ListNode* curr = groupStart;
 while (curr != nextGroupStart) {
 ListNode* next = curr->next;
 curr->next = prev;
 prev = curr; curr = next;
 }
 prevGroupEnd->next = kth; // new head of reversed segment
 prevGroupEnd = groupStart; // becomes end of this segment
 }
 return dummy.next;
}
```

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Input: head = [1,2,3,4]

Output: [2,1,4,3]



```
ListNode* swapPairs(ListNode* head) {
 ListNode dummy(0);
 dummy.next = head;
 ListNode* prev = &dummy;
 while (prev->next && prev->next->next) {
 ListNode* first = prev->next;
 ListNode* second = prev->next->next;
 // Swapping
 first->next = second->next;
 second->next = first;
 prev->next = second;
 // Move prev two nodes ahead
 prev = first;
 }
 return dummy.next;
}
```



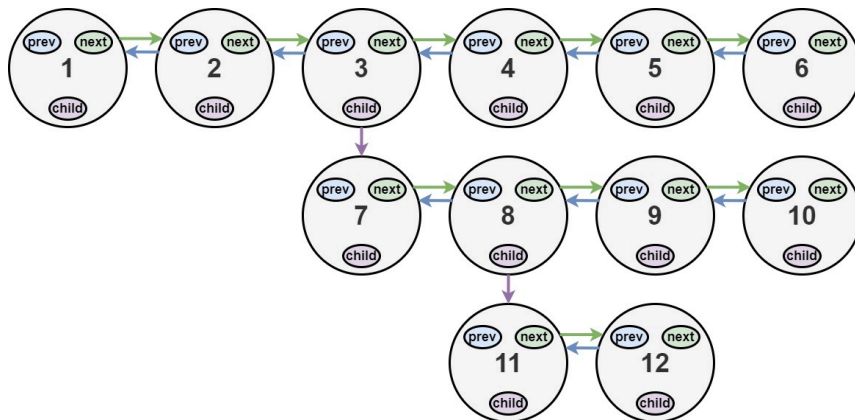
You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.

Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the child list should appear after curr and before curr.next in the flattened list.

Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]

Output: [1,2,3,7,8,11,12,9,10,4,5,6]



Explanation: The multilevel linked list in the input is shown.

After flattening the multilevel linked list it becomes:



```
Node* flatten(Node* head) {
 if (!head) return nullptr;
```

```

 flattenDFS(head);
 return head;
 }
 // returns the tail after flattening
 Node* flattenDFS(Node* node) {
 Node* curr = node;
 Node* last = nullptr;
 while (curr) {
 Node* nextNode = curr->next;
 // If current node has a child
 if (curr->child) {
 // Flatten the child list
 Node* childTail = flattenDFS(curr->child);
 // Insert child between curr and curr->next
 curr->next = curr->child;
 curr->child->prev = curr;
 curr->child = nullptr;
 // Connect child's tail to nextNode
 if (nextNode) {
 childTail->next = nextNode;
 nextNode->prev = childTail;
 }
 last = childTail;
 } else {
 last = curr;
 }
 curr = nextNode;
 }
 return last;
 }
}

```

# GenDS-MatrixManipulation

## General DS - Matrix Manipulation

**Description:** Problems involving 2D arrays (matrices) are often solved using row-column traversal or manipulation based on matrix properties.

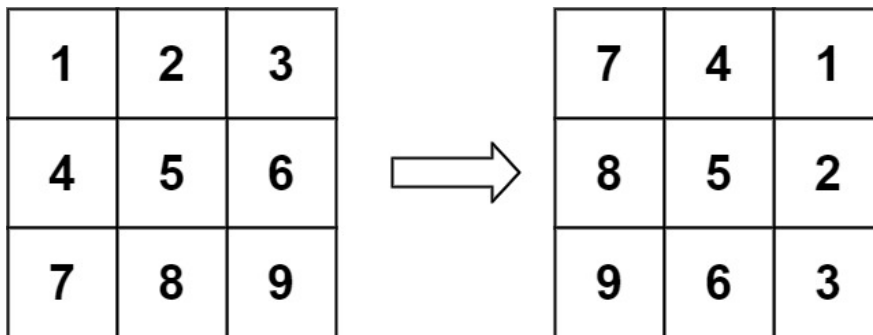
- [Rotate Image](#)
- [Spiral Matrix](#)
- [Set Matrix Zeroes](#)
- [Game of Life](#)

You are given an  $n \times n$  2D matrix representing an image, rotate the image by 90 degrees (clockwise).

You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Input: matrix = `[[1,2,3],[4,5,6],[7,8,9]]`

Output: `[[7,4,1],[8,5,2],[9,6,3]]`



```
void rotate(vector<vector<int>>& matrix) {
 int n = matrix.size();
 // Step 1: transpose
 for (int i = 0; i < n; i++) {
 for (int j = i + 1; j < n; j++) {
 swap(matrix[i][j], matrix[j][i]);
 }
 }
 // Step 2: reverse each row
 for (int i = 0; i < n; i++) {
 reverse(matrix[i].begin(), matrix[i].end());
 }
}
```

Given an m x n matrix, return all elements of the matrix in spiral order.

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]

Output: [1,2,3,6,9,8,7,4,5]

|       |     |     |
|-------|-----|-----|
| 1 →   | 2 → | 3 ↓ |
| 4 →   | 5   | 6 ↓ |
| ↑ 7 ← | 8 ← | 9   |

```
vector<int> spiralOrder(vector<vector<int>>& matrix) {
 int top = 0, bottom = matrix.size() - 1;
 int left = 0, right = matrix[0].size() - 1;
 vector<int> ans;
 while (top <= bottom && left <= right) {
 for (int i = left; i <= right; i++)
 ans.push_back(matrix[top][i]);
 top++;
 for (int i = top; i <= bottom; i++)
 ans.push_back(matrix[i][right]);
 right--;
 if (top <= bottom) {
 for (int i = right; i >= left; i--)
 ans.push_back(matrix[bottom][i]);
 bottom--;
 }
 if (left <= right) {
 for (int i = bottom; i >= top; i--)
 ans.push_back(matrix[i][left]);
 left++;
 }
 }
 return ans;
}
```

Given an  $m \times n$  integer matrix `matrix`, if an element is 0, set its entire row and column to 0's. You must do it in place.

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`.

Output: `[[1,0,1],[0,0,0],[1,0,1]]`

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | ⇒ | 1 | 0 | 1 |
| 1 | 0 | 1 |   | 0 | 0 | 0 |
| 1 | 1 | 1 |   | 1 | 0 | 1 |

```
void setZeroes(vector<vector<int>>& matrix) {
 int m = matrix.size(), n = matrix[0].size();
 bool firstColZero = false;
 for (int i = 0; i < m; i++) {
 if (matrix[i][0] == 0) firstColZero = true;
 for (int j = 1; j < n; j++) {
 if (matrix[i][j] == 0) {
 matrix[i][0] = 0;
 matrix[0][j] = 0;
 }
 }
 }
 for (int i = 1; i < m; i++) {
 for (int j = 1; j < n; j++) {
 if (matrix[i][0] == 0 || matrix[0][j] == 0) {
 matrix[i][j] = 0;
 }
 }
 }
 if (matrix[0][0] == 0) {
 for (int j = 0; j < n; j++) matrix[0][j] = 0;
 }
 if (firstColZero) {
 for (int i = 0; i < m; i++) matrix[i][0] = 0;
 }
}
```

According to Wikipedia's article: "The Game of Life, also known simply as Life, is a cellular automaton devised by the British mathematician John Horton Conway in 1970."

The board is made up of an  $m \times n$  grid of cells, where each cell has an initial state: live (represented by a 1) or dead (represented by a 0). Each cell interacts with its eight neighbors (horizontal, vertical, diagonal) using the following four rules (taken from the above Wikipedia article):

Any live cell with fewer than two live neighbors dies as if caused by under-population.

Any live cell with two or three live neighbors lives on to the next generation.

Any live cell with more than three live neighbors dies, as if by over-population.

Any dead cell with exactly three live neighbors becomes a live cell, as if by reproduction.

The next state of the board is determined by applying the above rules simultaneously to every cell in the current state of the  $m \times n$  grid board. In this process, births and deaths occur simultaneously. Given the current state of the board, update the board to reflect its next state.

Note that you do not need to return anything.

Input: board = `[[0,1,0],[0,0,1],[1,1,1],[0,0,0]]`

Output: `[[0,0,0],[1,0,1],[0,1,1],[0,1,0]]`

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 0 |   | 0 | 0 | 0 |
| 0 | 0 | 1 |   | 1 | 0 | 1 |
| 1 | 1 | 1 | → | 0 | 1 | 1 |
| 0 | 0 | 0 |   | 0 | 1 | 0 |

```
void gameOfLife(vector<vector<int>>& board) {
```

```

int m = board.size(); int n = board[0].size();
int dirs[8][2] = { // Directions (8 neighbors)
 {-1,-1}, {-1,0}, {-1,1}, {0,-1},
 {0,1}, {1,-1}, {1,0}, {1,1}
};
for (int i = 0; i < m; ++i) { // Step 1: Mark transitions
 for (int j = 0; j < n; ++j) {
 int liveNeighbors = 0;
 // Count live neighbors (1 or 2 means originally alive)
 for (auto &d : dirs) {
 int r = i + d[0];
 int c = j + d[1];
 if (r >= 0 && r < m && c >= 0 && c < n) {
 if (board[r][c] == 1 || board[r][c] == 2)
 liveNeighbors++;
 }
 }
 // Rule applications with encoded marking
 if (board[i][j] == 1) {
 // live cell dies → mark 2
 if (liveNeighbors < 2 || liveNeighbors > 3)
 board[i][j] = 2;
 } else {
 // dead cell becomes live → mark 3
 if (liveNeighbors == 3)
 board[i][j] = 3;
 }
 }
}
// Step 2: Convert 2→0 and 3→1
for (int i = 0; i < m; ++i) {
 for (int j = 0; j < n; ++j) {
 if (board[i][j] == 2) board[i][j] = 0;
 if (board[i][j] == 3) board[i][j] = 1;
 }
}
}

```



GenDS-BFS

## General DS - Breadth First Search (BFS)

Description: BFS explores nodes level by level using a queue. It is particularly useful for shortest path problems.

- [Shortest Path in Binary Matrix](#)
- [Rotten Oranges](#)
- [Word Ladder: Word Ladder](#)
- [As Far From Land as Possible](#)

Given an  $n \times n$  binary matrix grid, return the length of the shortest clear path in the matrix. If there is no clear path, return -1.

A clear path in a binary matrix is a path from the top-left cell (i.e.,  $(0, 0)$ ) to the bottom-right cell (i.e.,  $(n - 1, n - 1)$ ) such that:

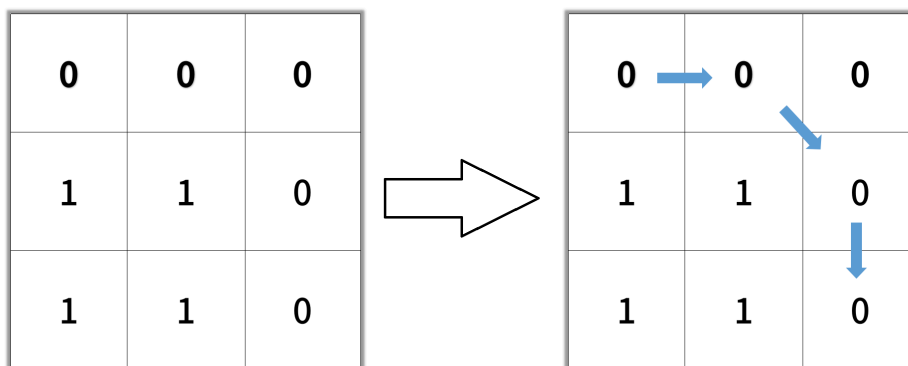
All the visited cells of the path are 0.

All the adjacent cells of the path are 8-directionally connected (i.e., they are different and they share an edge or a corner).

The length of a clear path is the number of visited cells of this path.

Input: grid =  $[[0,0,0],[1,1,0],[1,1,0]]$

Output: 4



```
int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
```

```

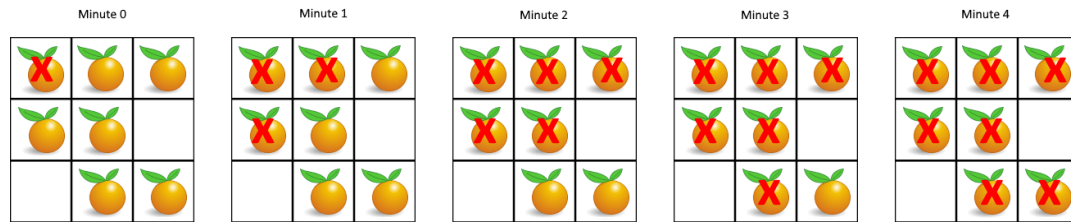
int n = grid.size();
if (grid[0][0] == 1 || grid[n-1][n-1] == 1) return -1;
queue<pair<int,int>> q;
q.push(make_pair(0, 0));
grid[0][0] = 1; // distance
int dirs[8][2] = {
 {1,0}, {-1,0}, {0,1}, {0,-1},
 {1,1}, {1,-1}, {-1,1}, {-1,-1}
};
while (!q.empty()) {
 pair<int,int> p = q.front();
 q.pop();
 int r = p.first, c = p.second;
 int dist = grid[r][c];
 if (r == n-1 && c == n-1) return dist;
 for (int i=0;i<8;i++) {
 int nr = r + dirs[i][0];
 int nc = c + dirs[i][1];
 if (nr>=0 && nr<n && nc>=0 && nc<n && grid[nr][nc]==0) {
 grid[nr][nc] = dist + 1;
 q.push(make_pair(nr, nc));
 }
 }
}
return -1;
}

```

You are given an  $m \times n$  grid where each cell can have one of three values: 0 representing an empty cell, 1 representing a fresh orange, or 2 representing a rotten orange. Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Input: grid = [[2,1,1],[1,1,0],[0,1,1]]

Output: 4



```
int orangesRotting(vector<vector<int>>& grid) {
 int m = grid.size(), n = grid[0].size();
 queue<pair<int,int>> q; int fresh = 0;
 for (int i=0;i<m;i++) {
 for (int j=0;j<n;j++) {
 if (grid[i][j] == 2) q.push(make_pair(i,j));
 else if (grid[i][j] == 1) fresh++;
 }
 }
 if (fresh == 0) return 0;
 int minutes = -1;
 int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
 while (!q.empty()) {
 int size = q.size();
 minutes++;
 while (size-->0) {
 pair<int,int> p = q.front();
 q.pop();
 int r = p.first, c = p.second;
 for (int i=0;i<4;i++) {
 int nr = r + dirs[i][0];
 int nc = c + dirs[i][1];
 if (nr<0 && nr<m && nc<0 && nc<n && grid[nr][nc]
== 1) {
 grid[nr][nc] = 2;
 fresh--;
 q.push(make_pair(nr,nc));
 }
 }
 }
 }
 return (fresh == 0 ? minutes : -1);
}
```

[hard]

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words `beginWord -> s1 -> s2 -> ... -> sk` such that: Every adjacent pair of words differs by a single letter. Every `si` for  $1 \leq i \leq k$  is in `wordList`. Note that `beginWord` does not need to be in `wordList`.

`sk == endWord`. Given two words, `beginWord` and `endWord`, and a dictionary `wordList`, return the number of words in the shortest transformation sequence from `beginWord` to `endWord`, or 0 if no such sequence exists. Input: `beginWord = "hit"`, `endWord = "cog"`, `wordList = ["hot","dot","dog","lot","log","cog"]`. Output: 5. Explanation: One shortest transformation sequence is `"hit" -> "hot" -> "dot" -> "dog" -> cog"`, which is 5 words long.

```
int ladderLength(string beginWord, string endWord, vector<string>&
wordList) {
 unordered_set<string> dict(wordList.begin(), wordList.end());
 if (!dict.count(endWord)) return 0;
 queue<pair<string,int>> q;
 q.push(make_pair(beginWord, 1));
 while (!q.empty()) {
 pair<string,int> p = q.front();
 q.pop();
 string word = p.first; int steps = p.second;
 if (word == endWord) return steps;
 for (int i=0;i<word.size();i++) {
 string temp = word;
 for (char c='a'; c<='z'; c++) {
 temp[i] = c;
 if (dict.count(temp)) {
 dict.erase(temp);
 q.push(make_pair(temp, steps + 1));
 }
 }
 }
 }
 return 0;
}
```

Given an  $n \times n$  grid containing only values 0 and 1, where 0 represents water and 1 represents land, find a water cell such that its distance to the nearest land cell is maximized, and return the distance. If no land or water exists in the grid, return -1. The distance used in this problem is the Manhattan distance: the distance between two cells  $(x_0, y_0)$  and  $(x_1, y_1)$  is  $|x_0 - x_1| + |y_0 - y_1|$ . Input: `grid = [[1,0,1],[0,0,0],[1,0,1]]`. Output: 2  
Explanation: The cell (1, 1) is as far as possible from all the land with distance 2.

|   |   |   |
|---|---|---|
| 1 | 0 | 1 |
| 0 | 0 | 0 |
| 1 | 0 | 1 |

```
int maxDistance(vector<vector<int>>& grid) {
 int n = grid.size(); queue<pair<int,int>> q;
 for (int i=0;i<n;i++) {
 for (int j=0;j<n;j++) {
 if (grid[i][j] == 1) {
 q.push(make_pair(i,j));
 }
 }
 }
 if (q.empty() || q.size() == n*n) return -1;
 int dirs[4][2] = {{1,0},{-1,0},{0,1},{0,-1}};
 int dist = -1;
 while (!q.empty()) {
 int size = q.size(); dist++;
 while (size-->0) {
 pair<int,int> p = q.front();
 q.pop();
 int r = p.first, c = p.second;
 for (int i=0;i<4;i++) {
 int nr = r + dirs[i][0];
 int nc = c + dirs[i][1];
 if (nr>=0 && nr<n && nc>=0 && nc<n && grid[nr][nc]
== 0) {
 grid[nr][nc] = 1; // visited
 q.push(make_pair(nr,nc));
 }
 }
 }
 }
 return dist;
}
```

GenDS-DFS

## General DS - Depth First Search (DFS)

Description: DFS explores as far as possible along a branch before backtracking. It's useful for graph traversal, pathfinding, and connected components.

- [Number of Closed Islands](#)
- [Coloring a Border](#)
- [DFS from boundary: Number of Enclaves](#)
- [Shortest time: Time Needed to Inform all Employees](#)
- [Cyclic Find: Find Eventual Safe States](#)

Given a 2D grid consists of 0s (land) and 1s (water). An island is a maximal 4-directionally connected group of 0s and a closed island is an island totally (all left, top, right, bottom) surrounded by 1s.

Return the number of closed islands.

Input: grid =

```
[[1,1,1,1,1,1,1,0],[1,0,0,0,0,1,1,0],[1,0,1,0,1,1,1,0],[1,0,0,0,0,1,0,1],[1,1,1,1,1,1,1,0]]
```

Output: 2

Explanation:

Islands in gray are closed because they are completely surrounded by water (group of 1s).

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 |



```

int closedIsland(vector<vector<int>>& grid) {
 int res = 0;
 for (int i = 0; i < grid.size(); i++){
 for (int j = 0; j < grid[0].size(); j++){
 if (grid[i][j] == 0){
 res += dfs(grid, i, j) ? 1 : 0;
 }
 }
 }
 return res;
}

bool dfs(vector<vector<int>>& g, int i, int j){
 if (i < 0 || j < 0 || i >= g.size() || j >= g[0].size()){
 return false;
 }
 if (g[i][j] == 1 || g[i][j] == 2){
 return true;
 }
 g[i][j] = 2;
 /* Note: Cannot use: `return dfs(g, i+1, j) && dfs(g, i, j+1) &&
 dfs(g, i-1, j) && dfs(g, i, j-1);` as if any first dfs() returns
 false, following ones will not execute, hence won't have chance to
 mark them as 0s/1s - Short circuit evaluation
 */
 bool down = dfs(g, i+1, j);
 bool right = dfs(g, i, j+1);
 bool up = dfs(g, i-1, j);
 bool left = dfs(g, i, j-1);
 return up && down && left && right;
}

```

You are given an  $m \times n$  integer matrix `grid`, and three integers `row`, `col`, and `color`. Each value in the grid represents the color of the grid square at that location.

Two squares are called adjacent if they are next to each other in any of the 4 directions.

Two squares belong to the same connected component if they have the same color and they are adjacent.

The border of a connected component is all the squares in the connected component that are either adjacent to (at least) a square not in the component, or on the boundary of the grid (the first or last row or column).

You should color the border of the connected component that contains the square `grid[row][col]` with `color`.

Return the final grid.

Input: `grid = [[1,1,1],[1,1,1],[1,1,1]]`, `row = 1`, `col = 1`, `color = 2`. Output: `[[2,2,2],[2,1,2],[2,2,2]]`

```
void dfs(int row, int col, int color, int orig, vector<vector<int>>&
grid, vector<vector<int>> &vis){
 int m= grid.size(), n = grid[0].size();
 vis[row][col]=1;
 int a[] = {0, -1,0,1}; int b[]= {1, 0, -1, 0};
 for(int k=0; k<4 ; k++){
 int newr = row + a[k];
 int newc = col + b[k];
 if(newr<0 || newc<0 || newr>=m || newc>=n){
 grid[row][col] = color;
 continue;
 }
 if(vis[newr][newc]) continue;
 if(grid[newr][newc] == orig){ // if it's connected
 dfs(newr, newc, color, orig, grid, vis);
 }
 else{
 grid[row][col] = color;
 }
 }
 return;
}
```

```

vector<vector<int>> colorBorder(vector<vector<int>>& grid, int row,
int col, int color) {
 int m= grid.size(), n = grid[0].size();
 vector<vector<int>> vis(m , vector<int>(n,0));
 dfs(row, col, color, grid[row][col], grid, vis);
 return grid;
}

```

You are given an  $m \times n$  binary matrix `grid`, where 0 represents a sea cell and 1 represents a land cell.

A move consists of walking from one land cell to another adjacent (4-directionally) land cell or walking off the boundary of the grid. Return the number of land cells in `grid` for which we cannot walk off the boundary of the grid in any number of moves.

Input: `grid = [[0,0,0,0],[1,0,1,0],[0,1,1,0],[0,0,0,0]]`

Output: 3

Explanation: There are three 1s that are enclosed by 0s, and one 1 that is not enclosed because its on the boundary.

|   |   |   |   |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 |

```

void dfs(int i, int j, vector<vector<int>>& grid) {
 int n = grid.size(), m = grid[0].size();
 if (i < 0 || j < 0 || i >= n || j >= m) return;
 if (grid[i][j] != 1) return; // water or already visited
 grid[i][j] = 2; // Mark land as visited (escaped land)
 dfs(i - 1, j, grid);
 dfs(i + 1, j, grid);
 dfs(i, j - 1, grid);
 dfs(i, j + 1, grid);
}

int numEnclaves(vector<vector<int>>& grid) {

```

```

int n = grid.size(), m = grid[0].size();
// Traverse boundary
for (int j = 0; j < m; j++) {
 dfs(0, j, grid);
 dfs(n - 1, j, grid);
}
for (int i = 0; i < n; i++) {
 dfs(i, 0, grid);
 dfs(i, m - 1, grid);
}
// Count land that never got visited
int ans = 0;
for (int i = 0; i < n; i++) {
 for (int j = 0; j < m; j++) {
 if (grid[i][j] == 1)
 ans++;
 }
}
return ans;
}

```

A company has  $n$  employees with a unique ID for each employee from 0 to  $n - 1$ . The head of the company is the one with headID.

Each employee has one direct manager given in the manager array where  $manager[i]$  is the direct manager of the  $i$ -th employee,  $manager[headID] = -1$ . Also, it is guaranteed that the subordination relationships have a tree structure.

The head of the company wants to inform all the company employees of an urgent piece of news. He will inform his direct subordinates, and they will inform their subordinates, and so on until all employees know about the urgent news.

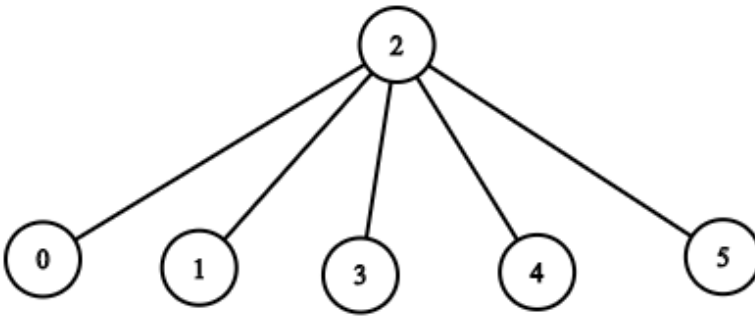
The  $i$ -th employee needs  $informTime[i]$  minutes to inform all of his direct subordinates (i.e., After  $informTime[i]$  minutes, all his direct subordinates can start spreading the news).

Return the number of minutes needed to inform all the employees about the urgent news.

Input:  $n = 6$ ,  $headID = 2$ ,  $manager = [2,2,-1,2,2,2]$ ,  $informTime = [0,0,1,0,0,0]$

Output: 1

Explanation: The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to inform them all. The tree structure of the employees in the company is shown.



Other eg:

1+3

Example 1:

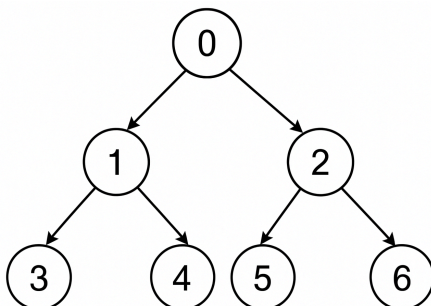
Input: n = 1, headID = 0, manager = [-1], informTime = [0]  
Output: 0  
Explanation: The head of the company is the only employee in the company.

Example 2:

```
graph TD; 2((2)) --- 0((0)); 2 --- 1((1)); 2 --- 3((3)); 2 --- 4((4)); 2 --- 5((5));
```

Input: n = 6, headID = 2, manager = [2,2,-1,2,2,2],  
informTime = [0,0,1,0,0,0]  
Output: 1  
Explanation: The head of the company with id = 2 is the direct manager of all the employees in the company and needs 1 minute to inform them all.  
The tree structure of the employees in the company is

Other eg 2: For n=7, headID=0, manager=[-1,0,0,1,1,2,2],  
informTime=[2,3,1,0,0,0,0]



```
int dfs(int u, vector<vector<int>>& adj, vector<int>& informTime) {
 int maxTime = 0;
```

```

 for (int v : adj[u]) {
 maxTime = max(maxTime, dfs(v, adj, informTime));
 }
 return informTime[u] + maxTime;
}
int numOfMinutes(int n, int headID, vector<int>& manager,
vector<int>& informTime) {
 vector<vector<int>> adj(n);
 for (int i = 0; i < n; i++) {
 if (manager[i] != -1)
 adj[manager[i]].push_back(i); // Build tree
 }
 return dfs(headID, adj, informTime);
}

```

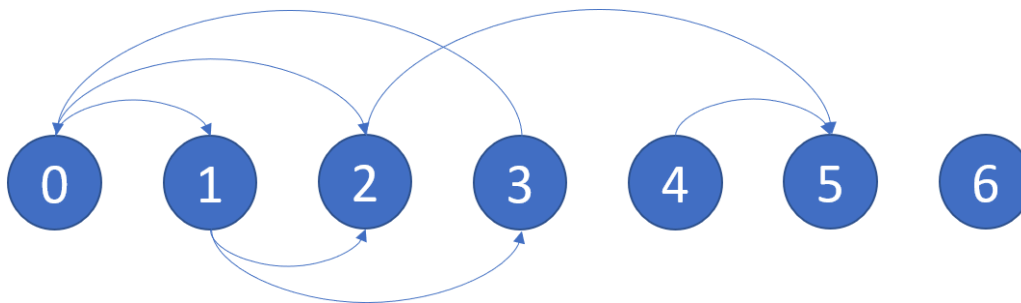
There is a directed graph of  $n$  nodes with each node labeled from  $0$  to  $n - 1$ . The graph is represented by a  $0$ -indexed  $2D$  integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node  $i$ , meaning there is an edge from node  $i$  to each node in `graph[i]`. A node is a terminal node if there are no outgoing edges. A node is a safe node if every possible path starting from that node leads to a terminal node (or another safe node).

Return an array containing all the safe nodes of the graph. The answer should be sorted in ascending order.

Input: `graph = [[1,2],[2,3],[5],[0],[5],[],[[]]`. Output: `[2,4,5,6]`

Explanation: The given graph is shown above.

Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them. Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.



```

bool dfs(int node, vector<vector<int>>& graph, vector<int>& vis,
vector<int>& dfsVis, vector<int>& safe) {

```

```

vis[node] = 1;
dfsVis[node] = 1;
for (int nxt : graph[node]) {
 if (!vis[nxt]) {
 bool hasCycle = dfs(nxt, graph, vis, dfsVis, safe);
 if (hasCycle) {
 return true; // cycle reachable → unsafe
 }
 }
 else if (dfsVis[nxt]) {
 return true; // back-edge → cycle
 }
}
dfsVis[node] = 0; // backtrack
safe[node] = 1; // no cycle found from this node
return false; // this path is safe
}

vector<int> eventualSafeNodes(vector<vector<int>>& graph) {
 // Similar logic to finding cycle in a graph
 int n = graph.size();
 vector<int> vis(n, 0), dfsVis(n, 0), safe(n, 0);
 for (int i = 0; i < n; i++) {
 if (!vis[i]) {
 dfs(i, graph, vis, dfsVis, safe);
 }
 }
 vector<int> ans;
 for (int i = 0; i < n; i++) {
 if (safe[i]) ans.push_back(i);
 }
 return ans;
}

```

# GenDS-Backtracking



## General DS - Backtracking

Description: Backtracking helps in problems where you need to explore all potential solutions, such as solving puzzles, generating combinations, or finding paths.

- [Permutation ii](#)
- [Combination Sum](#)
- [Generate Parenthesis](#)
- [N-Queens](#)
- [Sudoku Solver](#)
- [Palindrome Partitioning](#)
- [Word Search: Word Search](#)

Given a collection of numbers, `nums`, that might contain duplicates, return all possible unique permutations in any order.

Input: `nums = [1,1,2]`

Output:

```
[[1,1,2],
 [1,2,1],
 [2,1,1]]
```

```
vector<vector<int>> ans; vector<int> temp; vector<bool> used;
void backtrack(vector<int>& nums) {
 if (temp.size() == nums.size()) {
 ans.push_back(temp);
 return;
 }
 for (int i = 0; i < nums.size(); i++) {
 if (used[i]) continue; // Skip duplicates
 if (i > 0 && nums[i] == nums[i-1] && !used[i-1])
 continue;
 used[i] = true;
 temp.push_back(nums[i]);
 backtrack(nums);
 used[i] = false;
 temp.pop_back();
 }
}
```

```
vector<vector<int>> permuteUnique(vector<int>& nums) {
 sort(nums.begin(), nums.end());
 used.assign(nums.size(), false);
 backtrack(nums);
 return ans;
}
```

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input. Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]. Explanation: 2 and 3 are candidates, and  $2 + 2 + 3 = 7$ . Note that 2 can be used multiple times. 7 is a candidate, and  $7 = 7$ . These are the only two combinations.

```
vector<vector<int>> res; vector<int> curr;
void dfs(vector<int>& c, int target, int idx) {
 if (target == 0) {
 res.push_back(curr);
 return;
 }
 if (idx == c.size() || target < 0) return;
 // include
 curr.push_back(c[idx]);
 dfs(c, target - c[idx], idx);
 curr.pop_back();
 // skip
 dfs(c, target, idx + 1);
}
vector<vector<int>> combinationSum(vector<int>& c, int target) {
 dfs(c, target, 0);
 return res;
}
```

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Input: n = 3

Output: ["((()))","(()())","(())()","()(())","()()()"]

```
vector<string> ans;
void backtrack(int open, int close, string s, int n) {
 if (s.size() == 2*n) {
 ans.push_back(s);
 return;
 }
 if (open < n) backtrack(open + 1, close, s + "(", n);
 if (close < open) backtrack(open, close + 1, s + ")", n);
}
vector<string> generateParenthesis(int n) {
 backtrack(0, 0, "", n);
 return ans;
}
```

[hard]

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.

Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.

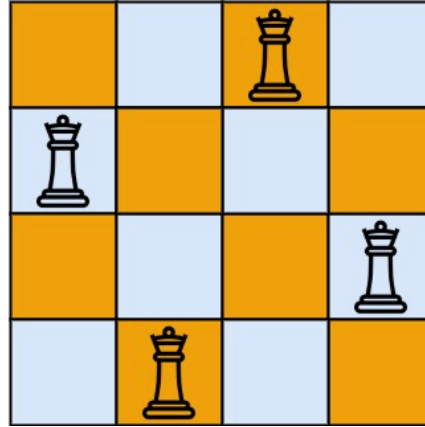
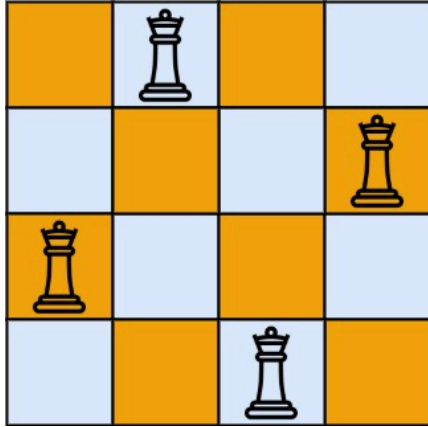
Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Input: n = 4

Output:

[[".Q...", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]

Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above



```
vector<vector<string>> res;
vector<string> board;
vector<bool> col, diag1, diag2;
void dfs(int r, int n) {
 if (r == n) {
 res.push_back(board);
 return;
 }
 for (int c = 0; c < n; c++) {
 if (col[c] || diag1[r+c] || diag2[r-c+n-1]) continue;
 board[r][c] = 'Q';
 col[c] = diag1[r+c] = diag2[r-c+n-1] = true;
 dfs(r + 1, n);
 board[r][c] = '.';
 col[c] = diag1[r+c] = diag2[r-c+n-1] = false;
 }
}
vector<vector<string>> solveNQueens(int n) {
 board.assign(n, string(n, '.')); /*Equivalent to: board.clear();
 then for (int i = 0; i < n; i++) board.push_back(string(n,
 '.'));...*/
 col.assign(n, false);
 diag1.assign(2*n, false);
 diag2.assign(2*n, false);
 dfs(0, n);
 return res;
}
```

[hard]

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

Each of the digits 1-9 must occur exactly once in each row.

Each of the digits 1-9 must occur exactly once in each column.

Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

Input: board =

```
[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".",
".","."],[".","9","8",".",".",".",".","6","."],["8",".",".",".","6",
".",".",".",3"],["4",".",".",8",".",3",".",1"],["7",".",.",
".","2",".",.",6"],[".",6",".",.",.",.",2","8","."],[".",
".",".",4","1","9",".",.",5"],[".",.",.",.",8",".",.",7","9
"]]
```

Output:

```
[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3",
"4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6",
"1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3",
"9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2",
"8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9
"]]
```

Explanation: The input board is shown above and the only valid solution is shown below:

|   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|
| 5 | 3 | 4 | 6 | 7 | 8 | 9 | 1 | 2 |
| 6 | 7 | 2 | 1 | 9 | 5 | 3 | 4 | 8 |
| 1 | 9 | 8 | 3 | 4 | 2 | 5 | 6 | 7 |
| 8 | 5 | 9 | 7 | 6 | 1 | 4 | 2 | 3 |
| 4 | 2 | 6 | 8 | 5 | 3 | 7 | 9 | 1 |
| 7 | 1 | 3 | 9 | 2 | 4 | 8 | 5 | 6 |
| 9 | 6 | 1 | 5 | 3 | 7 | 2 | 8 | 4 |
| 2 | 8 | 7 | 4 | 1 | 9 | 6 | 3 | 5 |
| 3 | 4 | 5 | 2 | 8 | 6 | 1 | 7 | 9 |

```
bool isValid(vector<vector<char>>& b, int r, int c, char ch) {
```

```

 for (int i = 0; i < 9; i++) {
 if (b[r][i] == ch) return false;
 if (b[i][c] == ch) return false;
 int br = 3*(r/3) + i/3;
 int bc = 3*(c/3) + i%3;
 if (b[br][bc] == ch) return false;
 }
 return true;
 }
 bool solve(vector<vector<char>>& b) {
 for (int r = 0; r < 9; r++) {
 for (int c = 0; c < 9; c++) {
 if (b[r][c] == '.') {
 for (char ch = '1'; ch <= '9'; ch++) {
 if (isValid(b, r, c, ch)) {
 b[r][c] = ch;
 if (solve(b)) return true;
 b[r][c] = '.';
 }
 }
 return false;
 }
 }
 }
 return true;
 }
}

void solveSudoku(vector<vector<char>>& board) {
 solve(board);
}

```

**Given a string *s*, partition *s* such that every substring of the partition is a palindrome. Return all possible palindrome partitioning of *s*.**

**Input:** *s* = "aab"

**Output:** `[["a","a","b"],["aa","b"]]`

`vector<vector<string>> ans;`

```

vector<string> curr;
bool isPal(string &s, int l, int r) {
 while (l < r) {
 if (s[l] != s[r]) return false;
 l++; r--;
 }
 return true;
}
void dfs(string &s, int idx) {
 if (idx == s.size()) {
 ans.push_back(curr);
 return;
 }
 for (int end = idx; end < s.size(); end++) {
 if (isPal(s, idx, end)) {
 curr.push_back(s.substr(idx, end - idx + 1));
 dfs(s, end + 1);
 curr.pop_back();
 }
 }
}
vector<vector<string>> partition(string s) {
 dfs(s, 0);
 return ans;
}

```

Given an  $m \times n$  grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Input: board =

[[ "A","B","C","E"],[ "S","F","C","S"],[ "A","D","E","E"]], word = "ABCCED"

Output: true

|          |          |          |          |
|----------|----------|----------|----------|
| <b>A</b> | <b>B</b> | <b>C</b> | <b>E</b> |
| <b>S</b> | <b>F</b> | <b>C</b> | <b>S</b> |
| <b>A</b> | <b>D</b> | <b>E</b> | <b>E</b> |

```

bool dfs(vector<vector<char>>& b, string& w, int i, int j, int idx)
{
 if (idx == w.size()) return true;
 if (i < 0 || j < 0 || i >= b.size() || j >= b[0].size()) return
false;
 if (b[i][j] != w[idx]) return false;
 char temp = b[i][j];
 b[i][j] = '#'; // mark visited
 bool found = dfs(b, w, i+1, j, idx+1)
 || dfs(b, w, i-1, j, idx+1)
 || dfs(b, w, i, j+1, idx+1)
 || dfs(b, w, i, j-1, idx+1);
 b[i][j] = temp; // unmark
 return found;
}

bool exist(vector<vector<char>>& board, string word) {
 int m = board.size(), n = board[0].size();
 for (int i = 0; i < m; i++) {
 for (int j = 0; j < n; j++) {
 if (dfs(board, word, i, j, 0))
 return true;
 }
 }
 return false;
}

```



# GenDS-BinarySearch

## General DS - Modified Binary Search

Description: A modified version of binary search that applies to rotated arrays, unsorted arrays, or specialized conditions.

- [Search in Rotated Sorted Array](#)
- [Find Minimum in Rotated Sorted Array](#)
- [Find Peak Element](#)
- [Single element in a sorted array](#)
- [Minimum Time to Arrive on Time](#)
- [Capacity to Ship Packages within 'd' Days](#)
- [Koko Eating Bananas](#)
- [Find in Mountain Array](#)
- [Median of Two Sorted Arrays](#)
- [\[Self added v2\] Magnetic force between two balls](#)
- [\[Self added v2\] Split Array Largest Sum](#)

There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).

Before being passed to your function, `nums` is rotated at an unknown pivot index `k` ( $0 \leq k < \text{nums.length}$ ) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (0-indexed). For example, `[0,1,2,4,4,4,5,6,6,7]` might be rotated at pivot index 5 and become `[4,5,6,6,7,0,1,2,4,4]`.

Given the array `nums` after the rotation and an integer `target`, return `true` if `target` is in `nums`, or `false` if it is not in `nums`.

You must decrease the overall operation steps as much as possible.

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0`

Output: `true`

```

bool search(vector<int>& nums, int target) {
 int l = 0, r = nums.size() - 1;
 while (l <= r) {
 int mid = l + (r - l) / 2;
 if (nums[mid] == target) return true;
 // Handle duplicates
 if (nums[l] == nums[mid] && nums[mid] == nums[r]) {
 l++;
 r--;
 }
 // Left is sorted
 else if (nums[l] <= nums[mid]) {
 if (nums[l] <= target && target < nums[mid])
 r = mid - 1;
 else
 l = mid + 1;
 }
 // Right is sorted
 else {
 if (nums[mid] < target && target <= nums[r])
 l = mid + 1;
 else
 r = mid - 1;
 }
 }
 return false;
}

```

Suppose an array of length  $n$  sorted in ascending order is rotated between 1 and  $n$  times. For example, the array `nums = [0,1,2,4,5,6,7]` might become: `[4,5,6,7,0,1,2]` if it was rotated 4 times.

`[0,1,2,4,5,6,7]` if it was rotated 7 times. Notice that rotating an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`. Given the sorted rotated array `nums` of unique elements, return the minimum element of this array. You must write an algorithm that runs in  $O(\log n)$  time.

Input: `nums = [3,4,5,1,2]`. Output: 1.

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

```

int findMin(vector<int>& nums) {
 int l = 0, r = nums.size() - 1;
 int ans = INT_MAX;
 while (l <= r) {
 int mid = l + (r - l) / 2;
 // If left part is sorted
 if (nums[l] <= nums[mid]) {
 // the minimum of this sorted region is nums[l]
 ans = min(ans, nums[l]);
 l = mid + 1;
 }
 // Else right part is sorted
 else {
 // the minimum of this sorted region is nums[mid]
 ans = min(ans, nums[mid]);
 r = mid - 1;
 }
 }
 return ans;
}

```

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in  $O(\log n)$  time.

Input: `nums = [1,2,1,3,5,6,4]`. Output: 5. Explanation: Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.

```

int findPeakElement(vector<int> &arr) {
 int n = arr.size();
 if(n == 1) return 0;
 if(arr[0] > arr[1]) return 0;
 if(arr[n-1] > arr[n-2]) return n - 1;
 int low = 1, high = n - 2;
 while(low <= high) {
 int mid = (low + high) / 2;
 // If mid is peak
 if(arr[mid] > arr[mid - 1] && arr[mid] > arr[mid + 1]) {
 return mid;
 }
 // Move to the side that has the bigger slope
 else if(arr[mid] > arr[mid - 1]) {
 low = mid + 1; // you're in an increasing slope
 }
 else {
 high = mid - 1; // you're in a decreasing slope
 }
 }
 return -1;
}

```

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once. Return the single element that appears only once. Your solution must run in  $O(\log n)$  time and  $O(1)$  space.

Input: nums = [1,1,2,3,3,4,4,8,8]. Output: 2

```

int singleNonDuplicate(vector<int>& nums) {
 int l = 0, r = nums.size() - 1;
 while (l < r) {
 int mid = l + (r - l) / 2;
 if (mid % 2 == 1) mid--; // make mid even
 if (nums[mid] == nums[mid + 1]) l = mid + 2;
 else r = mid;
 }
 return nums[l];
}

```

You are given a floating-point number `hour`, representing the amount of time you have to reach the office. To commute to the office, you must take `n` trains in sequential order. You are also given an integer array `dist` of length `n`, where `dist[i]` describes the distance (in kilometers) of the `i`th train ride.

Each train can only depart at an integer hour, so you may need to wait in between each train ride.

For example, if the 1st train ride takes 1.5 hours, you must wait for an additional 0.5 hours before you can depart on the 2nd train ride at the 2 hour mark.

Return the minimum positive integer speed (in kilometers per hour) that all the trains must travel at for you to reach the office on time, or -1 if it is impossible to be on time.

Tests are generated such that the answer will not exceed  $10^7$  and hour will have at most two digits after the decimal point.

Input: `dist = [1,3,2]`, `hour = 6`

Output: 1

Explanation: At speed 1:

- The first train ride takes  $1/1 = 1$  hour.
- Since we are already at an integer hour, we depart immediately at the 1 hour mark. The second train takes  $3/1 = 3$  hours.
- Since we are already at an integer hour, we depart immediately at the 4 hour mark. The third train takes  $2/1 = 2$  hours.
- You will arrive at exactly the 6 hour mark.

```

double calc(vector<int>& dist, int speed) {
 double t = 0.0;
 int n = dist.size();
 for (int i = 0; i < n - 1; i++) {
 t += ceil((double)dist[i] / speed);
 }
 t += (double)dist[n-1] / speed;
 return t;
}

int minSpeedOnTime(vector<int>& dist, double hour) {
 int l = 1, r = INT_MAX/2;
 int ans = -1;
 while (l <= r) {
 int m = l + (r - l) / 2;
 if (calc(dist, m) <= hour) {
 ans = m;
 r = m - 1;
 } else {
 l = m + 1;
 }
 }
 return ans;
}

```

A conveyor belt has packages that must be shipped from one port to another within  $days$  days. The  $i$ th package on the conveyor belt has a weight of  $weights[i]$ . Each day, we load the ship with packages on the conveyor belt (in the order given by  $weights$ ). We may not load more weight than the maximum weight capacity of the ship. Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within  $days$  days.

Input:  $weights = [1,2,3,4,5,6,7,8,9,10]$ ,  $days = 5$ . Output: 15

Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this: 1st day: 1, 2, 3, 4, 5; 2nd day: 6, 7; 3rd day: 8; 4th day: 9; 5th day: 10. Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.

```

bool canShip(vector<int>& w, int cap, int days) {
 int need = 1, cur = 0;
 for (int x : w) {
 if (x > cap) return false;
 if (cur + x > cap) {
 need++; cur = 0;
 }
 cur += x;
 }
 return need <= days;
}

int shipWithinDays(vector<int>& weights, int days) {
 int l = 1, r = 0;
 for (int x : weights) r += x;
 int ans = r;
 while (l <= r) {
 int mid = l + (r - l) / 2;
 if (canShip(weights, mid, days)) {
 ans = mid;
 r = mid - 1;
 } else {
 l = mid + 1;
 }
 }
 return ans;
}

```

Koko loves to eat bananas. There are  $n$  piles of bananas, the  $i$ th pile has  $\text{piles}[i]$  bananas. The guards have gone and will come back in  $h$  hours. Koko can decide her bananas-per-hour eating speed of  $k$ . Each hour, she chooses some pile of bananas and eats  $k$  bananas from that pile. If the pile has less than  $k$  bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return the minimum integer  $k$  such that she can eat all the bananas within  $h$  hours.

Input:  $\text{piles} = [3,6,7,11]$ ,  $h = 8$ . Output: 4



```

bool canEat(vector<int>& p, int h, int k) {
 long long t = 0;
 for (int x : p) {
 t += (x + k - 1) / k;
 }
 return t <= h;
}

int minEatingSpeed(vector<int>& piles, int h) {
 int l = 1, r = INT_MAX/2;
 int ans = r;
 while (l <= r) {
 int mid = l + (r - l) / 2;
 if (canEat(piles, h, mid)) {
 ans = mid;
 r = mid - 1;
 } else {
 l = mid + 1;
 }
 }
 return ans;
}

```

[hard]

You may recall that an array `arr` is a mountain array if and only if:  
`arr.length >= 3`

There exists some `i` with  $0 < i < arr.length - 1$  such that:

`arr[0] < arr[1] < ... < arr[i - 1] < arr[i]`

`arr[i] > arr[i + 1] > ... > arr[arr.length - 1]`

Given a mountain array `mountainArr`, return the minimum index such that `mountainArr.get(index) == target`. If such an index does not exist, return -1. You cannot access the mountain array directly. You may only access the array using a `MountainArray` interface:

`MountainArray.get(k)` returns the element of the array at index `k` (0-indexed). `MountainArray.length()` returns the length of the array. Submissions making more than 100 calls to `MountainArray.get` will be judged Wrong Answer. Also, any solutions that attempt to circumvent the judge will result in disqualification.

Input: `mountainArr = [1,2,3,4,5,3,1]`, `target = 3`. Output: 2

**Explanation:** 3 exists in the array, at index=2 and index=5. Return the minimum index, which is 2.

```
int findPeak(MountainArray &m) {
 int l = 0, r = m.length() - 1;
 while (l < r) {
 int mid = l + (r - l) / 2;
 if (m.get(mid) < m.get(mid + 1))
 l = mid + 1;
 else
 r = mid;
 }
 return l;
}

int bs(MountainArray &m, int l, int r, int t, bool asc) {
 while (l <= r) {
 int mid = l + (r - l) / 2;
 int v = m.get(mid);
 if (v == t) return mid;
 if (asc) {
 if (v < t) l = mid + 1;
 else r = mid - 1;
 } else {
 if (v > t) l = mid + 1;
 else r = mid - 1;
 }
 }
 return -1;
}

int findInMountainArray(int target, MountainArray &m) {
 int peak = findPeak(m);
 int left = bs(m, 0, peak, target, true);
 if (left != -1) return left;
 return bs(m, peak + 1, m.length() - 1, target, false);
}
```

**[hard]**

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays. The overall run time complexity should be  $O(\log(m+n))$ .

Input: nums1 = [1,3], nums2 = [2]

Output: 2.00000

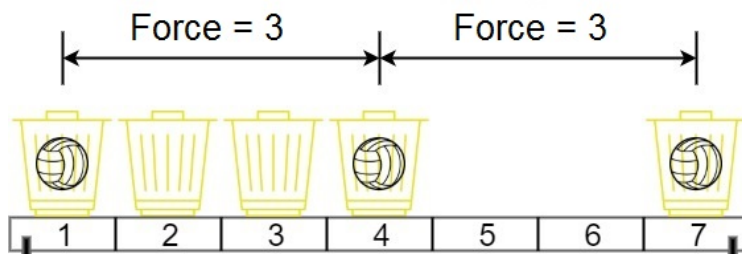
Explanation: merged array = [1,2,3] and median is 2.

```
double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
 if (A.size() > B.size()) return findMedianSortedArrays(B, A);
 int m = A.size(), n = B.size();
 int l = 0, r = m;
 while (l <= r) {
 int i = (l + r) / 2;
 int j = (m + n + 1) / 2 - i;
 int Aleft = (i == 0 ? INT_MIN : A[i-1]);
 int Aright = (i == m ? INT_MAX : A[i]);
 int Bleft = (j == 0 ? INT_MIN : B[j-1]);
 int Bright = (j == n ? INT_MAX : B[j]);
 if (Aleft <= Bright && Bleft <= Aright) {
 if ((m + n) % 2 == 0)
 return (max(Aleft, Bleft) + min(Aright, Bright))/2.0;
 else
 return max(Aleft, Bleft);
 }
 else if (Aleft > Bright)
 r = i - 1;
 else
 l = i + 1;
 }
 return 0.0;
}
```

In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has  $n$  empty baskets, the  $i$ th basket is at `position[i]`, Morty has  $m$  balls and needs to distribute the balls into the baskets such that the minimum magnetic force between any two balls is maximum. Rick stated that magnetic force between two different balls at positions  $x$  and  $y$  is  $|x - y|$ . Given the integer array `position` and the integer  $m$ . Return the required force.

Input: `position = [1,2,3,4,7]`,  $m = 3$ . Output: 3

Explanation: Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs  $[3, 3, 6]$ . The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.



```
bool canPlace(vector<int>& pos, int m, int dist) {
```

```

 int count = 1;
 int last = pos[0];
 for (int i = 1; i < pos.size(); i++) {
 if (pos[i] - last >= dist) {
 count++;
 last = pos[i];
 if (count >= m) return true;
 }
 }
 return false;
}

int maxDistance(vector<int>& position, int m) {
 sort(position.begin(), position.end());
 int low = 1; int high = position.back() - position[0];
 int ans = 1;
 while (low <= high) {
 int mid = low + (high - low) / 2;
 if (canPlace(position, m, mid)) {
 ans = mid;
 low = mid + 1;
 } else {
 high = mid - 1;
 }
 }
 return ans;
}

```

**[hard]**

Given an integer array `nums` and an integer `k`, split `nums` into `k` non-empty subarrays such that the largest sum of any subarray is minimized. Return the minimized largest sum of the split.

A subarray is a contiguous part of the array.

Input: `nums = [7,2,5,10,8]`, `k = 2`. Output: 18

Explanation: There are four ways to split `nums` into two subarrays. The best way is to split it into `[7,2,5]` and `[10,8]`, where the largest sum among the two subarrays is only 18.

```
bool canSplit(vector<int>& nums, int k, int limit) {
```

```

 int parts = 1;
 int curr = 0;
 for (int x : nums) {
 if (curr + x <= limit) {
 curr += x;
 } else {
 parts++;
 curr = x;
 if (parts > k) return false;
 }
 }
 return true;
}

int splitArray(vector<int>& nums, int k) {
 int low = *max_element(nums.begin(), nums.end());
 int high = accumulate(nums.begin(), nums.end(), 0);
 int ans = high;
 while (low <= high) {
 int mid = low + (high - low) / 2;

 if (canSplit(nums, k, mid)) {
 ans = mid;
 high = mid - 1;
 } else {
 low = mid + 1;
 }
 }
 return ans;
}

```

GenDS-BitwiseXOR

## General DS - Bitwise XOR

Description: XOR is a powerful bitwise operator that can solve problems like finding single numbers or efficiently pairing elements.

- [Single Number II](#)
- [Single Number III](#)
- [Find the Original array of Prefix XOR](#)
- [XOR Queries of a Subarray](#)

Given an integer array `nums` where every element appears three times except for one, which appears exactly once. Find the single element and return it.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Input: `nums = [2,2,3,2]`

Output: 3

```
int singleNumber(vector<int>& nums) {
 // For each bit (0-31), count how many numbers have that bit
 // set. If a bit count % 3 == 1 → that bit belongs to the single
 // number.
 int result = 0;
 for (int bit = 0; bit < 32; bit++) {
 int count = 0;
 for (int x: nums) {
 if (x & (1 << bit)) count++;
 }
 if (count % 3 != 0) {
 result |= (1 << bit);
 }
 }
 return result;
}
```



Given an integer array `nums`, in which exactly two elements appear only once and all the other elements appear exactly twice. Find the two elements that appear only once. You can return the answer in any order.

You must write an algorithm that runs in linear runtime complexity and uses only constant extra space.

Input: `nums = [1,2,1,3,2,5]`

Output: `[3,5]`

Explanation: `[5, 3]` is also a valid answer.

```
vector<int> singleNumber(vector<int>& nums) {
 // Let xorAll = a ^ b (a & b are the unique numbers). Find a
 rightmost set bit (mask). Split nums into 2 groups based on this bit
 and XOR separately.
 int xr = 0;
 for (int x : nums) xr ^= x;
 int mask = xr & (-xr); // rightmost set bit
 int a = 0, b = 0;
 for (int x : nums) {
 if (x & mask) a ^= x;
 else b ^= x;
 }
 return {a, b};
}
```

You are given an integer array `pref` of size `n`. Find and return the array `arr` of size `n` that satisfies:

`pref[i] = arr[0] ^ arr[1] ^ ... ^ arr[i]`.

Note that `^` denotes the bitwise-xor operation.

It can be proven that the answer is unique.

Input: `pref = [5,2,0,3,1]`

Output: `[5,7,2,3,2]`

Explanation: From the array `[5,7,2,3,2]` we have the following:

- `pref[0] = 5.`
- `pref[1] = 5 ^ 7 = 2.`
- `pref[2] = 5 ^ 7 ^ 2 = 0.`
- `pref[3] = 5 ^ 7 ^ 2 ^ 3 = 3.`
- `pref[4] = 5 ^ 7 ^ 2 ^ 3 ^ 2 = 1.`

```

vector<int> findArray(vector<int>& pref) {
 // Observe: pref[i] = arr[0] ^ arr[1] ^ ... ^ arr[i], arr[i] =
 pref[i] ^ pref[i-1]
 int n = pref.size();
 vector<int> arr(n);
 arr[0] = pref[0];
 for (int i = 1; i < n; i++) {
 arr[i] = pref[i] ^ pref[i-1];
 }
 return arr;
}

```

You are given an array `arr` of positive integers. You are also given the array `queries` where `queries[i] = [lefti, righti]`.

For each query `i` compute the XOR of elements from `lefti` to `righti` (that is, `arr[lefti] XOR arr[lefti + 1] XOR ... XOR arr[righti]` ).

Return an array `answer` where `answer[i]` is the answer to the `i`th query. Input: `arr = [1,3,4,8]`, `queries = [[0,1],[1,2],[0,3],[3,3]]`

Output: `[2,7,14,8]`. Explanation:

The binary representation of the elements in the array are:

`1 = 0001`, `3 = 0011`, `4 = 0100`, `8 = 1000`. The XOR values for queries are: `[0,1] = 1 xor 3 = 2`, `[1,2] = 3 xor 4 = 7`, `[0,3] = 1 xor 3 xor 4 xor 8 = 14`, `[3,3] = 8`.

```

vector<int> xorQueries(vector<int>& arr, vector<vector<int>>&
queries) {
 int n = arr.size();
 vector<int> px(n);
 px[0] = arr[0];
 for (int i = 1; i < n; i++) px[i] = px[i-1] ^ arr[i];
 vector<int> ans;
 for (auto &q : queries) {
 int l = q[0], r = q[1];
 if (l == 0) ans.push_back(px[r]);
 else ans.push_back(px[r] ^ px[l-1]);
 }
 return ans;
}

```

# Heaps-TopKElements

## Heaps - Top 'K' Elements

**Description:** This pattern uses heaps or quickselect to efficiently find the top 'K' largest/smallest elements from a dataset.

- [Top K Frequent Elements](#)
- [Kth Largest Element](#)
- [Ugly Number ii](#)
- [K Closest Points to Origin](#)

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`, `k = 2`. Output: `[1,2]`

```
vector<int> topKFrequent(vector<int>& nums, int k) {
 unordered_map<int,int> freq;
 for (int x : nums) freq[x]++;
 // min-heap of {frequency, value}
 priority_queue<pair<int,int>, vector<pair<int,int>>,
greater<pair<int,int>>> pq;
 for (auto &p : freq) {
 pq.push({p.second, p.first});
 if ((int)pq.size() > k) pq.pop();
 }
 vector<int> ans;
 while (!pq.empty()) {
 ans.push_back(pq.top().second);
 pq.pop();
 }
 return ans;
}
```

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

Note that it is the `k`th largest element in the sorted order, not the `k`th distinct element.

Can you solve it without sorting?

Input: `nums = [3,2,1,5,6,4]`, `k = 2`

Output: 5

```

int findKthLargest(vector<int>& nums, int k) {
 priority_queue<int, vector<int>, greater<int>> pq;
 for (int x : nums) {
 pq.push(x);
 if ((int)pq.size() > k) pq.pop();
 }
 return pq.top();
}

```

An ugly number is a positive integer whose prime factors are limited to 2, 3, and 5.

Given an integer n, return the nth ugly number.

Input: n = 10

Output: 12

Explanation: [1, 2, 3, 4, 5, 6, 8, 9, 10, 12] is the sequence of the first 10 ugly numbers.

```

int nthUglyNumber(int n) {
 vector<int> ugly(n);
 ugly[0] = 1;
 int i2 = 0, i3 = 0, i5 = 0;
 int n2 = 2, n3 = 3, n5 = 5;
 for (int i = 1; i < n; i++) {
 int next = min(n2, min(n3, n5));
 ugly[i] = next;
 if (next == n2) n2 = ugly[++i2] * 2;
 if (next == n3) n3 = ugly[++i3] * 3;
 if (next == n5) n5 = ugly[++i5] * 5;
 }
 return ugly[n - 1];
}

```

Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0). The distance between two points on the X-Y plane is the Euclidean distance (i.e.,  $\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$ ). You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Input: points = [[1,3],[-2,2]], k = 1. Output: [[-2,2]]

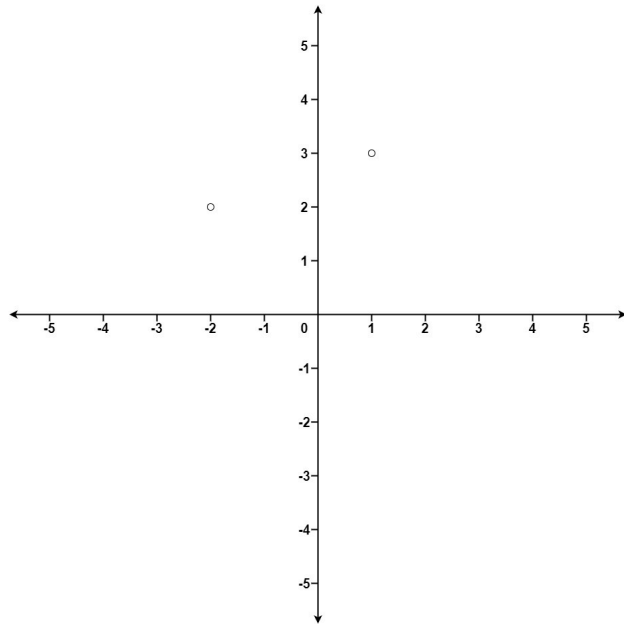
### Explanation:

The distance between (1, 3) and the origin is  $\sqrt{10}$ .

The distance between (-2, 2) and the origin is  $\sqrt{8}$ .

Since  $\sqrt{8} < \sqrt{10}$ , (-2, 2) is closer to the origin.

We only want the closest  $k = 1$  points from the origin, so the answer is just  $[[-2, 2]]$ .



```
vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
 // max-heap: {distance, index}
 priority_queue<pair<long long,int>> pq;
 for (int i = 0; i < points.size(); i++) {
 long long x = points[i][0];
 long long y = points[i][1];
 long long dist = x*x + y*y;
 pq.push({dist, i});
 if ((int)pq.size() > k) pq.pop();
 }
 vector<vector<int>> ans;
 while (!pq.empty()) {
 int idx = pq.top().second;
 pq.pop();
 ans.push_back(points[idx]);
 }
 return ans;
}
```

# Heaps-KWayMerge

## Heaps - K-way Merge

Description: The K-way merge technique uses a heap to efficiently merge multiple sorted lists or arrays.

- [Find K Pairs with Smallest Sums](#)
- [Kth Smallest Element in a Sorted Matrix](#)
- [Merge K Sorted Lists](#)
- [Smallest Range: Smallest Range Covering Elements from K Lists](#)

You are given two integer arrays `nums1` and `nums2` sorted in non-decreasing order and an integer `k`. Define a pair  $(u, v)$  which consists of one element from the first array and one element from the second array. Return the `k` pairs  $(u_1, v_1), (u_2, v_2), \dots, (u_k, v_k)$  with the smallest sums. Input: `nums1 = [1,7,11]`, `nums2 = [2,4,6]`, `k = 3`. Output: `[[1,2],[1,4],[1,6]]`. Explanation: The first 3 pairs are returned from the sequence:

```
[1,2],[1,4],[1,6],[7,2],[7,4],[11,2],[7,6],[11,4],[11,6]
vector<vector<int>> kSmallestPairs(vector<int>& nums1, vector<int>&
nums2, int k) {
 vector<vector<int>> ans;
 if (nums1.empty() || nums2.empty() || k == 0) return ans;
 priority_queue< vector<int>, vector<vector<int>>,
greater<vector<int>> > pq; // min heap: {sum, i, j}
 // start with (nums1[i], nums2[0])
 for (int i = 0; i < nums1.size() && i < k; i++) {
 pq.push({nums1[i] + nums2[0], i, 0});
 }
 while (!pq.empty() && ans.size() < k) {
 vector<int> top = pq.top(); pq.pop();
 int i = top[1], j = top[2];
 ans.push_back({nums1[i], nums2[j]});
 // push next pair (i, j+1)
 if (j + 1 < nums2.size()) {
 pq.push({nums1[i] + nums2[j+1], i, j+1});
 }
 }
 return ans;
}
```



Given an  $n \times n$  matrix where each of the rows and columns is sorted in ascending order, return the  $k$ th smallest element in the matrix. Note that it is the  $k$ th smallest element in the sorted order, not the  $k$ th distinct element.

You must find a solution with a memory complexity better than  $O(n^2)$ .

Input: matrix = `[[1,5,9],[10,11,13],[12,13,15]]`,  $k = 8$

Output: 13

Explanation: The elements in the matrix are

`[1,5,9,10,11,12,13,13,15]`, and the 8th smallest number is 13

```
int kthSmallest(vector<vector<int>>& matrix, int k) {
 int n = matrix.size();
 // min heap: {value, row, col}
 priority_queue< vector<int>, vector<vector<int>>,
greater<vector<int>> > pq;
 // push the first element of each row
 for (int r = 0; r < n; r++) {
 pq.push({matrix[r][0], r, 0});
 }
 int val = 0;
 while (k--) {
 vector<int> top = pq.top(); pq.pop();
 val = top[0];
 int r = top[1], c = top[2];
 if (c + 1 < n) {
 pq.push({matrix[r][c+1], r, c+1});
 }
 }
 return val;
}
```

[hard]

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order.

Merge all the linked-lists into one sorted linked-list and return it.

Input: lists = [[1,4,5],[1,3,4],[2,6]]

Output: [1,1,2,3,4,4,5,6]

Explanation: The linked-lists are:

```
[
 1->4->5,
 1->3->4,
 2->6
]
```

merging them into one sorted linked list:

1->1->2->3->4->4->5->6

```
struct Compare {
 bool operator()(ListNode* a, ListNode* b) {
 return a->val > b->val; // min-heap
 }
};

ListNode* mergeKLists(vector<ListNode*>& lists) {
 priority_queue<ListNode*, vector<ListNode*>, Compare> pq;
 for (int i = 0; i < lists.size(); i++) {
 if (lists[i]) pq.push(lists[i]);
 }
 ListNode dummy(0);
 ListNode* tail = &dummy;
 while (!pq.empty()) {
 ListNode* node = pq.top(); pq.pop();
 tail->next = node;
 tail = tail->next;
 if (node->next) pq.push(node->next);
 }
 return dummy.next;
}
```

[hard]

You have  $k$  lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the  $k$  lists. We define the range  $[a, b]$  is smaller than range  $[c, d]$  if  $b - a < d - c$  or  $a < c$  if  $b - a == d - c$ .

Input: `nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]`. Output: `[20,24]`. Explanation:

List 1: `[4, 10, 15, 24,26]`, 24 is in range `[20,24]`.

List 2: `[0, 9, 12, 20]`, 20 is in range `[20,24]`.

List 3: `[5, 18, 22, 30]`, 22 is in range `[20,24]`.

```
vector<int> smallestRange(vector<vector<int>>& nums) {
 int k = nums.size();
 // min-heap: {value, listIndex, elementIndex}
 priority_queue<vector<int>, vector<vector<int>>,
greater<vector<int>>> pq;
 int currentMax = INT_MIN;
 // Push first element from each list
 for (int i = 0; i < k; i++) {
 pq.push({nums[i][0], i, 0});
 currentMax = max(currentMax, nums[i][0]);
 }
 int bestL = 0, bestR = INT_MAX;
 while (true) {
 vector<int> top = pq.top(); pq.pop();
 int val = top[0], listIdx = top[1], elemIdx = top[2];
 if (currentMax - val < bestR - bestL) {
 bestL = val; // Update the best range
 bestR = currentMax;
 }
 // Move to next element in this list
 if (elemIdx + 1 == nums[listIdx].size())
 break; // can't expand this list → end
 int nextVal = nums[listIdx][elemIdx + 1];
 pq.push({nextVal, listIdx, elemIdx + 1});
 currentMax = max(currentMax, nextVal);
 }
 return {bestL, bestR};
}
```

# Heaps-TwoHeaps

## Heaps - Two Heaps

**Description:** This pattern uses two heaps (max heap and min heap) to solve problems involving tracking medians and efficiently managing dynamic data.

- [Find Median from Data Stream](#)
- [Sliding Window Median](#)
- [IPO](#)

[hard]

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

For example, for arr = [2,3,4], the median is 3.

For example, for arr = [2,3], the median is  $(2 + 3) / 2 = 2.5$ .

Implement the MedianFinder class:

MedianFinder() initializes the MedianFinder object.

void addNum(int num) adds the integer num from the data stream to the data structure.

double findMedian() returns the median of all elements so far.

Answers within  $10^{-5}$  of the actual answer will be accepted.

Input

```
["MedianFinder", "addNum", "addNum", "findMedian", "addNum",
"findMedian"]
```

```
[[], [1], [2], [], [3], []]
```

Output

```
[null, null, null, 1.5, null, 2.0]
```

Explanation

```
MedianFinder medianFinder = new MedianFinder();
```

```
medianFinder.addNum(1); // arr = [1]
```

```
medianFinder.addNum(2); // arr = [1, 2]
```

```
medianFinder.findMedian(); // return 1.5 (i.e., (1 + 2) / 2)
```

```
medianFinder.addNum(3); // arr[1, 2, 3]
```

```
medianFinder.findMedian(); // return 2.0
```

```

priority_queue<int> leftMax; // max-heap
priority_queue<int, vector<int>, greater<int>> rightMin; // min-heap
MedianFinder() {}
void addNum(int num) {
 if (leftMax.empty() || num <= leftMax.top())
 leftMax.push(num);
 else
 rightMin.push(num);
 // balance
 if (leftMax.size() > rightMin.size() + 1) {
 rightMin.push(leftMax.top());
 leftMax.pop();
 }
 else if (rightMin.size() > leftMax.size()) {
 leftMax.push(rightMin.top());
 rightMin.pop();
 }
}
double findMedian() {
 if (leftMax.size() == rightMin.size())
 return (leftMax.top() + rightMin.top()) / 2.0;
 return leftMax.top();
}

```

**[hard]**

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value. So the median is the mean of the two middle values. For examples, if arr = [2,3,4], the median is 3. For examples, if arr = [1,2,3,4], the median is  $(2 + 3) / 2 = 2.5$ . You are given an integer array nums and an integer k. There is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position. Return the median array for each window in the original array. Answers within  $10^{-5}$  of the actual value will be accepted.

Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [1.00000,-1.00000,-1.00000,3.00000,5.00000,6.00000]

Explanation:

| Window position     | Median |
|---------------------|--------|
| -----               | -----  |
| [1 3 -1] -3 5 3 6 7 | 1      |
| 1 [3 -1 -3] 5 3 6 7 | -1     |
| 1 3 [-1 -3 5] 3 6 7 | -1     |
| 1 3 -1 [-3 5 3] 6 7 | 3      |
| 1 3 -1 -3 [5 3 6] 7 | 5      |
| 1 3 -1 -3 5 [3 6 7] | 6      |

```

class DualHeap {
public:
 priority_queue<long> maxHeap;
 priority_queue<long, vector<long>, greater<long>> minHeap;
 unordered_map<long, int> delayed;
 int small = 0, large = 0;
 int k;
 DualHeap(int _k) : k(_k) {}
 void prune(priority_queue<long> &heap) {
 while (!heap.empty()) {
 long x = heap.top();
 if (delayed.count(x)) {
 if (--delayed[x] == 0) delayed.erase(x);
 heap.pop();
 } else break;
 }
 }
 void pruneMin(priority_queue<long, vector<long>, greater<long>>
&heap) {
 while (!heap.empty()) {
 long x = heap.top();
 if (delayed.count(x)) {
 if (--delayed[x] == 0) delayed.erase(x);
 heap.pop();
 } else break;
 }
 }
 void balance() {

```

```

 if (small > large + 1) {
 long x = maxHeap.top(); maxHeap.pop();
 minHeap.push(x);
 small--; large++;
 prune(maxHeap);
 } else if (large > small) {
 long x = minHeap.top(); minHeap.pop();
 maxHeap.push(x);
 large--; small++;
 pruneMin(minHeap);
 }
 }

 void insert(long num) {
 if (maxHeap.empty() || num <= maxHeap.top()) {
 maxHeap.push(num); small++;
 } else {
 minHeap.push(num); large++;
 }
 balance();
 }

 void erase(long num) {
 delayed[num]++;
 if (!maxHeap.empty() && num <= maxHeap.top()) {
 small--;
 if (num == maxHeap.top()) prune(maxHeap);
 } else {
 large--;
 if (!minHeap.empty() && num == minHeap.top())
pruneMin(minHeap);
 }
 balance();
 }

 double getMedian() {
 if (k % 2 == 1) return maxHeap.top();
 return (maxHeap.top() + minHeap.top()) / 2.0;
 }
};

vector<double> medianSlidingWindow(vector<int>& nums, int k) {

```



```

DualHeap dh(k);
vector<double> ans;

for (int i = 0; i < nums.size(); i++) {
 dh.insert(nums[i]);
 if (i >= k - 1) {
 ans.push_back(dh.getMedian());
 dh.erase(nums[i - k + 1]);
 }
}
return ans;
}

```

[hard]

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most  $k$  distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most  $k$  distinct projects. You are given  $n$  projects where the  $i$ th project has a pure profit  $profits[i]$  and a minimum capital of  $capital[i]$  is needed to start it. Initially, you have  $w$  capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital. Pick a list of at most  $k$  distinct projects from given projects to maximize your final capital, and return the final maximized capital. The answer is guaranteed to fit in a 32-bit signed integer. Input:  $k = 2$ ,  $w = 0$ ,  $profits = [1,2,3]$ ,  $capital = [0,1,1]$ . Output: 4

Explanation: Since your initial capital is 0, you can only start the project indexed 0. After finishing it you will obtain profit 1 and your capital becomes 1. With capital 1, you can either start the project indexed 1 or the project indexed 2.

Since you can choose at most 2 projects, you need to finish the project indexed 2 to get the maximum capital.

Therefore, output the final maximized capital, which is  $0 + 1 + 3 = 4$ .

```

int findMaximizedCapital(int k, int w, vector<int>& profits,
vector<int>& capital) {
 int n = profits.size();
 vector<pair<int,int>> projects; // (capital, profit)
 for (int i = 0; i < n; i++)
 projects.push_back({capital[i], profits[i]});
 sort(projects.begin(), projects.end());
 priority_queue<int> maxProfit;
 int idx = 0;
 while (k--> 0) {
 while (idx < n && projects[idx].first <= w) {
 maxProfit.push(projects[idx].second);
 idx++;
 }
 if (maxProfit.empty()) break;
 w += maxProfit.top();
 maxProfit.pop();
 }
 return w;
}

```

Stack

## Stack - Monotonic Stack

Description: A monotonic stack helps solve range queries by maintaining a stack of elements in increasing or decreasing order.

- [\[Self Added\] Next Greater Element I](#)
- [Next Greater Element II](#)
- [Next Greater Node in Linked List](#)
- [Daily Temperatures](#)
- [Online Stock Span](#)
- [Maximum Width Ramp](#)
- [Largest Rectangle in Histogram](#)
- [\[Self added v2\] Find Celebrity](#)

The next greater element of some element  $x$  in an array is the first greater element that is to the right of  $x$  in the same array.

You are given two distinct 0-indexed integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each  $0 \leq i < \text{nums1.length}$ , find the index  $j$  such that `nums1[i] == nums2[j]` and determine the next greater element of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the next greater element as described above.

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.
- 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is 3.
- 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the answer is `-1`.

```

vector<int> nextGreaterElement(vector<int>& nums1, vector<int>&
nums2) {
 stack<int> st;
 unordered_map<int,int> nge;
 for (int i = 0; i < nums2.size(); i++) {
 while (!st.empty() && nums2[st.top()] < nums2[i]) {
 nge[nums2[st.top()]] = nums2[i];
 st.pop();
 }
 st.push(i);
 }
 while (!st.empty()) {
 nge[nums2[st.top()]] = -1;
 st.pop();
 }
 vector<int> ans;
 for (int i = 0; i < nums1.size(); i++)
 ans.push_back(nge[nums1[i]]);
 return ans;
}

```

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the next greater number for every element in `nums`.

The next greater number of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Input: `nums = [1,2,1]`

Output: `[2,-1,2]`

Explanation: The first `1`'s next greater number is `2`;

The number `2` can't find next greater number.

The second `1`'s next greater number needs to search circularly, which is also `2`.

```

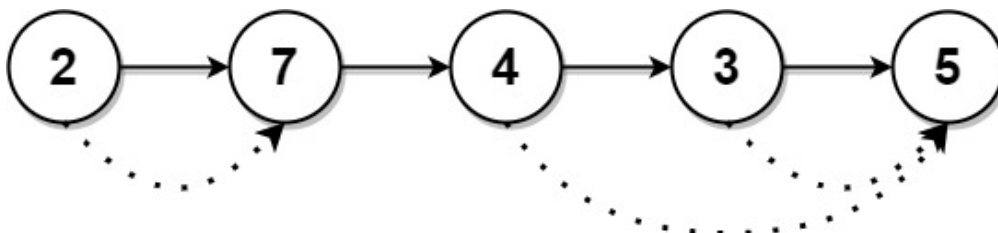
vector<int> nextGreaterElements(vector<int>& nums) {
 int n = nums.size();
 vector<int> ans(n, -1);
 stack<int> st;
 for (int i = 0; i < 2*n; i++) {
 int idx = i % n;
 while (!st.empty() && nums[st.top()] < nums[idx]) {
 ans[st.top()] = nums[idx];
 st.pop();
 }
 if (i < n) st.push(idx);
 }
 return ans;
}

```

You are given the head of a linked list with  $n$  nodes.  
 For each node in the list, find the value of the next greater node.  
 That is, for each node, find the value of the first node that is  
 next to it and has a strictly larger value than it.  
 Return an integer array `answer` where `answer[i]` is the value of the  
 next greater node of the  $i$ th node (1-indexed). If the  $i$ th node does  
 not have a next greater node, set `answer[i] = 0`.

Input: head = [2,7,4,3,5]

Output: [7,0,5,5,0]



```

struct ListNode {
 int val; ListNode *next;
 ListNode(int x) : val(x), next(NULL) {}
};

vector<int> nextLargerNodes(ListNode* head) {
 vector<int> arr;
 while (head) {
 arr.push_back(head->val);
 head = head->next;
 }
 int n = arr.size(); vector<int> ans(n, 0); stack<int> st;
 for (int i = 0; i < n; i++) {
 while (!st.empty() && arr[st.top()] < arr[i]) {
 ans[st.top()] = arr[i];
 st.pop();
 }
 st.push(i);
 }
 return ans;
}

```

Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead. Input: temperatures = [73,74,75,71,69,72,76,73]. Output: [1,1,4,2,1,1,0,0]

```

vector<int> dailyTemperatures(vector<int>& t) {
 int n = t.size(); vector<int> ans(n, 0); stack<int> st;
 for (int i = 0; i < n; i++) {
 while (!st.empty() && t[st.top()] < t[i]) {
 int idx = st.top(); st.pop();
 ans[idx] = i - idx;
 }
 st.push(i);
 }
 return ans;
}

```

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day. The span of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

For example, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.

Also, if the prices of the stock in the last four days is [7,34,1,2] and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

StockSpanner() Initializes the object of the class.

int next(int price) Returns the span of the stock's price given that today's price is price.

Input

```
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
```

```
[[], [100], [80], [60], [70], [60], [75], [85]]
```

Output

```
[null, 1, 1, 1, 2, 1, 4, 6]
```

Explanation

```
StockSpanner stockSpanner = new StockSpanner();
```

```
stockSpanner.next(100); // return 1
```

```
stockSpanner.next(80); // return 1
```

```
stockSpanner.next(60); // return 1
```

```
stockSpanner.next(70); // return 2
```

```
stockSpanner.next(60); // return 1
```

```
stockSpanner.next(75); // return 4, because the last 4 prices
(including today's price of 75) were less than or equal to today's
price.
```

```
stockSpanner.next(85); // return 6
```



```

class StockSpanner {
public:
 stack<pair<int,int> > st; // {price, span}
 StockSpanner() {}
 int next(int price) {
 int span = 1;
 while (!st.empty() && st.top().first <= price) {
 span += st.top().second;
 st.pop();
 }
 st.push(make_pair(price, span));
 return span;
 }
};

```

A ramp in an integer array `nums` is a pair  $(i, j)$  for which  $i < j$  and `nums[i] <= nums[j]`. The width of such a ramp is  $j - i$ .

Given an integer array `nums`, return the maximum width of a ramp in `nums`. If there is no ramp in `nums`, return 0.

Input: `nums = [6,0,8,2,1,5]`

Output: 4

Explanation: The maximum width ramp is achieved at  $(i, j) = (1, 5)$ :  
`nums[1] = 0` and `nums[5] = 5`.

```

int maxWidthRamp(vector<int>& nums) {
 int n = nums.size(); stack<int> st;
 for (int i = 0; i < n; i++)
 if (st.empty() || nums[i] < nums[st.top()])
 st.push(i);
 int ans = 0;
 for (int j = n - 1; j >= 0; j--) {
 while (!st.empty() && nums[st.top()] <= nums[j]) {
 ans = max(ans, j - st.top());
 st.pop();
 }
 }
 return ans;
}

```

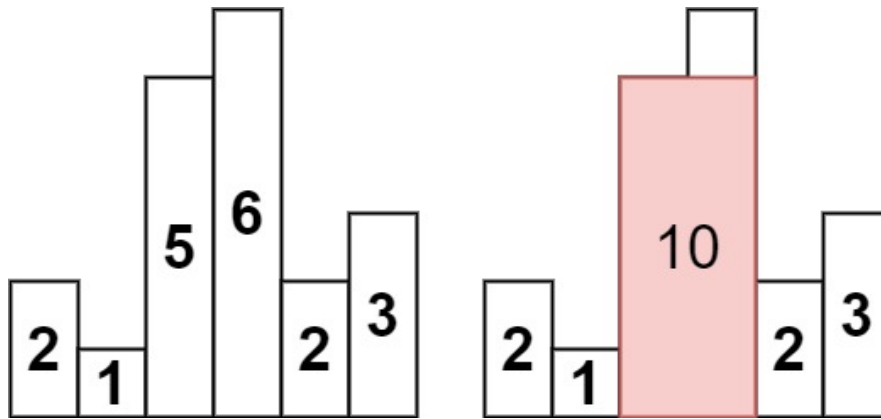
[hard]

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Input: heights = [2,1,5,6,2,3]

Output: 10

Explanation: The above is a histogram where width of each bar is 1. The largest rectangle is shown in the red area, which has an area = 10 units.



```
int largestRectangleArea(vector<int>& h) {
 int n = h.size();
 stack<int> st;
 int maxArea = 0;
 for (int i = 0; i <= n; i++) {
 int height = (i == n ? 0 : h[i]);
 while (!st.empty() && h[st.top()] > height) {
 int ht = h[st.top()]; st.pop();
 int left = st.empty() ? -1 : st.top();
 int width = i - left - 1;
 maxArea = max(maxArea, ht * width);
 }
 st.push(i);
 }
 return maxArea;
}
```

A celebrity is a person who is known to all but does not know anyone at a party. If you go to a party of N people, find if there is a celebrity in the party or not.

A square NxN matrix M[][] is used to represent people at the party such that if an element of row i and column j is set to 1 it means ith person knows jth person. Here M[i][i] will always be 0.

Note: Follow 0 based indexing. Input: N = 3

```
M[][] = {{0 1 0},
 {0 0 0},
 {0 1 0}}
```

Output: 1

```
bool knows(int x, int y, vector<vector<int> >& M) {
 return M[x][y];
 // it itself send if celebrity know a person or not...
}

int celebrity(vector<vector<int> >& M, int n)
{
 stack<int> st;
 for (int i = 0 ; i < n ; i++) {
 st.push(i);
 }
 while (st.size() > 1) {
 int x = st.top(); st.pop();
 int y = st.top(); st.pop();
 if (knows(x, y, M)) st.push(y); // we push back y as its a
potential celebrity..
 else st.push(x);
 }
 int pc = st.top(); // potential celebrity... pc...
 if (st.empty()) return -1;
 for (int i = 0 ; i < n ; i++) {
 if (i != pc and (knows(pc, i, M) or !knows(i, pc, M))) {
 return -1;
 }
 }

 return pc;
}
```

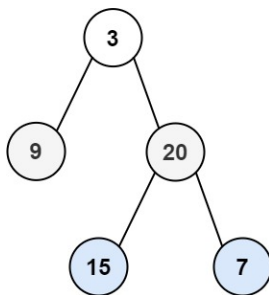
# Tree-LevelOrderTraversal

## Trees - Level Order Traversal (BFS in Binary Tree)

- [Level order Traversal](#)
- [Zigzag Level order Traversal](#)
- [Even Odd Tree](#)
- [Reverse odd Levels](#)
- [Deepest Leaves Sum](#)
- [Add one row to Tree](#)
- [Maximum width of Binary Tree](#)
- [All Nodes Distance K in Binary tree](#)

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Input: root = [3,9,20,null,null,15,7]. Output: [[3],[9,20],[15,7]]

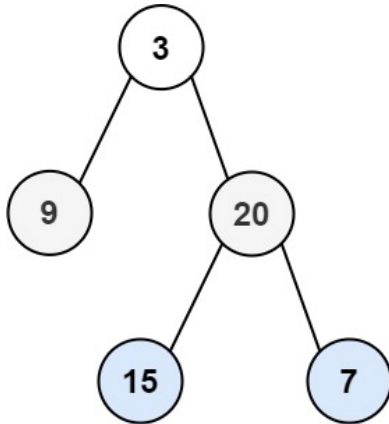


```
vector<vector<int>> levelOrder(TreeNode* root) {
 vector<vector<int>> ans; queue<TreeNode*> q;
 if(root==NULL) return ans;
 q.push(root);
 while(!q.empty()){
 int size = q.size(); vector<int> l;
 for(int i=0; i<size; i++){
 TreeNode* x = q.front();
 q.pop();
 l.push_back(x->val);
 if(x->left) q.push(x->left);
 if(x->right) q.push(x->right);
 }
 ans.push_back(l);
 }
 return ans;
}
```

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Input: root = [3,9,20,null,null,15,7]

Output: [[3],[20,9],[15,7]]



```
vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
 vector<vector<int>> ans;
 if(root==NULL) return ans;
 queue<TreeNode*> q;
 q.push(root);
 bool direction = true;
 while(!q.empty()){
 int size = q.size();
 vector<int> l(size);
 for(int i = 0; i<size; i++){
 TreeNode* x = q.front();
 q.pop();
 if(direction)l[i]=x->val;
 else l[size-i-1]=x->val;
 if(x->left) q.push(x->left);
 if(x->right) q.push(x->right);
 }
 direction=!direction;
 ans.push_back(l);
 }
 return ans;
}
```

A binary tree is named Even-Odd if it meets the following conditions:

The root of the binary tree is at level index 0, its children are at level index 1, their children are at level index 2, etc.

For every even-indexed level, all nodes at the level have odd integer values in strictly increasing order (from left to right).

For every odd-indexed level, all nodes at the level have even integer values in strictly decreasing order (from left to right).

Given the root of a binary tree, return true if the binary tree is Even-Odd, otherwise return false.

Input: root = [1,10,4,3,null,7,9,12,8,6,null,null,2]

Output: true

Explanation: The node values on each level are:

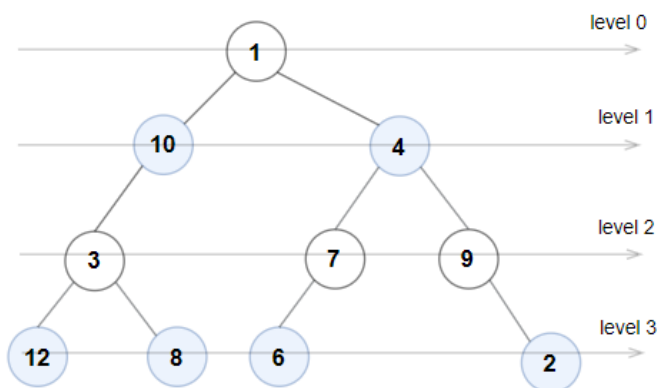
Level 0: [1]

Level 1: [10,4]

Level 2: [3,7,9]

Level 3: [12,8,6,2]

Since levels 0 and 2 are all odd and increasing and levels 1 and 3 are all even and decreasing, the tree is Even-Odd.



```

struct TreeNode {
 int val;
 TreeNode *left, *right;
 TreeNode(int x) : val(x), left(NULL), right(NULL) {}
};

bool isEvenOddTree(TreeNode* root) {
 queue<TreeNode*> q;
 q.push(root);
 int level = 0;
 while (!q.empty()) {
 int sz = q.size();
 int prev = (level % 2 == 0 ? INT_MIN : INT_MAX);
 for (int i = 0; i < sz; i++) {
 TreeNode* cur = q.front(); q.pop();
 int v = cur->val;
 if (level % 2 == 0) { // even level
 if (v % 2 == 0 || v <= prev) return false;
 } else { // odd level
 if (v % 2 == 1 || v >= prev) return false;
 }
 prev = v;
 if (cur->left) q.push(cur->left);
 if (cur->right) q.push(cur->right);
 }
 level++;
 }
 return true;
}

```

Given the root of a perfect binary tree, reverse the node values at each odd level of the tree.

For example, suppose the node values at level 3 are [2,1,3,4,7,11,29,18], then it should become [18,29,11,7,4,3,1,2]. Return the root of the reversed tree.

A binary tree is perfect if all parent nodes have two children and all leaves are on the same level.

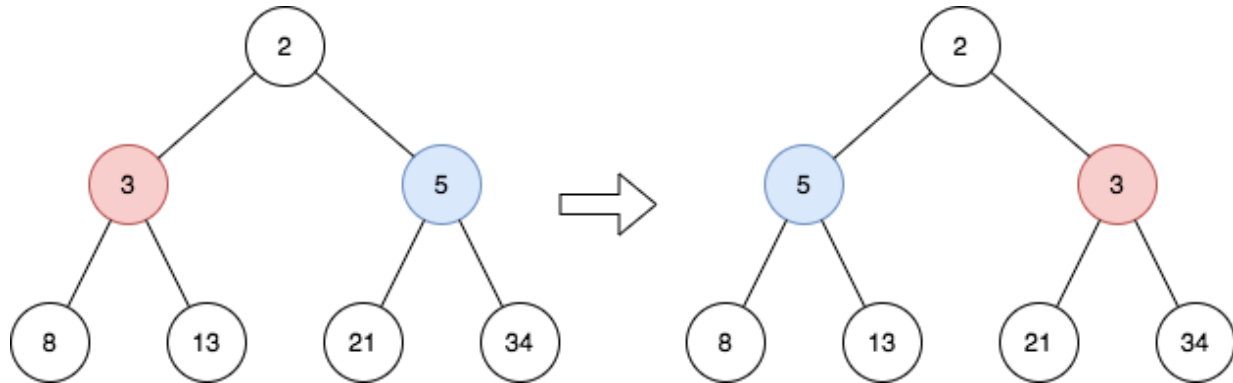
The level of a node is the number of edges along the path between it and the root node.



Input: root = [2,3,5,8,13,21,34]. Output: [2,5,3,8,13,21,34]

Explanation: The tree has only one odd level.

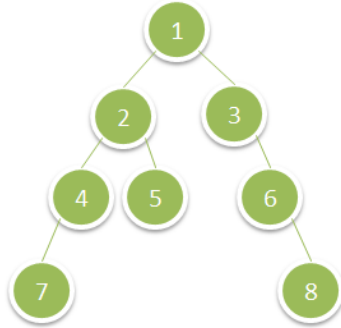
The nodes at level 1 are 3, 5 respectively, which are reversed and become 5, 3.



```
TreeNode* reverseOddLevels(TreeNode* root) {
 if (!root) return root;
 queue<TreeNode*> q; q.push(root);
 int level = 0;
 while (!q.empty()) {
 int sz = q.size(); vector<TreeNode*> nodes;
 for (int i = 0; i < sz; i++) {
 TreeNode* cur = q.front(); q.pop();
 nodes.push_back(cur);
 if (cur->left) q.push(cur->left);
 if (cur->right) q.push(cur->right);
 }
 // Reverse values at odd level
 if (level % 2 == 1) {
 int l = 0, r = nodes.size() - 1;
 while (l < r) {
 swap(nodes[l]->val, nodes[r]->val);
 l++; r--;
 }
 }
 level++;
 }
 return root;
}
```

Given the root of a binary tree, return the sum of values of its deepest leaves. Input: root =

[1,2,3,4,5,null,6,7,null,null,null,null,8]. Output: 15



```
int deepestLeavesSum(TreeNode* root) {
 queue<TreeNode*> q; q.push(root);
 int sum = 0;
 while (!q.empty()) {
 int sz = q.size(); sum = 0;
 for (int i = 0; i < sz; i++) {
 TreeNode* cur = q.front(); q.pop();
 sum += cur->val;
 if (cur->left) q.push(cur->left);
 if (cur->right) q.push(cur->right);
 }
 }
 return sum;
}
```

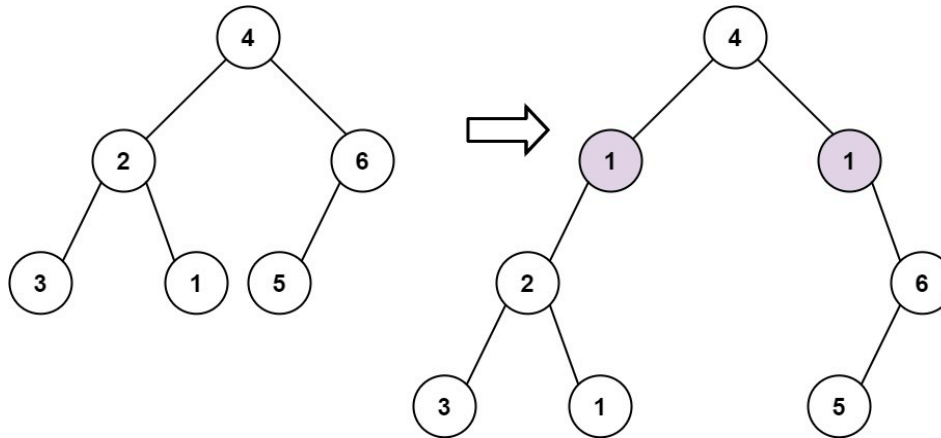
Given the root of a binary tree and two integers val and depth, add a row of nodes with value val at the given depth depth. Note that the root node is at depth 1.

The adding rule is: Given the integer depth, for each not null tree node cur at the depth depth - 1, create two tree nodes with value val as cur's left subtree root and right subtree root. cur's original left subtree should be the left subtree of the new left subtree root. cur's original right subtree should be the right subtree of the new right subtree root.

If  $\text{depth} == 1$  that means there is no depth  $\text{depth} - 1$  at all, then create a tree node with value  $\text{val}$  as the new root of the whole original tree, and the original tree is the new root's left subtree.

Input:  $\text{root} = [4, 2, 6, 3, 1, 5]$ ,  $\text{val} = 1$ ,  $\text{depth} = 2$ .

Output:  $[4, 1, 1, 2, \text{null}, \text{null}, 6, 3, 1, 5]$



Other ex:

Level 1:

10

Level 2:

5 15

Level 3:

3 7 12 18

Level 4:

1 4 13

Level 5:

0

Case: Row added at depth = 3,  $\text{val} = 99$

Level 1:

10

Level 2:

5 15

Level 3:

99 99 99 99

Level 4:

3 7 12 18

Level 5:

1 4 13

Level 6:

0

TreeNode\* addOneRow(TreeNode\* root, int val, int depth) {

```

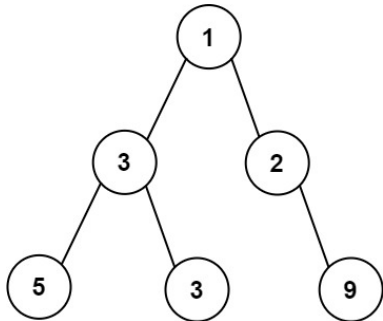
 if (depth == 1) {
 TreeNode* newRoot = new TreeNode(val);
 newRoot->left = root;
 return newRoot;
 }
 queue<TreeNode*> q;
 q.push(root);
 int d = 1;
 while (!q.empty()) {
 int sz = q.size();
 if (d == depth - 1) {
 for (int i = 0; i < sz; i++) {
 TreeNode* cur = q.front();
 q.pop();
 TreeNode* l = cur->left;
 TreeNode* r = cur->right;
 cur->left = new TreeNode(val);
 cur->left->left = l;
 cur->right = new TreeNode(val);
 cur->right->right = r;
 }
 break;
 }
 for (int i = 0; i < sz; i++) {
 TreeNode* cur = q.front();
 q.pop();
 if (cur->left) q.push(cur->left);
 if (cur->right) q.push(cur->right);
 }
 d++;
 }
 return root;
}

```

**Given the root of a binary tree, return the maximum width of the given tree. The maximum width of a tree is the maximum width among**

all levels. The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation. It is guaranteed that the answer will in the range of a 32-bit signed integer.

Input: root = [1,3,2,5,3,null,9]. Output: 4. Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).



```

long long widthOfBinaryTree(TreeNode* root) {
 if (!root) return 0;
 queue<pair<TreeNode*, long long>> q;
 q.push(make_pair(root, 1)); long long ans = 0;
 while (!q.empty()) {
 int sz = q.size();
 long long left = q.front().second, right = left;
 for (int i = 0; i < sz; i++) {
 pair<TreeNode*, long long> p = q.front(); q.pop();
 TreeNode* cur = p.first;
 long long idx = p.second;
 right = idx;
 if (cur->left) q.push(make_pair(cur->left, idx*2));
 if (cur->right) q.push(make_pair(cur->right, idx*2+1));
 }
 ans = max(ans, right - left + 1);
 }
 return ans;
}

```

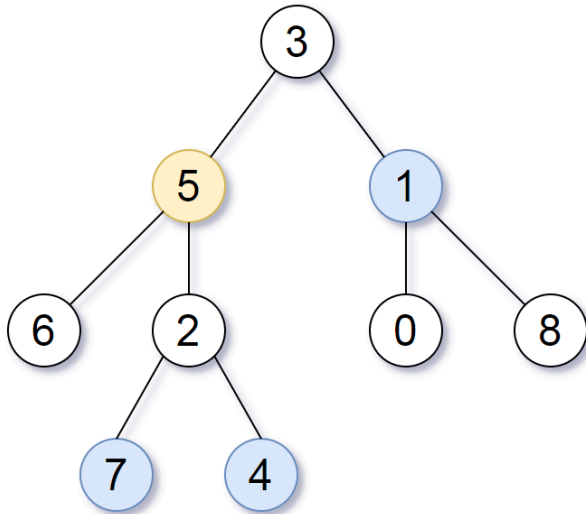
Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2

Output: [7,4,1]

Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.



```
void buildParents(TreeNode* root, unordered_map<TreeNode*,
TreeNode*>& parent) {
 queue<TreeNode*> q;
 q.push(root);
 parent[root] = NULL;
 while (!q.empty()) {
 TreeNode* cur = q.front(); q.pop();
 if (cur->left) {
 parent[cur->left] = cur;
 q.push(cur->left);
 }
 if (cur->right) {
 parent[cur->right] = cur;
 q.push(cur->right);
 }
 }
}

vector<int> distanceK(TreeNode* root, TreeNode* target, int k) {
```

```

unordered_map<TreeNode*, TreeNode*> parent;
buildParents(root, parent);
unordered_set<TreeNode*> vis;
queue<TreeNode*> q;
q.push(target);
vis.insert(target);
int dist = 0;
while (!q.empty()) {
 int sz = q.size();
 if (dist == k) break;
 dist++;
 for (int i = 0; i < sz; i++) {
 TreeNode* cur= q.front(); q.pop();
 TreeNode* arr[3]= {cur->left, cur->right, parent[cur]};
 for (int j = 0; j < 3; j++) {
 TreeNode* nxt = arr[j];
 if (nxt && !vis.count(nxt)) {
 vis.insert(nxt);
 q.push(nxt);
 }
 }
 }
}
vector<int> ans;
while (!q.empty()) {
 ans.push_back(q.front()->val);
 q.pop();
}
return ans;
}

```

# Tree-Traversals



[Self added] Tree Traversals

- [\[Self added v2\] Left, Right, Top View of Binary Tree](#)

**// Left View**

// recursive

```
void leftViewDFS(Node* curr, int level, int &maxLevel, vector<int>
&ans) {
```

```
 if (curr == NULL) return;
```

```
 if (level > maxLevel) {
```

```
 ans.push_back(curr->data);
```

```
 maxLevel = level;
```

```
 }
```

```
 leftViewDFS(curr->left, level + 1, maxLevel, ans);
```

```
 leftViewDFS(curr->right, level + 1, maxLevel, ans);
```

```
}
```

```
vector<int> leftView(Node *root) {
```

```
 if (root == NULL) return {};
```

```
 vector<int> ans;
```

```
 int maxLevel = -1;
```

```
 leftViewDFS(root, 0, maxLevel, ans);
```

```
 return ans;
```

```
}
```

// iterative bfs

```
vector<int> leftViewIterative(Node* root) {
```

```
 if (root == NULL) return {};
```

```
 queue<Node*> q; q.push(root); vector<int> ans;
```

```
 while (!q.empty()) {
```

```
 int sz = q.size();
```

```
 ans.push_back(q.front()->data); // leftmost element
```

```
 for (int i = 0; i < sz; i++) {
```

```
 Node* curr = q.front(); q.pop();
```

```
 if (curr->left) q.push(curr->left);
```

```
 if (curr->right) q.push(curr->right);
```

```
 }
```

```
 }
```

```
 return ans;
```

```
}
```

```

// Right View
// recursive
void rightViewDFS(Node* curr, int level, int &maxLevel, vector<int>
&ans) {
 if (curr == NULL) return;
 if (level > maxLevel) {
 ans.push_back(curr->data);
 maxLevel = level;
 }
 rightViewDFS(curr->right, level + 1, maxLevel, ans);
 rightViewDFS(curr->left, level + 1, maxLevel, ans);
}
vector<int> rightView(Node *root) {
 if (root == NULL) return {};
 vector<int> ans;
 int maxLevel = -1;
 rightViewDFS(root, 0, maxLevel, ans);
 return ans;
}
// iterative bfs
vector<int> rightViewIterative(Node* root) {
 if (root == NULL) return {};
 queue<Node*> q;
 q.push(root);
 vector<int> ans;
 while (!q.empty()) {
 int sz = q.size();
 ans.push_back(q.front()->data); // rightmost now!
 for (int i = 0; i < sz; i++) {
 Node* curr = q.front(); q.pop();
 if (curr->right) q.push(curr->right);
 if (curr->left) q.push(curr->left);
 }
 }
 return ans;
}

```

```

// Top View
// recursive
void topViewDFS(Node* curr, int dist, int level,
 map<int, pair<int,int>> &mp) {
 if (curr == NULL) return;
 if (mp.count(dist) == 0 || mp[dist].second > level) {
 mp[dist] = {curr->data, level};
 }
 topViewDFS(curr->left, dist - 1, level + 1, mp);
 topViewDFS(curr->right, dist + 1, level + 1, mp);
}

vector<int> topView(Node* root) {
 if (root == NULL) return {};
 map<int, pair<int,int>> mp; // dist -> {val, level}
 topViewDFS(root, 0, 0, mp);
 vector<int> ans;
 for (auto &p : mp) ans.push_back(p.second.first);
 return ans;
}

// iterative bfs
vector<int> topViewIterative(Node* root) {
 if (root == NULL) return {};
 map<int, int> mp; // dist -> node value
 queue<pair<Node*, int>> q; // node, horizontal dist
 q.push({root, 0}); vector<int> ans;
 while (!q.empty()) {
 auto front = q.front(); q.pop();
 Node* curr = front.first; int dist = front.second;
 if (mp.count(dist) == 0) {
 mp[dist] = curr->data;
 }
 if (curr->left) q.push({curr->left, dist - 1});
 if (curr->right) q.push({curr->right, dist + 1});
 }
 for (auto &p : mp) ans.push_back(p.second);
 return ans;
}

```

# Tree-Construction

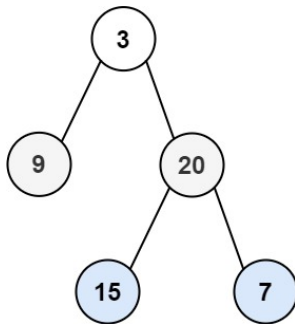
## Trees - Tree Construction

- [Construct BT from Preorder and Inorder](#)
- [Construct BT from Postorder and Inorder](#)
- [Maximum Binary Tree](#)
- [Construct BST from Preorder](#)

(Preorder-PLR, Inorder-LPR, Postorder-LRP)

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Input: root = [3,9,20,null,null,15,7]. Output: [[3],[9,20],[15,7]]



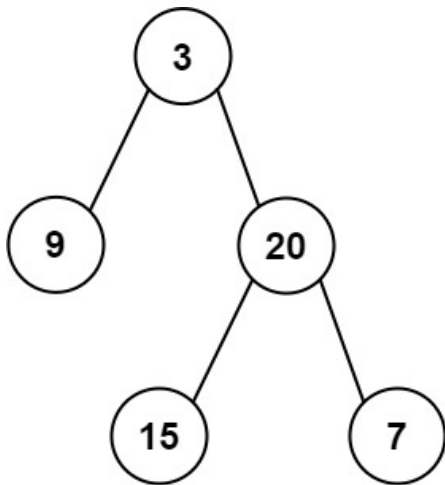
```
TreeNode* solve(vector<int>& preorder, vector<int>& inorder,
 unordered_map<int, int>& inomap, int& preo,
 int start, int end) {
 if (start > end) return nullptr;
 int data = preorder[preo++];
 TreeNode* root = new TreeNode(data);
 int mid = inomap[data];
 root->left = solve(preorder, inorder, inomap, preo, start, mid - 1);
 root->right = solve(preorder, inorder, inomap, preo, mid + 1, end);
 return root;
}

TreeNode* buildTree(vector<int>& preorder, vector<int>& inorder) {
 unordered_map<int, int> inomap;
 for (int i = 0; i < inorder.size(); i++)
 inomap[inorder[i]] = i;
 int preo = 0;
 return solve(preorder, inorder, inomap, preo, 0, inorder.size() - 1);
}
```

Given two integer arrays `inorder` and `postorder` where `inorder` is the inorder traversal of a binary tree and `postorder` is the postorder traversal of the same tree, construct and return the binary tree.

Input: `inorder = [9,3,15,20,7]`, `postorder = [9,15,7,20,3]`

Output: `[3,9,20,null,null,15,7]`



```
TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
 unordered_map<int,int> mp;
 for(int i = 0; i < inorder.size(); i++) mp[inorder[i]] = i;
 int postIdx = postorder.size() - 1;
 return build(inorder, postorder, mp, 0, inorder.size() - 1,
postIdx);
}

TreeNode* build(vector<int>& inorder, vector<int>& postorder,
 unordered_map<int,int>& mp,
 int inL, int inR, int &postIdx) {
 if(inL > inR) return NULL;
 int rootVal = postorder[postIdx--];
 TreeNode* root = new TreeNode(rootVal);
 int k = mp[rootVal];
 // Important: build right subtree first
 root->right = build(inorder, postorder, mp, k+1, inR, postIdx);
 root->left = build(inorder, postorder, mp, inL, k-1, postIdx);
 return root;
}
```

You are given an integer array `nums` with no duplicates. A maximum binary tree can be built recursively from `nums` using the following algorithm:

Create a root node whose value is the maximum value in `nums`.

Recursively build the left subtree on the subarray prefix to the left of the maximum value. Recursively build the right subtree on the subarray suffix to the right of the maximum value.

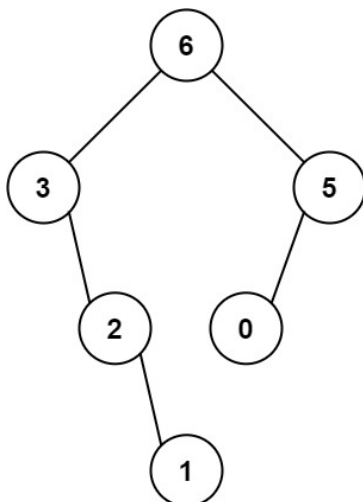
Return the maximum binary tree built from `nums`.

Input: `nums = [3,2,1,6,0,5]`

Output: `[6,3,5,null,2,0,null,null,1]`

Explanation: The recursive calls are as follow:

- The largest value in `[3,2,1,6,0,5]` is 6. Left prefix is `[3,2,1]` and right suffix is `[0,5]`.
  - The largest value in `[3,2,1]` is 3. Left prefix is `[]` and right suffix is `[2,1]`.
    - Empty array, so no child.
    - The largest value in `[2,1]` is 2. Left prefix is `[]` and right suffix is `[1]`.
      - Empty array, so no child.
      - Only one element, so child is a node with value 1.
  - The largest value in `[0,5]` is 5. Left prefix is `[0]` and right suffix is `[]`.
    - Only one element, so child is a node with value 0.
    - Empty array, so no child.



```

TreeNode* constructMaximumBinaryTree(vector<int>& nums) {
 return build(nums, 0, nums.size() - 1);
}
TreeNode* build(vector<int>& nums, int l, int r) {
 if(l > r) return NULL;
 int idx = l;
 for(int i = l; i <= r; i++)
 if(nums[i] > nums[idx]) idx = i;
 TreeNode* root = new TreeNode(nums[idx]);
 root->left = build(nums, l, idx - 1);
 root->right = build(nums, idx + 1, r);
 return root;
}

```

Given an array of integers preorder, which represents the preorder traversal of a BST (i.e., binary search tree), construct the tree and return its root.

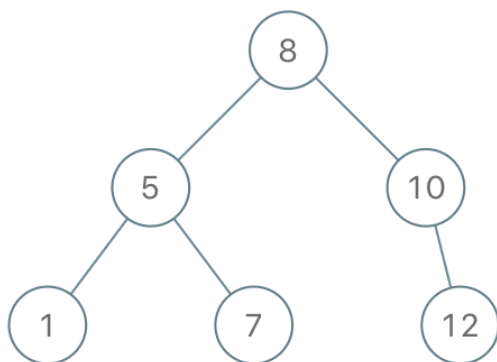
It is guaranteed that there is always possible to find a binary search tree with the given requirements for the given test cases.

A binary search tree is a binary tree where for every node, any descendant of Node.left has a value strictly less than Node.val, and any descendant of Node.right has a value strictly greater than Node.val.

A preorder traversal of a binary tree displays the value of the node first, then traverses Node.left, then traverses Node.right.

Input: preorder = [8,5,1,7,10,12]

Output: [8,5,10,1,7,null,12]





```
int idx;
TreeNode* bstFromPreorder(vector<int>& preorder) {
 idx = 0;
 return build(preorder, INT_MAX);
}
TreeNode* build(vector<int>& preorder, int bound) {
 if(idx == preorder.size() || preorder[idx] > bound)
 return NULL;
 int rootVal = preorder[idx++];
 TreeNode* root = new TreeNode(rootVal);
 root->left = build(preorder, rootVal);
 root->right = build(preorder, bound);
 return root;
}
```

# Tree-Height

## Trees - Height related Problems

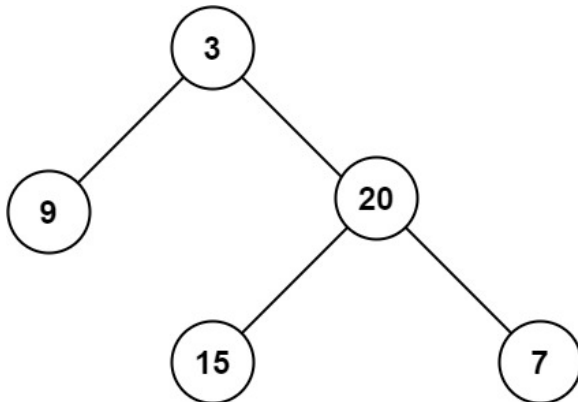
- [Maximum Depth of BT](#)
- [Balanced Binary Tree](#)
- [Diameter of Binary Tree](#)
- [Minimum Depth of BT](#)

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Input: root = [3,9,20,null,null,15,7]

Output: 3

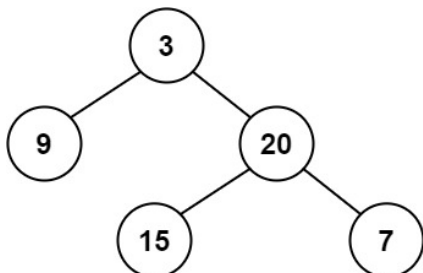


```
int maxDepth(TreeNode* root) {
 if(root == NULL) return 0;
 return 1 + max(maxDepth(root->left), maxDepth(root->right));
}
```

Given a binary tree, determine if it is height-balanced.

Input: root = [3,9,20,null,null,15,7]

Output: true



```

bool isBalanced(TreeNode* root) {
 return height(root) != -1;
}
int height(TreeNode* root) {
 if(root == NULL) return 0;
 int lh = height(root->left);
 if(lh == -1) return -1;
 int rh = height(root->right);
 if(rh == -1) return -1;
 if(abs(lh - rh) > 1) return -1;
 return 1 + max(lh, rh);
}

```

Given the root of a binary tree, return the length of the diameter of the tree.

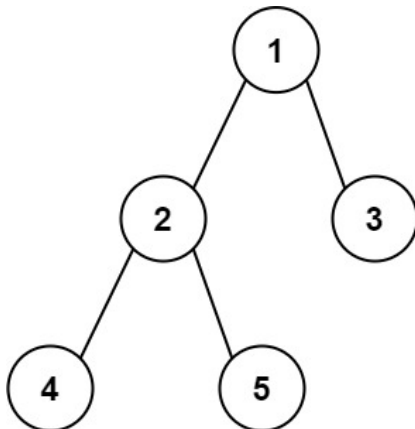
The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].



```

int ans = 0;
int diameterOfBinaryTree(TreeNode* root) {
 height(root);
 return ans;
}
int height(TreeNode* root) {
 if(root == NULL) return 0;
 int lh = height(root->left);
 int rh = height(root->right);
 ans = max(ans, lh + rh); // edges = nodes - 1
 return 1 + max(lh, rh);
}

```

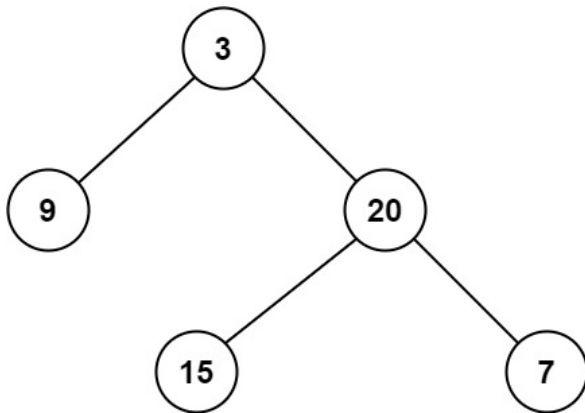
**Given a binary tree, find its minimum depth.**

The minimum depth is the number of nodes along the shortest path from the root node down to the nearest leaf node.

**Note:** A leaf is a node with no children.

**Input:** root = [3,9,20,null,null,15,7]

**Output:** 2



```

int minDepth(TreeNode* root) {
 if(root == NULL) return 0;
 int left = minDepth(root->left);
 int right = minDepth(root->right);
 if(root->left == NULL) return 1 + right;
 if(root->right == NULL) return 1 + left;
 return 1 + min(left, right);
}

```

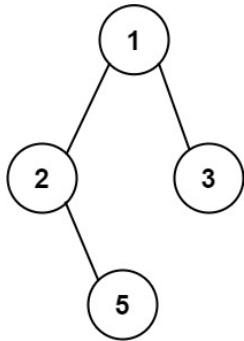
Tree-RootToLeaf

## Trees - Root to leaf path problems

- [Binary Tree Paths](#)
- [Path Sum ii](#)
- [Sum Root to Leaf numbers](#)
- [Smallest string starting from Leaf](#)
- [Insufficient nodes in root to Leaf](#)
- [Pseudo-Palindromic Paths in a Binary Tree](#)
- [Binary Tree Maximum Path Sum](#)

Given the root of a binary tree, return all root-to-leaf paths in any order. A leaf is a node with no children.

Input: root = [1,2,3,null,5]. Output: ["1->2->5","1->3"]

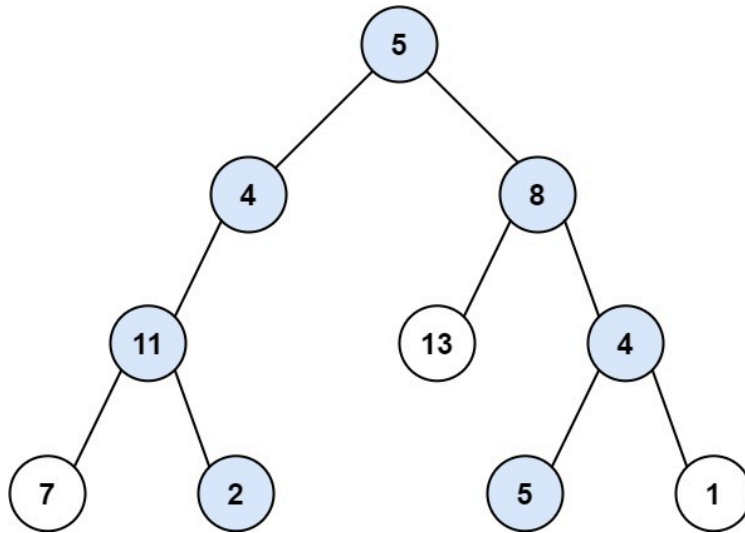


```
vector<string> ans;
vector<string> binaryTreePaths(TreeNode* root) {
 if(root == NULL) return ans;
 dfs(root, "");
 return ans;
}

void dfs(TreeNode* node, string path) {
 if(node == NULL) return;
 if(path.size() == 0) path = to_string(node->val);
 else path += "->" + to_string(node->val);
 if(node->left == NULL && node->right == NULL) {
 ans.push_back(path);
 return;
 }
 dfs(node->left, path);
 dfs(node->right, path);
}
```

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references. A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22. Output: [[5,4,11,2],[5,8,4,5]]. Explanation: There are two paths whose sum equals targetSum: 5+4+11+2=22, 5+8+4+5=22



```
vector<vector<int>> res;
vector<int> curr;
vector<vector<int>> pathSum(TreeNode* root, int targetSum) {
 dfs(root, targetSum);
 return res;
}
void dfs(TreeNode* root, int sum) {
 if(root == NULL) return;
 curr.push_back(root->val);
 if(root->left==NULL && root->right==NULL && sum==root->val)
 res.push_back(curr);
 dfs(root->left, sum - root->val);
 dfs(root->right, sum - root->val);
 curr.pop_back();
}
```



You are given the root of a binary tree containing digits from 0 to 9 only.

Each root-to-leaf path in the tree represents a number.

For example, the root-to-leaf path 1 -> 2 -> 3 represents the number 123.

Return the total sum of all root-to-leaf numbers. Test cases are generated so that the answer will fit in a 32-bit integer.

A leaf node is a node with no children.

Input: root = [4,9,0,5,1]

Output: 1026

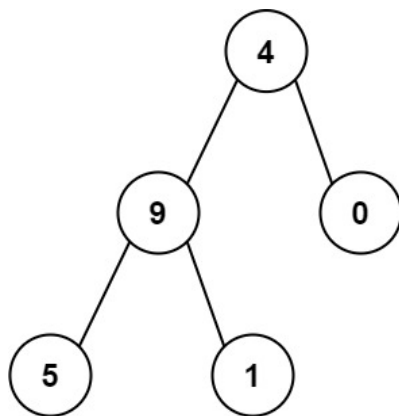
Explanation:

The root-to-leaf path 4->9->5 represents the number 495.

The root-to-leaf path 4->9->1 represents the number 491.

The root-to-leaf path 4->0 represents the number 40.

Therefore, sum = 495 + 491 + 40 = 1026.



```
int sumNumbers(TreeNode* root) {
 return dfs(root, 0);
}

int dfs(TreeNode* root, int num) {
 if(root == NULL) return 0;
 num = num * 10 + root->val;
 if(root->left == NULL && root->right == NULL)
 return num;
 return dfs(root->left, num) + dfs(root->right, num);
}
```

You are given the root of a binary tree where each node has a value in the range [0, 25] representing the letters 'a' to 'z'.

Return the lexicographically smallest string that starts at a leaf of this tree and ends at the root.

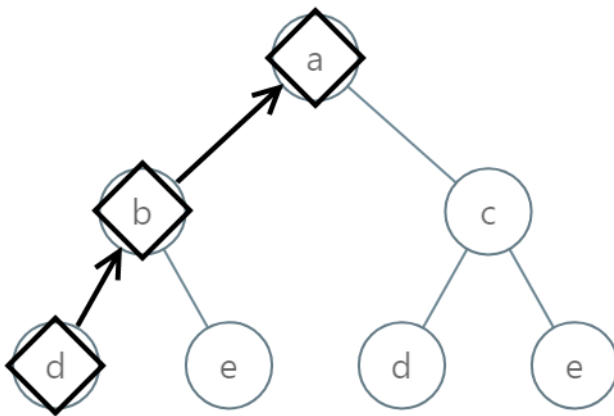
As a reminder, any shorter prefix of a string is lexicographically smaller.

For example, "ab" is lexicographically smaller than "aba".

A leaf of a node is a node that has no children.

Input: root = [0,1,2,3,4,3,4]

Output: "dba"



```
string ans = "~"; // '~' is lexicographically larger than any lowercase
```

```
string smallestFromLeaf(TreeNode* root) {
 string curr = "";
 dfs(root, curr);
 return ans;
}

void dfs(TreeNode* root, string curr) {
 if(root == NULL) return;
 char c = 'a' + root->val;
 curr = c + curr; // prepend
 if(root->left == NULL && root->right == NULL) {
 if(curr < ans) ans = curr;
 return;
 }
 dfs(root->left, curr);
 dfs(root->right, curr);
}
```

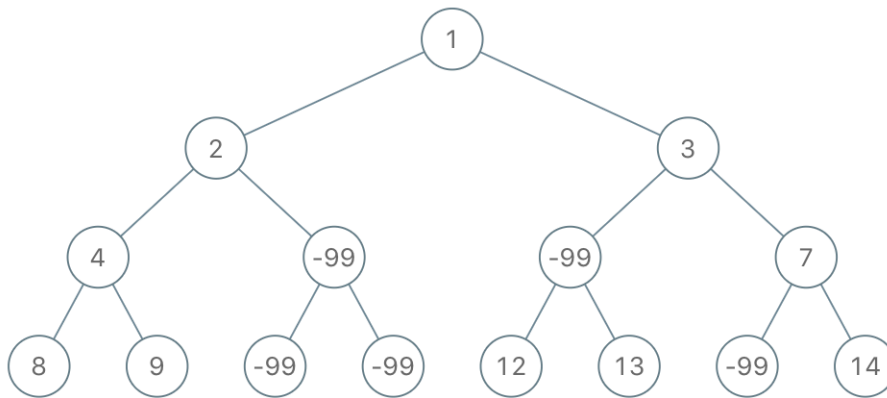
Given the root of a binary tree and an integer limit, delete all insufficient nodes in the tree simultaneously, and return the root of the resulting binary tree.

A node is insufficient if every root to leaf path intersecting this node has a sum strictly less than limit.

A leaf is a node with no children.

Input: root = [1,2,3,4,-99,-99,7,8,9,-99,-99,12,13,-99,14], limit = 1

Output: [1,2,3,4,null,null,7,8,9,null,14]



```
TreeNode* sufficientSubset(TreeNode* root, int limit) {
 return dfs(root, 0, limit);
}

TreeNode* dfs(TreeNode* root, int sum, int limit) {
 if (!root) return NULL;
 sum += root->val;
 // store original leaf status
 bool isLeaf = (!root->left && !root->right);
 root->left = dfs(root->left, sum, limit);
 root->right = dfs(root->right, sum, limit);
 if (isLeaf) {
 return (sum >= limit) ? root : NULL;
 }
 // internal node: keep only if at least one child survives
 return (root->left || root->right) ? root : NULL;
}
```

Given a binary tree where node values are digits from 1 to 9. A path in the binary tree is said to be pseudo-palindromic if at least one permutation of the node values in the path is a palindrome.

Return the number of pseudo-palindromic paths going from the root node to leaf nodes.

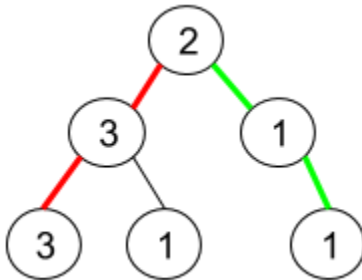
Input: root = [2,3,1,3,1,null,1]

Output: 2

Explanation: The figure above represents the given binary tree.

There are three paths going from the root node to leaf nodes: the red path [2,3,3], the green path [2,1,1], and the path [2,3,1].

Among these paths only red path and green path are pseudo-palindromic paths since the red path [2,3,3] can be rearranged in [3,2,3] (palindrome) and the green path [2,1,1] can be rearranged in [1,2,1] (palindrome).



```
int count = 0;
int pseudoPalindromicPaths (TreeNode* root) {
 dfs(root, 0);
 return count;
}

void dfs(TreeNode* root, int mask) {
 if(root == NULL) return;
 mask ^= (1 << root->val);
 if(root->left == NULL && root->right == NULL) {
 if((mask & (mask - 1)) == 0)
 count++;
 return;
 }
 dfs(root->left, mask);
 dfs(root->right, mask);
}
```

[hard]

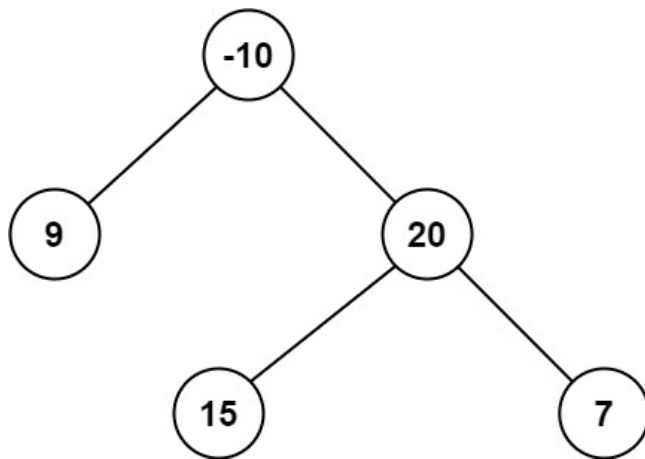
A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path. Given the root of a binary tree, return the maximum path sum of any non-empty path.

Input: root = [-10,9,20,null,null,15,7]

Output: 42

Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of  $15 + 20 + 7 = 42$ .



```
int ans;
int maxPathSum(TreeNode* root) {
 ans = INT_MIN;
 dfs(root);
 return ans;
}
int dfs(TreeNode* root) {
 if(root == NULL) return 0;
 int left = max(0, dfs(root->left));
 int right = max(0, dfs(root->right));
 ans = max(ans, root->val + left + right);
 return root->val + max(left, right);
}
```

Tree-Ancestor

## Trees - Ancestor problem

- [LCA of Binary Tree](#)
- [Maximum difference between node and ancestor](#)
- [LCA of deepest leaves](#)
- [Kth Ancestor of a Tree Node](#)

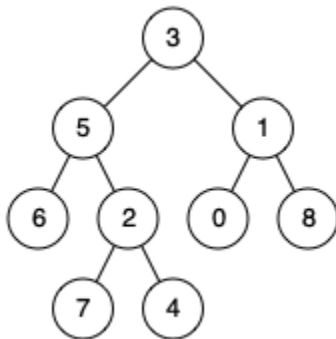
Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1

Output: 3

Explanation: The LCA of nodes 5 and 1 is 3.



```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
TreeNode* q) {
 if (!root || root == p || root == q) return root;
 TreeNode* left = lowestCommonAncestor(root->left, p, q);
 TreeNode* right = lowestCommonAncestor(root->right, p, q);
 if (left && right) return root;
 return left ? left : right;
}
```

Given the root of a binary tree, find the maximum value  $v$  for which there exist different nodes  $a$  and  $b$  where  $v = |a.val - b.val|$  and  $a$  is an ancestor of  $b$ .

A node  $a$  is an ancestor of  $b$  if either: any child of  $a$  is equal to  $b$  or any child of  $a$  is an ancestor of  $b$ .

Input: root = [8,3,10,1,6,null,14,null,null,4,7,13]

Output: 7

Explanation: We have various ancestor-node differences, some of which are given below :

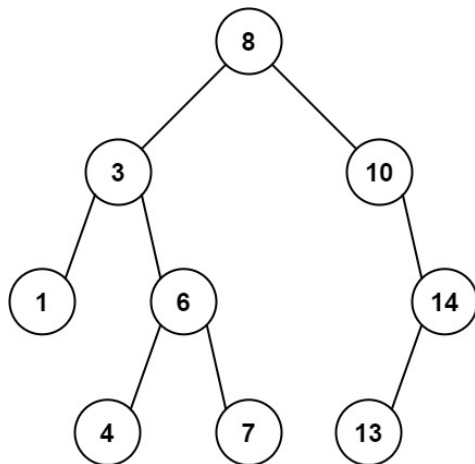
$$|8 - 3| = 5$$

$$|3 - 7| = 4$$

$$|8 - 1| = 7$$

$$|10 - 13| = 3$$

Among all possible differences, the maximum value of 7 is obtained by  $|8 - 1| = 7$ .



```
int dfs(TreeNode* node, int mn, int mx) {
 if (!node) return mx - mn;
 mn = min(mn, node->val);
 mx = max(mx, node->val);
 int left = dfs(node->left, mn, mx);
 int right = dfs(node->right, mn, mx);
 return max(left, right);
}

int maxAncestorDiff(TreeNode* root) {
 return dfs(root, root->val, root->val);
}
```



Given the root of a binary tree, return the lowest common ancestor of its deepest leaves.

Recall that:

The node of a binary tree is a leaf if and only if it has no children

The depth of the root of the tree is 0. if the depth of a node is  $d$ , the depth of each of its children is  $d + 1$ .

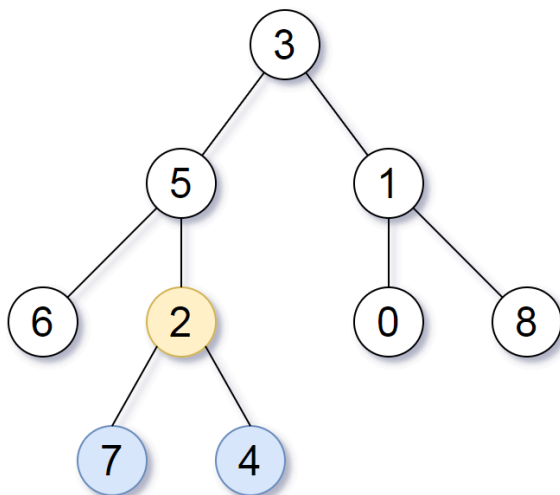
The lowest common ancestor of a set  $S$  of nodes, is the node  $A$  with the largest depth such that every node in  $S$  is in the subtree with root  $A$ .

Input: root = [3,5,1,6,2,0,8,null,null,7,4]

Output: [2,7,4]

Explanation: We return the node with value 2, colored in yellow in the diagram.

The nodes coloured in blue are the deepest leaf-nodes of the tree. Note that nodes 6, 0, and 8 are also leaf nodes, but the depth of them is 2, but the depth of nodes 7 and 4 is 3.



```

struct Info {
 int depth;
 TreeNode* lca;
};

Info solve(TreeNode* root) {
 if (!root) {
 Info base;
 base.depth = 0;
 base.lca = NULL;
 return base;
 }
 Info left = solve(root->left);
 Info right = solve(root->right);
 Info result;
 if (left.depth > right.depth) {
 result.depth = left.depth + 1;
 result.lca = left.lca;
 }
 else if (right.depth > left.depth) {
 result.depth = right.depth + 1;
 result.lca = right.lca;
 }
 else {
 // equal depth → current root is LCA
 result.depth = left.depth + 1;
 result.lca = root;
 }
 return result;
}

TreeNode* lcaDeepestLeaves(TreeNode* root) {
 return solve(root).lca;
}

```

[hard]

You are given a tree with  $n$  nodes numbered from  $0$  to  $n - 1$  in the form of a parent array `parent` where `parent[i]` is the parent of  $i$ th node. The root of the tree is node  $0$ . Find the  $k$ th ancestor of a given node.

The  $k$ th ancestor of a tree node is the  $k$ th node in the path from that node to the root node.

Implement the `TreeAncestor` class:

`TreeAncestor(int n, int[] parent)` Initializes the object with the number of nodes in the tree and the parent array.

`int getKthAncestor(int node, int k)` return the  $k$ th ancestor of the given node `node`. If there is no such ancestor, return `-1`.

Input

```
["TreeAncestor", "getKthAncestor", "getKthAncestor",
"getKthAncestor"]
```

```
[[7, [-1, 0, 0, 1, 1, 2, 2]], [3, 1], [5, 2], [6, 3]]
```

Output

```
[null, 1, 0, -1]
```

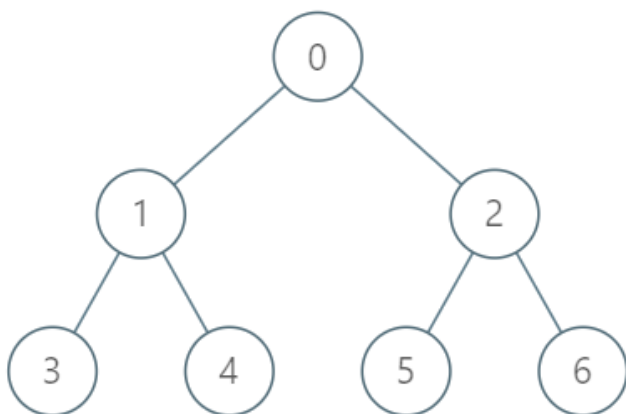
Explanation

```
TreeAncestor treeAncestor = new TreeAncestor(7, [-1, 0, 0, 1, 1, 2, 2]);
```

```
treeAncestor.getKthAncestor(3, 1); // returns 1 which is the parent
of 3
```

```
treeAncestor.getKthAncestor(5, 2); // returns 0 which is the
grandparent of 5
```

```
treeAncestor.getKthAncestor(6, 3); // returns -1 because there is no
such ancestor
```



```

class TreeAncestor {
/*
Do binary lifting
Constraints
 $n \leq 5e5$ often $\rightarrow$ binary lifting needed.
Preprocessing
 $up[i][j]$ = 2^j -th ancestor of node i .
Binary lifting table size
 $LOG = 20$ (since $2^{19} \approx 5e5$)
*/
public:
 vector<vector<int>> up;
 int LOG;
 TreeAncestor(int n, vector<int>& parent) {
 LOG = 20;
 up.assign(n, vector<int>(LOG, -1));
 for (int i = 0; i < n; i++)
 up[i][0] = parent[i];
 for (int j = 1; j < LOG; j++) {
 for (int i = 0; i < n; i++) {
 int prev = up[i][j-1];
 up[i][j] = (prev == -1) ? -1 : up[prev][j-1];
 }
 }
 }
 int getKthAncestor(int node, int k) {
 for (int j = 0; j < LOG; j++) {
 if (k & (1 << j)) {
 node = up[node][j];
 if (node == -1) return -1;
 }
 }
 return node;
 }
};

```

# Tree-BST

## Trees - Binary Search Tree

- [Validate BST](#)
- [Range Sum of BST](#)
- [Minimum Absolute Difference in BST](#)
- [Insert into a BST](#)
- [LCA of BST](#)

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

The left subtree of a node contains only nodes with keys strictly less than the node's key.

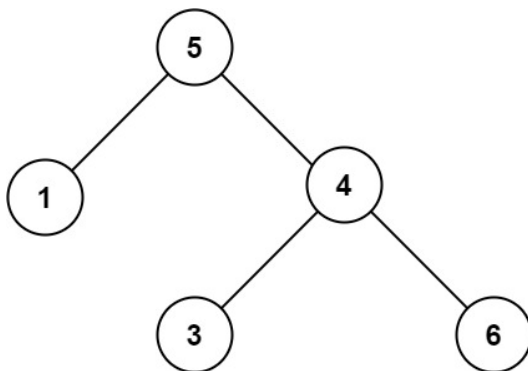
The right subtree of a node contains only nodes with keys strictly greater than the node's key.

Both the left and right subtrees must also be binary search trees.

Input: root = [5,1,4,null,null,3,6]

Output: false

Explanation: The root node's value is 5 but its right child's value is 4.



```
bool isValidBST(TreeNode* root) {
 return validate(root, LONG_MIN, LONG_MAX);
}

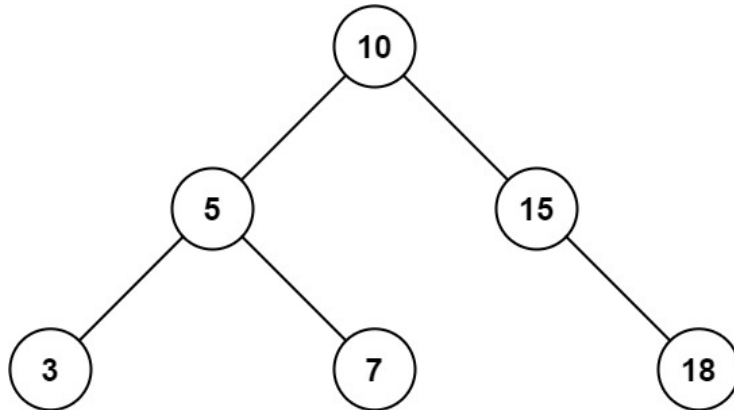
bool validate(TreeNode* node, long low, long high) {
 if (!node) return true;
 if (node->val <= low || node->val >= high) return false;
 return validate(node->left, low, node->val) &&
 validate(node->right, node->val, high);
}
```

Given the root node of a binary search tree and two integers low and high, return the sum of values of all nodes with a value in the inclusive range [low, high].

Input: root = [10,5,15,3,7,null,18], low = 7, high = 15

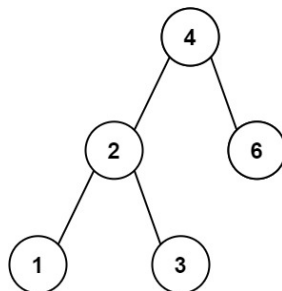
Output: 32

Explanation: Nodes 7, 10, and 15 are in the range [7, 15].  $7 + 10 + 15 = 32$ .



```
int rangeSumBST(TreeNode* root, int low, int high) {
 if (!root) return 0;
 if (root->val < low) return rangeSumBST(root->right, low, high);
 if (root->val > high) return rangeSumBST(root->left, low, high);
 return root->val +
 rangeSumBST(root->left, low, high) +
 rangeSumBST(root->right, low, high);
}
```

Given the root of a Binary Search Tree (BST), return the minimum absolute difference between the values of any two different nodes in the tree. Input: root = [4,2,6,1,3]. Output: 1



```
int res = INT_MAX;
TreeNode* prev = NULL;
```

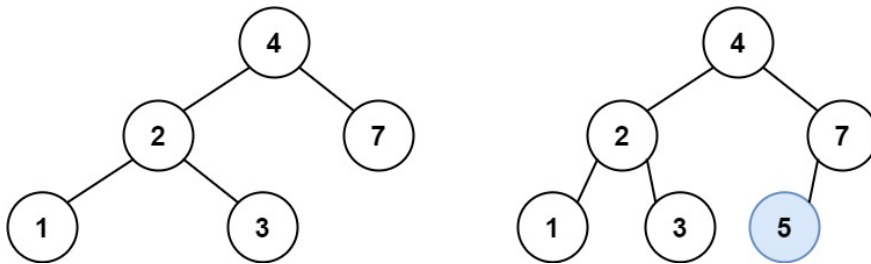
```

int getMinimumDifference(TreeNode* root) {
 inorder(root);
 return res;
}

void inorder(TreeNode* node) {
 if (!node) return;
 inorder(node->left);
 if (prev != NULL) {
 res = min(res, node->val - prev->val);
 }
 prev = node;
 inorder(node->right);
}

```

You are given the root node of a binary search tree (BST) and a value to insert into the tree. Return the root node of the BST after the insertion. It is guaranteed that the new value does not exist in the original BST. Notice that there may exist multiple valid ways for the insertion, as long as the tree remains a BST after insertion. You can return any of them. Input: root = [4,2,7,1,3], val = 5. Output: [4,2,7,1,3,5]. Explanation: Another accepted tree is: [5,2,7,1,3,null,null,null,null,4].



```

TreeNode* insertIntoBST(TreeNode* root, int val) {
 if (!root) return new TreeNode(val);
 if (val < root->val)
 root->left = insertIntoBST(root->left, val);
 else
 root->right = insertIntoBST(root->right, val);
 return root;
}

```



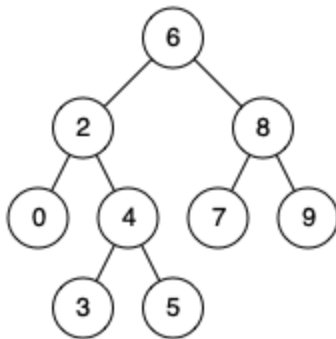
Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6. And for 2 & 4 is 2.



```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p,
TreeNode* q) {
 while (root) {
 if (p->val < root->val && q->val < root->val)
 root = root->left;
 else if (p->val > root->val && q->val > root->val)
 root = root->right;
 else
 return root;
 }
 return NULL;
}
```

DP-Take/Not Take

## DP - Take / Not take (DP)

Description: Solve optimization problems like selecting items with the max/min value under certain constraints.

- [\[Self Added\] House Robber i](#)
- [House Robber ii](#)
- [Target Sum](#)
- [Partition Equal Subset Sum](#)
- [Ones and Zeroes](#)
- [Last Stone Weight ii](#)

Given an integer array `nums` where each element represents the amount of money in a house arranged in a line – You cannot rob two adjacent houses. Find the maximum money you can rob.

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob = 2 + 9 + 1 = 12.

```
int solve(int i, vector<int>& nums, vector<int>& dp) {
 int n = nums.size();
 if (i >= n) return 0; // past last house
 if (dp[i] != -1) return dp[i]; // memoized
 int rob = nums[i] + solve(i + 2, nums, dp);
 int skip = solve(i + 1, nums, dp);
 return dp[i] = max(rob, skip);
}

int rob(vector<int>& nums) {
 int n = nums.size();
 vector<int> dp(n, -1);
 return solve(0, nums, dp);
}
```

```

/*
solve(0)
├─ rob = 2 + solve(2)
│ └─ rob = 9 + solve(4)
│ └─ rob = 1 + solve(6=0)
│ └─ skip = solve(5=0)
│ → max = 1
│ └─ skip = solve(3)
│ └─ rob = 3 + solve(5=0)
│ └─ skip = solve(4)=1
│ → max = 3
│ → max = 9 + 1 = 10
└─ skip = solve(1)
 └─ rob = 7 + solve(3)
 → 7 + 3 = 10
 └─ skip = solve(2)=10
 → max = 10
→ max = 2 + 10 = 12
*/

```

You're given an array `nums` where: Each element = amount of money in a house. Houses are arranged in a circle → first and last houses are adjacent. You cannot rob two adjacent houses. Return the maximum amount of money you can rob.

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob = 1 + 3 = 4.

```

int solve(int i, vector<int>& nums, vector<int>& dp) {
 if (i >= nums.size()) return 0; // beyond last index
 if (dp[i] != -1) return dp[i]; // memoized result
 int rob = nums[i] + solve(i + 2, nums, dp); // rob this house
 int skip = solve(i + 1, nums, dp); // skip this house
 return dp[i] = max(rob, skip);
}

int rob(vector<int>& nums) {
 int n = nums.size();
 if (n == 1) return nums[0];
 vector<int> nums1(nums.begin(), nums.end() - 1); // exclude last
 vector<int> nums2(nums.begin() + 1, nums.end()); // exclude first
 vector<int> dp1(nums1.size(), -1);
 vector<int> dp2(nums2.size(), -1);
 return max(solve(0, nums1, dp1), solve(0, nums2, dp2));
}

```

You are given an integer array `nums` and an integer `target`. You must assign either '+' or '-' before each number and evaluate the expression. Return the number of different expressions that evaluate to target. Input: `nums = [1,1,1,1,1]`, `target = 3`.

Output: 5. Explanation: There are 5 ways to assign symbols to make the sum of `nums` be target 3. Eg:  $-1 + 1 + 1 + 1 + 1 = 3$ ,  $+1 - 1 + 1 + 1 + 1 = 3$ ,  $+1 + 1 - 1 + 1 + 1 = 3$ ,  $+1 + 1 + 1 - 1 + 1 = 3$ ,  $+1 + 1 + 1 + 1 - 1 = 3$

```

unordered_map<string, int> dp; // global variable
int solve(vector<int>& nums, int i, int currSum, int target) {
 if (i == nums.size())
 return currSum == target ? 1 : 0;
 string key = to_string(i) + "," + to_string(currSum);
 if (dp.count(key)) return dp[key];
 int plus = solve(nums, i + 1, currSum + nums[i], target);
 int minus = solve(nums, i + 1, currSum - nums[i], target);
 return dp[key] = plus + minus;
}

int findTargetSumWays(vector<int>& nums, int target) {
 return solve(nums, 0, 0, target);
}

```

```

/*
 (i=0, sum=0)
 / \
 +1 -1
 (i=1,1) (i=1,-1)
 / \ / \
 +1 -1 +1 -1
(i=2,2) (i=2,0) (i=2,0) (i=2,-2)
 /\ /\ /\ /\
+1 -1 +1 -1 +1 -1 +1 -1
(3,3)(3,1) (3,1)(3,-1)(3,1)(3,-1) (3,-1)(3,-3)
... continue 2 more levels ...
*/

```

You are given an integer array `nums`. Return `true` if you can partition it into two subsets with equal sum, otherwise `false`.

Input: `nums = [1,5,11,5]`. Output: `true`. Explanation: The array can be partitioned as `[1, 5, 5]` and `[11]`.

// Let total sum =  $S$ . For equal partition, both subsets must sum to  $S/2$ . So, the problem becomes: Can we find a subset of `nums` whose sum =  $S/2$ ? If yes  $\rightarrow$  the remaining elements also sum to  $S/2$ .

```

bool solve(int i, int currSum, int target, vector<int>& nums,
vector<vector<int>>& dp) {
 if (currSum == target) return true; // found a subset
 if (i == nums.size() || currSum > target) return false;
 if (dp[i][currSum] != -1) return dp[i][currSum];
 // Option 1: Take the current element
 bool take = solve(i + 1, currSum + nums[i], target, nums, dp);
 // Option 2: Skip the current element
 bool notTake = solve(i + 1, currSum, target, nums, dp);
 return dp[i][currSum] = (take || notTake);
}

bool canPartition(vector<int>& nums) {
 int total = accumulate(nums.begin(), nums.end(), 0);
 if (total % 2 != 0) return false; // odd sum $\rightarrow$ cannot partition
 int target = total / 2; int n = nums.size();
 vector<vector<int>> dp(n, vector<int>(target + 1, -1));
 return solve(0, 0, target, nums, dp);
}

```

```

/*
(0, 0)
├── take 1 → (1, 1)
│ ├── take 5 → (2, 6)
│ │ ├── take 11 → (3, 17) ✗
│ │ └── skip → (3, 6)
│ └── skip 5 → (2, 1)
│ ├── take 11 → (3, 12) ✗
│ └── take 5 → (3, 6)
└── skip 1 → (1, 0)
 ├── take 5 → (2, 5)
 │ ├── take 11 → (3, 16) ✗
 │ └── take 5 → (3, 10)
 └── take 11 → (2, 11) ✓ FOUND!
*/

```

You are given an array of binary strings `strs` and two integers `m` and `n`. Return the size of the largest subset of `strs` such that there are at most `m` 0's and `n` 1's in the subset.

A set `x` is a subset of a set `y` if all elements of `x` are also elements of `y`.

Input: `strs = ["10","0001","111001","1","0"]`, `m = 5`, `n = 3`

Output: 4

Explanation: The largest subset with at most 5 0's and 3 1's is `{"10", "0001", "1", "0"}`, so the answer is 4.

Other valid but smaller subsets include `{"0001", "1"}` and `{"10", "1", "0"}`.

`{"111001"}` is an invalid subset because it contains 4 1's, greater than the maximum of 3.

```

vector<vector<int>> cnt;
vector<vector<vector<int>>> dp;
int solve(int i, int m, int n, const vector<string>& strs) {
 if (i == strs.size()) return 0;
 int &res = dp[i][m][n];
 if (res != -1) return res;
 int zeros = cnt[i][0];
 int ones = cnt[i][1];
 // skip
 int notTake = solve(i + 1, m, n, strs);
 // take (if fits)
 int take = 0;
 if (m >= zeros && n >= ones)
 take = 1 + solve(i + 1, m - zeros, n - ones, strs);
 return res = max(take, notTake);
}
int findMaxForm(vector<string>& strs, int m, int n) {
 int N = strs.size();
 cnt.assign(N, vector<int>(2, 0)); // Same as: cnt = [[0, 0], [0,
0], [0, 0],...N times]
 for (int i = 0; i < N; i++) {
 int z = 0, o = 0;
 for (char c : strs[i])
 (c == '0') ? z++ : o++;
 cnt[i][0] = z;
 cnt[i][1] = o;
 }
 dp.assign(N+1, vector<vector<int>>(m+1, vector<int>(n+1,-1)));
 return solve(0, m, n, strs);
}

```



You are given an array of integers stones where stones[i] is the weight of the ith stone. We are playing a game with the stones. On each turn, we choose any two stones and smash them together. Suppose the stones have weights x and y with  $x \leq y$ . The result of this smash is: If  $x == y$ , both stones are destroyed, and If  $x \neq y$ , the stone of weight x is destroyed, and the stone of weight y has new weight  $y - x$ . At the end of the game, there is at most one stone left. Return the smallest possible weight of the left stone. If there are no stones left, return 0. Input: stones = [2,7,4,1,8,1]. Output: 1 Explanation: We can combine 2 and 4 to get 2, so the array converts to [2,7,1,8,1] then, we can combine 7 and 8 to get 1, so the array converts to [2,1,1,1] then, we can combine 2 and 1 to get 1, so the array converts to [1,1,1] then, we can combine 1 and 1 to get 0, so the array converts to [1], then that's the optimal value.

```
bool solve(int i, int sum, vector<int>& stones, vector<vector<int>>& dp){
 if (sum == 0) return true;
 if (i == stones.size()) return false;
 if (dp[i][sum] != -1) return dp[i][sum];
 bool notTake = solve(i+1, sum, stones, dp);
 bool take = false;
 if (sum >= stones[i])
 take = solve(i+1, sum - stones[i], stones, dp);
 return dp[i][sum] = (take || notTake);
}

int lastStoneWeightII(vector<int>& stones) {
 int total = accumulate(stones.begin(), stones.end(), 0);
 int target = total / 2; int n = stones.size();
 vector<vector<int>> dp(n, vector<int>(target + 1, -1));
 int best = 0;
 for (int s = target; s >= 0; s--) {
 if (solve(0, s, stones, dp)) {
 best = s;
 break;
 }
 }
 return total - 2 * best;}
```

DP-InfiniteSupply

## DP - Infinite Supply (DP)

Description: Similar to the 0/1 knapsack, but items can be chosen multiple times.

- [Coin Change](#)
- [Coin Change II](#)
- [Perfect Squares](#)
- [Minimum Cost For Tickets](#)
- [\[Self added v2\] Rod Cutting Problem](#)

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Input: `coins = [1,2,5]`, `amount = 11`

Output: 3

Explanation:  $11 = 5 + 5 + 1$

```
vector<int> dp; // dp[x] = min coins for amount x
int INF = INT_MAX/2;
int solve(vector<int>& coins, int amount) {
 if (amount == 0) return 0;
 if (amount < 0) return INF;
 if (dp[amount] != -1) return dp[amount];
 int best = INF;
 for (int c : coins) {
 best = min(best, 1 + solve(coins, amount - c));
 }
 return dp[amount] = best;
}

int coinChange(vector<int>& coins, int amount) {
 dp.assign(amount + 1, -1);
 int ans = solve(coins, amount);
 return (ans >= INF ? -1 : ans);
}
```

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return the number of combinations that make up that amount. If that amount of money cannot be made up by any combination of the coins, return 0.

You may assume that you have an infinite number of each kind of coin.

The answer is guaranteed to fit into a signed 32-bit integer.

Input: `amount = 5, coins = [1,2,5]`

Output: 4

Explanation: there are four ways to make up the amount:

5=5

5=2+2+1

5=2+1+1+1

5=1+1+1+1+1

```
vector<vector<int>> dp;
int solve(int i, int amount, vector<int>& coins) {
 if (amount == 0) return 1;
 if (i == coins.size()) return 0;
 if (dp[i][amount] != -1) return dp[i][amount];
 long long ways = 0;
 // take unlimited
 if (amount >= coins[i])
 ways += solve(i, amount - coins[i], coins);
 // skip
 ways += solve(i + 1, amount, coins);
 return dp[i][amount] = (int)ways;
}
int change(int amount, vector<int>& coins) {
 dp.assign(coins.size(), vector<int>(amount + 1, -1));
 return solve(0, amount, coins);
}
```

Given an integer  $n$ , return the least number of perfect square numbers that sum to  $n$ .

A perfect square is an integer that is the square of an integer; in other words, it is the product of some integer with itself. For example, 1, 4, 9, and 16 are perfect squares while 3 and 11 are not.

Input:  $n = 12$ . Output: 3

Explanation:  $12 = 4 + 4 + 4$ .

```
vector<int> dp;
int INF = INT_MAX/2;
int solve(int n, vector<int>& sq) {
 if (n == 0) return 0;
 if (n < 0) return INF;
 if (dp[n] != -1) return dp[n];
 int best = INF;
 for (int s : sq)
 best = min(best, 1 + solve(n - s, sq));
 return dp[n] = best;
}
int numSquares(int n) {
 vector<int> sq;
 for (int i = 1; i * i <= n; i++)
 sq.push_back(i * i);
 dp.assign(n + 1, -1);
 return solve(n, sq);
}
```

You have planned some train traveling one year in advance. The days of the year in which you will travel are given as an integer array `days`. Each day is an integer from 1 to 365.

Train tickets are sold in three different ways:

a 1-day pass is sold for `costs[0]` dollars, a 7-day pass is sold for `costs[1]` dollars, and a 30-day pass is sold for `costs[2]` dollars.

The passes allow that many days of consecutive travel.

For example, if we get a 7-day pass on day 2, then we can travel for 7 days: 2, 3, 4, 5, 6, 7, and 8.

Return the minimum number of dollars you need to travel every day in the given list of days.

Input: `days = [1,4,6,7,8,20]`, `costs = [2,7,15]`

**Output: 11**

**Explanation:** For example, here is one way to buy passes that lets you travel your travel plan:

On day 1, you bought a 1-day pass for  $\text{costs}[0] = \$2$ , which covered day 1.

On day 3, you bought a 7-day pass for  $\text{costs}[1] = \$7$ , which covered days 3, 4, ..., 9.

On day 20, you bought a 1-day pass for  $\text{costs}[0] = \$2$ , which covered day 20.

In total, you spent \$11 and covered all the days of your travel.

```
vector<int> dp;
vector<int> d, cost;
int solve(int i) {
 if (i == d.size()) return 0;
 if (dp[i] != -1) return dp[i];
 // cost for 1-day
 int c1 = cost[0] + solve(i + 1);
 // cost for 7-day
 int j = i;
 while (j < d.size() && d[j] <= d[i] + 6) j++;
 int c7 = cost[1] + solve(j);
 // cost for 30-day
 j = i;
 while (j < d.size() && d[j] <= d[i] + 29) j++;
 int c30 = cost[2] + solve(j);
 return dp[i] = min(c1, min(c7, c30));
}
int mincostTickets(vector<int>& days, vector<int>& costs) {
 d = days;
 cost = costs;
 dp.assign(d.size(), -1);
 return solve(0);
}
```

**You are given:**

A rod of length N

length[] array → available cut lengths

price[] array → price of each cut size

You can cut the rod into pieces such that sum of lengths = N and you maximize the profit. (Unbounded Knapsack)

N = 8

length = [1,2,3,4]

price = [3,7,9,10]

Explain: If say: (8 cuts of size 1 →  $8 * 3 = 24$  profit). But (2 + 2 + 2 + 2 = 4 cuts of size 2 →  $4 * 7 = 28$ )

Hence best profit: 28.

```
int solve(int idx, int rem, vector<int> &len, vector<int> &price,
 vector<vector<int>> &dp) {
 // no more indices left
 if (idx == len.size()) return 0;
 if (dp[idx][rem] != -1) return dp[idx][rem];
 // Option 1: Do NOT take this cut size
 int notTake = solve(idx + 1, rem, len, price, dp);
 // Option 2: TAKE this cut size if it fits
 int take = INT_MIN;
 if (len[idx] <= rem) {
 take = price[idx] + solve(idx, rem-len[idx], len, price, dp);
 }
 return dp[idx][rem] = max(take, notTake);
}
```

```
int cutRod(vector<int> &price, int N) {
 vector<int> len(N);
 for (int i = 0; i < N; i++) len[i] = i + 1; // lengths = 1..N
 vector<vector<int>> dp(N, vector<int>(N + 1, -1));
 return solve(0, N, len, price, dp);
}
```

DP-LIS



## DP - Longest Increasing subsequence

**Description:** It involves finding the longest subsequence of a given sequence where the elements are in ascending order

- [Longest Increasing Subsequence](#)
- [Largest Divisible Subset](#)
- [Maximum Length of Pair Chain](#)
- [Number of LIS](#)
- [Longest String Chain](#)

Given an integer array `nums`, return the length of the longest strictly increasing subsequence

**Input:** `nums = [10,9,2,5,3,7,101,18]`

**Output:** 4

**Explanation:** The longest increasing subsequence is `[2,3,7,101]`, therefore the length is 4.

```
vector<vector<int>> dp;
int solve(int i, int prev, vector<int>& nums) {
 if (i == nums.size()) return 0;
 if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
 int skip = solve(i + 1, prev, nums);
 int take = 0;
 if (prev == -1 || nums[i] > nums[prev])
 take = 1 + solve(i + 1, i, nums);
 return dp[i][prev + 1] = max(skip, take);
}
int lengthOfLIS(vector<int>& nums) {
 int n = nums.size();
 dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
 return solve(0, -1, nums);
}
```

Given a set of distinct positive integers `nums`, return the largest subset `answer` such that every pair `(answer[i], answer[j])` of elements in this subset satisfies:

`answer[i] % answer[j] == 0`, or `answer[j] % answer[i] == 0`

If there are multiple solutions, return any of them.

Input: `nums = [1,2,3]`

Output: `[1,2]`

Explanation: `[1,3]` is also accepted.

```
vector<vector<int>> dp;
int solve(int i, int prev, vector<int>& nums) {
 if (i == nums.size()) return 0;
 if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
 int skip = solve(i + 1, prev, nums);
 int take = 0;
 if (prev == -1 || nums[i] % nums[prev] == 0)
 take = 1 + solve(i + 1, i, nums);
 return dp[i][prev + 1] = max(skip, take);
}
vector<int> largestDivisibleSubset(vector<int>& nums) {
 sort(nums.begin(), nums.end());
 int n = nums.size();
 dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
 solve(0, -1, nums);
 vector<int> ans; int i = 0, prev = -1;
 while (i < n) {
 int best = dp[i][prev + 1];
 int skip = solve(i + 1, prev, nums);
 if (best == skip) {
 i++;
 } else {
 ans.push_back(nums[i]);
 prev = i;
 i++;
 }
 }
 return ans;
}
```

You are given an array of  $n$  pairs `pairs` where `pairs[i] = [lefti, righti]` and `lefti < righti`.

A pair `p2 = [c, d]` follows a pair `p1 = [a, b]` if `b < c`. A chain of pairs can be formed in this fashion.

Return the length longest chain which can be formed.

You do not need to use up all the given intervals. You can select pairs in any order.

Input: `pairs = [[1,2],[2,3],[3,4]]`

Output: 2

Explanation: The longest chain is `[1,2] -> [3,4]`.

```
vector<vector<int>> dp;
int solve(int i, int prev, vector<vector<int>>& p) {
 if (i == p.size()) return 0;
 if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
 int skip = solve(i + 1, prev, p);
 int take = 0;
 if (prev == -1 || p[prev][1] < p[i][0])
 take = 1 + solve(i + 1, i, p);
 return dp[i][prev + 1] = max(skip, take);
}
int findLongestChain(vector<vector<int>>& pairs) {
 sort(pairs.begin(), pairs.end());
 int n = pairs.size();
 dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
 return solve(0, -1, pairs);
}
```

Given an integer array `nums`, return the number of longest increasing subsequences.

Notice that the sequence has to be strictly increasing.

Input: nums = [1,3,5,4,7]

Output: 2

Explanation: The two longest increasing subsequences are [1, 3, 4, 7] and [1, 3, 5, 7].

```
vector<vector<int>> lenDP, cntDP;
pair<int,int> solve(int i, int prev, vector<int>& nums) {
 if (i == nums.size()) return {0, 1};
 if (lenDP[i][prev+1] != -1)
 return {lenDP[i][prev+1], cntDP[i][prev+1]};
 // skip
 auto skip = solve(i + 1, prev, nums);
 // take
 pair<int,int> take = {0, 0};
 if (prev == -1 || nums[i] > nums[prev]) {
 auto r = solve(i + 1, i, nums);
 take.first = 1 + r.first;
 take.second = r.second;
 }
 // compare
 int L = max(skip.first, take.first);
 int C = 0;
 if (skip.first == L) C += skip.second;
 if (take.first == L) C += take.second;
 lenDP[i][prev+1] = L;
 cntDP[i][prev+1] = C;
 return {L, C};
}
int findNumberOfLIS(vector<int>& nums) {
 int n = nums.size();
 lenDP = cntDP = vector<vector<int>>(n, vector<int>(n + 1, -1));
 return solve(0, -1, nums).second;
}
```

You are given an array of words where each word consists of lowercase English letters. wordA is a predecessor of wordB if and only if we can insert exactly one letter anywhere in wordA without changing the order of the other characters to make it equal to

wordB. For example, "abc" is a predecessor of "abac", while "cba" is not a predecessor of "bcad". A word chain is a sequence of words [word1, word2, ..., wordk] with  $k \geq 1$ , where word1 is a predecessor of word2, word2 is a predecessor of word3, and so on. A single word is trivially a word chain with  $k = 1$ . Return the length of the longest possible word chain with words chosen from the given list of words. Input: words = ["a","b","ba","bca","bda","bdca"]. Output: 4. Explanation: One of the longest word chains is ["a","ba","bda","bdca"].

```
vector<vector<int>> dp;
bool isPred(string& a, string& b) {
 if (b.size() != a.size() + 1) return false;
 int i = 0, j = 0;
 while (j < b.size()) {
 if (i < a.size() && a[i] == b[j]) i++;
 j++;
 }
 return i == a.size();
}
int solve(int i, int prev, vector<string>& w) {
 if (i == w.size()) return 0;
 if (dp[i][prev + 1] != -1) return dp[i][prev + 1];
 int skip = solve(i + 1, prev, w);
 int take = 0;
 if (prev == -1 || isPred(w[prev], w[i])) take = 1 + solve(i + 1, i, w);
 return dp[i][prev + 1] = max(skip, take);
}
int longestStrChain(vector<string>& words) {
 sort(words.begin(), words.end(),
 [](string& a, string& b) { return a.size() < b.size(); });
 int n = words.size();
 dp = vector<vector<int>>(n, vector<int>(n + 1, -1));
 return solve(0, -1, words);
}
```

# DP-Grids

## DP - DP on Grids

Description: Dynamic Programming on matrices involves solving problems that can be broken down into smaller overlapping subproblems within a matrix.

- [\[Self Added\] Unique Paths i](#)
- [Unique Paths ii](#)
- [Minimum Path Sum](#)
- [Triangle](#)
- [Minimum Falling Path Sum](#)
- [Maximal Square](#)
- [Cherry Pickup](#)
- [Dungeon Game: Dungeon Game](#)

There is a robot on an  $m \times n$  grid. The robot is initially located at the top-left corner (i.e.,  $\text{grid}[0][0]$ ). The robot tries to move to the bottom-right corner (i.e.,  $\text{grid}[m - 1][n - 1]$ ). The robot can only move either down or right at any point in time.

Given the two integers  $m$  and  $n$ , return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to  $2 * 10^9$ .

Input:  $m = 3, n = 7$

Output: 28



```
vector<vector<int>> dp;
```

```

int solve(int i, int j, int m, int n) {
 if (i == m - 1 && j == n - 1) return 1;
 if (i >= m || j >= n) return 0;
 int &res = dp[i][j];
 if (res != -1) return res;
 return res = solve(i + 1, j, m, n) + solve(i, j + 1, m, n);
}

int uniquePaths(int m, int n) {
 dp.assign(m, vector<int>(n, -1));
 return solve(0, 0, m, n);
}

```

You are given an  $m \times n$  integer array `grid`. There is a robot initially located at the top-left corner (i.e., `grid[0][0]`). The robot tries to move to the bottom-right corner (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

An obstacle and space are marked as 1 or 0 respectively in `grid`. A path that the robot takes cannot include any square that is an obstacle.

Return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The testcases are generated so that the answer will be less than or equal to  $2 * 10^9$ .

Input: `obstacleGrid = [[0,0,0],[0,1,0],[0,0,0]]`

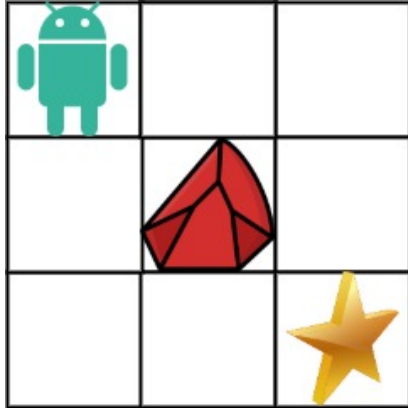
Output: 2

Explanation: There is one obstacle in the middle of the 3x3 grid above.

There are two ways to reach the bottom-right corner:

1. Right -> Right -> Down -> Down
2. Down -> Down -> Right -> Right





```
vector<vector<long long>> dp;
long long solve(int i, int j, vector<vector<int>>& g) {
 int m = g.size(), n = g[0].size();
 if (i >= m || j >= n) return 0;
 if (g[i][j] == 1) return 0;
 if (i == m - 1 && j == n - 1) return 1;
 long long &res = dp[i][j];
 if (res != -1) return res;
 return res = solve(i + 1, j, g) + solve(i, j + 1, g);
}

int uniquePathsWithObstacles(vector<vector<int>>& obstacleGrid) {
 int m = obstacleGrid.size(), n = obstacleGrid[0].size();
 dp.assign(m, vector<long long>(n, -1));
 return (int)solve(0, 0, obstacleGrid);
}
```

Given a  $m \times n$  grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Input: grid = [[1,3,1],[1,5,1],[4,2,1]]

Output: 7

Explanation: Because the path  $1 \rightarrow 3 \rightarrow 1 \rightarrow 1 \rightarrow 1$  minimizes the sum.

|   |   |   |
|---|---|---|
| 1 | 3 | 1 |
| 1 | 5 | 1 |
| 4 | 2 | 1 |

```

vector<vector<int>> dp;
int solve(int i, int j, vector<vector<int>>& grid) {
 int m = grid.size(), n = grid[0].size();
 if (i >= m || j >= n) return 1e9;
 if (i == m - 1 && j == n - 1) return grid[i][j];
 int &res = dp[i][j];
 if (res != -1) return res;
 int down = solve(i + 1, j, grid);
 int right = solve(i, j + 1, grid);
 return res = grid[i][j] + min(down, right);
}
int minPathSum(vector<vector<int>>& grid) {
 int m = grid.size(), n = grid[0].size();
 dp.assign(m, vector<int>(n, -1));
 return solve(0, 0, grid);
}

```

Given a triangle array, return the minimum path sum from top to bottom. For each step, you may move to an adjacent number of the row below. More formally, if you are on index  $i$  on the current row, you may move to either index  $i$  or index  $i + 1$  on the next row.

Input: triangle = [[2],[3,4],[6,5,7],[4,1,8,3]]

Output: 11

Explanation: The triangle looks like:

```

 2
 3 4
6 5 7
4 1 8 3

```

The minimum path sum from top to bottom is  $2 + 3 + 5 + 1 = 11$  (underlined above).

```

int minimumTotal(vector<vector<int>>& triangle) {
 // top down gives TLE, hence using bottom up dp
 int n = triangle.size();
 vector<int> dp = triangle.back(); // start with last row
 for (int row = n - 2; row >= 0; row--) {
 for (int col = 0; col <= row; col++) {
 dp[col] = triangle[row][col] + min(dp[col], dp[col+1]);
 }
 }
 return dp[0];
}

```

Given an  $n \times n$  array of integers matrix, return the minimum sum of any falling path through matrix.

A falling path starts at any element in the first row and chooses the element in the next row that is either directly below or diagonally left/right. Specifically, the next element from position (row, col) will be (row + 1, col - 1), (row + 1, col), or (row + 1, col + 1).

Input: matrix = [[2,1,3],[6,5,4],[7,8,9]]

Output: 13

Explanation: There are two falling paths with a minimum sum as shown.

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
| 6 | 5 | 4 |
| 7 | 8 | 9 |

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
| 6 | 5 | 4 |
| 7 | 8 | 9 |

|   |   |   |
|---|---|---|
| 2 | 1 | 3 |
| 6 | 5 | 4 |
| 7 | 8 | 9 |

```

int minFallingPathSum(vector<vector<int>>& matrix) {
 // top down TLE, hence bottom up
 int n = matrix.size();
 vector<vector<int>> dp = matrix;
 for (int i = 1; i < n; i++) {
 for (int j = 0; j < n; j++) {
 int best = dp[i-1][j];
 if (j > 0) best = min(best, dp[i-1][j-1]);
 if (j+1 < n) best = min(best, dp[i-1][j+1]);

 dp[i][j] = matrix[i][j] + best;
 }
 }

 return *min_element(dp[n-1].begin(), dp[n-1].end());
}

```

Given an  $m \times n$  binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Input: matrix =

```

[["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],[
"1","0","0","1","0"]]

```

Output: 4

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 0 | 1 | 0 | 0 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 1 | 0 |

```

int maximalSquare(vector<vector<char>>& matrix) {
 // top down TLE, hence bottom up
 int m = matrix.size(), n = matrix[0].size();
 vector<vector<int>> dp(m, vector<int>(n, 0));
 int best = 0;
 for (int i = 0; i < m; i++) {
 for (int j = 0; j < n; j++) {
 if (matrix[i][j] == '1') {
 if (i == 0 || j == 0) dp[i][j] = 1;
 else dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1],
dp[i-1][j-1]});
 best = max(best, dp[i][j]);
 }
 }
 }
 return best * best; // return area
}

```

[hard]

You are given an  $n \times n$  grid representing a field of cherries, each cell is one of three possible integers.

0 means the cell is empty, so you can pass through,

1 means the cell contains a cherry that you can pick up and pass through, or

-1 means the cell contains a thorn that blocks your way.

Return the maximum number of cherries you can collect by following the rules below:

Starting at the position  $(0, 0)$  and reaching  $(n - 1, n - 1)$  by moving right or down through valid path cells (cells with value 0 or 1).

After reaching  $(n - 1, n - 1)$ , returning to  $(0, 0)$  by moving left or up through valid path cells.

When passing through a path cell containing a cherry, you pick it up, and the cell becomes an empty cell 0.

If there is no valid path between  $(0, 0)$  and  $(n - 1, n - 1)$ , then no cherries can be collected.

Input: `grid = [[0,1,-1],[1,0,-1],[1,1,1]]`

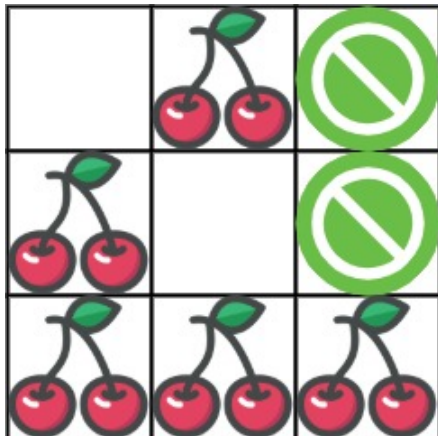
Output: 5

Explanation: The player started at  $(0, 0)$  and went down, down, right right to reach  $(2, 2)$ .

4 cherries were picked up during this single trip, and the matrix becomes `[[0,1,-1],[0,0,-1],[0,0,0]]`.

Then, the player went left, up, up, left to return home, picking up one more cherry.

The total number of cherries picked up is 5, and this is the maximum possible.



```

int n;
vector<vector<int>> g;
int dp[51][51][51];
int solve(int r1, int c1, int r2) {
 // top down TLE, hence bottom up
 int c2 = r1 + c1 - r2;
 if (r1 >= n || c1 >= n || r2 >= n || c2 >= n) return -1e9;
 if (g[r1][c1] == -1 || g[r2][c2] == -1) return -1e9;
 if (r1 == n-1 && c1 == n-1) return g[r1][c1];
 int &res = dp[r1][c1][r2];
 if (res != -1) return res;
 int best = max({
 solve(r1+1, c1, r2+1),
 solve(r1, c1+1, r2),
 solve(r1+1, c1, r2),
 solve(r1, c1+1, r2+1)
 });
 if (best < 0) return res = -1e9;
 int cherries = g[r1][c1];
 if (r1 != r2 || c1 != c2) cherries += g[r2][c2];
 return res = cherries + best;
}
int cherryPickup(vector<vector<int>>& grid) {
 n = grid.size();
 g = grid;
 memset(dp, -1, sizeof(dp));
 int ans = solve(0,0,0);
 return max(0, ans);
}

```

[hard]

The demons had captured the princess and imprisoned her in the bottom-right corner of a dungeon. The dungeon consists of  $m \times n$  rooms laid out in a 2D grid. Our valiant knight was initially positioned in the top-left room and must fight his way through dungeon to rescue the princess.

The knight has an initial health point represented by a positive integer. If at any point his health point drops to 0 or below, he dies immediately.

Some of the rooms are guarded by demons (represented by negative integers), so the knight loses health upon entering these rooms; other rooms are either empty (represented as 0) or contain magic orbs that increase the knight's health (represented by positive integers).

To reach the princess as quickly as possible, the knight decides to move only rightward or downward in each step.

Return the knight's minimum initial health so that he can rescue the princess.

Note that any room can contain threats or power-ups, even the first room the knight enters and the bottom-right room where the princess is imprisoned.

Input: `dungeon = [[-2,-3,3],[-5,-10,1],[10,30,-5]]`

Output: 7

Explanation: The initial health of the knight must be at least 7 if he follows the optimal path: RIGHT-> RIGHT -> DOWN -> DOWN.

|    |     |    |
|----|-----|----|
| -2 | -3  | 3  |
| -5 | -10 | 1  |
| 10 | 30  | -5 |



```

vector<vector<int>> dp;
int solve(int i, int j, vector<vector<int>>& d) {
 int m = d.size(), n = d[0].size();
 if (i >= m || j >= n) return 1e9;
 if (i == m - 1 && j == n - 1)
 return max(1, 1 - d[i][j]);
 int &res = dp[i][j];
 if (res != -1) return res;
 int down = solve(i + 1, j, d);
 int right = solve(i, j + 1, d);
 int need = min(down, right) - d[i][j];
 return res = max(1, need);
}
int calculateMinimumHP(vector<vector<int>>& dungeon) {
 int m = dungeon.size(), n = dungeon[0].size();
 dp.assign(m, vector<int>(n, -1));
 return solve(0, 0, dungeon);
}

```

# DP-Strings

## DP - DP on Strings

Description: It Involves 2 strings, whenever you are considering two substrings/subsequence from given two strings, concentrate on what happens when the last characters of the two substrings are same, i.e, matching.

- [Longest Common Subsequence](#)
- [Longest Palindromic Subsequence](#)
- [Palindromic Substrings](#)
- [Longest Palindromic Substrings](#)
- [Edit Distance](#)
- [Minimum ASCII Delete Sum for Two Strings](#)
- [Distinct Subsequences](#)
- [Shortest Common Supersequence](#)
- [Wildcard Matching](#)

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

For example, "ace" is a subsequence of "abcde".

A common subsequence of two strings is a subsequence that is common to both strings.

Input: text1 = "abcde", text2 = "ace"

Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

```

string a, b;
vector<vector<int>> dp;
int solve(int i, int j) {
 if (i == a.size() || j == b.size()) return 0;
 int &res = dp[i][j];
 if (res != -1) return res;
 if (a[i] == b[j])
 return res = 1 + solve(i+1, j+1);
 return res = max(solve(i+1, j), solve(i, j+1));
}
int longestCommonSubsequence(string s1, string s2) {
 a = s1; b = s2;
 dp.assign(a.size(), vector<int>(b.size(), -1));
 return solve(0, 0);
}

```

**Given a string s, find the longest palindromic subsequence's length in s. A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements. Input: s = "bbbab". Output: 4.**

**Explanation: One possible longest palindromic subsequence is "bbbb".**

```

string s; vector<vector<int>> dp;
int solve(int i, int j) {
 if (i > j) return 0;
 if (i == j) return 1;
 int &res = dp[i][j];
 if (res != -1) return res;
 if (s[i] == s[j])
 return res = 2 + solve(i+1, j-1);
 return res = max(solve(i+1, j), solve(i, j-1));
}
int longestPalindromeSubseq(string str) {
 s = str; int n = s.size();
 dp.assign(n, vector<int>(n, -1));
 return solve(0, n-1);
}

```

Given a string *s*, return the number of palindromic substrings in it. A string is a palindrome when it reads the same backward as forward. A substring is a contiguous sequence of characters within the string.

Input: *s* = "aaa"

Output: 6

Explanation: Six palindromic strings: "a", "a", "a", "aa", "aa", "aaa".

```
int countSubstrings(string s) {
 // bottom up better than top down
 int n = s.size();
 vector<vector<bool>> dp(n, vector<bool>(n, false));
 int ans = 0;
 for (int len = 1; len <= n; len++) {
 for (int i = 0; i + len - 1 < n; i++) {
 int j = i + len - 1;
 if (s[i] == s[j] && (len <= 2 || dp[i+1][j-1])) {
 dp[i][j] = true;
 ans++;
 }
 }
 }
 return ans;
}
```

Given a string *s*, return the longest palindromic substring in *s*.

Input: *s* = "babad"

Output: "bab"

Explanation: "aba" is also a valid answer.

```
bool isPalin(const string &s, int i, int j) {
 while (i < j) {
```

```

 if (s[i] != s[j]) return false;
 i++;
 j--;
 }
 return true;
}
string longestPalindrome(string s) {
 int n = s.size();
 int maxLen = 0, start = 0;
 for (int i = 0; i < n; i++) {
 for (int j = i; j < n; j++) {
 if (isPalin(s, i, j)) {
 int len = j - i + 1;
 if (len > maxLen) {
 maxLen = len;
 start = i;
 }
 }
 }
 }
 return s.substr(start, maxLen);
}

```

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

Insert a character

Delete a character

Replace a character

Input: word1 = "horse", word2 = "ros"

Output: 3

Explanation:

horse -> rorse (replace 'h' with 'r')

rorse -> rose (remove 'r')

rose -> ros (remove 'e')

string a, b;

vector<vector<int>> dp;

```

int solve(int i, int j) {
 if (i == a.size()) return b.size() - j; // insert all remaining
 if (j == b.size()) return a.size() - i; // delete all remaining
 int &res = dp[i][j];
 if (res != -1) return res;
 if (a[i] == b[j])
 return res = solve(i+1, j+1);
 int insertOp = 1 + solve(i, j+1);
 int deleteOp = 1 + solve(i+1, j);
 int replaceOp = 1 + solve(i+1, j+1);
 return res = min({insertOp, deleteOp, replaceOp});
}

int minDistance(string s1, string s2) {
 a = s1; b = s2;
 dp.assign(a.size(), vector<int>(b.size(), -1));
 return solve(0, 0);
}

```

Given two strings s1 and s2, return the lowest ASCII sum of deleted characters to make two strings equal.

Input: s1 = "sea", s2 = "eat"

Output: 231

Explanation: Deleting "s" from "sea" adds the ASCII value of "s" (115) to the sum.

Deleting "t" from "eat" adds 116 to the sum.

At the end, both strings are equal, and  $115 + 116 = 231$  is the minimum sum possible to achieve this.

string a, b;

vector<vector<int>> dp;

```

int solve(int i, int j) {
 if (i == a.size()) {
 int sum = 0;
 for (int k = j; k < b.size(); k++) sum += b[k];
 return sum;
 }
 if (j == b.size()) {
 int sum = 0;
 for (int k = i; k < a.size(); k++) sum += a[k];
 return sum;
 }
 int &res = dp[i][j];
 if (res != -1) return res;
 if (a[i] == b[j])
 return res = solve(i+1, j+1);
 int deleteA = a[i] + solve(i+1, j);
 int deleteB = b[j] + solve(i, j+1);
 return res = min(deleteA, deleteB);
}

int minimumDeleteSum(string s1, string s2) {
 a = s1; b = s2;
 dp.assign(a.size(), vector<int>(b.size(), -1));
 return solve(0, 0);
}

```

**[hard]**

Given two strings *s* and *t*, return the number of distinct subsequences of *s* which equals *t*.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Input: *s* = "rabbbit", *t* = "rabbit"

Output: 3

Explanation:

As shown below, there are 3 ways you can generate "rabbit" from *s*.  
rabbbit, rabbbit, rabbbit

string *s*, *t*;

vector<vector<long long>> dp;



```

long long solve(int i, int j) {
 if (j == t.size()) return 1; // matched full t
 if (i == s.size()) return 0; // s ended but t not finished
 long long &res = dp[i][j];
 if (res != -1) return res;
 long long ways = 0;
 if (s[i] == t[j])
 ways += solve(i+1, j+1);
 ways += solve(i+1, j); // skip char in s
 return res = ways;
}
int numDistinct(string S, string T) {
 s = S; t = T;
 dp.assign(s.size(), vector<long long>(t.size(), -1));
 return solve(0, 0);
}

```

**[hard]**

Given two strings `str1` and `str2`, return the shortest string that has both `str1` and `str2` as subsequences. If there are multiple valid strings, return any of them.

A string `s` is a subsequence of string `t` if deleting some number of characters from `t` (possibly 0) results in the string `s`.

Input: `str1 = "abac"`, `str2 = "cab"`

Output: `"cabac"`

Explanation:

`str1 = "abac"` is a subsequence of `"cabac"` because we can delete the first `"c"`.

`str2 = "cab"` is a subsequence of `"cabac"` because we can delete the last `"ac"`.

The answer provided is the shortest such string that satisfies these properties.

```

string shortestCommonSupersequence(string a, string b) {
 int n = a.size(), m = b.size();

```

```

vector<vector<int>> dp(n+1, vector<int>(m+1, 0));
// LCS table
for (int i = 1; i <= n; i++) {
 for (int j = 1; j <= m; j++) {
 if (a[i-1] == b[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
 else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
 }
}
// Construct SCS
string res = "";
int i = n, j = m;
while (i > 0 && j > 0) {
 if (a[i-1] == b[j-1]) {
 res.push_back(a[i-1]);
 i--, j--;
 } else if (dp[i-1][j] >= dp[i][j-1]) {
 res.push_back(a[i-1]);
 i--;
 } else {
 res.push_back(b[j-1]);
 j--;
 }
}
while (i > 0) res.push_back(a[i-1]), i--;
while (j > 0) res.push_back(b[j-1]), j--;
reverse(res.begin(), res.end());
return res;
}

```

**[hard]**

Given an input string (s) and a pattern (p), implement wildcard pattern matching with support for '?' and '\*' where:

'?' Matches any single character.

'\*' Matches any sequence of characters (including the empty sequence). The matching should cover the entire input string (not partial). Input: s = "aa", p = "\*". Output: true

Explanation: '\*' matches any sequence.

Input: s = "cb", p = "?a"

Output: false

Explanation: '?' matches 'c', but the second letter is 'a', which does not match 'b'.

```
string s, p;
vector<vector<int>> dp;
bool solve(int i, int j) {
 if (j == p.size()) return i == s.size();
 if (i == s.size()) {
 // s ended → only valid if remaining p is all '*'
 for (int k = j; k < p.size(); k++)
 if (p[k] != '*') return false;
 return true;
 }
 int &res = dp[i][j];
 if (res != -1) return res;
 bool ans = false;
 if (p[j] == s[i] || p[j] == '?')
 ans = solve(i+1, j+1);
 else if (p[j] == '*') {
 ans = solve(i, j+1) // '*' = empty
 || solve(i+1, j); // '*' consumes s[i]
 }
 return res = ans;
}
bool isMatch(string S, string P) {
 s = S; p = P;
 dp.assign(s.size(), vector<int>(p.size(), -1));
 return solve(0, 0);
}
```

DP-Stocks

## DP - DP on Stocks

**Description:** It focuses on maximizing profit from buying and selling stocks over time while considering constraints.

- [\[Self Added\] Buy and Sell Stocks i](#)
- [Buy and Sell Stocks ii](#)
- [Buy and Sell Stocks iii](#)
- [Buy and Sell Stocks iv](#)
- [Buy and Sell Stocks with Cooldown](#)
- [Buy and Sell Stocks with Transaction fee](#)

You are given an array prices where prices[i] is the price of a given stock on the ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Input: prices = [7,1,5,3,6,4]

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.

Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

```
int maxProfit(vector<int>& prices) {
 int mn = prices[0];
 int ans = 0;
 for (int i = 1; i < prices.size(); i++) {
 ans = max(ans, prices[i] - mn);
 mn = min(mn, prices[i]);
 }
 return ans;
}
```

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

Input: prices = [7,1,5,3,6,4]

Output: 7

Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit =  $5 - 1 = 4$ .

Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit =  $6 - 3 = 3$ .

Total profit is  $4 + 3 = 7$ .

```
int maxProfit(vector<int>& prices) {
 int ans = 0;
 for (int i = 1; i < prices.size(); i++) {
 if (prices[i] > prices[i-1])
 ans += prices[i] - prices[i-1];
 }
 return ans;
}
```

[hard]

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Input: prices = [3,3,5,0,0,3,1,4]

Output: 6

Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit =  $3 - 0 = 3$ .

Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit =  $4 - 1 = 3$ .

```
vector<vector<vector<int>>> dp;
vector<int> prices;
```

```

int solve(int i, int buy, int t) {
 if (i == prices.size() || t == 0) return 0;
 int &res = dp[i][buy][t];
 if (res != -1) return res;
 if (buy) {
 int take = -prices[i] + solve(i+1, 0, t);
 int skip = solve(i+1, 1, t);
 return res = max(take, skip);
 } else {
 int sell = prices[i] + solve(i+1, 1, t-1);
 int skip = solve(i+1, 0, t);
 return res = max(sell, skip);
 }
}

int maxProfit(vector<int>& p) {
 prices = p;
 int n = prices.size();
 dp.assign(n, vector<vector<int>>(2, vector<int>(3, -1)));
 return solve(0, 1, 2);
}

```

[hard]

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the *i*th day, and an integer *k*. Find the maximum profit you can achieve. You may complete at most *k* transactions: i.e. you may buy at most *k* times and sell at most *k* times.

**Note:** You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Input: *k* = 2, `prices` = [3,2,6,5,0,3]

Output: 7

Explanation: Buy on day 2 (price = 2) and sell on day 3 (price = 6), profit = 6-2 = 4. Then buy on day 5 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.

```

vector<vector<vector<int>>> dp;
vector<int> prices;

```

```

int K;
int solve(int i, int buy, int t) {
 if (i == prices.size() || t == 0) return 0;
 int &res = dp[i][buy][t];
 if (res != -1) return res;
 if (buy) {
 int take = -prices[i] + solve(i+1, 0, t);
 int skip = solve(i+1, 1, t);
 return res = max(take, skip);
 } else {
 int sell = prices[i] + solve(i+1, 1, t-1);
 int skip = solve(i+1, 0, t);
 return res = max(sell, skip);
 }
}

int maxProfit(int k, vector<int>& p) {
 prices = p;
 K = k;
 int n = prices.size();
 dp.assign(n, vector<vector<int>>(2, vector<int>(k+1, -1)));
 return solve(0, 1, k);
}

```

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

**Note:** You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

**Input:** prices = [1,2,3,0,2]

**Output:** 3

**Explanation:** transactions = [buy, sell, cooldown, buy, sell]

vector<vector<int>> dp;

vector<int> prices;



```

int solve(int i, int state) {
 if (i >= prices.size()) return 0;
 int &res = dp[i][state];
 if (res != -1) return res;
 if (state == 0) { // can buy
 int buy = -prices[i] + solve(i+1, 1);
 int skip = solve(i+1, 0);
 return res = max(buy, skip);
 }
 else if (state == 1) { // have stock
 int sell = prices[i] + solve(i+1, 2);
 int skip = solve(i+1, 1);
 return res = max(sell, skip);
 }
 else { // cooldown
 return res = solve(i+1, 0);
 }
}

int maxProfit(vector<int>& p) {
 prices = p;
 int n = prices.size();
 dp.assign(n, vector<int>(3, -1));
 return solve(0, 0);
}

```

You are given an array prices where prices[i] is the price of a given stock on the ith day, and an integer fee representing a transaction fee.

Find the maximum profit you can achieve. You may complete as many transactions as you like, but you need to pay the transaction fee for each transaction.

**Note:**

You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

The transaction fee is only charged once for each stock purchase and sale.

Input: prices = [1,3,2,8,4,9], fee = 2

Output: 8

**Explanation:** The maximum profit can be achieved by:

- Buying at prices[0] = 1
- Selling at prices[3] = 8
- Buying at prices[4] = 4
- Selling at prices[5] = 9

The total profit is  $((8 - 1) - 2) + ((9 - 4) - 2) = 8$ .

```
vector<vector<int>> dp;
```

```
vector<int> prices;
```

```
int fee;
```

```
int solve(int i, int buy) {
 if (i == prices.size()) return 0;
 int &res = dp[i][buy];
 if (res != -1) return res;
 if (buy) {
 int take = -prices[i] + solve(i+1, 0);
 int skip = solve(i+1, 1);
 return res = max(take, skip);
 } else {
 int sell = prices[i] - fee + solve(i+1, 1);
 int skip = solve(i+1, 0);
 return res = max(sell, skip);
 }
}

int maxProfit(vector<int>& p, int f) {
 prices = p;
 fee = f;
 int n = prices.size();
 dp.assign(n, vector<int>(2, -1));
 return solve(0, 1);
}
```

DP-MCM

## DP - Partition DP (MCM)

**Description:** It involves a sequence that needs to be divided into partitions in an optimal way. The goal is often to minimize or maximize a cost function, such as computation time, multiplications, or some other metric, by exploring all possible partitions and combining results from subproblems.

- [Partition array for Maximum Sum](#)
- [Minimum Cost to Cut a Stick](#)
- [Burst Balloons](#)
- [Palindrome Partitioning ii](#)
- [\[Self added v2\] Egg Dropping](#)

Given an integer array `arr`, partition the array into (contiguous) subarrays of length at most `k`. After partitioning, each subarray has their values changed to become the maximum value of that subarray. Return the largest sum of the given array after partitioning. Test cases are generated so that the answer fits in a 32-bit integer.

Input: `arr = [1,15,7,9,2,5,10]`, `k = 3`

Output: 84

Explanation: `arr` becomes `[15,15,15,9,10,10,10]`

```
int n;
vector<int> arr;
vector<int> memo;
int solve(int i, int k) {
 if (i >= n) return 0;
 int &res = memo[i];
 if (res != -1) return res;
 int mx = 0;
 res = 0;
 for (int len = 1; len <= k && i + len <= n; ++len) {
 mx = max(mx, arr[i + len - 1]);
 res = max(res, mx * len + solve(i + len, k));
 }
 return res;
}
int maxSumAfterPartitioning(vector<int>& A, int K) {
```

```

arr = A;
n = arr.size();
memo.assign(n, -1);
return solve(0, K);
}

```

[hard]

Given a wooden stick of length  $n$  units. The stick is labelled from  $0$  to  $n$ . For example, a stick of length  $6$  is labelled as follows:

Given an integer array `cuts` where `cuts[i]` denotes a position you should perform a cut at.

You should perform the cuts in order, you can change the order of the cuts as you wish.

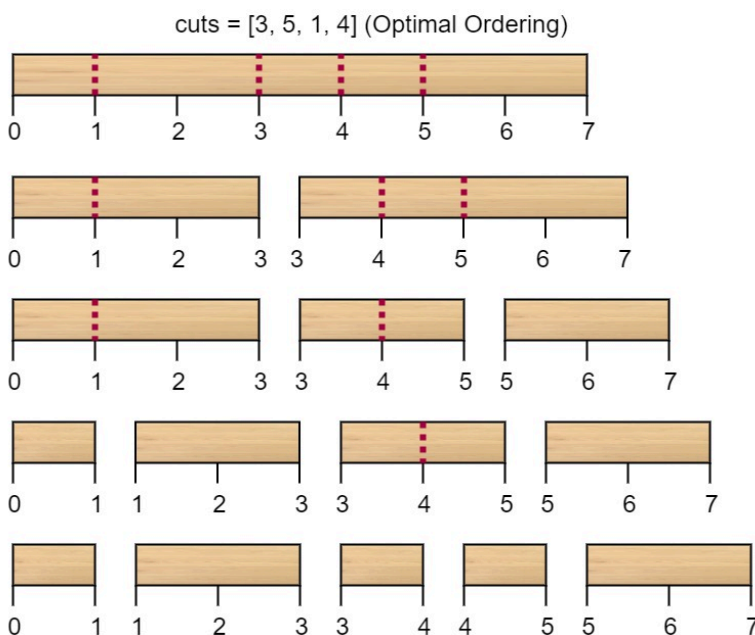
The cost of one cut is the length of the stick to be cut, the total cost is the sum of costs of all cuts. When you cut a stick, it will be split into two smaller sticks (i.e. the sum of their lengths is the length of the stick before the cut). Please refer to the first example for a better explanation.

Return the minimum total cost of the cuts.

Input:  $n = 7$ , `cuts = [1,3,4,5]`

Output: 16

Explanation: Using cuts order = `[1, 3, 4, 5]` as in the input leads to the following scenario:



```
vector<int> cuts2;
```

```

vector<vector<int>> memo;
int solve(int l, int r) {
 if (l + 1 == r) return 0; // no cuts between
 int &res = memo[l][r];
 if (res != -1) return res;
 res = INT_MAX/4;
 for (int k = l + 1; k <= r - 1; ++k) {
 int cost = cuts2[r] - cuts2[l] + solve(l, k) + solve(k, r);
 res = min(res, cost);
 }
 return res;
}
int minCost(int n, vector<int>& cuts) {
 sort(cuts.begin(), cuts.end());
 cuts2.clear();
 cuts2.push_back(0);
 for (int c : cuts) cuts2.push_back(c);
 cuts2.push_back(n);
 int m = cuts2.size();
 memo.assign(m, vector<int>(m, -1));
 return solve(0, m-1);
}

```

[hard]

You are given  $n$  balloons, indexed from 0 to  $n - 1$ . Each balloon is painted with a number on it represented by an array `nums`. You are asked to burst all the balloons.

If you burst the  $i$ th balloon, you will get  $\text{nums}[i - 1] * \text{nums}[i] * \text{nums}[i + 1]$  coins. If  $i - 1$  or  $i + 1$  goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it. Return the maximum coins you can collect by bursting the balloons wisely.

Input: `nums = [3,1,5,8]`. Output: 167

Explanation:

`nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []`

`coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167`

`vector<int> a; // padded`

`vector<vector<int>> memo;`

```

int m;
int solve(int l, int r) {
 if (l + 1 == r) return 0; // no balloons between
 int &res = memo[l][r];
 if (res != -1) return res;
 res = 0;
 for (int k = l + 1; k <= r - 1; ++k) {
 int gain = a[l] * a[k] * a[r];
 res = max(res, gain + solve(l, k) + solve(k, r));
 }
 return res;
}
int maxCoins(vector<int>& nums) {
 int n = nums.size();
 a.clear();
 a.push_back(1);
 for (int x : nums) a.push_back(x);
 a.push_back(1);
 m = a.size();
 memo.assign(m, vector<int>(m, -1));
 return solve(0, m-1);
}

```

[hard]

Given a string *s*, partition *s* such that every substring of the partition is a palindrome.

Return the minimum cuts needed for a palindrome partitioning of *s*.

Input: *s* = "aab"

Output: 1

Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.

```
string s;
```

```
int n;
```

```

vector<vector<char>> isPal;
vector<int> memo;
int solve(int i) {
 if (i >= n) return 0; // no cuts needed for empty suffix
 int &res = memo[i];
 if (res != -1) return res;
 int best = INT_MAX/4;
 for (int j = i; j < n; ++j) {
 if (isPal[i][j]) {
 // if we cut after j, we add 1 cut, except when j is at
end => 0 cuts needed
 int cost = (j == n-1) ? 0 : 1 + solve(j+1);
 best = min(best, cost);
 }
 }
 return res = best;
}

int minCut(string str) {
 s = str;
 n = s.size();
 isPal.assign(n, vector<char>(n, 0));
 for (int i = n-1; i >= 0; --i) {
 for (int j = i; j < n; ++j) {
 if (s[i] == s[j] && (j - i < 2 || isPal[i+1][j-1]))
 isPal[i][j] = 1;
 }
 }
 memo.assign(n, -1);
 return solve(0);
}

```

**[hard]**



You are given  $k$  identical eggs and you have access to a building with  $n$  floors labeled from 1 to  $n$ .

You know that there exists a floor  $f$  where  $0 \leq f \leq n$  such that any egg dropped at a floor higher than  $f$  will break, and any egg dropped at or below floor  $f$  will not break.

Each move, you may take an unbroken egg and drop it from any floor  $x$  (where  $1 \leq x \leq n$ ). If the egg breaks, you can no longer use it.

However, if the egg does not break, you may reuse it in future moves. Return the minimum number of moves that you need to determine with certainty what the value of  $f$  is.

Input:  $k = 1$ ,  $n = 2$ . Output: 2. Explanation: Drop the egg from floor 1. If it breaks, we know that  $f = 0$ . Otherwise, drop the egg from floor 2. If it breaks, we know that  $f = 1$ . If it does not break, then we know  $f = 2$ . Hence, we need at minimum 2 moves to determine with certainty what the value of  $f$  is.

Input:  $k = 3$ ,  $n = 14$ . Output: 4

```
int superEggDrop(int k, int n) {
 // Top down may give TLE
 if (n == 0 || n == 1) return n;
 if (k == 1) return n;
 // dp[e] will represent dp[m][e] for current m in the loop
 vector<long long> dp(k + 1, 0);
 int moves = 0;
 while (dp[k] < n) {
 moves++;
 // update dp[e] for new moves value from high e to low e
 for (int e = k; e >= 1; --e) {
 dp[e] = dp[e] + dp[e - 1] + 1;
 }
 // dp[0] stays 0
 }
 return moves;
}
```

# Graphs-BasicQs

## [Self added] Basic Questions

- [\[Self added v2\] Clone graph](#)
- [\[Self added v2\] Number of islands](#)

Given a reference of a node in a connected undirected graph.  
Return a deep copy (clone) of the graph.

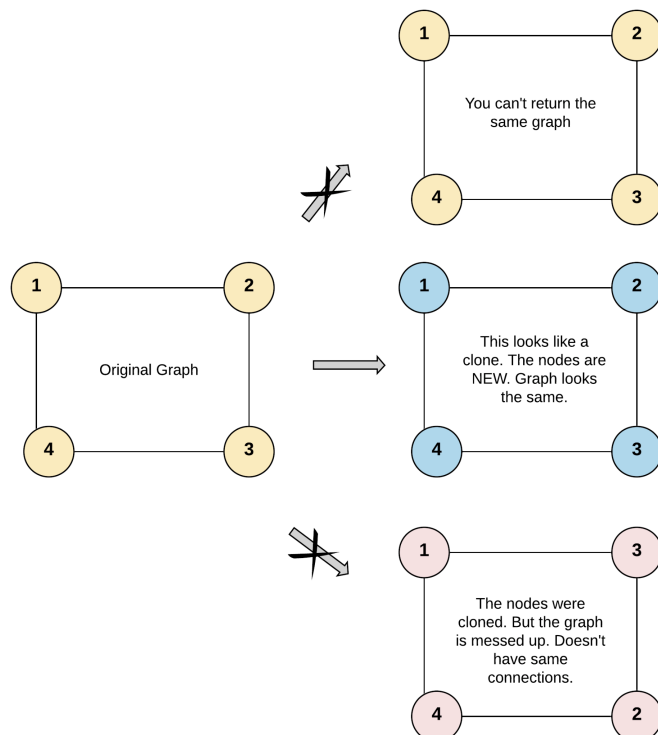
Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {
 public int val;
 public List<Node> neighbors;
}
```

Input: adjList = [[2,4],[1,3],[2,4],[1,3]]. Output:

[[2,4],[1,3],[2,4],[1,3]]

Explanation: There are 4 nodes in the graph. 1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4). 2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3). 3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4). 4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).



```

// DFS
unordered_map<Node*, Node*> mp; // maps original → cloned
Node* cloneGraph(Node* node) {
 if (!node) return nullptr;
 // If node is already created, return it
 if (mp.count(node)) return mp[node];
 // Create clone node
 Node* clone = new Node(node->val);
 mp[node] = clone;
 // Clone all neighbors
 for (auto neigh : node->neighbors) {
 clone->neighbors.push_back(cloneGraph(neigh));
 }
 return clone;
}

// BFS
Node* cloneGraph(Node* node) {
 if (!node) return nullptr;
 unordered_map<Node*, Node*> mp;
 queue<Node*> q;
 mp[node] = new Node(node->val);
 q.push(node);
 while (!q.empty()) {
 Node* curr = q.front();
 q.pop();
 for (auto next : curr->neighbors) {
 if (!mp.count(next)) {
 mp[next] = new Node(next->val);
 q.push(next);
 }
 mp[curr]->neighbors.push_back(mp[next]);
 }
 }
 return mp[node];
}

```

Given an  $m \times n$  2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

```
Input: grid = [
 ["1","1","1","1","0"],
 ["1","1","0","1","0"],
 ["1","1","0","0","0"],
 ["0","0","0","0","0"]
]
```

Output: 1

```
void dfs(vector<vector<char>>& grid, int i, int j) {
 int n = grid.size();
 int m = grid[0].size();
 if (i < 0 || j < 0 || i >= n || j >= m || grid[i][j] == '0')
 return;
 grid[i][j] = '0'; // mark visited
 dfs(grid, i+1, j);
 dfs(grid, i-1, j);
 dfs(grid, i, j+1);
 dfs(grid, i, j-1);
}

int numIslands(vector<vector<char>>& grid) {
 int n = grid.size();
 int m = grid[0].size();
 int count = 0;
 for (int i = 0; i < n; i++) {
 for (int j = 0; j < m; j++) {
 if (grid[i][j] == '1') {
 count++;
 dfs(grid, i, j); // flood-fill this island
 }
 }
 }
 return count;
}
```

# Graphs-TopologicalSort

## Graphs - Topological Sort

Description: Topological sorting is useful for tasks that require dependency resolution (InDegree) in directed acyclic graphs (DAGs).

- [Course Schedule](#)
- [Course Schedule II](#)
- [Strange Printer ii](#)
- [Sequence Reconstruction](#)
- [Alien Dictionary](#)
- [\[Self added v2\] Cycle detection in Directed & Undirected graph](#)

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Input: numCourses = 2, prerequisites = [[1,0]]

Output: true

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
```

```

// Kahn's Topological Sort
vector<vector<int>> adj(numCourses);
vector<int> indeg(numCourses, 0);
for (auto &p : prerequisites) {
 int a = p[0], b = p[1];
 adj[b].push_back(a);
 indeg[a]++;
}
queue<int> q;
for (int i = 0; i < numCourses; i++)
 if (indeg[i] == 0) q.push(i);
int count = 0;
while (!q.empty()) {
 int u = q.front(); q.pop();
 count++;
 for (int v : adj[u]) {
 indeg[v]--;
 if (indeg[v] == 0) q.push(v);
 }
}
return count == numCourses;
}

```

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

Output: [0,2,1,3]

Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.



**So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].**

```
vector<int> findOrder(int numCourses, vector<vector<int>>&
prerequisites) {
 vector<vector<int>> adj(numCourses);
 vector<int> indeg(numCourses, 0);
 for (auto &p : prerequisites) {
 adj[p[1]].push_back(p[0]);
 indeg[p[0]]++;
 }
 queue<int> q;
 for (int i = 0; i < numCourses; i++)
 if (indeg[i] == 0) q.push(i);
 vector<int> topo;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 topo.push_back(u);
 for (int v : adj[u]) {
 indeg[v]--;
 if (indeg[v] == 0) q.push(v);
 }
 }
 if (topo.size() != numCourses) return {};
 return topo;
}
```

[hard]

There is a strange printer with the following two special requirements:

On each turn, the printer will print a solid rectangular pattern of a single color on the grid. This will cover up the existing colors in the rectangle.

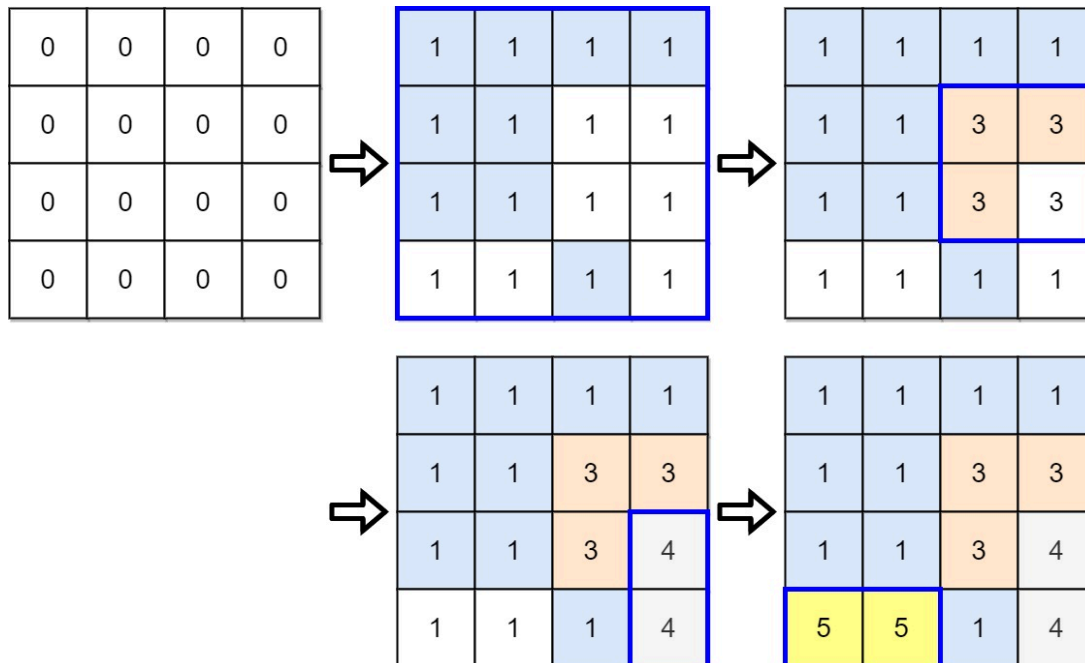
Once the printer has used a color for the above operation, the same color cannot be used again.

You are given a  $m \times n$  matrix `targetGrid`, where `targetGrid[row][col]` is the color in the position (row, col) of the grid.

Return true if it is possible to print the matrix `targetGrid`, otherwise, return false.

Input: `targetGrid = [[1,1,1,1],[1,1,3,3],[1,1,3,4],[5,5,1,4]]`

Output: true



```
bool isPrintable(vector<vector<int>>& g) {
 int m = g.size(), n = g[0].size();
 // Find actual max color
```

```

int maxColor = 0;
for (auto &row : g)
 for (int c : row)
 maxColor = max(maxColor, c);
maxColor++; // colors are 1..maxColor-1
vector<int> minr(maxColor, INT_MAX), minc(maxColor, INT_MAX);
vector<int> maxr(maxColor, INT_MIN), maxc(maxColor, INT_MIN);
vector<bool> present(maxColor, false);
// bounding boxes
for (int r = 0; r < m; r++) {
 for (int c = 0; c < n; c++) {
 int col = g[r][c];
 present[col] = true;
 minr[col] = min(minr[col], r);
 minc[col] = min(minc[col], c);
 maxr[col] = max(maxr[col], r);
 maxc[col] = max(maxc[col], c);
 }
}
// Build dependency graph
vector<vector<int>> adj(maxColor);
vector<int> indeg(maxColor, 0);
for (int col = 1; col < maxColor; col++) {
 if (!present[col]) continue;

 for (int r = minr[col]; r <= maxr[col]; r++) {
 for (int c = minc[col]; c <= maxc[col]; c++) {
 int inside = g[r][c];
 if (inside != col) {
 adj[inside].push_back(col);
 indeg[col]++;
 }
 }
 }
}
// topo-sort
queue<int> q;
for (int col = 1; col < maxColor; col++)

```

```

 if (present[col] && indeg[col] == 0)
 q.push(col);
 int printed = 0;
 while (!q.empty()) {
 int u = q.front(); q.pop();
 printed++;
 for (int v : adj[u]) {
 if (--indeg[v] == 0)
 q.push(v);
 }
 }
 int total = 0;
 for (int i = 1; i < maxColor; i++)
 if (present[i]) total++;
 return printed == total;
}

```

(premium)

### Sequence reconstruction

You are given an integer array `nums` which is the original sequence, and an array of sequences `seqs` where each `seqs[i]` is a subsequence of `org`. Check whether the `org` sequence can be uniquely reconstructed from the sequences in `seqs`.

Reconstruction means:

The sequence is the shortest common supersequence of all `seqs[i]`.

Return:

true if `org` is the only sequence that can be reconstructed.

false otherwise.

Input:

`org = [1,2,3]`

`seqs = [[1,2],[1,3]]`

Output: false

Explanation: Possible sequences: [1,2,3] or [1,3,2]

Both satisfy `seqs`  $\Rightarrow$  reconstruction not unique.

```

bool sequenceReconstruction(vector<int>& org, vector<vector<int>>&
seqs) {

```

```

 int n = org.size();

```

```

 if (seqs.empty()) return false;

```

```

unordered_map<int, vector<int>> adj;
unordered_map<int, int> indeg; unordered_set<int> seen;
for (auto& s : seqs) {
 for (int x : s) {
 indeg[x] = 0;
 adj[x] = {};
 seen.insert(x);
 }
}
if (seen.size() != n) return false; // missing elements
// build graph from seqs
for (auto& s : seqs) {
 for (int i = 1; i < (int)s.size(); i++) {
 int u = s[i-1], v = s[i];
 adj[u].push_back(v);
 indeg[v]++;
 }
}
queue<int> q;
for (auto& p : indeg)
 if (p.second == 0)
 q.push(p.first);
vector<int> result;
while (!q.empty()) {
 if (q.size() > 1) return false; // not unique
 int u = q.front(); q.pop();
 result.push_back(u);
 for (int v : adj[u]) {
 if (--indeg[v] == 0)
 q.push(v);
 }
}
return result == org;
}

```

### Alien Dictionary - Premium

There is a new alien language and you are given a list of words sorted lexicographically according to this new language.

You need to derive the order of letters in this alien language.

Return:

A string representing the unique character order

"" if no valid order exists or if multiple orders are possible

Rules:

Words are sorted according to the alien alphabet.

If w1 is a prefix of w2, then w2 must not come before w1.

Input:

["wrt","wrf","er","ett","rftt"]

Output:

"wertf"

```
string alienOrder(vector<string>& words) {
 unordered_map<char, vector<char>>> adj;
 unordered_map<char, int> indeg;
 // Register all characters
 for (auto& w : words)
 for (char c : w)
 indeg[c] = 0;
 // Build graph by comparing adjacent words
 for (int i = 0; i + 1 < words.size(); i++) {
 string& w1 = words[i];
 string& w2 = words[i+1];
 int len = min(w1.size(), w2.size());
```

```

 bool found = false;
 for (int j = 0; j < len; j++) {
 if (w1[j] != w2[j]) {
 adj[w1[j]].push_back(w2[j]);
 indeg[w2[j]]++;
 found = true;
 break;
 }
 }
 // prefix rule: invalid
 if (!found && w1.size() > w2.size())
 return "";
}
queue<char> q;
for (auto& p : indeg)
 if (p.second == 0)
 q.push(p.first);
string order;
while (!q.empty()) {
 if (q.size() > 1) {
 // Multiple valid orders → return any? No → return all
 // deterministic?
 // LeetCode expects ANY valid deterministic topo.
 }
 char c = q.front(); q.pop();
 order.push_back(c);
 for (char nxt : adj[c]) {
 if (--indeg[nxt] == 0)
 q.push(nxt);
 }
}
if (order.size() != indeg.size()) return ""; // cycle found
return order;
}

```

### Cycle detection in directed/undirected graph

// Undirected - DFS

```
bool dfsCycle(int node, int parent, vector<vector<int>>& adj,
```

```

 vector<int>& vis) {
vis[node] = 1;
for (int nxt : adj[node]) {
 if (!vis[nxt]) {
 bool hasCycle = dfsCycle(nxt, node, adj, vis);
 if (hasCycle) {
 return true;
 }
 }
 else if (nxt != parent) {
 return true; // visited neighbor that's not parent
 }
}
return false;
}

bool isCycle(int V, vector<vector<int>>& adj) {
 vector<int> vis(V, 0);
 for (int i = 0; i < V; i++) {
 if (!vis[i]) {
 if (dfsCycle(i, -1, adj, vis))
 return true;
 }
 }
 return false;
}

// Undirected - BFS
bool bfsCycle(int start, vector<int> adj[], vector<int> &vis) {
 // Use - vector<vector<int>>& adj preferably
 queue<pair<int,int>> q;
 q.push({start, -1});
 vis[start] = 1;
 while (!q.empty()) {
 int node = q.front().first;
 int parent = q.front().second;
 q.pop();
 for (int nxt : adj[node]) {
 if (!vis[nxt]) {
 vis[nxt] = 1;

```



```

 q.push({nxt, node});
 }
 else if (nxt != parent) {
 return true;
 }
}
}
return false;
}
bool isCycle(int V, vector<int> adj[]) {
 vector<int> vis(V, 0);
 for (int i = 0; i < V; i++) {
 if (!vis[i]) {
 if (bfsCycle(i, adj, vis))
 return true;
 }
 }
 return false;
}
// Directed - DFS
bool dfsDirected(int node, vector<vector<int>>& adj, vector<int>& vis, vector<int>& dfsVis) {
 vis[node] = 1;
 dfsVis[node] = 1;
 for (int nxt : adj[node]) {
 if (!vis[nxt]) {
 bool hasCycle = dfsDirected(nxt, adj, vis, dfsVis);
 if (hasCycle) {
 return true;
 }
 }
 else if (dfsVis[nxt]) {
 return true; // back edge
 }
 }
 dfsVis[node] = 0; // backtrack
 return false;
}

```

```

bool isCyclic(int V, vector<vector<int>>& adj) {
 vector<int> vis(V, 0), dfsVis(V, 0);
 for (int i = 0; i < V; i++) {
 if (!vis[i]) {
 if (dfsDirected(i, adj, vis, dfsVis))
 return true;
 }
 }
 return false;
}

// Directed - BFS + Indegree (Kahn's Algorithm)
// A Directed Acyclic Graph (DAG) always has a valid topological
// ordering. If graph has a cycle, Kahn's algo won't be able to remove
// all nodes. If no. of removed nodes (count) < V → cycle exists.
bool isCyclicKahn(int V, vector<int> adj[]) {
 // Use - vector<vector<int>>& adj preferably
 vector<int> indegree(V, 0);
 for (int i = 0; i < V; i++) {
 for (int nxt : adj[i]) {
 indegree[nxt]++;
 }
 }
 queue<int> q;
 for (int i = 0; i < V; i++) {
 if (indegree[i] == 0) q.push(i);
 }
 int count = 0; // number of nodes processed
 while (!q.empty()) {
 int node = q.front(); q.pop();
 count++;
 for (int nxt : adj[node]) {
 indegree[nxt]--;
 if (indegree[nxt] == 0)
 q.push(nxt);
 }
 }
 // If topological sort cannot include all nodes → cycle exists
 return (count != V);
}

```

}

# Graphs-UnionFind

## Graphs - Union Find (Disjoint Set)

Description: Union-Find (or Disjoint Set) is used to solve problems involving connectivity or grouping, often in graphs.

- [Number of Operations to Make Network Connected](#)
- [Redundant Connection](#)
- [Accounts Merge](#)
- [Satisfiability of Equality Equations](#)

There are  $n$  computers numbered from  $0$  to  $n - 1$  connected by ethernet cables connections forming a network where  $\text{connections}[i] = [a_i, b_i]$  represents a connection between computers  $a_i$  and  $b_i$ . Any computer can reach any other computer directly or indirectly through the network.

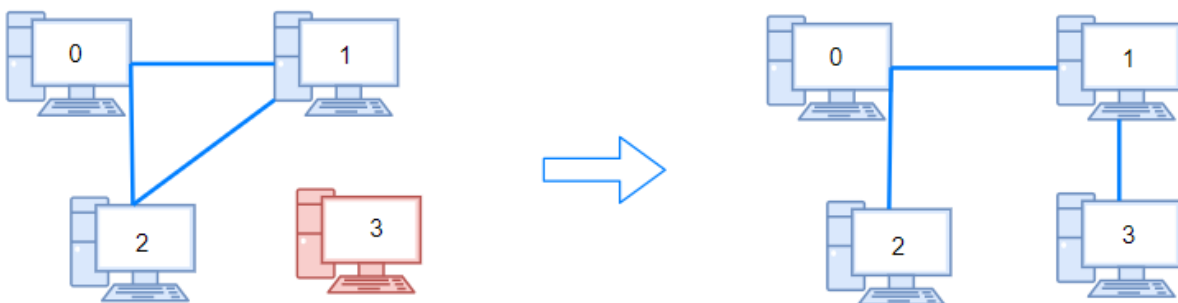
You are given an initial computer network connections. You can extract certain cables between two directly connected computers, and place them between any pair of disconnected computers to make them directly connected.

Return the minimum number of times you need to do this in order to make all the computers connected. If it is not possible, return  $-1$ .

Input:  $n = 4$ ,  $\text{connections} = [[0,1],[0,2],[1,2]]$

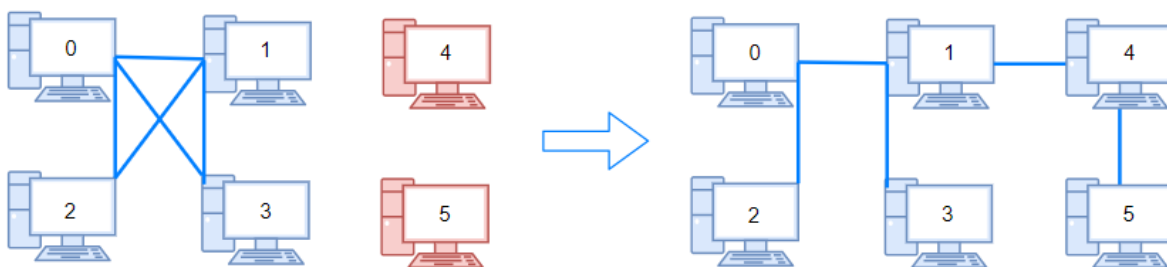
Output: 1

Explanation: Remove cable between computer 1 and 2 and place between computers 1 and 3.



Input:  $n = 6$ ,  $\text{connections} = [[0,1],[0,2],[0,3],[1,2],[1,3]]$

Output: 2



```

struct DSU {
 vector<int> parent, rankv;
 DSU(int n) {
 parent.resize(n);
 rankv.assign(n, 0);
 for (int i = 0; i < n; i++) parent[i] = i;
 }
 int find(int x) {
 if (parent[x] != x)
 parent[x] = find(parent[x]);
 return parent[x];
 }
 bool unite(int a, int b) {
 int pa = find(a), pb = find(b);
 if (pa == pb) return false;
 if (rankv[pa] < rankv[pb]) swap(pa, pb);
 parent[pb] = pa;
 if (rankv[pa] == rankv[pb]) rankv[pa]++;
 return true;
 }
};

int makeConnected(int n, vector<vector<int>>& connections) {
 if (connections.size() < n - 1)
 return -1;
 DSU dsu(n);
 int components = n;
 for (auto &e : connections)
 if (dsu.unite(e[0], e[1]))
 components--;
 return components - 1;
}

```

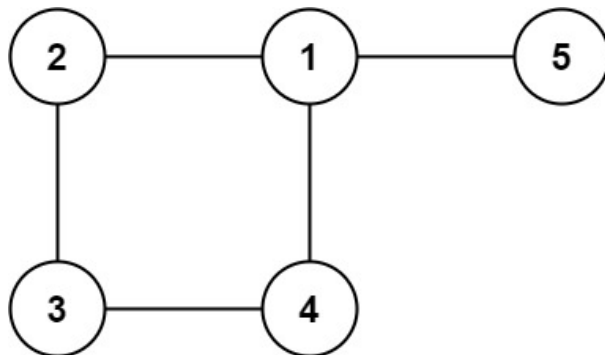
In this problem, a tree is an undirected graph that is connected and has no cycles.

You are given a graph that started as a tree with  $n$  nodes labeled from 1 to  $n$ , with one additional edge added. The added edge has two different vertices chosen from 1 to  $n$ , and was not an edge that already existed. The graph is represented as an array `edges` of length  $n$  where `edges[i] = [ai, bi]` indicates that there is an edge between nodes  $ai$  and  $bi$  in the graph.

Return an edge that can be removed so that the resulting graph is a tree of  $n$  nodes. If there are multiple answers, return the answer that occurs last in the input.

Input: `edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]`

Output: `[1,4]`



```
struct DSU {
vector<int> parent, rankv;
DSU(int n) {
 parent.resize(n);
 rankv.resize(n, 0);
 for (int i = 0; i < n; i++)
 parent[i] = i;
}
int find(int x) {
 if (parent[x] != x)
 parent[x] = find(parent[x]); // path compression
 return parent[x];
}
```

```

bool unite(int a, int b) {
 int pa = find(a), pb = find(b);
 if (pa == pb) return false; // already connected → cycle
 if (rankv[pa] < rankv[pb]) swap(pa, pb);
 parent[pb] = pa;
 if (rankv[pa] == rankv[pb]) rankv[pa]++;
 return true;
}
};

vector<int> findRedundantConnection(vector<vector<int>>& edges) {
 int n = edges.size();
 DSU dsu(n + 1);
 for (auto &e : edges) {
 int u = e[0], v = e[1];
 if (!dsu.unite(u, v))
 return e; // this edge forms the cycle
 }
 return {};
}

```

Given a list of accounts where each element `accounts[i]` is a list of strings, where the first element `accounts[i][0]` is a name, and the rest of the elements are emails representing emails of the account. Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Input: `accounts =`

`[["John", "johnsmith@mail.com", "john_newyork@mail.com"], ["John", "john`



```
smith@mail.com", "john00@mail.com"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]
```

Output:

```
[["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"], ["Mary", "mary@mail.com"], ["John", "johnnybravo@mail.com"]]
```

Explanation:

The first and second John's are the same person as they have the common email "johnsmith@mail.com".

The third John and Mary are different people as none of their email addresses are used by other accounts.

We could return these lists in any order, for example the answer

```
[['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.
```

```
struct DSU {
 vector<int> parent, rankv;
 DSU(int n) {
 parent.resize(n);
 rankv.resize(n, 0);
 for (int i = 0; i < n; i++)
 parent[i] = i;
 }
 int find(int x) {
 if (parent[x] != x)
 parent[x] = find(parent[x]); // path compression
 return parent[x];
 }
 bool unite(int a, int b) {
 int pa = find(a), pb = find(b);
 if (pa == pb) return false; // already connected → cycle
 if (rankv[pa] < rankv[pb]) swap(pa, pb);
 parent[pb] = pa;
 if (rankv[pa] == rankv[pb]) rankv[pa]++;
 return true;
 }
};

vector<vector<string>> accountsMerge(vector<vector<string>>&
accounts) {
```

```

unordered_map<string, int> emailID;
int id = 0;
for (auto &acc : accounts)
 for (int i = 1; i < acc.size(); i++)
 if (!emailID.count(acc[i]))
 emailID[acc[i]] = id++; // assign id to each email
DSU dsu(id);
// union all emails in each account
for (auto &acc : accounts) {
 string rootEmail = acc[1];
 int rootID = emailID[rootEmail];
 for (int i = 2; i < acc.size(); i++)
 dsu.unite(rootID, emailID[acc[i]]);
}
// group emails by DSU parent
unordered_map<int, vector<string>> groups;
for (auto &p : emailID) {
 int parent = dsu.find(p.second);
 groups[parent].push_back(p.first);
}
vector<vector<string>> ans;
for (auto &acc : accounts)
 acc[0]; // name used below
unordered_map<string, string> emailToName;
for (auto &acc : accounts)
 for (int i = 1; i < acc.size(); i++)
 emailToName[acc[i]] = acc[0];
for (auto &g : groups) {
 vector<string> emails = g.second;
 sort(emails.begin(), emails.end());
 string name = emailToName[emails[0]];
 emails.insert(emails.begin(), name);
 ans.push_back(emails);
}
return ans;
}

```

**You are given an array of strings equations that represent relationships between variables where each string equations[i] is of**

length 4 and takes one of two different forms: "xi==yi" or "xi!=yi". Here, xi and yi are lowercase letters (not necessarily different) that represent one-letter variable names.

Return true if it is possible to assign integers to variable names so as to satisfy all the given equations, or false otherwise.

Input: equations = ["a==b","b!=a"]

Output: false

Explanation: If we assign say, a = 1 and b = 1, then the first equation is satisfied, but not the second.

There is no way to assign the variables to satisfy both equations.

Input: equations = ["b==a","a==b"]

Output: true

Explanation: We could assign a = 1 and b = 1 to satisfy both equations.

```
struct DSU {
 vector<int> parent, rankv;
 DSU(int n) {
 parent.resize(n);
 rankv.resize(n, 0);
 for (int i = 0; i < n; i++)
 parent[i] = i;
 }
 int find(int x) {
 if (parent[x] != x)
 parent[x] = find(parent[x]); // path compression
 return parent[x];
 }
 bool unite(int a, int b) {
 int pa = find(a), pb = find(b);
 if (pa == pb) return false; // already connected → cycle
 if (rankv[pa] < rankv[pb]) swap(pa, pb);
 parent[pb] = pa;
 if (rankv[pa] == rankv[pb]) rankv[pa]++;
 return true;
 }
};

bool equationsPossible(vector<string>& equations) {
 DSU dsu(26);
```

```
// 1. union all equalities
for (auto &eq : equations)
 if (eq[1] == '=')
 dsu.unite(eq[0] - 'a', eq[3] - 'a');
// 2. check inequalities
for (auto &eq : equations)
 if (eq[1] == '!=')
 if (dsu.find(eq[0] - 'a') == dsu.find(eq[3] - 'a'))
 return false;
return true;
}
```

# Graphs-Algos

## Graphs - Graph Algorithms

Description: Advanced graph algorithms are used to solve complex problems involving shortest paths, minimum spanning trees, and graph cycles.

- [Kruskal's Algorithm: Minimum Cost to connect all Points](#)
- [Dijkstra's Algorithm: Cheapest Flights Within K Stops](#)
- [Floyd-Warshall: Find the City with Smallest Number of Neighbours at a Threshold Distance](#)
- [Bellman Ford: Network Delay time](#)

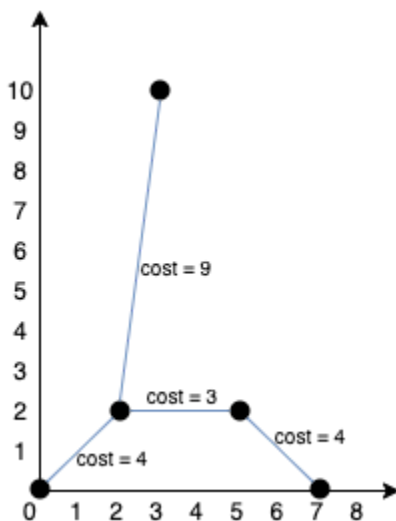
You are given an array points representing integer coordinates of some points on a 2D-plane, where  $\text{points}[i] = [x_i, y_i]$ .

The cost of connecting two points  $[x_i, y_i]$  and  $[x_j, y_j]$  is the manhattan distance between them:  $|x_i - x_j| + |y_i - y_j|$ , where  $|val|$  denotes the absolute value of  $val$ .

Return the minimum cost to make all points connected. All points are connected if there is exactly one simple path between any two points.

Input:  $\text{points} = [[0,0],[2,2],[3,10],[5,2],[7,0]]$

Output: 20



```

struct DSU {
 vector<int> parent, rankv;
 DSU(int n) {
 parent.resize(n);
 rankv.assign(n, 0);
 for (int i = 0; i < n; i++) parent[i] = i;
 }
 int find(int x) {
 if (parent[x] != x) parent[x] = find(parent[x]);
 return parent[x];
 }
 bool unite(int a, int b) {
 int pa = find(a), pb = find(b);
 if (pa == pb) return false;
 if (rankv[pa] < rankv[pb]) swap(pa, pb);
 parent[pb] = pa;
 if (rankv[pa] == rankv[pb]) rankv[pa]++;
 return true;
 }
};

int minCostConnectPoints(vector<vector<int>>& points) {
 int n = points.size();
 vector<vector<int>> edges;
 // Build all edges: O(n^2)
 for (int i = 0; i < n; i++) {
 for (int j = i + 1; j < n; j++) {
 int dist = abs(points[i][0] - points[j][0]) +
 abs(points[i][1] - points[j][1]);
 edges.push_back({dist, i, j});
 }
 }
 sort(edges.begin(), edges.end());
 DSU dsu(n);
 int cost = 0, used = 0;

```

```

 for (auto &e : edges) {
 int w = e[0], a = e[1], b = e[2];
 if (dsu.unite(a, b)) {
 cost += w;
 used++;
 if (used == n - 1) break;
 }
 }
 return cost;
}

```

There are  $n$  cities connected by some number of flights. You are given an array `flights` where `flights[i] = [fromi, toi, pricei]` indicates that there is a flight from city `fromi` to city `toi` with cost `pricei`.

You are also given three integers `src`, `dst`, and `k`, return the cheapest price from `src` to `dst` with at most `k` stops. If there is no such route, return `-1`.

Input: `n = 4`, `flights =`

`[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]]`, `src = 0`, `dst = 3`, `k = 1`

Output: `700`

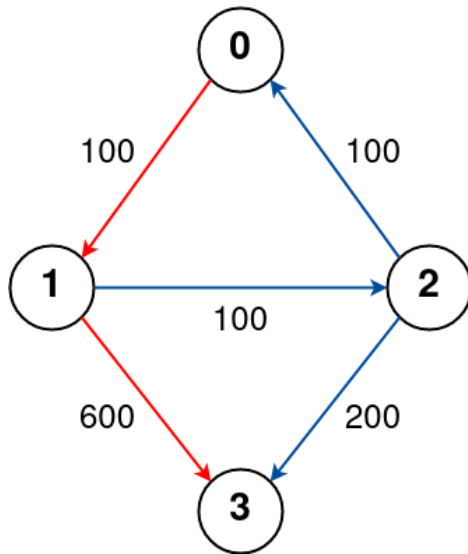
Explanation:

The graph is shown above.

The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost  $100 + 600 = 700$ .

Note that the path through cities `[0,1,2,3]` is cheaper but is invalid because it uses 2 stops.





```

int findCheapestPrice(int n, vector<vector<int>>& flights,
 int src, int dst, int K) {
 vector<vector<pair<int,int>>> adj(n);
 for (auto &f : flights)
 adj[f[0]].push_back({f[1], f[2]});
 vector<vector<int>> dist(n, vector<int>(K+2, INT_MAX));
 dist[src][0] = 0;
 queue<array<int,3>> q;
 q.push({src, 0, 0}); // node, cost, stops
 int ans = INT_MAX;
 while (!q.empty()) {
 auto cur = q.front(); q.pop();
 int u = cur[0], cost = cur[1], stops = cur[2];
 if (u == dst) ans = min(ans, cost);
 if (stops > K) continue;
 for (auto &nx : adj[u]) {
 int v = nx.first, w = nx.second;
 if (cost + w < dist[v][stops+1]) {
 dist[v][stops+1] = cost + w;
 q.push({v, cost + w, stops+1});
 }
 }
 }
 return ans == INT_MAX ? -1 : ans;
}

```

There are  $n$  cities numbered from 0 to  $n-1$ . Given the array `edges` where `edges[i] = [fromi, toi, weighti]` represents a bidirectional and weighted edge between cities `fromi` and `toi`, and given the integer `distanceThreshold`.

Return the city with the smallest number of cities that are reachable through some path and whose distance is at most `distanceThreshold`. If there are multiple such cities, return the city with the greatest number.

Notice that the distance of a path connecting cities  $i$  and  $j$  is equal to the sum of the edges' weights along that path.

Input:  $n = 4$ , `edges = [[0,1,3],[1,2,1],[1,3,4],[2,3,1]]`,  
`distanceThreshold = 4`

Output: 3

Explanation: The figure above describes the graph.

The neighboring cities at a `distanceThreshold = 4` for each city are:

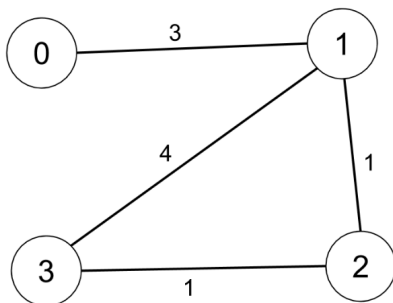
City 0 -> [City 1, City 2]

City 1 -> [City 0, City 2, City 3]

City 2 -> [City 0, City 1, City 3]

City 3 -> [City 1, City 2]

Cities 0 and 3 have 2 neighboring cities at a `distanceThreshold = 4`, but we have to return city 3 since it has the greatest number.



```
int findTheCity(int n, vector<vector<int>>& edges, int threshold) {
```

```

const int INF = 1e9;
vector<vector<int>> dist(n, vector<int>(n, INF));
for (int i = 0; i < n; i++) dist[i][i] = 0;
for (auto &e : edges) {
 int u = e[0], v = e[1], w = e[2];
 dist[u][v] = dist[v][u] = min(dist[u][v], w);
}
// Floyd-Warshall
for (int k = 0; k < n; k++)
 for (int i = 0; i < n; i++)
 for (int j = 0; j < n; j++)
 if (dist[i][k] + dist[k][j] < dist[i][j])
 dist[i][j] = dist[i][k] + dist[k][j];
int bestCity = -1, bestCnt = 1e9;
for (int i = 0; i < n; i++) {
 int cnt = 0;
 for (int j = 0; j < n; j++)
 if (dist[i][j] <= threshold) cnt++;
 if (cnt <= bestCnt) {
 bestCnt = cnt;
 bestCity = i; // choose largest i on tie
 }
}
return bestCity;
}

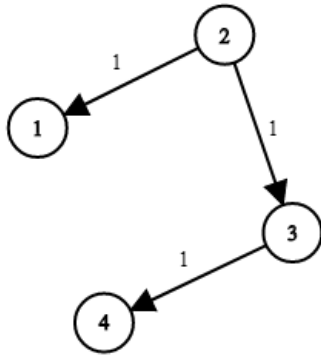
```

You are given a network of  $n$  nodes, labeled from 1 to  $n$ . You are also given times, a list of travel times as directed edges  $\text{times}[i] = (u_i, v_i, w_i)$ , where  $u_i$  is the source node,  $v_i$  is the target node, and  $w_i$  is the time it takes for a signal to travel from source to target.

We will send a signal from a given node  $k$ . Return the minimum time it takes for all the  $n$  nodes to receive the signal. If it is impossible for all the  $n$  nodes to receive the signal, return -1.

Input:  $\text{times} = [[2,1,1],[2,3,1],[3,4,1]]$ ,  $n = 4$ ,  $k = 2$

Output: 2



```

int networkDelayTime(vector<vector<int>>& times, int n, int k) {
 const int INF = 1e9;
 vector<int> dist(n + 1, INF);
 dist[k] = 0;
 // Relax edges n-1 times
 for (int i = 1; i < n; i++) {
 bool updated = false;
 for (auto &e : times) {
 int u = e[0], v = e[1], w = e[2];
 if (dist[u] != INF && dist[u] + w < dist[v]) {
 dist[v] = dist[u] + w;
 updated = true;
 }
 }
 if (!updated) break;
 }
 int ans = 0;
 for (int i = 1; i <= n; i++)
 ans = max(ans, dist[i]);
 return ans == INF ? -1 : ans;
}

```

GenDS-Greedy

## General DS - Greedy

Description: Greedy algorithms make local optimal choices at each step, which lead to a global optimal solution for problems like scheduling and resource allocation.

- [\[Can add\] Jump Game](#)
- [Jump Game ii](#)
- [Gas Station](#)
- [Bag of Tokens](#)
- [Boats to Save People](#)
- [Wiggle Subsequence](#)
- [Car Pooling](#)
- [Candy](#)

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

Input: `nums = [2,3,1,1,4]`

Output: `true`

Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.

```
bool canJump(vector<int>& nums) {
 int n = nums.size();
 int maxReach = 0;
 for (int i = 0; i < n; i++) {
 if (i > maxReach) return false; // can't reach this
position
 maxReach = max(maxReach, i + nums[i]);
 if (maxReach >= n - 1) return true; // can reach the end
 }
 return true;
}
```

You are given a 0-indexed array of integers `nums` of length `n`. You are initially positioned at index 0.

Each element `nums[i]` represents the maximum length of a forward jump from index `i`. In other words, if you are at index `i`, you can jump to any index `(i + j)` where:

$0 \leq j \leq \text{nums}[i]$  and  $i + j < n$

Return the minimum number of jumps to reach index `n - 1`. The test cases are generated such that you can reach index `n - 1`.

Input: `nums = [2,3,1,1,4]`

Output: 2

Explanation: The minimum number of jumps to reach the last index is 2. Jump 1 step from index 0 to 1, then 3 steps to the last index.

```
int jump(vector<int>& nums) {
 int jumps = 0, curEnd = 0, curFarthest = 0;
 int n = nums.size();
 for (int i = 0; i < n - 1; i++) { // no need to jump from last
index
 curFarthest = max(curFarthest, i + nums[i]);

 if (i == curEnd) { // end of current jump range
 jumps++;
 curEnd = curFarthest;
 }
 }
 return jumps;
}
```

There are `n` gas stations along a circular route, where the amount of gas at the `i`th station is `gas[i]`.

You have a car with an unlimited gas tank and it costs `cost[i]` of gas to travel from the `i`th station to its next `(i + 1)`th station.

You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays `gas` and `cost`, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1. If there exists a solution, it is guaranteed to be unique.

Input: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]`

Output: 3

Explanation:

Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank =  $0 + 4 = 4$

Travel to station 4. Your tank =  $4 - 1 + 5 = 8$

Travel to station 0. Your tank =  $8 - 2 + 1 = 7$

Travel to station 1. Your tank =  $7 - 3 + 2 = 6$

Travel to station 2. Your tank =  $6 - 4 + 3 = 5$

Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.

Therefore, return 3 as the starting index.

```
int canCompleteCircuit(vector<int>& gas, vector<int>& cost) {
 int totalTank = 0, currTank = 0, start = 0;
 int n = gas.size();
 for (int i = 0; i < n; i++) {
 int diff = gas[i] - cost[i];
 totalTank += diff;
 currTank += diff;
 if (currTank < 0) {
 start = i + 1;
 currTank = 0;
 }
 }
 return (totalTank >= 0) ? start : -1;
}
```

You start with an initial power of power, an initial score of 0, and a bag of tokens given as an integer array tokens, where each tokens[i] denotes the value of tokeni. Your goal is to maximize the total score by strategically playing these tokens. In one move, you can play an unplayed token in one of the two ways (but not both for the same token): Face-up: If your current power is at least tokens[i], you may play tokeni, losing tokens[i] power and gaining 1 score; Face-down: If your current score is at least 1, you may play tokeni, gaining tokens[i] power and losing 1 score. Return the maximum possible score you can achieve after playing any number of tokens.

Input: tokens = [100], power = 50



**Output: 0**

**Explanation:** Since your score is 0 initially, you cannot play the token face-down. You also cannot play it face-up since your power (50) is less than tokens[0] (100).

```
int bagOfTokensScore(vector<int>& tokens, int power) {
 sort(tokens.begin(), tokens.end());
 int i = 0, j = tokens.size() - 1;
 int score = 0, maxScore = 0;
 while (i <= j) {
 if (power >= tokens[i]) {
 power -= tokens[i++];
 score++;
 maxScore = max(maxScore, score);
 } else if (score > 0) {
 power += tokens[j--];
 score--;
 } else {
 break;
 }
 }
 return maxScore;
}
```

You are given an array people where people[i] is the weight of the ith person, and an infinite number of boats where each boat can carry a maximum weight of limit. Each boat carries at most two people at the same time, provided the sum of the weight of those people is at most limit.

Return the minimum number of boats to carry every given person.

**Input:** people = [3,5,3,4], limit = 5

**Output:** 4

**Explanation:** 4 boats (3), (3), (4), (5)

```
int numRescueBoats(vector<int>& people, int limit) {
```

```

 sort(people.begin(), people.end());
 int i = 0, j = people.size() - 1, boats = 0;
 while (i <= j) {
 if (people[i] + people[j] <= limit) i++;
 j--;
 boats++;
 }
 return boats;
}

```

A wiggle sequence is a sequence where the differences between successive numbers strictly alternate between positive and negative. The first difference (if one exists) may be either positive or negative. A sequence with one element and a sequence with two non-equal elements are trivially wiggle sequences.

For example, [1, 7, 4, 9, 2, 5] is a wiggle sequence because the differences (6, -3, 5, -7, 3) alternate between positive and negative.

In contrast, [1, 4, 7, 2, 5] and [1, 7, 4, 5, 5] are not wiggle sequences. The first is not because its first two differences are positive, and the second is not because its last difference is zero. A subsequence is obtained by deleting some elements (possibly zero) from the original sequence, leaving the remaining elements in their original order.

Given an integer array nums, return the length of the longest wiggle subsequence of nums.

Input: nums = [1,7,4,9,2,5]

Output: 6

Explanation: The entire sequence is a wiggle sequence with differences (6, -3, 5, -7, 3).

```

int wiggleMaxLength(vector<int>& nums) {
 if (nums.size() < 2) return nums.size();

```

```

 int up = 1, down = 1;
 for (int i = 1; i < nums.size(); i++) {
 if (nums[i] > nums[i-1]) up = down + 1;
 else if (nums[i] < nums[i-1]) down = up + 1;
 }
 return max(up, down);
}

```

There is a car with capacity empty seats. The vehicle only drives east (i.e., it cannot turn around and drive west).

You are given the integer capacity and an array trips where trips[i] = [numPassengersi, fromi, toi] indicates that the ith trip has numPassengersi passengers and the locations to pick them up and drop them off are fromi and toi respectively. The locations are given as the number of kilometers due east from the car's initial location. Return true if it is possible to pick up and drop off all passengers for all the given trips, or false otherwise.

Input: trips = [[2,1,5],[3,3,7]], capacity = 5

Output: true

```

bool carPooling(vector<vector<int>>& trips, int capacity) {
 vector<int> changes(1001, 0); // constraints: locations <= 1000
 for (auto &t : trips) {
 changes[t[1]] += t[0]; // pickup
 changes[t[2]] -= t[0]; // drop
 }
 int curr = 0;
 for (int p : changes) {
 curr += p;
 if (curr > capacity) return false;
 }
 return true;
}

```

[hard]

There are n children standing in a line. Each child is assigned a rating value given in the integer array ratings.

You are giving candies to these children subjected to the following requirements:

Each child must have at least one candy.

Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

Input: ratings = [1,0,2]

Output: 5

Explanation: You can allocate to the first, second and third child with 2, 1, 2 candies respectively.

```
int candy(vector<int>& ratings) {
 int n = ratings.size();
 vector<int> candies(n,1);
 for(int i=1;i<n;i++){
 if(ratings[i]>ratings[i-1]) candies[i] = candies[i-1]+1;
 }
 for(int i=n-2;i>=0;i--){
 if(ratings[i]>ratings[i+1]) candies[i] = max(candies[i],
candies[i+1]+1);
 }
 return accumulate(candies.begin(), candies.end(), 0);
}
```

# DesignDataStructure

## Design Data Structure

**Description:** It involves building custom data structures to efficiently handle specific operations, like managing data access, updates, and memory usage. Focusing on optimizing performance and resource management.

- [Design Twitter](#)
- [Design Browser History](#)
- [Design Circular Deque](#)
- [Snapshot Array](#)
- [LRU Cache](#)
- [LFU Cache](#)

Design a simplified version of Twitter where users can post tweets, follow/unfollow another user, and is able to see the 10 most recent tweets in the user's news feed.

Implement the Twitter class:

Twitter() Initializes your twitter object.

void postTweet(int userId, int tweetId) Composes a new tweet with ID tweetId by the user userId. Each call to this function will be made with a unique tweetId.

List<Integer> getNewsFeed(int userId) Retrieves the 10 most recent tweet IDs in the user's news feed. Each item in the news feed must be posted by users who the user followed or by the user themselves. Tweets must be ordered from most recent to least recent.

void follow(int followerId, int followeeId) The user with ID followerId started following the user with ID followeeId.

void unfollow(int followerId, int followeeId) The user with ID followerId started unfollowing the user with ID followeeId.

Input

```
["Twitter", "postTweet", "getNewsFeed", "follow", "postTweet",
"getNewsFeed", "unfollow", "getNewsFeed"]
```

```
[[], [1, 5], [1], [1, 2], [2, 6], [1], [1, 2], [1]]
```

Output

```
[null, null, [5], null, null, [6, 5], null, [5]]
```

Explanation

```
Twitter twitter = new Twitter();
```

```

twitter.postTweet(1, 5); // User 1 posts a new tweet (id = 5).
twitter.getNewsFeed(1); // User 1's news feed should return a list
with 1 tweet id -> [5]. return [5]
twitter.follow(1, 2); // User 1 follows user 2.
twitter.postTweet(2, 6); // User 2 posts a new tweet (id = 6).
twitter.getNewsFeed(1); // User 1's news feed should return a list
with 2 tweet ids -> [6, 5]. Tweet id 6 should precede tweet id 5
because it is posted after tweet id 5.
twitter.unfollow(1, 2); // User 1 unfollows user 2.
twitter.getNewsFeed(1); // User 1's news feed should return a list
with 1 tweet id -> [5], since user 1 is no longer following user 2.
class Twitter {
private:
 int timestamp;
 unordered_map<int, unordered_set<int>> follows;
 unordered_map<int, vector<pair<int,int>>> tweets; // user ->
{(time, tweetId)}
public:
 Twitter() {
 timestamp = 0;
 }
 void postTweet(int userId, int tweetId) {
 tweets[userId].push_back({timestamp++, tweetId});
 }
 vector<int> getNewsFeed(int userId) {
 vector<pair<int,int>> feed;
 if(tweets.count(userId)) feed.insert(feed.end(),
tweets[userId].begin(), tweets[userId].end());
 for(auto f : follows[userId]) {
 if(tweets.count(f))
 feed.insert(feed.end(), tweets[f].begin(),
tweets[f].end());
 }
 sort(feed.rbegin(), feed.rend()); // most recent first
 vector<int> ans;
 for(int i=0;i<min(10,(int)feed.size());i++)
 ans.push_back(feed[i].second);
 return ans;
 }
};

```

```

 }
 void follow(int followerId, int followeeId) {
 if(followerId != followeeId)
 follows[followerId].insert(followeeId);
 }
 void unfollow(int followerId, int followeeId) {
 follows[followerId].erase(followeeId);
 }
};

```

You have a browser of one tab where you start on the homepage and you can visit another url, get back in the history number of steps or move forward in the history number of steps.

Implement the BrowserHistory class:

BrowserHistory(string homepage) Initializes the object with the homepage of the browser.

void visit(string url) Visits url from the current page. It clears up all the forward history.

string back(int steps) Move steps back in history. If you can only return x steps in the history and steps > x, you will return only x steps. Return the current url after moving back in history at most steps.

string forward(int steps) Move steps forward in history. If you can only forward x steps in the history and steps > x, you will forward only x steps. Return the current url after forwarding in history at most steps.

Input:

```

["BrowserHistory","visit","visit","visit","back","back","forward","visit","forward","back","back"]
[["leetcode.com"],["google.com"],["facebook.com"],["youtube.com"],[1],[1],[1],["linkedin.com"],[2],[2],[7]]

```

Output:

```

[null,null,null,null,"facebook.com","google.com","facebook.com",null,"linkedin.com","google.com","leetcode.com"]

```

Explanation:

```

BrowserHistory browserHistory = new BrowserHistory("leetcode.com");

```



```

browserHistory.visit("google.com"); // You are in
"leetcode.com". Visit "google.com"
browserHistory.visit("facebook.com"); // You are in
"google.com". Visit "facebook.com"
browserHistory.visit("youtube.com"); // You are in
"facebook.com". Visit "youtube.com"
browserHistory.back(1); // You are in
"youtube.com", move back to "facebook.com" return "facebook.com"
browserHistory.back(1); // You are in
"facebook.com", move back to "google.com" return "google.com"
browserHistory.forward(1); // You are in
"google.com", move forward to "facebook.com" return "facebook.com"
browserHistory.visit("linkedin.com"); // You are in
"facebook.com". Visit "linkedin.com"
browserHistory.forward(2); // You are in
"linkedin.com", you cannot move forward any steps.
browserHistory.back(2); // You are in
"linkedin.com", move back two steps to "facebook.com" then to
"google.com". return "google.com"
browserHistory.back(7); // You are in
"google.com", you can move back only one step to "leetcode.com".
return "leetcode.com"
class BrowserHistory {
 vector<string> history;
 int curr;
public:
 BrowserHistory(string homepage) {
 history.push_back(homepage);
 curr = 0;
 }
 void visit(string url) {
 history.resize(curr+1); // remove forward
 history.push_back(url);
 curr++;
 }
 string back(int steps) {
 curr = max(0, curr-steps);
 return history[curr];
 }

```

```

 }
 string forward(int steps) {
 curr = min((int)history.size()-1, curr+steps);
 return history[curr];
 }
};

```

Design your implementation of the circular double-ended queue (deque).

Implement the MyCircularDeque class:

MyCircularDeque(int k) Initializes the deque with a maximum size of k.

boolean insertFront() Adds an item at the front of Deque. Returns true if the operation is successful, or false otherwise.

boolean insertLast() Adds an item at the rear of Deque. Returns true if the operation is successful, or false otherwise.

boolean deleteFront() Deletes an item from the front of Deque. Returns true if the operation is successful, or false otherwise.

boolean deleteLast() Deletes an item from the rear of Deque. Returns true if the operation is successful, or false otherwise.

int getFront() Returns the front item from the Deque. Returns -1 if the deque is empty.

int getRear() Returns the last item from Deque. Returns -1 if the deque is empty.

boolean isEmpty() Returns true if the deque is empty, or false otherwise.

boolean isFull() Returns true if the deque is full, or false otherwise.

Input

```

["MyCircularDeque", "insertLast", "insertLast", "insertFront",
"insertFront", "getRear", "isFull", "deleteLast", "insertFront",
"getFront"]

```

```

[[3], [1], [2], [3], [4], [], [], [], [4], []]

```

Output

```

[null, true, true, true, false, 2, true, true, true, 4]

```

### Explanation

```
MyCircularDeque myCircularDeque = new MyCircularDeque(3);
myCircularDeque.insertLast(1); // return True
myCircularDeque.insertLast(2); // return True
myCircularDeque.insertFront(3); // return True
myCircularDeque.insertFront(4); // return False, the queue is full.
myCircularDeque.getRear(); // return 2
myCircularDeque.isFull(); // return True
myCircularDeque.deleteLast(); // return True
myCircularDeque.insertFront(4); // return True
myCircularDeque.getFront(); // return 4
class MyCircularDeque {
 vector<int> dq;
 int frontIndex, rearIndex, capacity, size;
public:
 MyCircularDeque(int k) {
 capacity = k;
 dq.resize(k);
 frontIndex = 0;
 rearIndex = 0;
 size = 0;
 }
 bool insertFront(int value) {
 if(isFull()) return false;
 frontIndex = (frontIndex - 1 + capacity) % capacity;
 dq[frontIndex] = value;
 size++;
 return true;
 }
 bool insertLast(int value) {
 if(isFull()) return false;
 dq[rearIndex] = value;
 rearIndex = (rearIndex + 1) % capacity;
 size++;
 return true;
 }
 bool deleteFront() {
 if(isEmpty()) return false;
```

```

 frontIndex = (frontIndex + 1) % capacity;
 size--;
 return true;
 }
 bool deleteLast() {
 if(isEmpty()) return false;
 rearIndex = (rearIndex - 1 + capacity) % capacity;
 size--;
 return true;
 }
 int getFront() {
 if(isEmpty()) return -1;
 return dq[frontIndex];
 }
 int getRear() {
 if(isEmpty()) return -1;
 return dq[(rearIndex - 1 + capacity) % capacity];
 }
 bool isEmpty() { return size == 0; }
 bool isFull() { return size == capacity; }
};

```

Implement a SnapshotArray that supports the following interface:  
**SnapshotArray(int length)** initializes an array-like data structure with the given length. Initially, each element equals 0.  
**void set(index, val)** sets the element at the given index to be equal to val.

**int snap()** takes a snapshot of the array and returns the snap\_id: the total number of times we called snap() minus 1.

**int get(index, snap\_id)** returns the value at the given index, at the time we took the snapshot with the given snap\_id

Input: ["SnapshotArray","set","snap","set","get"]

[[3],[0,5],[],[0,6],[0,0]]

Output: [null,null,0,null,5]

Explanation:

```
SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3
```

```
snapshotArr.set(0,5); // Set array[0] = 5
```

```
snapshotArr.snap(); // Take a snapshot, return snap_id = 0
```

```
snapshotArr.set(0,6);
```

```
snapshotArr.get(0,0); // Get the value of array[0] with snap_id = 0, return 5
```

```
class SnapshotArray {
 int snap_id;
 vector<vector<pair<int,int>>> arr; // index -> (snap_id, value)
public:
 SnapshotArray(int length) {
 snap_id = 0;
 arr.resize(length);
 for(int i=0;i<length;i++)
 arr[i].push_back({-1,0});
 }
 void set(int index, int val) {
 arr[index].push_back({snap_id, val});
 }
 int snap() {
 return snap_id++;
 }
 int get(int index, int id) {
 auto &v = arr[index];
 int l=0,r=v.size()-1, ans=0;
 while(l<=r){
 int m = (l+r)/2;
 if(v[m].first <= id){
 ans = v[m].second;
 l = m+1;
 } else r = m-1;
 }
 return ans;
 }
};
```

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

LRUCache(int capacity) Initialize the LRU cache with positive size capacity.

int get(int key) Return the value of the key if the key exists, otherwise return -1.

void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.

The functions get and put must each run in  $O(1)$  average time complexity.

Input

["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]

[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

Output

[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation

```
LRUCache lRUCache = new LRUCache(2);
```

```
lRUCache.put(1, 1); // cache is {1=1}
```

```
lRUCache.put(2, 2); // cache is {1=1, 2=2}
```

```
lRUCache.get(1); // return 1
```

```
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
```

```
lRUCache.get(2); // returns -1 (not found)
```

```
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
```

```
lRUCache.get(1); // return -1 (not found)
```

```
lRUCache.get(3); // return 3
```

```
lRUCache.get(4); // return 4
```

```

class LRUCache {
 int cap;
 list<pair<int,int>> dq;
 unordered_map<int,list<pair<int,int>>::iterator> mp;
public:
 LRUCache(int capacity) { cap = capacity; }
 int get(int key) {
 if(mp.find(key) == mp.end()) return -1;
 dq.splice(dq.begin(), dq, mp[key]); // move to front
 return mp[key]->second;
 }
 void put(int key, int value) {
 if(mp.find(key) != mp.end()){ // logic same as get()
 dq.splice(dq.begin(), dq, mp[key]);
 mp[key]->second = value;
 } else {
 if(dq.size() == cap){
 mp.erase(dq.back().first);
 dq.pop_back();
 }
 dq.push_front({key,value});
 mp[key] = dq.begin();
 }
 }
};

```

**[hard]**

Design and implement a data structure for a Least Frequently Used (LFU) cache. Implement the LFUCache class:

**LFUCache(int capacity)** Initializes the object with the capacity of the data structure. **int get(int key)** Gets the value of the key if the key exists in the cache. Otherwise, returns -1.

**void put(int key, int value)** Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it. The functions get and put must each run in  $O(1)$  average time complexity.

Input: ["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]

[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]

Output: [null, null, null, 1, null, -1, 3, null, -1, 3, 4]

Explanation: // cnt(x) = the use counter for key x. // cache=[] will show the last used order for tiebreakers (leftmost element is most recent)

```
LFUCache lfu = new LFUCache(2);
lfu.put(1, 1); // cache=[1,_], cnt(1)=1
lfu.put(2, 2); // cache=[2,1], cnt(2)=1, cnt(1)=1
lfu.get(1); // return 1
 // cache=[1,2], cnt(2)=1, cnt(1)=2
lfu.put(3, 3); // 2 is the LFU key because cnt(2)=1 is the
 // smallest, invalidate 2.
 // cache=[3,1], cnt(3)=1, cnt(1)=2
lfu.get(2); // return -1 (not found)
lfu.get(3); // return 3
 // cache=[3,1], cnt(3)=2, cnt(1)=2
lfu.put(4, 4); // Both 1 and 3 have the same cnt, but 1 is LRU,
 // invalidate 1.
 // cache=[4,3], cnt(4)=1, cnt(3)=2
lfu.get(1); // return -1 (not found)
lfu.get(3); // return 3
 // cache=[3,4], cnt(4)=1, cnt(3)=3
lfu.get(4); // return 4
 // cache=[4,3], cnt(4)=2, cnt(3)=3
```



```

class LFUCache {
 int cap, minFreq;
 unordered_map<int, pair<int, int>> keyValFreq;
 unordered_map<int, list<int>> freqKeys;
 unordered_map<int, list<int>::iterator> keyIter;
public:
 LFUCache(int capacity) { cap = capacity; minFreq = 0; }
 int get(int key) {
 if(keyValFreq.find(key)==keyValFreq.end()) return -1;
 int val = keyValFreq[key].first, freq =
keyValFreq[key].second;
 freqKeys[freq].erase(keyIter[key]);
 if(freqKeys[freq].empty()){
 freqKeys.erase(freq);
 if(minFreq == freq) minFreq++;
 }
 freq++;
 freqKeys[freq].push_front(key);
 keyIter[key] = freqKeys[freq].begin();
 keyValFreq[key].second = freq;
 return val;
 }
 void put(int key, int value) {
 if(cap==0) return;
 if(get(key)!=-1){
 keyValFreq[key].first = value;
 return;
 }
 if(keyValFreq.size()==cap){
 int k = freqKeys[minFreq].back();
 freqKeys[minFreq].pop_back();
 keyValFreq.erase(k);
 keyIter.erase(k);
 }
 keyValFreq[key] = {value, 1};
 freqKeys[1].push_front(key);
 keyIter[key] = freqKeys[1].begin();
 minFreq = 1;}};

```

SQL\_Qs

Fundamentals & Qs.

# Fundamentals

## Fundamentals

---

### Table of Contents

1. Sample Tables for Examples
2. SQL Basics
3. Aggregate Functions
4. Window Functions Deep Dive
5. Time/Date Functions
6. Common Table Expressions (CTEs)
7. Joins

### 1. Sample Tables for Examples {#sample-tables}

Let's create sample tables that we'll use throughout this guide:

#### employees table

| emp_id | name    | department | salary | hire_date  | manager_id |
|--------|---------|------------|--------|------------|------------|
| 1      | Alice   | Sales      | 60000  | 2020-01-15 | NULL       |
| 2      | Bob     | IT         | 75000  | 2019-03-20 | 1          |
| 3      | Charlie | Sales      | 55000  | 2021-06-10 | 1          |
| 4      | Diana   | IT         | 80000  | 2018-11-05 | 1          |
| 5      | Eve     | HR         | 50000  | 2020-08-25 | 1          |
| 6      | Frank   | Sales      | 62000  | 2019-12-01 | 1          |

|   |       |    |       |            |   |
|---|-------|----|-------|------------|---|
| 7 | Grace | IT | 78000 | 2021-02-14 | 4 |
| 8 | Henry | HR | 52000 | 2022-04-30 | 5 |

### sales table

| sale_id | emp_id | product  | amount | sale_date  |
|---------|--------|----------|--------|------------|
| 1       | 1      | Laptop   | 1200   | 2023-01-10 |
| 2       | 3      | Mouse    | 25     | 2023-01-12 |
| 3       | 1      | Keyboard | 75     | 2023-01-15 |
| 4       | 6      | Monitor  | 350    | 2023-01-18 |
| 5       | 3      | Laptop   | 1100   | 2023-02-01 |
| 6       | 1      | Mouse    | 30     | 2023-02-05 |
| 7       | 6      | Keyboard | 80     | 2023-02-10 |
| 8       | 3      | Monitor  | 400    | 2023-02-15 |

## 2. SQL Basics {#sql-basics}

### SELECT Statement

```
sql
-- Basic SELECT
SELECT * FROM employees;
```

```

-- Select specific columns
SELECT name, department, salary FROM employees;

-- With WHERE clause
SELECT name, salary
FROM employees
WHERE department = 'IT' AND salary > 75000;

-- With ORDER BY
SELECT name, salary
FROM employees
ORDER BY salary DESC;

-- With LIMIT
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;

```

## INSERT, UPDATE, DELETE

```

sql
-- INSERT
INSERT INTO employees (emp_id, name, department, salary,
hire_date) VALUES (9, 'Ivan', 'Sales', 58000, '2023-03-01');

-- UPDATE
UPDATE employees SET salary = salary * 1.1 WHERE department =
'IT';

-- DELETE
DELETE FROM employees WHERE emp_id = 9;

```

## DISTINCT and GROUP BY

```

sql
-- DISTINCT values

```

```
SELECT DISTINCT department FROM employees;

-- GROUP BY with HAVING
SELECT department, COUNT(*) as emp_count, AVG(salary) as
avg_salary FROM employees GROUP BY department
HAVING COUNT(*) > 2;
```

---

### 3. Aggregate Functions {#aggregate-functions}

#### Common Aggregate Functions

```
sql
-- COUNT
SELECT COUNT(*) as total_employees FROM employees;
SELECT COUNT(DISTINCT department) as unique_departments FROM
employees;

-- SUM
SELECT SUM(salary) as total_payroll FROM employees;

-- AVG
SELECT AVG(salary) as average_salary FROM employees;

-- MIN/MAX
SELECT
 MIN(salary) as lowest_salary,
 MAX(salary) as highest_salary
FROM employees;

-- GROUP_CONCAT / STRING_AGG (varies by database)
-- MySQL
SELECT department, GROUP_CONCAT(name) as employees_list
FROM employees
GROUP BY department;
```

#### Combining Aggregates



```
sql
SELECT
 department,
 COUNT(*) as emp_count,
 MIN(salary) as min_salary,
 MAX(salary) as max_salary,
 AVG(salary) as avg_salary,
 SUM(salary) as total_salary
FROM employees
GROUP BY department
ORDER BY avg_salary DESC;
```

#### 4. Window Functions Deep Dive {#window-functions}

##### Understanding OVER()

The **OVER()** clause defines a window (a set of rows) for the function to operate on. **OVER() with no partition -> Window = entire table**

```
sql
-- Aggregate function with OVER() - calculates for ALL rows
SELECT
 name,
 department,
 salary,
 AVG(salary) OVER() as company_avg_salary
FROM employees;
```

Result:

| name  | department | salary | company_avg_salary |
|-------|------------|--------|--------------------|
| Alice | Sales      | 60000  | 63625              |
| Bob   | IT         | 75000  | 63625              |

|         |       |       |       |
|---------|-------|-------|-------|
| Charlie | Sales | 55000 | 63625 |
| ...     | ...   | ...   | 63625 |

## OVER() PARTITION BY

**PARTITION BY** divides the result set into partitions (groups) and performs calculations within each partition.

```
sql
SELECT
 name,
 department,
 salary,
 AVG(salary) OVER(PARTITION BY department) as dept_avg_salary
FROM employees;
```

Result:

| name    | department | salary | dept_avg_salary |
|---------|------------|--------|-----------------|
| Alice   | Sales      | 60000  | 59000           |
| Charlie | Sales      | 55000  | 59000           |
| Frank   | Sales      | 62000  | 59000           |
| Bob     | IT         | 75000  | 77667           |
| Diana   | IT         | 80000  | 77667           |
| Grace   | IT         | 78000  | 77667           |
| Eve     | HR         | 50000  | 51000           |

|       |    |       |       |
|-------|----|-------|-------|
| Henry | HR | 52000 | 51000 |
|-------|----|-------|-------|

## ROW\_NUMBER()

Assigns a unique sequential integer to rows within a partition.

```
sql
-- Simple ROW_NUMBER()
SELECT
 name,
 department,
 salary,
 ROW_NUMBER() OVER(ORDER BY salary DESC) as salary_rank
FROM employees;
```

## ROW\_NUMBER() OVER() PARTITION BY

```
sql
-- ROW_NUMBER() with PARTITION BY
SELECT
 name,
 department,
 salary,
 ROW_NUMBER() OVER(PARTITION BY department ORDER BY salary
DESC) as dept_rank
FROM employees;
```

Result:

| name  | department | salary | dept_rank |
|-------|------------|--------|-----------|
| Henry | HR         | 52000  | 1         |
| Eve   | HR         | 50000  | 2         |
| Diana | IT         | 80000  | 1         |

|         |       |       |   |
|---------|-------|-------|---|
| Grace   | IT    | 78000 | 2 |
| Bob     | IT    | 75000 | 3 |
| Frank   | Sales | 62000 | 1 |
| Alice   | Sales | 60000 | 2 |
| Charlie | Sales | 55000 | 3 |

### **RANK() vs DENSE\_RANK() vs ROW\_NUMBER()**

Key differences when there are ties:

- ROW\_NUMBER(): Always assigns unique numbers (1,2,3,4...)
- RANK(): Assigns same rank to ties, then skips numbers (1,2,2,4...)
- DENSE\_RANK(): Assigns same rank to ties, no gaps (1,2,2,3...)

```
sql
-- Let's see the difference with an example having duplicate
salaries
WITH sales_summary AS (
 SELECT
 emp_id,
 SUM(amount) as total_sales
 FROM sales
 GROUP BY emp_id
)
SELECT
 e.name,
 s.total_sales,
 ROW_NUMBER() OVER(ORDER BY s.total_sales DESC) as row_num,
 RANK() OVER(ORDER BY s.total_sales DESC) as rank_num,
 DENSE_RANK() OVER(ORDER BY s.total_sales DESC) as
dense_rank_num
FROM employees e
JOIN sales_summary s ON e.emp_id = s.emp_id;
```

## Advanced Window Functions

```
sql
-- LEAD and LAG - access next/previous row
SELECT
 name,
 salary,
 LAG(salary, 1) OVER(ORDER BY salary) as previous_salary,
 LEAD(salary, 1) OVER(ORDER BY salary) as next_salary
FROM employees;

-- FIRST_VALUE and LAST_VALUE
SELECT
 name,
 department,
 salary,
 FIRST_VALUE(name) OVER(PARTITION BY department ORDER BY salary
DESC) as highest_paid_in_dept,
 LAST_VALUE(name) OVER(
 PARTITION BY department
 ORDER BY salary DESC
 ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
) as lowest_paid_in_dept
FROM employees;

-- NTILE - divide rows into n groups
SELECT
 name,
 salary,
 NTILE(4) OVER(ORDER BY salary DESC) as salary_quartile
FROM employees;

-- Running totals with SUM() OVER()
SELECT
 sale_date,
 amount,
 SUM(amount) OVER(ORDER BY sale_date) as running_total
FROM sales
```

```

ORDER BY sale_date;

-- Moving averages
SELECT
 sale_date,
 amount,
 AVG(amount) OVER(
 ORDER BY sale_date
 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) as moving_avg_3_days
FROM sales;

```

## Window Frame Specification

```

sql
-- Different window frames
SELECT
 name,
 salary,
 department,
 -- Default frame: RANGE BETWEEN UNBOUNDED PRECEDING AND
 CURRENT ROW
 SUM(salary) OVER(PARTITION BY department ORDER BY salary) as
 default_sum,

 -- All rows in partition
 SUM(salary) OVER(
 PARTITION BY department
 ORDER BY salary
 ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
) as total_dept_salary,

 -- Current row and 1 before
 AVG(salary) OVER(
 PARTITION BY department
 ORDER BY salary
 ROWS BETWEEN 1 PRECEDING AND CURRENT ROW
) as avg_with_prev,

```

```
-- Current row and 1 after
AVG(salary) OVER(
 PARTITION BY department
 ORDER BY salary
 ROWS BETWEEN CURRENT ROW AND 1 FOLLOWING
) as avg_with_next
FROM employees;
```

---

## 5. Time/Date Functions {#time-functions}

### Common Date Functions

```
sql
-- Current date/time
SELECT
 CURRENT_DATE,
 CURRENT_TIME,
 CURRENT_TIMESTAMP,
 NOW(); -- MySQL

-- Extract parts
SELECT
 hire_date,
 YEAR(hire_date) as year,
 MONTH(hire_date) as month,
 DAY(hire_date) as day,
 DAYOFWEEK(hire_date) as day_of_week, -- MySQL
 EXTRACT(YEAR FROM hire_date) as year_extracted -- Standard SQL
FROM employees;

-- Date arithmetic
SELECT
 name,
 hire_date,
 DATE_ADD(hire_date, INTERVAL 1 YEAR) as first_anniversary, --
```

```

MySQL
 DATE_SUB(hire_date, INTERVAL 30 DAY) as month_before, --
MySQL
 DATEDIFF(CURRENT_DATE, hire_date) as days_employed, -- MySQL
FROM employees;

-- Formatting dates
SELECT
 name,
 DATE_FORMAT(hire_date, '%W, %M %d, %Y') as formatted_date, --
MySQL
FROM employees;

-- Date truncation
SELECT
 DATE_FORMAT(hire_date, '%Y-%m-01') as month_start_mysql --
MySQL
FROM employees;

```

## Time-based Analysis

```

sql
-- Employees hired by year and quarter
SELECT
 YEAR(hire_date) as hire_year,
 QUARTER(hire_date) as hire_quarter,
 COUNT(*) as employees_hired
FROM employees
GROUP BY YEAR(hire_date), QUARTER(hire_date)
ORDER BY hire_year, hire_quarter;

-- Calculate tenure
SELECT
 name,
 hire_date,
 TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE) as
years_employed,
 TIMESTAMPDIFF(MONTH, hire_date, CURRENT_DATE) % 12 as

```



```
additional_months
FROM employees
ORDER BY years_employed DESC;
```

---

## 6. Common Table Expressions (CTEs) {#ctes}

CTEs make complex queries more readable and maintainable.

```
sql
-- Basic CTE
WITH high_earners AS (
 SELECT * FROM employees WHERE salary > 70000
)
SELECT department, COUNT(*) as high_earner_count
FROM high_earners
GROUP BY department;

-- Multiple CTEs
WITH
dept_stats AS (
 SELECT
 department,
 AVG(salary) as avg_salary,
 COUNT(*) as emp_count
 FROM employees
 GROUP BY department
) ,
company_stats AS (
 SELECT AVG(salary) as company_avg FROM employees
)
SELECT
 d.department,
 d.avg_salary,
 d.emp_count,
 c.company_avg,
 d.avg_salary - c.company_avg as diff_from_company_avg
FROM dept_stats d
```

```

CROSS JOIN company_stats c
ORDER BY diff_from_company_avg DESC;

-- Recursive CTE (employee hierarchy)
WITH RECURSIVE emp_hierarchy AS (
 -- Anchor: start with top-level employees
 SELECT
 emp_id,
 name,
 manager_id,
 1 as level
 FROM employees
 WHERE manager_id IS NULL

 UNION ALL

 -- Recursive: find employees of current level
 SELECT
 e.emp_id,
 e.name,
 e.manager_id,
 h.level + 1
 FROM employees e
 JOIN emp_hierarchy h ON e.manager_id = h.emp_id
)
SELECT
 level,
 emp_id,
 name,
 manager_id
FROM emp_hierarchy
ORDER BY level, emp_id;

```

---

## 7. Joins {#joins}

### Types of Joins

```
sql
-- INNER JOIN
SELECT
 e.name,
 e.department,
 s.product,
 s.amount
FROM employees e
INNER JOIN sales s ON e.emp_id = s.emp_id;

-- LEFT JOIN
SELECT
 e.name,
 e.department,
 COALESCE(SUM(s.amount), 0) as total_sales
FROM employees e
LEFT JOIN sales s ON e.emp_id = s.emp_id
GROUP BY e.emp_id, e.name, e.department;

-- RIGHT JOIN (less common)
SELECT
 e.name,
 s.product,
 s.amount
FROM employees e
RIGHT JOIN sales s ON e.emp_id = s.emp_id;

-- FULL OUTER JOIN (not in MySQL)
SELECT
 COALESCE(e.name, 'No Employee') as name,
 COALESCE(s.product, 'No Sale') as product
FROM employees e
FULL OUTER JOIN sales s ON e.emp_id = s.emp_id;

-- CROSS JOIN
SELECT
 e.name,
```

```
 d.department
FROM employees e
CROSS JOIN (SELECT DISTINCT department FROM employees) d;

-- SELF JOIN
SELECT
 e1.name as employee,
 e2.name as manager
FROM employees e1
LEFT JOIN employees e2 ON e1.manager_id = e2.emp_id;
```

---

## Practical Examples Combining Everything

### Example 1: Top N per Group

```
sql
-- Get top 2 salaries per department
WITH ranked_salaries AS (
 SELECT
 name,
 department,
 salary,
 DENSE_RANK() OVER(PARTITION BY department ORDER BY salary
DESC) as salary_rank
 FROM employees
)
SELECT name, department, salary
FROM ranked_salaries
WHERE salary_rank <= 2
ORDER BY department, salary_rank;
```

### Example 2: Year-over-Year Comparison

```
sql
WITH monthly_sales AS (
 SELECT
 DATE_FORMAT(sale_date, '%Y-%m') as month,
```

```

 SUM(amount) as total_sales
 FROM sales
 GROUP BY DATE_FORMAT(sale_date, '%Y-%m')
)
SELECT
 month,
 total_sales,
 LAG(total_sales, 12) OVER(ORDER BY month) as
same_month_last_year,
 total_sales - LAG(total_sales, 12) OVER(ORDER BY month) as
yoy_change,
 ROUND(
 (total_sales - LAG(total_sales, 12) OVER(ORDER BY month))
 /
 LAG(total_sales, 12) OVER(ORDER BY month) * 100,
 2
) as yoy_percent_change
FROM monthly_sales
ORDER BY month;

```

### Example 3: Percentile Calculation

```

sql
SELECT
 name,
 salary,
 PERCENT_RANK() OVER(ORDER BY salary) as percentile_rank,
 CUME_DIST() OVER(ORDER BY salary) as cumulative_distribution,
 NTILE(100) OVER(ORDER BY salary) as percentile_bucket
FROM employees;

```

### Example 4: Gap Analysis

```

sql
-- Find gaps in sequential IDs
WITH id_gaps AS (
 SELECT
 emp_id,

```

```
 LEAD(emp_id) OVER(ORDER BY emp_id) as next_id,
 LEAD(emp_id) OVER(ORDER BY emp_id) - emp_id as gap
FROM employees
)
SELECT
 emp_id as last_before_gap,
 next_id as first_after_gap,
 gap - 1 as missing_ids_count
FROM id_gaps
WHERE gap > 1;
```

---

## Key Takeaways

1. Window Functions vs GROUP BY: Window functions don't reduce rows like GROUP BY; they add calculated columns while preserving all rows.
  2. PARTITION BY vs GROUP BY: PARTITION BY divides data for window function calculations, GROUP BY aggregates and reduces rows.
  3. ROW\_NUMBER vs RANK vs DENSE\_RANK: Use ROW\_NUMBER for unique numbering, RANK when you want gaps after ties, DENSE\_RANK for continuous ranking.
  4. Performance: Window functions can be more efficient than self-joins or subqueries for calculations across rows.
  5. Frame Clauses: Understanding ROWS vs RANGE and frame boundaries is crucial for moving averages and running totals.
  6. CTEs: Use them to break complex queries into manageable, readable parts.
  7. Combining Techniques: Real-world queries often combine multiple concepts - CTEs with window functions, joins with aggregates, etc.
-

Qs

Write a solution to report the products that were only sold in the first quarter of 2019. That is, between 2019-01-01 and 2019-03-31 inclusive.

```
select distinct p.product_id as product_id, p.product_name as
product_name
from Product p join Sales s
on p.product_id = s.product_id
where s.sale_date between '2019-01-01' and '2019-03-31'
and p.product_id not in (
 select sx.product_id from Sales sx where
 sx.sale_date < '2019-01-01' or sx.sale_date > '2019-03-31'
)
-- select p.product_id, p.product_name
-- from Product p
-- join Sales s on p.product_id = s.product_id
-- group by p.product_id, p.product_name having min(s.sale_date)
>= '2019-01-01' and max(s.sale_date) <= '2019-03-31';
```

Write a solution to get the names of products that have at least 100 units ordered in February 2020 and their amount.

```
select p.product_name as product_name, SUM(ofil.unit) as unit
from
(
 select o.product_id, o.unit
 from Orders o
 where year(order_date) = 2020
 and month(order_date) = 2
) ofil
join Products p
on ofil.product_id = p.product_id
group by ofil.product_id
having sum(ofil.unit) >=100
```

Find all numbers that appear at least three times consecutively.

```
select distinct(l1.num) as ConsecutiveNums
```



```

from
Logs l1 join Logs l2 join Logs l3
on l1.id = l2.id-1 and l2.id = l3.id-1
and l1.num=l2.num
and l2.num=l3.num
-- SELECT DISTINCT num AS ConsecutiveNums
-- FROM (
-- SELECT num, a.id - ROW_NUMBER() OVER (PARTITION BY num ORDER BY
id) AS label
-- FROM Logs a
--) cte
-- GROUP BY label, num
-- HAVING COUNT(num) >= 3;

```

Write a solution to find the people who have the most friends and the most friends number. The test cases are generated so that only one person has the most friends.

```

with base as
(
 select requester_id as id /* idr */
 from RequestAccepted
 union all
 select acceptor_id as id /* ida */
 from RequestAccepted
)
select id, count(*) as num from base
/* name taken from first expression only, i.e idr, but for sake of
error safety, we will name it same*/
group by 1
order by 2 desc
limit 1

```

Write a solution to find the employees who earn more than their managers.

```

select e.name as Employee

```

```

from Employee e
inner join Employee m
on e.managerId = m.id
where e.salary > m.salary
| e.id | e.name | e.salary | e.managerId | m.id | m.name |
m.salary |
| ---- | -|-----| -|-----| -|----| -|-----|
-----|
| 1 | Joe | 70000 | 3 | 3 | Sam | 60000 |
| 2 | Henry | 80000 | 4 | 4 | Max | 90000 |

```

Write a solution to report the movies with an odd-numbered ID and a description that is not "boring".

```

select * from
Cinema
where id%2!=0 and description != "boring"
order by rating desc

```

Write a solution to find the rank of the scores. The ranking should be calculated according to the following rules:

The scores should be ranked from the highest to the lowest.

If there is a tie between two scores, both should have the same ranking. After a tie, the next ranking number should be the next consecutive integer value. In other words, there should be no holes between ranks.

```

select score,
dense_rank() over (
 order by score desc
) as 'rank'
from Scores

```

Write a solution to report all the duplicate emails. Note that it's guaranteed that the email field is not NULL.

```
-- select email as Email
-- from Person
-- group by 1
-- having count(email) > 1
select distinct(p.email) as Email
from (
select email, row_number() over (partition by email) r
from Person
) p
where r > 1;
```

Write a solution to find all customers who never order anything.

```
select c.name as Customers
from Customers c left join Orders o
on c.id = o.customerId where o.customerId is null
```

Write a solution to find all dates' id with higher temperatures compared to its previous dates (yesterday).

```
select w2.id
from Weather w1 cross join Weather w2
where w2.temperature > w1.temperature
and datediff(w2.recordDate, w1.recordDate)=1;
```

Write a solution to report the name and bonus amount of each employee with a bonus less than 1000.

```
select e.name, b.bonus
from Employee e left join Bonus b
on e.empId = b.empId
where b.bonus < 1000 or b.bonus is null
```

---

LLD

LLD: Reference:

- <https://github.com/ashishps1/awesome-low-level-design>
- <https://www.hellointerview.com/learn/low-level-design/in-a-hurry/introduction>
- GPT

## Basics

- A class is a blueprint, template, or recipe for creating objects. It defines what an object will contain (its data) and what it will be able to do (its behavior).
- Datatype Refreshers:

```
Primary datatypes: int, long, double, float, char, boolean
Wrapper classes: Integer, Boolean, Double
Other datatypes:
List<String> names = new ArrayList<>(); // or new LinkedList<>();
// Add
names.add("Alice");
// Insert at index
names.add(1, "Zara");
// Modify
names.set(2, "Bobby"); // replace element at index 2
// Remove
names.remove("Charlie"); // by value
names.remove(0); // by index
// Iterate
for (String name : names)
 System.out.println(name);
// Contains
names.contains("Alice");
// Sort
Collections.sort(names);
names.sort(Comparator.reverseOrder());
// Sublist
List<String> sub = names.subList(0, 2);

Set<String> countries = new HashSet<>();
```

```

// Add
countries.add("India");
countries.add("India"); // ignored (duplicate)
// Remove
countries.remove("USA");
// Iterate
for (String c : countries)
 System.out.println(c);
// Contains
countries.contains("India");

Map<Integer, String> students = new HashMap<>();
// Add
students.put(101, "Alice");
// Modify (same key overwrites)
students.put(102, "Bobby");
// Remove
students.remove(102);
// Iterate:
for (Map.Entry<Integer, String> entry : students.entrySet()) {
 System.out.println(entry.getKey() + "->" + entry.getValue());
} // Over everything
for (Integer id: students.keySet()) { } // Iterate over key
for (String name: students.values()) { } // Iterate over value
// Using Map.of()
Map<String, Integer> ages = Map.of(
 "Alice", 25, "Bob", 30) // Immutable- Only supports 10 entries
Map<String, Integer> big = Map.ofEntries(
 Map.entry("A", 1),
 Map.entry("B", 2)) // Mutable - Supports > 10 entries
// Contains
students.containsKey(101);
students.containsValue("Alice");
// Get Or Default. Better than using get().
students.getDefault(105, "Unknown");
// Comparing operator
For primitives → use ==

```

```

int a = 5, b = 5;
System.out.println(a==b);
For objects→ ALWAYS use .equals(). '==' references memory address
String s1 = new String("hello"); String s2 = new String("hello");
System.out.println(s1==s2); // false (different memory)
System.out.println(s1.equals(s2)); // true (same content)

Queue<String> queue = new LinkedList<>();
queue.add("Task1"); // add head
System.out.println(queue.poll()); // removes head (Task1)
System.out.println(queue.peek()); // view head/front element
// Iterate
for (String t : queue)
 System.out.println(t);

Deque<Integer> stack = new ArrayDeque<>();
stack.push(10);
stack.pop();
stack.peek();
// Also has methods like addLast/addFirst, etc...

Min-heap (default): PriorityQueue<Integer> pq = new
PriorityQueue<>();
Max-heap: PriorityQueue<Integer> pq = new
PriorityQueue<>(Comparator.reverseOrder());

Have others like LinkedHashMap, EnumMap, etc., as well.

```

- Exceptions: Only methods (functions) and constructors can explicitly throw exceptions in Java. A class itself does not throw an exception.

```

public void writeFile() throws IOException {
 // ... code that might fail ...
 if (condition) {
 throw new IOException("File access denied.");
 }
}

```

```

}

/*
try {
 int a = 10;
 int b = 0;
 int result = a / b; // This will cause ArithmeticException
 System.out.println(result);
}
catch (ArithmeticException e) {
 System.out.println("Error: xyz");
}
catch (Exception e) {
 System.out.println("General error: " + e.getMessage());
}
finally {
 System.out.println("This block always runs.");
}
*/

```

- **Java Streams:** Java stream is a pipeline that processes a collection in a functional way. Can think of it as a conveyor belt.
  - It has 3 parts: source -> intermediate operations that return stream -> terminal operations that produce result.
  - Source: `list.stream()`, `array.stream()`, [Stream.of\(...\)](#)
  - Intermediate operations: `filter`, `map`, `sorted`, `distinct`, `limit`, `skip`
  - Terminal operations: `collect`, `toList`, `findFirst`, `min`, `max`, `count`, `anyMatch`, `allMatch`, `forEach`. Eg:

Eg:

```

List<Integer> nums = List.of(5, 3, 8, 3, 1);
Optional<Integer> smallestEven =
 nums.stream()
 .filter(n -> n % 2 == 0) // keep only even numbers
 .sorted() // sorted stream
 .findFirst(); // terminal op → executes

```



Important ones:

```
list.stream().filter(x -> x > 10) // Keeps only elements that
match a condition.
list.stream().map(x -> x * 2) // Transforms each element into
another value (or object).
list.stream().sorted(Comparator.comparing(Person::getAge)) //
sort
list.stream().distinct()
list.stream().limit(5)
List<Integer> ans =
list.stream().collect(Collectors.toList()); // or toSet()
list.stream().anyMatch(x -> x == 5)
list.stream().allMatch(x -> x > 0)
```

Other egs:

```
eg1: return spots.values().stream().filter(spot ->
!spot.isOccupied() &&
spot.canFitVehicle(vehicle)).min(Comparator.comparing(ParkingSpot:
:getSpotSize));
eg2: int sum = orders.stream().mapToInt(Order::getCost)
.sum();

```

- Normal vs Static vs Abstract vs Interface Class
  - **Normal Class:** A regular class that: Can be instantiated using new, Provides full implementation. May contain: fields, constructors, fully implemented methods, overridden methods (optional). It cannot have abstract methods. When to use:
    - When objects of the class should exist directly.
    - When full implementation is known.
    - When no forced method implementation rule is needed.

Eg:

```
class Dog {
 void bark() { System.out.println("Woof"); }
```

```
}
Dog d = new Dog(); // ✓ valid
```

- **Static:** Static members belong to the class, not the object. Used without creating an object. Shared across all objects. Cannot access non-static variables/methods. You can have a static field/method/inner class (NOT parent outer class).
  - Use static when something is shared, does not depend on object state, or should be accessible without creating an object.

```
class Counter {
 static int count = 0;
}
// Other ex:
class MathUtil {
 static int square(int n) {
 return n * n;
 }
}
MathUtil.square(5); // used directly
```

- **Abstract Class:** A class declared with abstract keyword that: Cannot be instantiated (most important), Can contain: abstract methods (no body), normal methods (with body), fields, constructors. Must be extended by child classes.
  - Why abstract classes exist: Because sometimes a parent class is: Conceptual (not concrete); Common base for multiple subclasses. Examples: Vehicle → Car, Truck; Animal → Dog, Cat; Account → Savings, Current.
  - When to use abstract class: Subclasses share common fields and logic; You want a structured base for all subclasses; Parent should never be instantiated; Some methods require compulsory overriding; Codebase

is large → avoids duplication (DRY) & enforces patterns.

Eg:

```
abstract class Vehicle {
 String plate;
 int wheels;
 Vehicle(String plate, int wheels) {
 this.plate = plate;
 this.wheels = wheels;
 }
 abstract void move(); // must be implemented later
 void printInfo() { // normal method
 System.out.println("General vehicle");
 }
}
class Car extends Vehicle {
 Car(String plate) { super(plate, 4); }
 @Override
 void move() { System.out.println("Car moving"); }
}
```

- **Abstract Methods:** Declared without body; Can only exist inside an abstract class; Must be implemented by the first concrete subclass; Abstract class cannot include abstract method with body, only signature/declaration. If subclass doesn't implement abstract methods then compile-time error (unless subclass is also abstract).
- Do all abstract methods need implementation?: Yes, but ONLY in a concrete subclass. If subclass is also abstract → it may skip implementation.
- Can abstract class contain constructor?: Yes. Reason: When you create a child object, the parent portion must be initialized.

Eg:

```
abstract class Vehicle {...
 Vehicle(String plate) { this.plate = plate; }
```

```

}
class Car extends Vehicle {
 Car() { super("KA-01"); }
}
// Abstract class cannot be instantiated directly, but its
// constructor is used during child object creation.

```

- Constructor rules?: If parent constructor takes (a, b) and child passes (a, b, c) → compile error. Constructor signatures must match exactly. Also as we know, constructor has no return type. And child class can be imagined as superset of parent as Child object contains: All parent fields; All parent methods; Its own fields and methods; It is one single object → parent + child portions.
- **Interface**: A pure behavior contract. Defines what a class must do, not how. Interfaces contain: abstract methods; default methods (Java 8+); static methods; constants (public static final). A class can implement multiple interfaces but extend only single class. (Hence multiple inheritance done via interface). Interface cannot hold common state (fields except constants). Interface = no identity / no shared data, only behavior. Interface cannot be instantiated. No constructors.
  - Use interface when: Unrelated classes need same behavior (Dog, Car, Drone can all be Movable); You want a pure contract; You need multiple inheritance of behavior; No shared fields or common state is needed.

```

Eg:
interface Drivable {
 void drive();
}
class Car implements Drivable {
 @Override
 public void drive() { System.out.println("Car driving"); }
}
Another Eg:

```

```
code
public interface PaymentGateway {
 void initiatePayment(double amount);
}
public class StripePayment implements PaymentGateway {
 public void initiatePayment(double amount) {
 System.out.println("Processing payment via Stripe: $" +
amount);
 }
}
public class RazorpayPayment implements PaymentGateway {
 public void initiatePayment(double amount) {
 System.out.println("Processing payment via Razorpay: ₹" +
amount);
 }
}
public class CheckoutService {
 private PaymentGateway paymentGateway;
 public CheckoutService(PaymentGateway paymentGateway) {
 this.paymentGateway = paymentGateway;
 }
 public void checkout(double amount) {
 paymentGateway.initiatePayment(amount);
 }
}
public class Main {
 public static void main(String[] args) {
 PaymentGateway stripeGateway = new StripePayment();
 CheckoutService service = new
CheckoutService(stripeGateway);
 service.checkout(120.50); // Output: Processing payment
via Stripe: $120.5
 // Switch to Razorpay
 PaymentGateway razorpayGateway = new RazorpayPayment();
 service.setPaymentGateway(razorpayGateway);
 service.checkout(150.50); // Output: Processing payment
via Razorpay: ₹150.5
 }
}
```

```
}
}
```

- A Combined/Mix Example:

```
abstract class Vehicle {
 String plate;
 int wheels;
 Vehicle(String plate, int wheels) {
 this.plate = plate;
 this.wheels = wheels;
 }
}
interface Drivable {
 void drive();
}
class Car extends Vehicle implements Drivable {
 Car(String plate) { super(plate, 4); }
 @Override
 public void drive() { System.out.println("Car driving"); }
}
```

- Misc stuff:

- Instant entry = Instant.now();
- UUID.randomUUID().toString()
- new Date().getTime()
- A final field must be assigned once and only once. Can be assigned during declaration time or inside constructor.  
Case: OK  
private final int x = 10; // Assigned at declaration  
Case: OK  
private final int x;  
public MyClass() {  
 x = 10; // Assigned in constructor  
}  
Case: FALSE

```

private final int x; // ✗ not assigned here
public MyClass() {
 // ✗ not assigned in constructor → compile error
}
- printf is the easiest way to format numbers, strings, and
 floats in Java -- just like C.
Integers: int n = 10; System.out.printf("Number=%d",n);
Float: double price = 12.346;
System.out.printf("Price=%f",price);
String: String name = "Suraj"; System.out.printf("Hi,%s",
name);
Char: System.out.printf("First letter: %c %n", 'A');
Escape sequences:
| Sequence | Meaning |
| ----- | ----- |
| `\\n` | new line (platform independent) |
| `\\%` | print a literal % |
| `\\t` | tab |
| `\\n` | new line (Unix) |

```

- **Enum:** It's a special data type that defines a collection of constant values under a single name. Are type-safe.

```

code
public enum VehicleSize {
 SMALL,
 MEDIUM,
 LARGE;
}
// Can access like: VehicleSize.MEDIUM -> Enum type. Other ways:
.name()/toString(): Returns the exact enum constant name as
written in the code. But toString() can be override to customize
display. Eg: VehicleSize.SMALL.name();
.ordinal(): Returns the index of the constant, based on
declaration order. Shouldn't be used in prod as if order changes
index changes. Safer option below: Eg:

```

```

public enum VehicleSize {
 SMALL(1),
 MEDIUM(2),
 LARGE(3);
 private final int level;
 VehicleSize(int level) {
 this.level = level;
 }
 public int getLevel() {
 return level;
 }
}

```

Eg: `VehicleSize.SMALL.getLevel()` can be used. Live eg., would look like:

```

spotA.getSpotSize().getLevel() < spotB.getSpotSize().getLevel()
/*

```

SMALL, MEDIUM, and LARGE are objects. Each enum constant is an object of type `VehicleSize` just like: `new VehicleSize(1)`. But you cannot call `new`. Java automatically creates these objects for you. When Java loads this enum, it does: `VehicleSize SMALL = new VehicleSize(1)`. Fundamentally, these are singletons.

Why can't I write `new VehicleSize(1)` myself?: Because enums are a special restricted class, etc.

```

*/

```

- **Encapsulation:**

- Data hiding + Controlled access, via: Access Modifiers - `private`, `public`, `protected` | Getters & Setters.

Encapsulation focuses on how to protect and control access to data within a class. But if we extend that idea not just hiding data, but also hiding complexity itself; That's where Abstraction comes in.

code

```

class PaymentProcessor {
 private String cardNumber;

```



```

private double amount;
public PaymentProcessor(String cardNumber, double amount) {
 this.cardNumber = maskCardNumber(cardNumber);
 this.amount = amount;
}
public void processPayment() {
 System.out.println("Processing payment of " + amount + "
for card " + cardNumber);
}
}
public class Main {
 public static void main(String[] args) {
 PaymentProcessor payment = new
PaymentProcessor("1234567812345678", 250.00);
 payment.processPayment();
 }
}

```

- **Abstraction:**

- Hiding Complexity + Showing Essentials. Achieved via abstract class or interfaces. Either of them cannot be instantiated. A class can extend only one abstract class but can implement multiple interfaces.

```

code
abstract class Vehicle {
 String brand;
 // Constructor
 Vehicle(String brand) {
 this.brand = brand;
 }
 // Abstract method (must be implemented by subclasses)
 abstract void start();
 // Concrete method (can be inherited)
 void displayBrand() {
 System.out.println("Brand: " + brand);
 }
}

```

```
// Subclass implementing the abstract method
class Car extends Vehicle {
 Car(String brand) {
 super(brand);
 }
 @Override
 void start() {
 System.out.println("Car is starting...");
 }
}
```

- **Inheritance:**

- Allows one class (called the subclass or child class) to inherit the properties and behaviors of another class (called the superclass or parent class).

```
code
class ElectricCar extends Car {
 public void chargeBattery() {
 System.out.println("Battery charging");
 }
}
```

- **Polymorphism:**

- Allows the same method name or interface to exhibit different behaviors depending on the object that is invoking it.

Compile-time Polymorphism (Static Binding), Also known as method overloading

```
code
public class Calculator {
 public int add(int a, int b) {
 return a + b;
 }
 public double add(double a, double b) {
 return a + b;
 }
}
```

```

 public int add(int a, int b, int c) {
 return a + b + c;
 }
 }
Runtime Polymorphism (Dynamic Binding), Also known as method
overriding
code
public interface NotificationSender {
 void sendNotification(String message);
}
public class EmailSender implements NotificationSender {
 public void sendNotification(String message) {
 System.out.println("Sending EMAIL: " + message);
 }
}
public class SmsSender implements NotificationSender {
 public void sendNotification(String message) {
 System.out.println("Sending SMS: " + message);
 }
}
public void notifyUser(NotificationSender sender, String message)
{
 sender.sendNotification(message);
}

```

## Class Relations: Association, Aggregation, Composition, Dependency

### Association

Definition: The most general and structural relationship, showing that instances of two classes are connected. It means one class is somehow related to another. No ownership is implied; classes are generally independent. Typically bidirectional (but can be unidirectional)

Dashed line in UML (---)

Example: Teacher Student Interaction

code

```

class Teacher {
 private String name;
 private List<Student> students; // Teacher knows students
 public void teach(Student student) {
 System.out.println(name + " teaching " + student.getName());
 }
}
class Student {
 private String name;
 private List<Teacher> teachers; // Student knows teachers
 public String getName() {
 return name;
 }
}
// Both Teacher and Student can exist independently

```

## Aggregation

Definition: A special form of Association that represents a "part-whole" or "has-a" relationship, but with weak ownership. The part (the contained object) can exist independently of the whole (the container object).

Hollow diamond in UML (◇---)

Example: Department has Employees

```

code
class Department {
 private String name;
 private List<Employee> employees; // Department has employees
 public Department(String name) {
 this.name = name;
 this.employees = new ArrayList<>();
 }
 public void addEmployee(Employee emp) {
 employees.add(emp);
 }
}
class Employee {
 private String name;

```

```

 private String id;
 public Employee(String name, String id) {
 this.name = name;
 this.id = id;
 }
}
// Usage:
Employee emp1 = new Employee("John", "E001");
Employee emp2 = new Employee("Jane", "E002");
Department dept = new Department("Engineering");
dept.addEmployee(emp1);
dept.addEmployee(emp2);
// Employees can exist even if department is destroyed

```

## Composition

Definition: A more restrictive and stronger form of Aggregation where the whole is solely responsible for the lifecycle of its part. The part cannot exist without the whole.

Filled diamond in UML (◆---)

Example: House owns Rooms

```

code
class House {
 private String address;
 private List<Room> rooms; // House owns rooms
 public House(String address) {
 this.address = address;
 this.rooms = new ArrayList<>();
 // Creating rooms inside constructor - strong ownership
 rooms.add(new Room("Living Room", 300));
 rooms.add(new Room("Bedroom", 200));
 rooms.add(new Room("Kitchen", 150));
 }
 // Inner class - Room cannot exist without House
 private class Room {
 private String name;
 private int squareFeet;
 public Room(String name, int squareFeet) {

```

```

 this.name = name;
 this.squareFeet = squareFeet;
 }
}
// When House object is garbage collected, all Room objects are
// destroyed too

```

## Dependency

Definition: The weakest form of relationship, indicating that one class uses another class, usually on a temporary basis, such as a local variable in a method or as a parameter to a method.

Dashed arrow in UML (---->)

No field reference maintained

```

code
class EmailService {
 // EmailService depends on EmailMsg and EmailServer
 public void sendEmail(EmailMsg message, EmailServer server) {
 // Uses EmailMessage & EmailServer but doesn't store them
 server.connect();
 server.send(message);
 server.disconnect();
 }
}
class EmailMessage {
 private String to;
 private String subject;
 private String body;
 // Constructor and getters
}
class EmailServer {
 public void connect() { }
 public void send(EmailMessage msg) { }
 public void disconnect() { }
}
// EmailService doesn't maintain references to EmailMsg or
// EmailServer

```

| Feature           | Association                | Aggregation<br>(Weak<br>"Has-A")               | Composition<br>(Strong<br>"Has-A")                 | Dependency<br>("Uses-A")                |
|-------------------|----------------------------|------------------------------------------------|----------------------------------------------------|-----------------------------------------|
| Relationship Type | General connection         | Part-of<br>(Optional)                          | Part-of<br>(Mandatory)                             | Temporary link (E.g., Method parameter) |
| Ownership         | None implied; just a link. | Weak ownership; the part is independent.       | Strong/Exclusive ownership; the part is dependent. | None; just temporary use.               |
| Part's Lifecycle  | Independent                | Independent                                    | Dependent (Part dies with the whole).              | Independent                             |
| Part Sharing      | Possible                   | Possible (Part can belong to multiple wholes). | Not possible (Part belongs to only one whole).     | Possible                                |

#### Design principles:

- Don't Repeat Yourself (DRY) (Every piece of knowledge should have a single, unambiguous representation in the system.)
- Keep It Simple Stupid (KISS) (Systems work best when they are kept simple rather than made complex.)
- You Aren't Gonna Need It (YAGNI) (Don't add functionality until it's actually needed.)
- Law of Demeter (LoD) / Principle of Least Knowledge (A method should only talk to its immediate friends, not strangers.)

## SOLID Principles:

### Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change. It should have only one job or responsibility.

BAD:

```
class Employee {
 private String name;
 private double salary;
 public double calculateSalary() { return salary * 12; } //
calc
 public void saveToDatabase() { /* DB logic */ } //
persist
 public String generateReport() { return name + salary; } //
report
 public void sendEmail(String msg) { /* email logic */ } //
notify
}
```

GOOD:

```
class Employee { String name; double salary; }
class SalaryCalculator {
 double calcAnnual(Employee e) { return e.salary * 12; }
}
class EmployeeRepository {
 void save(Employee e) { /* DB ops */ }
}
class ReportGenerator {
 String generate(Employee e) { return e.name + " " + e.salary;
}
}
class EmailService {
 void send(String to, String msg) { /* email logic */ }
}
```

### Open/Closed Principle (OCP)

Definition: Software entities should be open for extension but



closed for modification.

BAD:

```
class PaymentProcessor {
 public void processPayment(String paymentType, double amount)
 {
 if (paymentType.equals("CREDIT_CARD")) {
 // Credit card processing
 System.out.println("Processing credit card payment: "
+ amount);
 } else if (paymentType.equals("PAYPAL")) {
 // PayPal processing
 System.out.println("Processing PayPal payment: " +
amount);
 } else if (paymentType.equals("BITCOIN")) {
 // Bitcoin processing - had to modify this class!
 System.out.println("Processing Bitcoin payment: " +
amount);
 }
 // Every new payment method requires modifying this class
 }
}
```

GOOD:

```
// Open for extension, closed for modification
interface PaymentMethod {
 void processPayment(double amount);
}

class CreditCardPayment implements PaymentMethod {
 @Override
 public void processPayment(double amount) {
 System.out.println("Processing credit card payment: " +
amount);
 }
}

class PayPalPayment implements PaymentMethod {
 @Override
 public void processPayment(double amount) {
 System.out.println("Processing PayPal payment: " +
```

```

amount);
 }
}
// New payment method - just add new class, no modification needed
class BitcoinPayment implements PaymentMethod {
 @Override
 public void processPayment(double amount) {
 System.out.println("Processing Bitcoin payment: " +
amount);
 }
}
class PaymentProcessor {
 public void processPayment(PaymentMethod paymentMethod, double
amount) {
 paymentMethod.processPayment(amount);
 // No modification needed for new payment types!
 }
}
// Usage
PaymentProcessor processor = new PaymentProcessor();
processor.processPayment(new CreditCardPayment(), 100.0);
processor.processPayment(new BitcoinPayment(), 0.5);

```

## Liskov Substitution Principle (LSP)

Definition: Objects of a superclass should be replaceable with objects of its subclasses without breaking the application.

```

BAD:
class Rectangle {
 protected int width;
 protected int height;
 public void setWidth(int width) {
 this.width = width;
 }
 public void setHeight(int height) {
 this.height = height;
 }
}

```

```

 public int getArea() {
 return width * height;
 }
}
// Violates LSP!
class Square extends Rectangle {
 @Override
 public void setWidth(int width) {
 this.width = width;
 this.height = width; // Changes behavior!
 }
 @Override
 public void setHeight(int height) {
 this.width = height;
 this.height = height; // Changes behavior!
 }
}
// This breaks!
void testArea(Rectangle rect) {
 rect.setWidth(5);
 rect.setHeight(4);
 assert rect.getArea() == 20; // Fails for Square! Gets 16
}
GOOD:
interface Shape {
 int getArea();
}
class Rectangle implements Shape {
 private int width;
 private int height;
 public Rectangle(int width, int height) {
 this.width = width;
 this.height = height;
 }
 public void setWidth(int width) {
 this.width = width;
 }
}

```

```

 public void setHeight(int height) {
 this.height = height;
 }
 @Override
 public int getArea() {
 return width * height;
 }
 }
 class Square implements Shape {
 private int side;
 public Square(int side) {
 this.side = side;
 }
 public void setSide(int side) {
 this.side = side;
 }
 @Override
 public int getArea() {
 return side * side;
 }
 }
 // Now both can be used correctly
 void printArea(Shape shape) {
 System.out.println("Area: " + shape.getArea());
 }
}

```

## Interface Segregation Principle (ISP)

Definition: No client should be forced to depend on methods it doesn't use. Create specific interfaces rather than one general-purpose interface.

```

BAD:
// Fat interface - forces implementation of unused methods
interface Worker {
 void work();
 void eat();
 void sleep();
}

```

```
 void attendMeeting();
 void submitReport();
}
class HumanWorker implements Worker {
 @Override
 public void work() {
 System.out.println("Working...");
 }
 @Override
 public void eat() {
 System.out.println("Eating lunch...");
 }
 @Override
 public void sleep() {
 System.out.println("Sleeping...");
 }
 @Override
 public void attendMeeting() {
 System.out.println("Attending meeting...");
 }
 @Override
 public void submitReport() {
 System.out.println("Submitting report...");
 }
}
class RobotWorker implements Worker {
 @Override
 public void work() {
 System.out.println("Working 24/7...");
 }
 @Override
 public void eat() {
 // Forced to implement but doesn't make sense!
 throw new UnsupportedOperationException("Robots don't
eat");
 }
 @Override
```

```

 public void sleep() {
 // Forced to implement but doesn't make sense!
 throw new UnsupportedOperationException("Robots don't
sleep");
 }
 @Override
 public void attendMeeting() {
 throw new UnsupportedOperationException("Robots don't
attend meetings");
 }
 @Override
 public void submitReport() {
 System.out.println("Generating automated report...");
 }
}

```

GOOD:

```

// Segregated interfaces
interface Workable {
 void work();
}
interface Eatable {
 void eat();
}
interface Sleepable {
 void sleep();
}
interface Reportable {
 void submitReport();
}
interface Attendable {
 void attendMeeting();
}
// Human implements all
class HumanWorker implements Workable, Eatable, Sleepable,
Attendable, Reportable {
 @Override
 public void work() {

```

```

 System.out.println("Working...");
 }
 @Override
 public void eat() {
 System.out.println("Eating lunch...");
 }
 @Override
 public void sleep() {
 System.out.println("Sleeping...");
 }
 @Override
 public void attendMeeting() {
 System.out.println("Attending meeting...");
 }
 @Override
 public void submitReport() {
 System.out.println("Submitting report...");
 }
}
// Robot only implements what it needs
class RobotWorker implements Workable, Reportable {
 @Override
 public void work() {
 System.out.println("Working 24/7...");
 }
 @Override
 public void submitReport() {
 System.out.println("Generating automated report...");
 }
 // No need to implement eat, sleep, or attendMeeting!
}

```

## Dependency Inversion Principle (DIP)

Definition:

High-level modules should not depend on low-level modules. Both should depend on abstractions.

Abstractions should not depend on details. Details should depend on abstractions.

BAD:

```
// Low-level module
class MySQLDatabase {
 public void save(String data) {
 System.out.println("Saving to MySQL: " + data);
 }
}
// High-level module depends on low-level concrete class
class UserService {
 private MySQLDatabase database; // Tight coupling!
 public UserService() {
 this.database = new MySQLDatabase(); // Hard dependency
 }
 public void saveUser(String userData) {
 database.save(userData);
 // If we want to switch to PostgreSQL, we have to modify
 this class!
 }
}
```

GOOD:

```
// Abstraction
interface Database {
 void save(String data);
}
// Low-level modules implement abstraction
class MySQLDatabase implements Database {
 @Override
 public void save(String data) {
 System.out.println("Saving to MySQL: " + data);
 }
}
class PostgreSQLDatabase implements Database {
 @Override
 public void save(String data) {
 System.out.println("Saving to PostgreSQL: " + data);
 }
}
```



```

 }
}
class MongoDBDatabase implements Database {
 @Override
 public void save(String data) {
 System.out.println("Saving to MongoDB: " + data);
 }
}
// High-level module depends on abstraction
class UserService {
 private Database database; // Depends on abstraction!
 // Dependency injection
 public UserService(Database database) {
 this.database = database;
 }
 public void saveUser(String userData) {
 database.save(userData);
 // Can switch databases without modifying this class!
 }
}
// Usage
Database mysql = new MySQLDatabase();
UserService service1 = new UserService(mysql);
Database mongo = new MongoDBDatabase();
UserService service2 = new UserService(mongo);

```

## Design Patterns - Creational

### **\*\*Singleton Pattern**

Definition: Ensures a class has only one instance and provides a global point of access to it. Few ways; (Note: Singleton can be broken by reflection, serialization, and cloning. To protect it, we check instance in constructor, use readResolve(), and override clone())

 Not Thread-Safe (Lazy):

```

class Singleton {

```

```

private static Singleton instance;
private Singleton() {} // Private constructor
public static Singleton getInstance() {
 if (instance == null) {
 instance = new Singleton(); // Race condition!
 }
 return instance;
}
}

```

✓ Thread-Safe (Synchronized Method):

```

public class Singleton {
 private static Singleton instance;
 private Singleton() {}
 // Synchronized - thread safe but slow
 public static synchronized Singleton getInstance() {
 if (instance == null) {
 instance = new Singleton();
 }
 return instance;
 }
}

```

✓ Thread-Safe (Double-Checked Locking) - BEST for lazy initialization:

```

public class Singleton {
 // volatile ensures visibility across threads
 private static volatile Singleton instance;
 private Singleton() {}
 public static Singleton getInstance() {
 if (instance == null) { // First check (no locking)
 synchronized (Singleton.class) {
 if (instance == null) { // Second check (with
locking)
 instance = new Singleton();
 }
 }
 }
 return instance;
 }
}

```

```

 }
}
✓ Thread-Safe (Bill Pugh - Inner Static Class) - RECOMMENDED:
public class Singleton {
 private Singleton() {}
 // Inner static class - loaded only when getInstance() is
 called
 private static class SingletonHelper {
 private static final Singleton INSTANCE = new Singleton();
 }
 public static Singleton getInstance() {
 return SingletonHelper.INSTANCE;
 }
}
✓ Thread-Safe (Eager Initialization):
public class Singleton {
 // Created at class loading time
 private static final Singleton INSTANCE = new Singleton();
 private Singleton() {}
 public static Singleton getInstance() {
 return INSTANCE;
 }
}
✓ Enum Singleton - BEST for serialization safety:
public enum Singleton {
 INSTANCE;
 public void doSomething() {
 System.out.println("Doing something...");
 }
}
// Usage
Singleton.INSTANCE.doSomething();

```

## **\*\*Factory Method Pattern**

Definition: Defines an interface for creating objects, but lets subclasses decide which class to instantiate.

```
// Product
interface Notification {
 void send(String message);
}

// Concrete Products
class EmailNotification implements Notification {
 @Override
 public void send(String message) {
 System.out.println("Sending Email: " + message);
 }
}

class SMSNotification implements Notification {
 @Override
 public void send(String message) {
 System.out.println("Sending SMS: " + message);
 }
}

class PushNotification implements Notification {
 @Override
 public void send(String message) {
 System.out.println("Sending Push Notification: " +
message);
 }
}

// Factory
class NotificationFactory {
 public static Notification createNotification(String type) {
 switch (type.toUpperCase()) {
 case "EMAIL":
 return new EmailNotification();
 case "SMS":
 return new SMSNotification();
 case "PUSH":
 return new PushNotification();
 default:
 throw new IllegalArgumentException("Unknown
notification type");
 }
 }
}
```

```

 }
}
// Usage
Notification notification =
NotificationFactory.createNotification("EMAIL");
notification.send("Hello World!");

```

## **\*\*Builder Pattern**

Definition: Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

```

public class Employee {
 // Required fields
 private final String name;
 private final String department;
 // Optional fields
 private final int age;
 private final Map<String, String> preferences;
 // Private constructor
 private Employee(Builder builder) {
 this.name = builder.name;
 this.department = builder.department;
 this.age = builder.age;
 this.preferences = builder.preferences;
 }
 // Getters
 public String getName(){ return name; }
 public String getDepartment(){ return department; }
 public int getAge(){ return age; }
 public Map<String, String> getPreferences(){ return
preferences;}
 // Builder class
 public static class Builder {
 // Required
 private final String name;

```

```

 private final String department;
 // Optional
 private int age = 0;
 private Map<String, String> preferences = Map.of();
 // Constructor for required params
 public Builder(String name, String department) {
 this.name = name;
 this.department = department;
 }
 public Builder age(int age) {
 this.age = age;
 return this;
 }
 public Builder preferences(Map<String, String>
preferences) {
 this.preferences = preferences;
 return this;
 }
 public Employee build() {
 return new Employee(this);
 }
}
@Override
public String toString() {
 return "Employee{" +
 "name='" + name + '\'' +
 ", department='" + department + '\'' +
 ", age=" + age +
 ", preferences=" + preferences +
 '}';
}
// Example usage
public static void main(String[] args) {
 Employee emp1 = new Employee.Builder("Alice",
"Engineering").age(28).preferences(Map.of("Shift", "Morning",
"Laptop", "Mac")).build();
 Employee emp2 = new Employee.Builder("Bob", "HR").build();
}

```

```
}
}
```

## Abstract Factory Pattern

Definition: Provides an interface for creating families of related objects (like UI components, database connections, or devices) without specifying their concrete classes.

```
// Abstract Products
interface Button {
 void render();
}
interface Checkbox {
 void render();
}
// Concrete Products
class LightButton implements Button {
 public void render() { System.out.println("Light Button"); }
}
class DarkButton implements Button {
 public void render() { System.out.println("Dark Button"); }
}
class LightCheckbox implements Checkbox {
 public void render() { System.out.println("Light Checkbox"); }
}
class DarkCheckbox implements Checkbox {
 public void render() { System.out.println("Dark Checkbox"); }
}
// Abstract Factory
interface UIFactory {
 Button createButton();
 Checkbox createCheckbox();
}
// Concrete Factories
class LightUIFactory implements UIFactory {
 public Button createButton() { return new LightButton(); }
 public Checkbox createCheckbox() { return new LightCheckbox(); }
}
```

```

}
}
class DarkUIFactory implements UIFactory {
 public Button createButton() { return new DarkButton(); }
 public Checkbox createCheckbox() { return new DarkCheckbox(); }
}
}
// Client
public class AbstractFactoryDemo {
 public static void main(String[] args) {
 UIFactory factory = new DarkUIFactory(); // can switch to
 LightUIFactory easily
 Button btn = factory.createButton();
 Checkbox chk = factory.createCheckbox();
 btn.render();
 chk.render();
 }
}

```

## Prototype Pattern

Definition: Creates new objects by copying an existing object (prototype) rather than creating from scratch.

```

// import java.util.*;
// Prototype Interface
interface Prototype extends Cloneable {
 Prototype clone();
}
// Concrete Prototypes
class Circle implements Prototype {
 private int radius;
 public Circle(int radius) { this.radius = radius; }
 @Override
 public Prototype clone() {
 return new Circle(this.radius); // deep copy
 }
 public void draw() {

```



```

 System.out.println("Drawing Circle with radius " +
radius);
 }
}
class Rectangle implements Prototype {
 private int width, height;
 public Rectangle(int width, int height) {
 this.width = width; this.height = height;
 }
 @Override
 public Prototype clone() {
 return new Rectangle(this.width, this.height);
 }
 public void draw() {
 System.out.println("Drawing Rectangle " + width + "x" +
height);
 }
}
// Client
public class PrototypeDemo {
 public static void main(String[] args) {
 Prototype circle1 = new Circle(10);
 Prototype circle2 = circle1.clone(); // copy instead of
new
 Prototype rect1 = new Rectangle(5, 8);
 Prototype rect2 = rect1.clone();
 ((Circle) circle2).draw();
 ((Rectangle) rect2).draw();
 }
}

```

## Design Patterns - Structural

### **\*\*Adapter Pattern**

Definition: Converts the interface of a class into another interface

that clients expect. Allows classes with incompatible interfaces to work together.

```
// Target interface
interface MediaPlayer {
 void play(String fileName);
}

// Adaptee (incompatible interface)
class VLCPlayer {
 void playVlcFile(String fileName) {
 System.out.println("Playing VLC: " + fileName);
 }
}

// Adapter (composition)
class MediaAdapter implements MediaPlayer {
 private VLCPlayer vlc = new VLCPlayer();

 @Override
 public void play(String fileName) {
 vlc.playVlcFile(fileName); // adapts method name
 }
}

// Client
public class AdapterDemo {
 public static void main(String[] args) {
 MediaPlayer player = new MediaAdapter();
 player.play("song.vlc");
 }
}

// Here, MediaAdapter is tightly coupled to VlcPlayer (it knows
// exactly how to call it).
// But the client (AudioPlayer) only depends on the MediaPlayer
// interface -- not VlcPlayer or MediaAdapter.
// Inside the adapter class, there is a dependency on the adaptee
// -- but that coupling is localized, not spread to client code.
```

## **\*\*Facade Pattern**

Definition: Provides a unified, simplified interface to a complex subsystem. Makes the subsystem easier to use.

```
// Subsystem classes
class CPU {
 void start() { System.out.println("CPU starting..."); }
}
class Memory {
 void load() { System.out.println("Memory loading..."); }
}
class Disk {
 void read() { System.out.println("Disk reading data..."); }
}
// Facade
class ComputerFacade {
 private final CPU cpu = new CPU();
 private final Memory memory = new Memory();
 private final Disk disk = new Disk();

 public void startComputer() {
 System.out.println("Starting computer...");
 cpu.start();
 memory.load();
 disk.read();
 System.out.println("Computer ready!");
 }
}
// Client
public class FacadeDemo {
 public static void main(String[] args) {
 ComputerFacade computer = new ComputerFacade();
 computer.startComputer(); // One simple call
 }
}
```

## **\*\*Decorator Pattern**

Definition: Attaches additional responsibilities to an object dynamically. Provides a flexible alternative to subclassing for extending functionality.

```
// Base
interface Coffee {
 String getDescription();
 double getCost();
}

class SimpleCoffee implements Coffee {
 public String getDescription() { return "Simple Coffee"; }
 public double getCost() { return 2.0; }
}

// Decorator
class MilkDecorator implements Coffee {
 private Coffee coffee;
 public MilkDecorator(Coffee coffee) { this.coffee = coffee; }
 // Use @override for safety
 public String getDescription() { return
coffee.getDescription() + ", Milk"; }
 public double getCost() { return coffee.getCost() + 0.5; }
}

class SugarDecorator implements Coffee {
 private Coffee coffee;
 public SugarDecorator(Coffee coffee) { this.coffee = coffee; }
 // Use @override for safety
 public String getDescription() { return
coffee.getDescription() + ", Sugar"; }
 public double getCost() { return coffee.getCost() + 0.2; }
}

// Use
public class Main {
 public static void main(String[] args) {
 Coffee coffee = new SimpleCoffee(); // base
 // Memory: SimpleCoffee{ }
 coffee = new MilkDecorator(coffee); // add milk
 // Memory: MilkDecorator{coffee: SimpleCoffee{ }}
 }
}
```

```

 coffee = new SugarDecorator(coffee); // add sugar
 // Memory: SugarDecorator{ coffee: MilkDecorator{ coffee:
SimpleCoffee{ }}}
 System.out.println(coffee.getDescription()); // Simple
Coffee, Milk, Sugar
 System.out.println(coffee.getCost()); // 2.7
 }
}

```

## **\*\*Composite Pattern**

Definition: Composes objects into tree structures to represent part-whole hierarchies. Allows clients to treat individual objects and compositions uniformly.

Structure:

Component (interface)

└─ Leaf (File / Developer)

└─ Composite (Directory / Manager) → contains list of Components

```

interface FileSystem {
 void showDetails();
}
// Leaf
class File implements FileSystem {
 private String name;
 public File(String name) { this.name = name; }
 public void showDetails() { System.out.println("File: " +
name); }
}
// Composite
class Folder implements FileSystem {
 private String name;
 private List<FileSystem> items = new ArrayList<>();
 public Folder(String name) { this.name = name; }
 public void add(FileSystem item) { items.add(item); }
 public void showDetails() {
 System.out.println("Folder: " + name);
 for (FileSystem item : items) item.showDetails();
 }
}

```

```

 }
}
// Usage
public class CompositeDemo {
 public static void main(String[] args) {
 FileSystem file1 = new File("resume.pdf");
 FileSystem file2 = new File("photo.jpg");
 Folder docs = new Folder("Documents");
 docs.add(file1);
 docs.add(file2);
 docs.showDetails();
 }
}

```

## Proxy Pattern

Definition: Provides a surrogate or placeholder for another object to control access, add security, caching, or lazy loading without changing the original object.

```

// Subject Interface
interface Image {
 void display();
}
// Real Subject
class RealImage implements Image {
 private String filename;
 public RealImage(String filename) {
 this.filename = filename;
 loadFromDisk(); // heavy operation
 }
 private void loadFromDisk() {
 System.out.println("Loading image from disk: " +
filename);
 }
 @Override
 public void display() {
 System.out.println("Displaying image: " + filename);
 }
}

```

```

}
// Proxy
class ProxyImage implements Image {
 private RealImage realImage;
 private String filename;
 public ProxyImage(String filename) {
 this.filename = filename;
 }
 @Override
 public void display() {
 // Lazy loading - load only when needed
 if (realImage == null) {
 realImage = new RealImage(filename);
 }
 realImage.display();
 }
}
// Client
public class ProxyDemo {
 public static void main(String[] args) {
 Image image = new ProxyImage("photo.jpg");
 System.out.println("First display:");
 image.display(); // Loads + displays
 System.out.println("\nSecond display:");
 image.display(); // Cached, no reload
 }
}

```

## Bridge Pattern

Definition: Decouples an abstraction from its implementation so that the two can vary independently.

```

// Implementor
interface Device {
 void turnOn();
 void turnOff();
 boolean isEnabled();
}

```

```

}
// Concrete Implementors
class TV implements Device {
 private boolean on = false;
 public void turnOn() { on = true; System.out.println("TV
turned ON"); }
 public void turnOff() { on = false; System.out.println("TV
turned OFF"); }
 public boolean isEnabled() { return on; }
}
class Radio implements Device {
 private boolean on = false;
 public void turnOn() { on = true; System.out.println("Radio
turned ON"); }
 public void turnOff() { on = false; System.out.println("Radio
turned OFF"); }
 public boolean isEnabled() { return on; }
}
// Abstraction
abstract class Remote {
 protected Device device;
 protected Remote(Device device) {
 this.device = device;
 }
 public abstract void togglePower();
}
// Refined Abstraction
class AdvancedRemote extends Remote {
 public AdvancedRemote(Device device) {
 super(device);
 }
 @Override
 public void togglePower() {
 if (device.isEnabled()) device.turnOff();
 else device.turnOn();
 }
}
}

```



```
// Client
public class BridgeDemo {
 public static void main(String[] args) {
 Remote remote = new AdvancedRemote(new TV());
 remote.togglePower(); // TV ON
 remote.togglePower(); // TV OFF
 Remote radioRemote = new AdvancedRemote(new Radio());
 radioRemote.togglePower(); // Radio ON
 }
}
```

## Flyweight Pattern

Definition: Uses sharing to support large numbers of fine-grained objects efficiently. Reduces memory usage by sharing common state.

```
// import java.util.*;
// Flyweight
interface Shape {
 void draw(int x, int y);
}
// Concrete Flyweight
class Circle implements Shape {
 private String color; // intrinsic state (shared)
 public Circle(String color) {
 this.color = color;
 }
 @Override
 public void draw(int x, int y) {
 System.out.println("Drawing " + color + " circle at (" + x
+ ", " + y + ")");
 }
}
// Flyweight Factory
class ShapeFactory {
 private static final Map<String, Shape> circleMap = new
HashMap<>();
 public static Shape getCircle(String color) {
```

```

 return circleMap.computeIfAbsent(color, c -> {
 System.out.println("Creating new circle of color: " +
c);
 return new Circle(c);
 });
 }
}
// Client
public class FlyweightDemo {
 public static void main(String[] args) {
 String[] colors = {"Red", "Green", "Blue", "Red",
"Green"};

 for (String color : colors) {
 Shape circle = ShapeFactory.getCircle(color);
 circle.draw(new Random().nextInt(100), new
Random().nextInt(100));
 }
 }
}

```

## Design Patterns - Behavioral

### **\*\*Observer Pattern**

Definition: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.

```

// Observer interface
interface Subscriber {
 void update(String videoTitle);
}
// Subject interface
interface Channel {
 void subscribe(Subscriber subscriber);
 void unsubscribe(Subscriber subscriber);
 void notifySubscribers();
}

```

```

}
// Concrete Subject
class YouTubeChannel implements Channel {
 private List<Subscriber> subscribers = new ArrayList<>();
 private String latestVideo;
 @Override
 public void subscribe(Subscriber subscriber) {
 subscribers.add(subscriber);
 }
 @Override
 public void unsubscribe(Subscriber subscriber) {
 subscribers.remove(subscriber);
 }
 @Override
 public void notifySubscribers() {
 for (Subscriber s : subscribers) {
 s.update(latestVideo);
 }
 }
 public void uploadVideo(String title) {
 this.latestVideo = title;
 System.out.println("\nNew video uploaded: " + title);
 notifySubscribers();
 }
}

// Concrete Observers
class User implements Subscriber {
 private String name;
 public User(String name) {
 this.name = name;
 }
 @Override
 public void update(String videoTitle) {
 System.out.println(name + " got notified: New video - " +
videoTitle);
 }
}

```

```
// Usage
public class ObserverDemo {
 public static void main(String[] args) {
 YouTubeChannel channel = new YouTubeChannel();
 Subscriber user1 = new User("Alice");
 Subscriber user2 = new User("Bob");
 Subscriber user3 = new User("Charlie");
 channel.subscribe(user1);
 channel.subscribe(user2);
 channel.subscribe(user3);
 channel.uploadVideo("Design Patterns in Java");
 channel.unsubscribe(user2);
 channel.uploadVideo("Observer Pattern Simplified!");
 }
}

// Output
New video uploaded: Design Patterns in Java
Alice got notified: New video - Design Patterns in Java
Bob got notified: New video - Design Patterns in Java
Charlie got notified: New video - Design Patterns in Java
New video uploaded: Observer Pattern Simplified!
Alice got notified: New video - Observer Pattern Simplified!
Charlie got notified: New video - Observer Pattern Simplified!
```

## **\*\*Strategy Pattern**

Definition: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. The client can switch strategy (behavior) without changing its code.

```
// Strategy interface
interface TextFormatter {
 String format(String text);
}

// Concrete Strategies
class UpperCaseFormatter implements TextFormatter {
 public String format(String text) {
 return text.toUpperCase();
 }
}
```

```

}
class LowerCaseFormatter implements TextFormatter {
 public String format(String text) {
 return text.toLowerCase();
 }
}
class TitleCaseFormatter implements TextFormatter {
 public String format(String text) {
 return Arrays.stream(text.split(" "))
 .map(w -> Character.toUpperCase(w.charAt(0))
+ w.substring(1).toLowerCase())
 .collect(Collectors.joining(" "));
 }
}
// Context
class TextEditor {
 private TextFormatter formatter;
 public void setFormatter(TextFormatter formatter) {
 this.formatter = formatter;
 }
 public void publish(String text) {
 System.out.println("Formatted: " +
formatter.format(text));
 }
}
// Usage
public class StrategyDemo {
 public static void main(String[] args) {
 TextEditor editor = new TextEditor();
 editor.setFormatter(new UpperCaseFormatter());
 editor.publish("strategy pattern made easy");
 editor.setFormatter(new TitleCaseFormatter());
 editor.publish("strategy pattern made easy");
 }
}

```

## **\*\*Command Pattern**

Definition: Encapsulates a request as an object, allowing you to parameterize clients with different requests, queue requests, and support undoable operations.

```
// Command interface
interface Command {
 void execute();
}

// Receiver
class Light {
 void on() { System.out.println("Light is ON"); }
 void off() { System.out.println("Light is OFF"); }
}

// Concrete Commands
class LightOnCommand implements Command {
 private Light light;
 public LightOnCommand(Light light) { this.light = light; }
 public void execute() { light.on(); }
}

class LightOffCommand implements Command {
 private Light light;
 public LightOffCommand(Light light) { this.light = light; }
 public void execute() { light.off(); }
}

// Invoker
class RemoteButton {
 private Command command;
 public void setCommand(Command command) { this.command = command; }
 public void press() { command.execute(); }
}

// Usage
public class CommandDemo {
 public static void main(String[] args) {
 Light livingRoomLight = new Light();
 RemoteButton remote = new RemoteButton();
 remote.setCommand(new LightOnCommand(livingRoomLight));
 }
}
```

```

 remote.press(); // Light is ON
 remote.setCommand(new LightOffCommand(livingRoomLight));
 remote.press(); // Light is OFF
 }
}
// Key Points to Remember
// Command → encapsulates a request (execute() method).
// Receiver → performs the real action.
// Invoker → triggers the command (doesn't know how it's done).
// Undo feature → can be added by storing executed commands.

```

## **\*\*State Pattern**

Definition: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

```

// State interface
interface TrafficLightState {
 void handleRequest(TrafficLight light);
}
// Concrete States
class RedLight implements TrafficLightState {
 @Override
 public void handleRequest(TrafficLight light) {
 System.out.println("RED → Stop");
 light.setState(new GreenLight());
 }
}
class GreenLight implements TrafficLightState {
 @Override
 public void handleRequest(TrafficLight light) {
 System.out.println("GREEN → Go");
 light.setState(new YellowLight());
 }
}
class YellowLight implements TrafficLightState {
 @Override
 public void handleRequest(TrafficLight light) {
 System.out.println("YELLOW → Slow down");
 }
}

```

```

 light.setState(new RedLight());
 }
}
// Context
class TrafficLight {
 private TrafficLightState state;
 public TrafficLight() {
 state = new RedLight(); // initial state
 }
 public void setState(TrafficLightState state) {
 this.state = state;
 }
 public void change() {
 state.handleRequest(this);
 }
}
// Usage
public class StatePatternDemo {
 public static void main(String[] args) {
 TrafficLight light = new TrafficLight();

 // simulate cycles
 for (int i = 0; i < 5; i++) {
 light.change();
 }
 }
}

```

## **\*\*Chain of Responsibility Pattern**

Definition: Passes request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain.

```

// Handler interface
interface SupportHandler {
 void setNext(SupportHandler next);
 void handleRequest(String issue);
}

```



```
// Concrete Handlers
class BasicSupport implements SupportHandler {
 private SupportHandler next;
 @Override
 public void setNext(SupportHandler next) {
 this.next = next;
 }
 @Override
 public void handleRequest(String issue) {
 if (issue.equalsIgnoreCase("password reset")) {
 System.out.println("Basic Support: Handling password
reset.");
 } else if (next != null) {
 next.handleRequest(issue);
 } else {
 System.out.println("No handler available for: " +
issue);
 }
 }
}

class TechnicalSupport implements SupportHandler {
 private SupportHandler next;
 @Override
 public void setNext(SupportHandler next) {
 this.next = next;
 }
 @Override
 public void handleRequest(String issue) {
 if (issue.equalsIgnoreCase("network issue")) {
 System.out.println("Technical Support: Handling
network issue.");
 } else if (next != null) {
 next.handleRequest(issue);
 } else {
 System.out.println("No handler available for: " +
issue);
 }
 }
}
```

```

 }
}
class ManagerSupport implements SupportHandler {
 @Override
 public void setNext(SupportHandler next) {
 // Last handler - no next
 }
 @Override
 public void handleRequest(String issue) {
 System.out.println("Manager: Escalating " + issue + " to
higher authorities.");
 }
}
// Usage
public class ChainDemo {
 public static void main(String[] args) {
 SupportHandler basic = new BasicSupport();
 SupportHandler tech = new TechnicalSupport();
 SupportHandler manager = new ManagerSupport();
 basic.setNext(tech);
 tech.setNext(manager);
 basic.handleRequest("password reset");
 basic.handleRequest("network issue");
 basic.handleRequest("billing problem");
 }
}

```

## Iterator Pattern

Definition: Provides a way to access elements of a collection sequentially without exposing its underlying representation.

```

// import java.util.*;
// Iterator interface
interface Iterator<T> {
 boolean hasNext();
 T next();
}
// Aggregate interface

```

```

interface Container<T> {
 Iterator<T> getIterator();
}
// Concrete Collection
class NameCollection implements Container<String> {
 private String[] names = {"Alice", "Bob", "Charlie", "Diana"};
 @Override
 public Iterator<String> getIterator() {
 return new NameIterator();
 }
 // Concrete Iterator
 private class NameIterator implements Iterator<String> {
 int index = 0;
 @Override
 public boolean hasNext() {
 return index < names.length;
 }
 @Override
 public String next() {
 return hasNext() ? names[index++] : null;
 }
 }
}
// Usage
public class IteratorDemo {
 public static void main(String[] args) {
 NameCollection collection = new NameCollection();
 Iterator<String> iterator = collection.getIterator();

 while (iterator.hasNext()) {
 System.out.println(iterator.next());
 }
 }
}

```

## Template Method Pattern

Definition: Defines the skeleton of an algorithm in a base class,

letting subclasses override specific steps without changing the algorithm's structure.

```
// Abstract class (Template)
abstract class DataParser {
 // Template method (final to prevent override)
 public final void parseData() {
 readData();
 processData();
 saveData();
 }
 abstract void readData();
 abstract void processData();
 // Common step
 void saveData() {
 System.out.println("Saving processed data to database.");
 }
}

// Concrete classes
class CSVParser extends DataParser {
 void readData() {
 System.out.println("Reading data from CSV file.");
 }
 void processData() {
 System.out.println("Processing CSV data.");
 }
}

class JSONParser extends DataParser {
 void readData() {
 System.out.println("Reading data from JSON file.");
 }
 void processData() {
 System.out.println("Processing JSON data.");
 }
}

// Usage
public class TemplateDemo {
 public static void main(String[] args) {
```

```

 DataParser csvParser = new CSVParser();
 csvParser.parseData();
 System.out.println();
 DataParser jsonParser = new JSONParser();
 jsonParser.parseData();
 }
}

```

## Visitor Pattern

Definition: Lets you define a new operation without changing the classes of the elements on which it operates. Separates algorithms from the objects they operate on.

```

// Element interface
interface Shape {
 void accept(ShapeVisitor visitor);
}

// Concrete elements
class Circle implements Shape {
 double radius;
 Circle(double radius) { this.radius = radius; }
 public void accept(ShapeVisitor visitor) {
 visitor.visit(this);
 }
}

class Rectangle implements Shape {
 double width, height;
 Rectangle(double width, double height) {
 this.width = width;
 this.height = height;
 }
 public void accept(ShapeVisitor visitor) {
 visitor.visit(this);
 }
}

// Visitor interface
interface ShapeVisitor {
 void visit(Circle circle);
}

```

```

 void visit(Rectangle rectangle);
 }
 // Concrete visitors (operations)
 class AreaCalculator implements ShapeVisitor {
 public void visit(Circle circle) {
 double area = Math.PI * circle.radius * circle.radius;
 System.out.println("Circle area: " + area);
 }
 public void visit(Rectangle rectangle) {
 double area = rectangle.width * rectangle.height;
 System.out.println("Rectangle area: " + area);
 }
 }
 class ShapePrinter implements ShapeVisitor {
 public void visit(Circle circle) {
 System.out.println("Drawing a Circle with radius " +
circle.radius);
 }
 public void visit(Rectangle rectangle) {
 System.out.println("Drawing a Rectangle (" +
rectangle.width + "x" + rectangle.height + ")");
 }
 }
 // Usage
 public class VisitorDemo {
 public static void main(String[] args) {
 Shape[] shapes = {
 new Circle(5),
 new Rectangle(4, 6)
 };
 ShapeVisitor areaCalc = new AreaCalculator();
 ShapeVisitor printer = new ShapePrinter();
 for (Shape s : shapes) {
 s.accept(areaCalc);
 s.accept(printer);
 }
 }
 }

```

```
}
```

## Mediator Pattern

Definition: Defines an object that encapsulates how a set of objects interact. Promotes loose coupling by keeping objects from referring to each other explicitly.

```
// Mediator interface
interface ChatMediator {
 void showMessage(User user, String message);
}
// Concrete Mediator
class ChatRoom implements ChatMediator {
 public void showMessage(User user, String message) {
 System.out.println(user.getName() + ": " + message);
 }
}
// Colleague class
class User {
 private String name;
 private ChatMediator mediator;
 public User(String name, ChatMediator mediator) {
 this.name = name;
 this.mediator = mediator;
 }
 public String getName() { return name; }
 public void send(String message) {
 mediator.showMessage(this, message);
 }
}
// Usage
public class MediatorDemo {
 public static void main(String[] args) {
 ChatMediator chatRoom = new ChatRoom();

 User alice = new User("Alice", chatRoom);
 User bob = new User("Bob", chatRoom);
 }
}
```

```

 alice.send("Hi Bob!");
 bob.send("Hey Alice, how are you?");
 }
}

```

## Memento Pattern

Definition: Captures and externalizes an object's internal state without violating encapsulation, allowing the object to be restored to this state later.

```

// Memento - stores state
class TextMemento {
 private final String state;
 public TextMemento(String state) { this.state = state; }
 public String getState() { return state; }
}

// Originator - creates/restores memento
class TextEditor {
 private String content = "";
 public void type(String words) { content += words; }
 public String getContent() { return content; }
 public TextMemento save() { return new TextMemento(content); }
 public void restore(TextMemento memento) { content =
memento.getState(); }
}

// Caretaker - manages mementos
class EditorHistory {
 private final java.util.Stack<TextMemento> history = new
java.util.Stack<>();
 public void save(TextEditor editor) {
history.push(editor.save()); }
 public void undo(TextEditor editor) {
 if (!history.isEmpty()) editor.restore(history.pop());
 }
}

// Usage
public class MementoDemo {
 public static void main(String[] args) {

```



```
 TextEditor editor = new TextEditor();
 EditorHistory history = new EditorHistory();
 editor.type("Hello ");
 history.save(editor);
 editor.type("World!");
 System.out.println(editor.getContent()); // Hello World!
 history.undo(editor);
 System.out.println(editor.getContent()); // Hello
 }
}
```

---

HLD

## HLD

Good HLD Reference:

- <https://github.com/ashishps1/awesome-system-design-resources?tab=readme-ov-file#-system-design-interview-problems>
- [https://www.youtube.com/watch?v=Qi3Kzq\\_rlKQ&list=PLjTveVh7FakLGZ36GbWak\\_DMf\\_0xBZpGv&index=9](https://www.youtube.com/watch?v=Qi3Kzq_rlKQ&list=PLjTveVh7FakLGZ36GbWak_DMf_0xBZpGv&index=9)
- GPT

## 1. NETWORKING FUNDAMENTALS

### IP Addressing

- **IPv4:** 32-bit addresses (4 billion addresses), format: 192.168.1.1
- **IPv6:** 128-bit addresses (340 undecillion addresses), format: 2001:0db8:85a3::8a2e:0370:7334
- **Types:** Public (internet-facing), Private (internal network), Static (fixed), Dynamic (DHCP-assigned)

### OSI Model (7 Layers)

- **Application (L7):** User-facing protocols (HTTP, SMTP, FTP)
- **Presentation (L6):** Data translation, encryption, compression
- **Session (L5):** Connection management between applications
- **Transport (L4):** End-to-end communication (TCP/UDP)
- **Network (L3):** Routing, IP addressing
- **Data Link (L2):** Frame transmission, MAC addressing
- **Physical (L1):** Physical hardware, cables, signals

### TCP vs UDP

#### TCP (Transmission Control Protocol)

- Connection-oriented with 3-way handshake
- Reliable delivery with acknowledgments
- Ordered packets, flow control, congestion control

- Higher overhead, slower
- Use cases: HTTP, HTTPS, FTP, email

### **UDP (User Datagram Protocol)**

- Connectionless, no handshake
- No delivery guarantee, packets may arrive out of order
- Lower overhead, faster
- Use cases: Video streaming, VoIP, gaming, DNS

### **Domain Name System (DNS)**

Translates domain names to IP addresses through hierarchical lookup.

#### **DNS Hierarchy:**

1. Recursive Resolver (ISP DNS)
2. Root Nameserver (13 global root servers)
3. TLD Nameserver (.com, .org, .net)
4. Authoritative Nameserver (actual IP)

#### **DNS Records:**

- **A Record:** Domain → IPv4
- **AAAA Record:** Domain → IPv6
- **CNAME:** Alias → Another domain
- **MX Record:** Mail server routing
- **NS Record:** Nameserver for domain
- **TXT Record:** Text info (SPF, DKIM)

**DNS Caching:** TTL-based caching at browser, OS, and resolver levels

---

## **2. SCALABILITY & PERFORMANCE**

### **Performance vs Scalability**

- **Performance Problem:** System slow for single user

- **Scalability Problem:** System fast for one user but slow under load
- Goal: Maximize throughput with acceptable latency

### Latency vs Throughput

- **Latency:** Time to perform an action (response time)
- **Throughput:** Number of actions per unit time (requests/sec)
- Trade-off: Higher throughput may increase latency

### Vertical vs Horizontal Scaling

#### Vertical Scaling (Scale Up)

- Add more power (CPU, RAM) to existing machine
- Pros: Simple, no code changes, data consistent
- Cons: Hardware limits, single point of failure, expensive, downtime

#### Horizontal Scaling (Scale Out)

- Add more machines to pool
- Pros: No limits, fault tolerant, cost-effective
- Cons: Complex, data consistency challenges, network overhead

---

## 3. LOAD BALANCING

Distributes incoming traffic across multiple servers.

### Algorithms

- **Round Robin:** Sequential distribution
- **Weighted Round Robin:** Based on server capacity
- **Least Connections:** Send to server with fewest active connections
- **Least Response Time:** Fastest response + fewest connections
- **IP Hash:** Based on client IP (sticky sessions)

### Layer 4 vs Layer 7

### Layer 4 (Transport Layer)

- Routes based on IP + Port (no content inspection)
- Faster, less resource intensive
- Simple NAT forwarding

### Layer 7 (Application Layer)

- Routes based on content (headers, cookies, URL path)
- Content-based routing, SSL termination
- More features but higher overhead

### High Availability

- **Active-Passive:** Standby server takes over on failure
  - **Active-Active:** All servers handle traffic simultaneously
- 

## 4. CACHING

Stores frequently accessed data in fast storage (RAM) to reduce latency.

### Cache Levels

- **Client-side:** Browser cache, OS cache
- **CDN:** Geographically distributed edge servers
- **Web Server:** Reverse proxy cache (Varnish, Nginx)
- **Application:** In-memory cache (Redis, Memcached)
- **Database:** Query cache, buffer pool

### Cache Strategies

#### Cache-Aside (Lazy Loading)

- App checks cache first, then DB on miss
- App responsible for loading cache
- Pros: Only requested data cached

- Cons: Cache miss penalty (3 trips), stale data possible

### **Write-Through**

- Write to cache and DB synchronously
- Pros: No stale data, cache always consistent
- Cons: Higher write latency, unused data cached

### **Write-Behind (Write-Back)**

- Write to cache, async write to DB
- Pros: Fast writes, high throughput
- Cons: Data loss risk if cache crashes

### **Refresh-Ahead**

- Automatically refresh before expiration
- Pros: Reduced latency for predicted data
- Cons: Wasteful if predictions wrong

### **Cache Eviction Policies**

- **LRU (Least Recently Used)**: Remove oldest unused
- **LFU (Least Frequently Used)**: Remove least accessed
- **FIFO**: Remove first added
- **Random**: Random removal

### **Cache Invalidation**

- **TTL (Time To Live)**: Expiration-based
- **Event-based**: Invalidate on data change
- **Manual**: Explicit deletion

---

## **5. CONTENT DELIVERY NETWORK (CDN)**

Globally distributed servers serving content from locations closest to users.

## Push CDN

- Content uploaded to CDN when changed
- Pros: Low latency, control over updates
- Cons: Higher storage costs, manual management
- Use: Low traffic sites, infrequent updates

## Pull CDN

- CDN fetches content on first user request
  - Pros: Minimal storage, automatic updates
  - Cons: Slower first request, redundant traffic
  - Use: High traffic sites, frequent changes
- 

## 6. PROXY SERVERS

**Forward Proxy:** Sits between clients and internet, makes requests on behalf of clients.

- **Uses:** Content filtering, anonymity, bypass restrictions, caching

**Reverse Proxy:** Sits between internet and servers, receives client requests.

- **Benefits:** Load balancing, SSL termination, caching, security, compression
  - **Examples:** Nginx, HAProxy
- 

## 7. AVAILABILITY & RELIABILITY

### Availability Metrics

- **99% (2 nines):** 3.65 days downtime/year
- **99.9% (3 nines):** 8.77 hours downtime/year
- **99.99% (4 nines):** 52.6 minutes downtime/year
- **99.999% (5 nines):** 5.25 minutes downtime/year



## Availability Patterns

### Failover

- **Active-Passive:** Heartbeat monitoring, passive takes over on failure
- **Active-Active:** Both servers handle traffic, better resource utilization

### Replication

- Multiple copies across servers for redundancy
- Improves availability and read performance

**Components in Sequence:** Total availability =  $A \times B$  (decreases) **Components in Parallel:** Total availability =  $1 - (1-A)(1-B)$  (increases)

---

## 8. CONSISTENCY MODELS

### CAP Theorem

In distributed systems, choose 2 of 3:

- **Consistency:** All nodes see same data simultaneously
- **Availability:** Every request gets response
- **Partition Tolerance:** System works despite network failures

**CP Systems:** Consistency + Partition Tolerance (MongoDB, HBase, Redis)

**AP Systems:** Availability + Partition Tolerance (Cassandra, CouchDB, DynamoDB)

**CA Systems:** Not realistic in distributed systems (traditional RDBMS in single node)

### PACELC Theorem

Extension of CAP: If Partition, choose A or C; Else choose Latency or Consistency

### Consistency Patterns

- **Weak Consistency:** No guarantee when data will be visible (VoIP, gaming)
  - **Eventual Consistency:** Data will be consistent eventually (DNS, email, S3)
  - **Strong Consistency:** Immediate consistency, reads always get latest (RDBMS, file systems)
- 

## 9. DATABASES - SQL

Structured data with predefined schema, ACID transactions.

### ACID Properties

- **Atomicity:** All or nothing transaction
- **Consistency:** Valid state transitions only
- **Isolation:** Concurrent transactions don't interfere
- **Durability:** Committed data persists

### Database Scaling Techniques

#### Master-Slave Replication

- Master: Handles writes + reads
- Slaves: Handle reads only (replicate from master)
- Pros: Scalable reads, backups
- Cons: Replication lag, master failure downtime

#### Master-Master Replication

- Both handle reads + writes
- Pros: Better write scalability, no single point
- Cons: Conflict resolution, consistency issues

#### Federation (Functional Partitioning)

- Split by function (users DB, products DB, orders DB)
- Pros: Less traffic per DB, more cache hits
- Cons: Complex joins across DBs, not for large tables

## Sharding (Horizontal Partitioning)

- Split data across multiple databases by key
- Strategies: Hash-based, List-based, Range-based
- Pros: Horizontal scalability, better performance
- Cons: Complex queries, rebalancing difficult, no joins

## Denormalization

- Add redundant data to avoid expensive joins
- Pros: Faster reads
- Cons: Data duplication, complex writes

## Indexing

- B-tree indexes for fast lookups
- Speeds up SELECT, WHERE, JOIN, ORDER BY
- Tradeoff: Faster reads, slower writes, more storage

## Normalization (1NF, 2NF, 3NF, BCNF)

- Remove data redundancy
  - Ensures data integrity
  - Reduces anomalies (insert, update, delete)
  - May require more joins
- 

## 10. DATABASES - NoSQL

Schema-less, horizontally scalable, BASE model.

### BASE Properties

- **Basically Available:** System guarantees availability
- **Soft State:** State may change over time
- **Eventual Consistency:** Consistency achieved eventually

## **NoSQL Types**

### **Key-Value Store**

- Simple hash table,  $O(1)$  reads/writes
- Examples: Redis, DynamoDB, Memcached
- Use: Caching, session storage, user preferences

### **Document Store**

- JSON/XML documents with flexible schema
- Examples: MongoDB, CouchDB, Firestore
- Use: Content management, catalogs, user profiles

### **Column-Family Store**

- Data stored in column families, not rows
- Examples: Cassandra, HBase, Google Bigtable
- Use: Time-series data, analytics, big data

### **Graph Database**

- Nodes and relationships
- Examples: Neo4j, Amazon Neptune
- Use: Social networks, recommendation engines, fraud detection

## **SQL vs NoSQL Decision**

### **Choose SQL when:**

- Structured data, strict schema needed
- Complex joins required
- ACID transactions critical
- Well-established patterns

### **Choose NoSQL when:**

- Massive scale (TB/PB data)
- Flexible/dynamic schema

- High throughput needed
  - Horizontal scaling required
  - Simple access patterns
- 

## 11. DISTRIBUTED SYSTEMS CONCEPTS

### Consistent Hashing

Distributes keys across nodes with minimal redistribution when nodes added/removed.

#### Process:

1. Hash nodes and keys onto ring (0 to  $2^{32}$ )
2. Key goes to first node clockwise
3. Adding/removing node only affects adjacent keys

**Virtual Nodes:** Each physical node mapped multiple times to improve load distribution

**Benefits:** Minimal redistribution, better load balance, handles failures gracefully

### Database Transactions

**Transaction States:** Active → Partially Committed → Committed/Failed → Aborted/Terminated

#### Distributed Transactions:

##### Two-Phase Commit (2PC)

- Phase 1: Prepare (all nodes vote)
- Phase 2: Commit (if all yes) or Abort
- Problem: Blocking protocol, coordinator failure

##### Three-Phase Commit (3PC)

- Adds pre-commit phase between prepare and commit
- Non-blocking, handles coordinator failure better

### **Saga Pattern**

- Sequence of local transactions
  - Choreography: Event-driven
  - Orchestration: Central coordinator
  - Compensating transactions for rollback
- 

## **12. ARCHITECTURAL PATTERNS**

### **N-Tier Architecture**

Separates application into logical layers and physical tiers.

#### **Common Tiers:**

- **3-Tier:** Presentation → Business Logic → Data
- **2-Tier:** Client → Server (no middle tier)
- **1-Tier:** All on single machine

**Pros:** Scalable layers independently, better security, separation of concerns

**Cons:** Network latency, complexity, expensive

### **Monolith vs Microservices**

#### **Monolithic**

- Single deployable unit, tightly coupled
- Pros: Simple development/deployment, easy debugging, ACID transactions
- Cons: Hard to scale, technology lock-in, single point of failure

#### **Microservices**

- Independent services, loosely coupled, separate databases

- Pros: Independent scaling/deployment, technology flexibility, fault isolation
- Cons: Complex, distributed system challenges, eventual consistency

**Distributed Monolith:** Looks like microservices but tightly coupled (worst of both)

## **Event-Driven Architecture (EDA)**

Services communicate through events via message broker.

### **Components:**

- Event Producers: Publish events
- Event Router: Message broker (Kafka, RabbitMQ)
- Event Consumers: Subscribe and react

**Pros:** Loose coupling, scalable, easy to add consumers **Cons:** Complex, difficult error handling, eventual consistency

## **Event Sourcing**

Store events (state changes) instead of current state.

- Append-only event log
- Rebuild state by replaying events
- Complete audit trail
- Use: Financial systems, audit requirements

## **CQRS (Command Query Responsibility Segregation)**

Separate read and write models.

- Commands: Change state (writes)
- Queries: Read state (reads)
- Different databases for reads/writes
- Often used with Event Sourcing

**Pros:** Independent scaling, optimized models **Cons:** Complexity, eventual

consistency, more infrastructure

---

## 13. API DESIGN & COMMUNICATION

### REST (Representational State Transfer)

- Stateless, resource-based (nouns)
- HTTP methods: GET (read), POST (create), PUT (update/replace), PATCH (partial update), DELETE
- Status codes: 2xx (success), 3xx (redirect), 4xx (client error), 5xx (server error)

#### Principles:

- Uniform interface
- Stateless
- Cacheable
- Client-server separation
- Layered system

### GraphQL

- Query language for APIs
- Client specifies exact data needed
- Single endpoint, no over/under-fetching
- Pros: Flexible, efficient, strong typing
- Cons: Complexity, caching harder, learning curve

### gRPC

- Remote Procedure Call using Protocol Buffers
- HTTP/2 based, binary protocol
- Strong typing, code generation
- Pros: Fast, efficient, bi-directional streaming
- Cons: Not browser-friendly, binary format harder to debug



## RPC vs REST

- **RPC:** Action-oriented (verbs), tight coupling, faster
  - **REST:** Resource-oriented (nouns), loose coupling, standard HTTP
- 

## 14. MESSAGE-DRIVEN ARCHITECTURE

### Message Queue

Point-to-point (1:1), FIFO delivery, each message consumed once.

- **Use:** Task distribution, decoupling, load leveling
- **Examples:** RabbitMQ, Amazon SQS, ActiveMQ

#### Features:

- Dead letter queues
- Message persistence
- Priority queues
- Exactly-once delivery

### Publish-Subscribe (Pub/Sub)

One-to-many (1:N), messages pushed to all subscribers.

- **Use:** Event notification, fanout, real-time updates
- **Examples:** Apache Kafka, Google Pub/Sub, Redis Pub/Sub

#### Benefits:

- Dynamic subscribers
- Decoupled publishers/subscribers
- Message filtering

### Message Broker

Intermediary for message exchange, translation between protocols.

- Validates, stores, routes messages
- Supports both queue and pub/sub patterns

## **Enterprise Service Bus (ESB)**

Centralized integration hub for applications.

- Transformation, routing, orchestration
  - Pros: Standardized integration
  - Cons: Single point of failure, complexity, bottleneck
- 

## **15. API GATEWAY & BFF**

### **API Gateway**

Single entry point for all clients, handles cross-cutting concerns.

#### **Responsibilities:**

- Authentication & authorization
- Rate limiting & throttling
- Load balancing
- Request/response transformation
- Caching
- Logging & monitoring
- Service discovery
- Circuit breaking

**Pros:** Simplified client, centralized logic, security **Cons:** Single point of failure, complexity, latency

### **Backend for Frontend (BFF)**

Separate backend for each client type (web, mobile, IoT).

- Optimized for specific frontend needs

- Formats data appropriately
  - Reduces client-side logic
- 

## 16. RELIABILITY PATTERNS

### Circuit Breaker

Prevents cascading failures by stopping calls to failing service.

**States:**

- **Closed:** Normal operation
- **Open:** Requests fail immediately (service down)
- **Half-Open:** Test if service recovered

### Rate Limiting

Controls request rate to prevent abuse and overload.

**Algorithms:**

- **Token Bucket:** Tokens refill at constant rate
- **Leaky Bucket:** Requests leak at constant rate
- **Fixed Window:** Counter per time window
- **Sliding Window:** Overlapping windows
- **Sliding Window Log:** Precise but memory-intensive

### Bulkhead Pattern

Isolate resources to prevent total system failure.

- Separate connection pools per service
- Limit threads per operation
- Prevents resource exhaustion

### Service Discovery

Automatically detect service locations in dynamic environments.

- **Client-side:** Client queries registry (Consul, Eureka)
  - **Server-side:** Load balancer queries registry
  - Health checks, automatic registration/deregistration
- 

## 17. STORAGE SYSTEMS

### Storage Types

#### File Storage

- Hierarchical directory structure
- Examples: NFS, Amazon EFS
- Use: Shared file access

#### Block Storage

- Raw storage blocks, fixed size
- Examples: Amazon EBS, SAN
- Use: Databases, VMs

#### Object Storage

- Flat namespace, metadata-rich
- Examples: Amazon S3, Azure Blob
- Use: Unstructured data, backups, media

#### RAID (Redundant Array of Independent Disks)

- **RAID 0:** Striping (performance, no redundancy)
- **RAID 1:** Mirroring (redundancy, 50% capacity)
- **RAID 5:** Striping + parity (balance)
- **RAID 6:** Striping + double parity (2 disk failures)
- **RAID 10:** Striping + mirroring (best performance + redundancy)

## **Distributed File Systems**

### **HDFS (Hadoop Distributed File System)**

- Block-based, replication for fault tolerance
  - Write-once, read-many
  - Use: Big data processing
- 

## **18. ASYNC PROCESSING**

### **Long Polling**

Client requests, server holds until data available.

- Less overhead than polling
- Still blocking connection

### **WebSockets**

Full-duplex, persistent connection over single TCP.

- Real-time, bi-directional communication
- Use: Chat, gaming, live updates

### **Server-Sent Events (SSE)**

Server pushes updates to client over HTTP.

- Unidirectional (server → client)
  - Automatic reconnection
  - Use: Live feeds, notifications
- 

## **19. SECURITY**

### **Best Practices**

- **Encryption:** TLS in transit, AES at rest
  - **Authentication:** OAuth 2.0, JWT, API keys
  - **Authorization:** RBAC, ABAC
  - **Input Validation:** Prevent XSS, SQL injection
  - **Rate Limiting:** Prevent DoS
  - **Least Privilege:** Minimal necessary permissions
  - **Security Headers:** CORS, CSP, HSTS
- 

## 20. IMPORTANT NUMBERS

### Latency Numbers

- L1 cache: 0.5 ns
- L2 cache: 7 ns
- RAM: 100 ns
- SSD random read: 150  $\mu$ s
- SSD sequential read: 1 ms
- HDD seek: 10 ms
- Network same datacenter: 0.5 ms
- Network CA  $\rightarrow$  Netherlands: 150 ms

### Capacity Estimates

- 1 KB =  $10^3$  bytes
- 1 MB =  $10^6$  bytes
- 1 GB =  $10^9$  bytes
- 1 TB =  $10^{12}$  bytes
- 1 PB =  $10^{15}$  bytes

### Throughput

- RAM: 4 GB/s sequential
- SSD: 1 GB/s sequential
- HDD: 30 MB/s sequential
- 1 Gbps network: 100 MB/s

# InterviewQsLearnings

## Interview Qs asked:

- (Some more job interview question/assignments solved saved in drive)

---

### Problem: API Marketplace Analytics Backend

We are building the backend for a cloud-based API Marketplace (similar to RapidAPI or AWS API Gateway) where developers can discover and use third-party APIs.

Let's start with the following functional requirements:

-> registerApi(apiName [String]) -> An API is given an incremental integer ID when it is registered, starting with 1.

-> invokeApi(apiId [Integer], developerId [Integer]) -> Assume any developer ID is valid and that the given API ID will have already been registered. -> This represents a developer successfully making a call to an API.

-> printAnalyticsSummary() -> This is used for an end-of-day billing and analytics report. -> The summary should be sorted (descending) by the number of unique developers who have invoked each API. -> The summary should include the API names and the number of unique developers, but the exact string formatting does not matter.

-> printLastThreeInvokedApis(developerId [Integer]) -> Prints the names of the last three unique APIs the given developer has invoked. -> Must be ordered with the most recently invoked API first.

---

### Objective:

Design and implement a rate limiter using Erlang's GenServer that restricts the number of allowed requests from a given user (identified by a unique ID) within a specified time window.

### Requirements:

#### 1. Rate Limit Logic:

- Each user is allowed a maximum of 5 requests within a 10 second window.
  - If the user exceeds the limit, additional requests should be rejected until the window resets.
-



Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

TimeMap() Initializes the object of the data structure.

void set(String key, String value, int timestamp) Stores the key key with the value value at the given time timestamp.

All the timestamps timestamp of set call are strictly increasing.

String get(String key, int timestamp) Returns a value such that set was called previously, with timestamp\_prev <= timestamp. If there are multiple such values, it returns the value associated with the largest timestamp\_prev. If there are no values, it returns "".

Example 1:

Input

```
["TimeMap", "set", "get", "get", "set", "get", "get"]
```

```
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
```

Output

```
[null, null, "bar", "bar", null, "bar2", "bar2"]
```

```
map<string, vector<pair<string, int>>> mp
```

Explanation

```
TimeMap timeMap = new TimeMap();
```

```
timeMap.set("foo", "bar", 1); // store the key "foo" and value "bar" along with timestamp = 1.
```

```
timeMap.get("foo", 1); // return "bar"
```

```
timeMap.get("foo", 3); // return "bar", since there is no value corresponding to foo at timestamp 3 and timestamp 2, then the only value is at timestamp 1 is "bar".
```

```
timeMap.set("foo", "bar2", 4); // store the key "foo" and value "bar2" along with timestamp = 4.
```

```
timeMap.get("foo", 4); // return "bar2"
```

```
timeMap.get("foo", 5); // return "bar2"
```

SOLN:

```
class TimeMap {
```

```

unordered_map<string, vector<pair<string,int>> timeDataSet;
void set(string key, string value, int time){
 timeDataSet[key].push_back({value, time});
}
void get(string key, int time){
 if(timeDataSet[key]==timeDataSet.end()) return NULL;
 vector<pair<int,int>> keyValue = timeDataSet[key]
 int size = keyValue.size();
 int lowValue = 0; int highValue = size-1;
 int targetTime = time;
 int globalLow = INT_MIN;
 while(lowValue<=highValue){
 int midValue = lowValue + (highValue-lowValue)/2
 pair<int,int> tempPair = timeDataSet[mid]
 if(tempPair.second == targetTime){
 return tempPair.first;
 }
 if(tempPair.second > target){
 highValue = midValue;
 }
 else{
 lowValue = midValue;
 globalLow = lowValue;
 }
 }
 if(globalLow!=INT_MIN){
 return vector[globalLow].first;
 }
}
};

```

Time complexity for get() -> O(1) since inserting in the Map

Time complexity for set() -> O(1) + O(LogN) -> Since using map and binary search approach

---

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night. Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).

Total amount you can rob =  $1 + 3 = 4$ .

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).

Total amount you can rob =  $2 + 9 + 1 = 12$ .

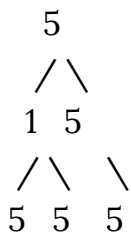
`[2,7,9,3,1]`

---

A subtree is univalue if:

All nodes in the subtree have the same value.

AND its left and right subtrees (if exist) are also univalue and match the current node's value. Your task is to write a function that counts how many univalue subtrees are present in a given binary tree



Input: `root = [5,1,5,5,5,null,5]`

Output: 4

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

Example 1:

Input: nums = [100,4,200,1,3,2]

Output: 4

Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.

Example 2:

Input: nums = [0,3,7,2,5,8,4,6,0,1]

Output: 9

Example 3:

Input: nums = [1,0,1,2]

Output: 3

---

Given two words w1 and w2. The baby can either erase a word or write a new word,

what is the minimum no of action he will perform to make these 2 words same.

S1 = "heap", s2 = "pea"

Output: 3

P and h deleted - 2 operation

P added in prior - 1 operation

---

You are building spotify shuffle feature

You have been given a playlist of songs

List songs = [s1,s2,s3,s4];

You have to play songs in shuffled manner in such a way that a song should be played again only after all other songs have been played.

---

Design a campaign management system (eg: samvad - multi tenant).

There could be 1000s of organizations, and each organization can have millions of users (scale).

Organizations could be having enterprise plan or other tier plans.

Organizations will send attributes related to user and campaign data.

And will tell, say campaign needs (schedule campaign) to run from 11 Dec to 16 Dec from 10am-5pm. Then the campaign would be sent to the user via whatsapp/telegram (basically some API call), etc. Also have SLAs, and check for throttling related areas. How to set up observability?. And 1 campaign should not affect other campaigns.

---

Leetcode qs asked:

<https://leetcode.com/problems/find-peak-element/description/>

<https://leetcode.com/problems/boats-to-save-people/description/>

<https://leetcode.com/problems/split-array-largest-sum/description/>

<https://leetcode.com/problems/number-of-visible-people-in-a-queue/description/>

<https://leetcode.com/problems/path-in-zigzag-labelled-binary-tree/description/>

<https://leetcode.com/problems/univalued-binary-tree/description/>

<https://algo.monster/liteproblems/250>

<https://www.geeksforgeeks.org/dsa/find-count-of-singly-subtrees/>

<https://leetcode.com/problems/longest-consecutive-sequence/description/>

---